

UNIVERSITATEA POLITEHNICA BUCURESTI

Catedra de Calculatoare

Adina Magda Florea

Elemente de Inteligenta Artificiala

Volumul I. Principii si Modele

Bucuresti 1993

Prefata

Inteligența artificială este un domeniu nou al științei calculatoarelor care a evoluat rapid în ultimii ani de la stadiul cercetărilor de laborator la acela al unui domeniu cu multiple sfere de aplicatii. La ora actuală există un întreg arsenal de tehnici care permit construirea programelor și sistemelor de inteligență artificială, multe dintre aceste sisteme fiind produse comerciale aplicate cu succes în diverse domenii. Scopul acestei cărți este acela de a oferi o introducere în problemele și tehnicile inteligenței artificiale, prezentând fundamentarea teoretică a disciplinei și modalitățile prin care tehnicile existente pot fi utilizate în dezvoltarea programelor.

Această carte a început a fi scrisă ca text al cursurilor de "Inteligența artificială" și "Elemente de inteligență artificială" predate studenților facultății de Automatică și Calculatoare, specializarea Calculatoare. Materialul rezultat necesită mai mult de un curs de un semestru pentru a putea fi cuprins. Din acest motiv, la predarea acestui curs s-a selectat un subset de subiecte pentru a fi prezentate, alte elemente, cum ar fi dezvoltarea aplicațiilor de inteligență artificială în Prolog, fiind abordate în partea aplicativă a cursului. Cursul presupune existența unor cunoștințe de matematică, în special matematică discretă, cunoștințe de structuri de date și algoritmi, în special arbori, stive, cozi și grafuri, și familiaritate cu tehnicile de programare recursivă și cu complexitatea calculului.

Cartea este structurată în patru părți: "Rezolvarea problemelor în inteligența artificială", "Reprezentarea cunoștințelor", "Sisteme de inteligență artificială" și "Programarea logică", primele două părți fiind continute în volumul de față, iar următoarele două formând volumul al doilea.

Partea I face o introducere în inteligența artificială, examinând natura problemelor și tehnicilor de inteligență artificială. Apoi se prezintă modalități de rezolvare a problemelor prin căutare, descriind algoritmi pentru diverse strategii de căutare, cu accent pe căutarea euristica.

În ultimii 20 de ani cercetările de inteligență artificială au demonstrat că inteligența implică mai mult decât capacitatea de a raționa. Un program poate simula un comportament inteligent numai dacă conține o cantitate semnificativă de cunoștințe. Partea a II-a prezintă o serie de modele de reprezentare a cunoștințelor într-un program de inteligență artificială. Aceste modele includ logica cu predicate, regulile de producție, rețelele semantice și unitățile. Se prezintă, de asemenea, tehnici de raționament statistic pentru rezolvarea problemelor cu cunoștințe incerte.

Primele două părți ale textului reprezintă fundamentarea domeniului și oferă suportul necesar pentru următoarele două părți. Partea a III-a descrie o serie de aplicații tipice ale inteligenței artificiale, cum ar fi sistemele bazate pe cunoștințe, sisteme cu cunoștințe distribuite și învățarea automată. Rețelele neuronale, ca model subsimolic alternativ al raționamentului, sunt prezentate pe scurt tot în această parte. O serie de aplicații ale inteligenței artificiale, cum ar fi prelucrarea limbajului natural, înțelegerea vorbirii și teoria jocurilor, nu sunt tratate în textul de față. Ele sunt însă abordate pe scurt, sub forma de paradigme de implementare în capitolele 6, 8 și, respectiv, capitolele 1 și 2.

Limbajul Prolog este unul dintre limbajele preferentiale ale inteligentei artificiale. Partea a IV-a contine descrierea limbajului Prolog si exemple de programe specifice inteligentei artificiale. Aceasta parte ofera cititorului un instrument de implementare a tehnicilor prezentate si, in anumite cazuri, o reformulare recursiva a unor algoritmi nerecursivi prezentati.

Textul contine algoritmi descrisi intr-o forma relaxata de pseudocod, cu teste si operatii descrise in limbaj natural, dar cu precizia necesara specificarii cu acuratete a prelucrarilor. In pseudocodul utilizat s-a introdus o structura de control suplimentara, "intoarce valoare", care semnifica intreruperea executiei algoritmului si obtinerea valorii indicate ca rezultat. Algoritmii prezentati pot fi usor tradusi in limbaje de programare cum ar fi Lisp, Prolog, C, si chiar Pascal, cu un mic efort suplimentar.

Deoarece inteligenta artificiala este un domeniu despre care s-au scris putine carti la noi in tara, numerosi termeni din jargonul in limba engleza a disciplinei nu au un echivalent general acceptat in limba romana. Pentru a facilita accesul cititorului care parcurge cunostintele prezentate in text la bibliografia de specialitate, s-a introdus, in anexa, un dictionar roman-englez de termeni semnificativi.

Construirea formei finale a acestei carti a fost posibila numai datorita mamei mele si lui Adrian Boangiu. Societatea M.T.IL ROM m-a ajutat la tiparirea textului de fata.

Bucuresti, 1993

Adina Magda Florea

Partea I	- 6 -
Capitolul 1	- 7 -
1.1 Inteligenta artificiala: o incercare de definire	- 7 -
1.2 Natura problemelor de inteligenta artificiala	- 8 -
1.3 Dezvoltarea domeniului	- 11 -
1.4 Structura sistemelor de inteligenta artificiala	- 15 -
1.5 Tehnici de inteligenta artificiala	- 17 -
Capitolul 2	- 24 -
2.1 Reprezentarea solutiei problemei	- 24 -
2.2 Strategii de cautare de baza	- 33 -
2.3 Strategii de cautare euristica	- 42 -
2.4 Consideratii de complexitate a strategiilor de cautare	- 58 -
2.5 Exercitii si probleme	- 59 -
Partea a II-a	- 63 -
Capitolul 3	- 64 -
3.1 Logica propozitionala	- 65 -
3.2 Logica cu predicate de ordinul I	- 72 -
3.3 Demonstrarea teoremelor utilizind rezolutia	- 80 -
3.4 Exercitii si probleme	- 95 -
Capitolul 4	- 98 -
4.1 Reprezentarea cunostintelor sub forma regulilor de productie	- 98 -
4.2 Sisteme bazate pe reguli de productie	- 102 -
4.3 Paradigme de sisteme bazate pe reguli	- 111 -
4.4 Exercitii si probleme	- 123 -
Capitolul 5	- 125 -
5.1 Modelul probabilistic Bayesian	- 125 -
5.2 Modelul factorilor de certitudine din sistemul MYCIN	- 128 -
5.3 Teoria Dempster-Shafer	- 133 -
5.4 Retele Bayesiene	- 140 -
5.5 Exercitii si probleme	- 142 -
Capitolul 6	- 144 -
6.1 De la reprezentarea logica la modelul cunostintelor structurate	- 145 -
6.2 Retele semantice	- 149 -
6.3 Unitati	- 162 -

6.4	Structuri "tari" de reprezentare a cunostintelor	- 173 -
6.5	Exercitii si probleme	- 185 -

Partea I

Rezolvarea problemelor in inteligenta artificiala

Capitolul 1

Ce este inteligenta artificiala?

Dezvoltarea spectaculoasa a calculatoarelor in ultimii treizeci de ani a permis cercetarilor in domeniu sa incerce utilizarea calculatoarelor pentru rezolvarea unor probleme din ce in ce mai dificile, din ce in ce mai apropiate de complexitatea problemelor solutionate de om. Pe masura ce problemele de viteza si capacitate de memorare au fost rezolvate la nivelul tehnologiei constructiei calculatoarelor, limitarea utilizarii tehnicii de calcul in locul expertului uman se datoreaza mai ales incapacitatii oamenilor de a instrui si programa adecvat calculatoarele. Incercarea extinderii utilizarii calculatoarelor la tot ceea ce poate fi solutionat de om a dus la aparitia unui nou domeniu al stiintei calculatoarelor: inteligenta artificiala.

1.1 Inteligenta artificiala: o incercare de definire

Ce este inteligenta artificiala? Exista numeroase definitii sau incercari de a defini inteligenta artificiala. Multitudinea acestor definitii provine tocmai din faptul ca domeniul, fiind legat de insasi esenta naturii umane, este deosebit de provocator. O incercare de definire a inteligentei artificiale ar trebui sa porneasca de la definitia inteligentei, definitie departe de a fi banal de formulat. Multe din abordarile caracterizarii domeniului au eludat sau au atins doar partial acest aspect, altele l-au considerat implicit. Citeva definitii ale inteligentei artificiale, dintre cele mai cunoscute si relevante, sint date in continuare:

- Inteligenta artificiala este domeniul stiintei calculatoarelor care se ocupa de studiul si crearea sistemelor de calcul si a programelor care prezinta o forma de inteligenta: sisteme care invata noi concepte, care pot rationa si deduce concepte utile intr-un domeniu al lumii inconjuratoare, sisteme care pot intelege limbajul natural sau percepe si intelege un peisaj, intr-un cuvint sisteme care necesita capacitati inteligente specifice omului.
- Inteligenta artificiala este studiul ideilor care permit calculatoarelor sa fie inteligente.
- Inteligenta artificiala este studiul facultatilor mentale pe baza modelelor computationale.
- Inteligenta artificiala se distinge prin subiectele pe care le trateaza, nu prin istorie sau metode specifice. Subiectul tratat de inteligenta artificiala este mintea, considerata ca un sistem de prelucrare a informatiei.
- Un program inteligent este un program care manifesta o comportare similara cu aceea a omului cind este confruntat cu o problema similara. Nu este necesar ca programul sa rezolve sau sa incerce sa rezolve problema in acelasi mod in care ar rezolva-o oamenii.
- Inteligenta artificiala este studiul procesului prin care calculatoarele pot fi instruite sa faca lucruri care, pentru moment, sint facute mai bine de oameni.
- Marvin Minsky, intrebata ce este inteligenta artificiala, a raspuns: "Exista intotdeauna persoane care au nevoie sa defineasca totul pentru a realiza ceva. De ce?"

Se poate observa din aceste definitii ca anumite curente de opinii privesc inteligenta artificiala ca o modalitate de cercetare, descoperire si simulare (copiere) a modului de functionare a inteligentei umane. Aceasta perspectiva a condus la numeroase cercetari in inteligenta artificiala si la dezvoltarea unor noi domenii cum ar fi stiinta cunoasterii, domeniu studiat de psihologi, lingvisti, informaticieni, filozofi, si domeniul retelelor neuronale, numit si inteligenta artificiala la nivel subsimbolic.

O a doua perspectiva asupra inteligentei artificiale considera domeniul dintr-un punct de vedere pragmatic. Nu conteaza daca inteligenta artificiala utilizeaza modelele si mecanismele comportamentului inteligent uman, importanta este capacitatea sistemelor de calcul de a rezolva aceleasi probleme cu performante similare cu cele ale oamenilor. Textul de fata se orienteaza preponderent spre aceasta opinie asupra inteligentei artificiale.

Ca orice stiinta, inteligenta artificiala se ocupa de o serie de probleme cu caracteristici generale comune si dezvolta tehnici specifice de rezolvare a acestor probleme. Sectiunile urmatoare vor delimita specificul problemelor si tehnicilor de inteligenta artificiala, acestea fiind reluate pe larg in capitolele ce urmeaza.

1.2 Natura problemelor de inteligenta artificiala

Majoritatea cercetarilor in domeniul inteligentei artificiale efectuate la inceputul aparitiei disciplinei s-au orientat spre rezolvarea unor probleme usor formalizabile dar considerate ca necesitind un comportament inteligent: demonstrarea teoremelor si jocurile. Programul "The Logic Theorist" al lui A. Newell, J. Shaw si H. Simon [1963] putea sa demonstreze mai multe teoreme din primul capitol al lucrarii "Principia Mathematica" a lui Whitehead si Russell. Programul care juca sah al lui A. Samuel [1967] isi imbunatatea performantele de joc dupa fiecare partida jucata. Problemele rezolvate de aceste programe au o caracteristica comuna: ele sint probleme grele, deci NP-complete. Oricit de performant ar fi un calculator, acesta nu poate rezolva o problema cu un algoritm de complexitate timp $O(e^N)$, pentru o valoare semnificativa a dimensiunii N a intrarii, intr-un interval de timp rezonabil. Cercetarile in inteligenta artificiala s-au orientat tocmai spre incercarea de a reduce explozia combinationala implicata de cautarea solutiei acestui tip de probleme. Una din contributiile mari aduse de inteligenta artificiala stiintei calculatoarelor este utilizarea euristicilor in rezolvarea problemelor grele, transformind astfel probleme inabordabile din punct de vedere al timpului de rezolvare in probleme posibil de rezolvat in numeroase cazuri.

Incercarea de a simula comportamentul inteligent uman a dus la investigarea unor alte tipuri de probleme, probleme care implica un grad inalt de expertiza umana, cum ar fi diagnosticul medical, managementul, proiectarea si rezolvarea problemelor ingineresti generale. Aceste probleme necesita modelarea unor cantitati mari de informatii, informatii greu de formalizat, cum ar fi experienta si intuitia. Sistemele expert, rezultat direct al acestor cercetari, sint acum mult utilizate in domeniile amintite.

O alta directie a cercetarilor de inteligenta artificiala a fost aceea a rezolvarii problemelor banale, cotidiene, care necesita cunostinte de bun simt. Aceste probleme includ rationamentul

despre obiecte fizice si relatiile intre ele, si rationamentul despre actiuni si consecintele acestora. Oricine stie, de exemplu, ca un obiect nu poate sa fie simultan in doua locuri diferite sau ca nu trebuie sa dea drumul unui pahar din mina deoarece poate sa cada si sa se sparga. Aceste comportamente pot fi greu caracterizate ca necesitind inteligenta si, totusi, ele sint cele mai greu de modelat intr-un program. Cunostintele de bun simt sint la indemina oricarui om dar ele trebuie reprezentate explicit intr-un program, iar volumul lor este impresionant. Surprinzator, cercetarile de inteligenta artificiala au avut rezultate cu mult mai bune in domenii ca rezolvarea problemelor formale dificile cum ar fi jocurile, demonstrarea teoremelor, sau a problemelor care necesita expertiza umana intr-un anumit domeniu, decit in domeniile care necesita cunostinte de bun simt. S-a reusit construirea unui program care sa demonstreze teoreme matematice complicate si care sa descopere chiar concepte matematice noi, dar nu s-a reusit construirea unui program care sa stie tot ceea ce stie un copil de doi ani!

- Reprezentarea cunostintelor • Demonstrarea automata a teoremelor
- Sisteme expert • Teoria jocurilor
- Achizitia cunostintelor si invatare • Planificare automata
- Prelucrarea limbajului natural
- Perceptie: intelegerea imaginilor si a vorbirii • Robotica
- Limbaje si medii de dezvoltare pentru inteligenta artificiala

Figura 1.1 Domeniile inteligentei artificiale

In final, multe cercetari in domeniul inteligentei artificiale s-au orientat spre imitarea altor capacitati umane cum ar limbajul, vazul, auzul. Perceptia si recunoasterea imaginilor, intelegerea limbajului vorbit sau scris, sinteza limbajului natural si a vocii sint probleme deosebit de provocatoare care implica atat algoritmi sofisticati de prelucrare cit si dispozitive tehnice complicate.

Inteligenta artificiala s-a organizat astfel intr-un numar de subdomenii care, desi au in comun aceleasi principii de reprezentare a informatiei si aceleasi tehnici de rezolvare a problemelor, s-au concentrat pe diverse aplicatii. Aceste subdomenii sint prezentate in Figura 1.1 si multe dintre ele vor constitui subiectul capitolelor urmatoare.

O discutie precisa despre natura, specificul si modalitatile de rezolvare a problemelor de inteligenta artificiala trebuie sa considere urmatoarele patru intrebari importante:

- (1) Este posibila simularea comportamentului inteligent pe calculator?
- (2) La ce nivel se incearca modelarea comportamentului inteligent?
- (3) Care este criteriul pe baza caruia se apreciaza inteligenta unui program?
- (4) Care sint reprezentarile si tehnicile utilizate in rezolvarea problemelor de inteligenta artificiala?

In limitele definitiilor anterioare ale domeniului, raspunsul la prima intrebare poate fi conjectura lui McCarthy sau ipoteza lui Newell si Simon. John McCarthy, una dintre personalitatile celebre ale inteligentei artificiale, a facut urmatoarea conjectura:

"Orice aspect al invatarii sau orice alta caracteristica a inteligentei umane poate fi descrisa suficient de precis astfel incit o masina sa o poata simula."

Pe de alta parte, Allen Newell si Herbert Simon, alte doua nume semnificative in inteligenta artificiala, au incercat sa raspunda aceleiasi intrebari pornind de la definitia unui *sistem fizic de simboluri* [Newell,Simon,1976]:

"Un sistem fizic de simboluri este format dintr-o multime de entitati, numite simboluri, care sint sabloane fizice ce pot apare drept componente ale altor tipuri de entitati numite structuri (sau structuri simbolice). Astfel, o structura simbolica este compusa dintr-un numar de instante (particularizari) ale simbolurilor legate intr-un mod fizic. In orice moment de timp sistemul va contine o colectie de astfel de structuri simbolice. Pe linga aceste structuri, sistemul contine deasemenea o colectie de procese care opereaza asupra expresiilor pentru a produce alte expresii: procese de creare, modificare, reproducere si distrugere. Un sistem fizic de simboluri este o masina care produce in timp o colectie de structuri simbolice care evolueaza. Un astfel de sistem exista intr-o lume de obiecte mai cuprinzatoare decit aceea a expresiilor simbolice."

Pornind de la aceasta definitie, Newell si Simon au facut urmatoarea ipoteza, numita *ipoteza sistemului fizic de simboluri*:

"Un sistem fizic de simboluri are capacitatile necesare si suficiente pentru a produce actiuni general inteligente."

Evident, atit conjectura lui McCarthy cit si ipoteza sistemului fizic de simboluri nu au fost demonstrate si validarea lor ramine un subiect deschis. Ele au reprezentat insa un punct de plecare al inteligentei artificiale si, in acelasi timp, provocarea esentiala adusa disciplinei. Discutia acestor ipoteze se leaga inevitabil atit de raspunsul la cea de a doua intrebare, intrebare discutata deja in prima sectiune, cit si de raspunsul la cea de a treia intrebare.

Cel mai celebru criteriu de apreciere a inteligentei unui program, deci criteriul care ar putea raspunde la intrebarea (3), este *testul Turing*. In 1950, Alan Turing (1912-1954), celebru matematician britanic si unul dintre intemeietorii stiintei calculatoarelor, a propus un test pentru a determina daca o masina poate avea sau nu un comportament inteligent. In articolul sau "Computing Machinery and Intelligence", Turing a pus pentru prima oara problema posibilitatii simularii gindirii umane cu ajutorul calculatorului. In acelasi articol, Turing descrie si testul care, afirma el, poate discerne intre o comportare inteligenta si una neinteligenta.

Testul Turing consta in principiu din urmatorul experiment. Un om comunica prin intermediul unui terminal al calculatorului cu alte doua terminale, situate in camere invecinate, punind intrebari si obtinind raspunsuri. Raspunsurile sint date de o persoana la unul din cele doua terminale ascunse celui de la care se intreaba si de un program la celalalt terminal. Daca persoana care intreaba nu poate stabili in urma dialogului care este terminalul de unde i-a raspuns omul si care este cel de la care i-a raspuns programul atunci programul are o comportare inteligenta. Programul poate fi astfel facut incit sa incerce sa pacaleasca pe cel care intreaba. De exemplu, daca programul este intrebat cit fac 12.120×32.425 , se asteapta citeva minute pina se primeste raspunsul. Problema cea mai importanta ridicata de realizarea unui program care sa treaca un astfel de test este cantitatea de informatie necesara programului tinind cont de spectrul larg al intrebarilor posibil de pus, cit si de faptul ca acestea sint puse in limbaj natural.

Pina la ora actuala nici un program nu a reusit sa treaca testul Turing. Cu toate acestea, testul Turing sugereaza o masura a performantelor unui program daca se considera domenii restrinse ale discursului. De exemplu, un program de jucat sah poate ajunge sa aiba performante similare cu cele ale unui maestru iar oponentul sa nu stie daca joaca sah cu o persoana sau cu un program. In alte domenii este posibil sa se compare performantele rezolvarii unei probleme de catre un program cu cele ale unui expert in domeniu, atat din punct de vedere al calitatii solutiei cit si din punct de vedere al vitezei de rezolvare.

Testul Turing este insa vulnerabil, in ciuda sarmului lui intelectual, din mai multe motive. In primul rind testul verifica numai capacitatile pur simbolice de rezolvare a problemelor, neluind in considerare aspecte cum ar fi perceptia sau dexteritatea manuala. Pe de alta parte exista posibila obiectie ca acest test impune ca inteligenta masinilor sa modeleze inteligenta umana. Unii cercetatori sustin ca inteligenta masinilor este o forma diferita de inteligenta si ca este o gresala sa incercam a o evalua in termenii inteligentei umane. Se doreste oare construirea unei masini capabile sa simuleze activitatea sociala a unui om si care sa fie la fel de lenta ca acesta in efectuarea calculelor matematice? Toate aceste intrebari ramin pentru moment fara un raspuns definitiv, unele depasind cu mult sfera inteligentei artificiale.

Raspunsurile la intrebarea (4) vor fi prezentate pe scurt in Sectiunile 1.4 si 1.5 si discutate pe larg in capitolele care urmeaza. Discutarea acestor raspunsuri reprezinta de fapt nucleul textului de fata.

1.3 Dezvoltarea domeniului

Ideea construirii unei masini inteligente precede cu multe sute de ani aparitia domeniului inteligentei artificiale. O superba prezentare a evolutiei acestei idei de-a lungul timpului poate fi gasita in cartea autoarei Pamela McCorduck [1979] "Machines who Think." Inceputurile inteligentei artificiale ca domeniu al stiintei calculatoarelor pot fi situate in jurul anului 1950. Cercetarile si rezultatele lui Alonzo Church, Kurt Goedel, Emil Post si Alan Turing din perioada anilor '20-'40 au pus in evidenta posibilitatea utilizarii metodelor formale de rationament si a reprezentarilor simbolice, creind astfel cadrul nasterii inteligentei artificiale. Cercetarile din aceeasi perioada in domeniul ciberneticii, legate de numele lui Norbert Wiener, prin modelarea comunicarii atat la nivel uman cit si la nivelul masinii, au fost un alt factor care a favorizat aparitia inteligentei artificiale.

In timpul anilor '50-'60 mai multe evenimente au marcat momentul crearii acestui nou domeniu. Asa cum s-a spus, in 1950 Alan Turing publica articolul care, pentru prima oara, pune problema posibilitatii simularii gindirii umane cu ajutorul calculatorului. Momentul nasterii oficiale a inteligentei artificiale este considerat a fi Conferinta de la Dartmouth College din 1956, organizata de John McCarthy de la Dartmouth College si Marvin Minsky de la Massachusetts Institute for Technology. La aceasta conferinta se intilnesc primii patru mari initiatori ai domeniului: John McCarthy, Marvin Minsky, Alen Newell si Herbert Simon, ultimii doi de la Carnegie Mellon University.

Se pare ca termenul de inteligenta artificiala a fost inventat in timpul acestei conferinte de catre McCarthy. Tot el este acela care a enuntat si conjectura posibilitatii simularii cu ajutorul calculatorului a oricarui comportament inteligent uman. Intre 1956 si 1957 A. Newell, J. Shaw si H. Simon dezvolta primul program de demonstrare automata a teoremelor, "The Logic Theorist." Incepind din 1960 apar primele programe de inteligenta artificiala. A. L. Samuel dezvolta in 1961 un program de jucat sah care isi imbunatateste performantele dupa fiecare partida jucata. In 1962 si 1963 A. Newell si H. Simon construiesc programul "General Problem Solver" (GPS) pe care incearca sa-l aplice in rezolvarea mai multor probleme, cum ar fi planificarea actiunilor sau manipularea simbolica a expresiilor logice. In 1965 J. A. Robinson introduce rezolutia ca metoda simpla si eficienta de demonstrare automata a teoremelor. Tot in 1965 incepe la Stanford University construirea primului sistem expert, DENDRAL, de catre J. Lederberg si E. Feigenbaum. DENDRAL [Lindsay,s.a.,1980] era un sistem expert capabil sa sintetizeze structura moleculelor organice pe baza formulelor chimice si a spectogramelor de masa. In 1968 C. Engleman, de la Massachusetts Institute for Technology, incepe dezvoltarea limbajului MACSYMA destinat rezolvarii simbolice a ecuatiilor.

In acelasi timp apare necesitatea existentei unor limbaje de programare mai puternice, capabile sa exprime la nivel simbolic informatiile necesare programelor de inteligenta artificiala. Se dezvolta astfel o noua filozofie a programarii, programarea descriptiva sau declarativa in care rezolvarea unei probleme se face pe baza descrierii universului problemei in termenii obiectelor, atributelor si a relatiilor existente intre acestea. In scopul cresterii eficientei activitatii de programare simbolica se dezvolta astfel o noua clasa de limbaje: limbajele declarative. Limbajele functionale, limbajele logice si limbajele orientate pe obiecte sint exemple de astfel de limbaje.

Limbajul Lisp (LISt Processing) a fost creat de John McCarthy in 1959. Limbajul Lisp [Siklossy,1977;Norvig,1992] este un limbaj functional destinat prelucrarii simbolice a informatiei, cu o sintaxa simpla, cu tipuri de date simple si gestiunea dinamica a memoriei. Limbajul Lisp a fost si este considerat limbajul preferential al inteligentei artificiale. Limbajul Prolog (PROgrammation et LOGique) a fost inventat de Alain Colmerauer la universitatea Marseille-Aix in 1972. Limbajul Prolog [Clocksin,Mellish,1981;Sterling,Shapiro,1986] este cel mai raspindit limbaj de programare logica si va fi prezentat in Capitolele 11 si 12.

O alta categorie de limbaje care au contribuit la dezvoltarea rezolvarii problemelor de inteligenta artificiala este aceea a limbajelor orientate pe obiecte. Smalltalk [Goldberg,Robson,1983] este limbajul reprezentativ pentru acest tip de programare si caracteristici ale limbajelor orientate pe obiecte au fost incluse si in limbajele procedurale de nivel inalt.

Studiile si cercetarile de inceput in inteligenta artificiala s-au orientat mai ales spre incercarea de a construi sisteme generale, bazate pe metode de inferenta puternice, care incercau sa functioneze si in cazul unor cunostinte limitate despre domeniul problemei. Exemple de astfel de sisteme sint programele de jucat sah, "The Logic Theorist", GPS si altele din perioada anilor '60. In scurt timp s-a dovedit ca ipoteza conform careia un program care contine metode de rezolvare puternice dar generale, si care se bazeaza in rezolvarea problemelor numai pe viteza de calcul a masinii, nu era capabil sa rezolve decit probleme foarte simple. De indata ce un astfel de

sistem trebuia sa rezolve probleme reale, complexe, explozia combinationala implicata de solutionarea problemelor facea ca programul sa devina inefectiv.

In urma acestor rezultate, la inceputul anilor '70 s-a produs o modificare fundamentala in modul de abordare si dezvoltare a sistemelor de inteligenta artificiala. Cercetatorii au ajuns la concluzia ca realizarea unui sistem inteligent capabil sa rezolve probleme reale, complexe, necesita un volum substantial de *cunostinte* specifice domeniului. In legatura cu aceasta idee a fost frecvent citata maxima lui Francis Bacon: "Scientia et potentia in idem coincidunt." Perspectiva importanței cunostintelor in programele de inteligenta artificiala este sustinuta si de observatia ca un specialist intr-un anumit domeniu nu poate lucra performant in alte domenii, oricit de puternica ar fi capacitatea lui de rationament. Descoperirea rolului cunostintelor specifice domeniului in rezolvarea unei probleme dificile a generat o noua conceptie a sistemelor de inteligenta artificiala: *sistemele bazate pe cunostinte*. Eduard Feigenbaum [1977] a rezumat aceasta noua conceptie intr-o lucrare prezentata la "The International Joint Conference on Artificial Intelligence" in 1977. El a pus in evidenta faptul ca adevarata putere de rezolvare a unui program este determinata in primul rind de cantitatea de cunostinte pe care o posedea si numai in al doilea rind de modalitatile de rationament general utilizate.

Sistemul expert DENDRAL este unul din primele exemple de sisteme bazate pe cunostinte. El a fost caracterizat de Feigenbaum ca "o masina de aplicare a cunostintelor." Sistemul MYCIN [Buchanan, Shortliffe, 1984], sistem expert pentru diagnosticarea infectiilor bacteriene ale singelui, a carui dezvoltare a inceput la Stanford University in jurul anilor '74-'75, este un alt exemplu de sistem care a pus in evidenta rolul important al cunostintelor specifice domeniului. Incepind din aceasta perioada si pina in prezent, toata comunitatea cercetatorilor in domeniul inteligentei artificiale a recunoscut rolul esential al cunostintelor in rezolvarea inteligenta a problemelor.

Din acest motiv o parte importanta a cercetarilor de inteligenta artificiala din ultimele doua decade s-au orientat spre dezvoltarea metodelor si instrumentelor de modelare a cunostintelor (Figura 1.2). Prima etapa a fost marcata de evolutia limbajelor generale de inteligenta artificiala, cum ar fi Lisp si Prolog. A doua etapa a marcat dezvoltarea de sisteme expert dedicate, specializate intr-un anumit domeniu, care aveau propriul lor limbaj de reprezentare a cunostintelor. Din aceste limbaje specializate au evoluat limbaje de reprezentare a cunostintelor independente de domeniu, numite si *limbaje de nivel foarte inalt*. Aceste limbaje, in general tot limbaje declarative, permit exprimarea modulara a cunostintelor in forme mult apropiate de limbajul natural. Desi aceste limbaje sint independente de domeniu, reprezentarea cunostintelor si metodele de rationament incorporate favorizeaza utilizarea fiecarui limbaj pentru o anumita clasa de probleme, facind dificila sau aproape imposibila utilizarea limbajului pentru probleme cu alte caracteristici. In jurul acestor limbaje de nivel foarte inalt s-au dezvoltat medii de programare care au condus la aparitia *sistemelor cadru de dezvoltare a sistemelor bazate pe cunostinte*. Limbajele de programare ale inteligentei artificiale au jucat un rol important in dezvoltarea cercetarilor din domeniu. De multe ori, idei si tehnici noi au fost insotite de un nou limbaj care sustinea in mod natural aceste idei si tehnici. Exista la ora actuala un peisaj complex si baroc al limbajelor de inteligenta artificiala si a mediilor de dezvoltare a sistemelor bazate pe cunostinte. Dintre acestea

cele care s-au impus in ultimul timp sint mai ales KEE [Kikes,Kehler,1985; Filman,s.a.,1992; Filman,1992] si OPS5 [Cooper,Wogrin,1988].

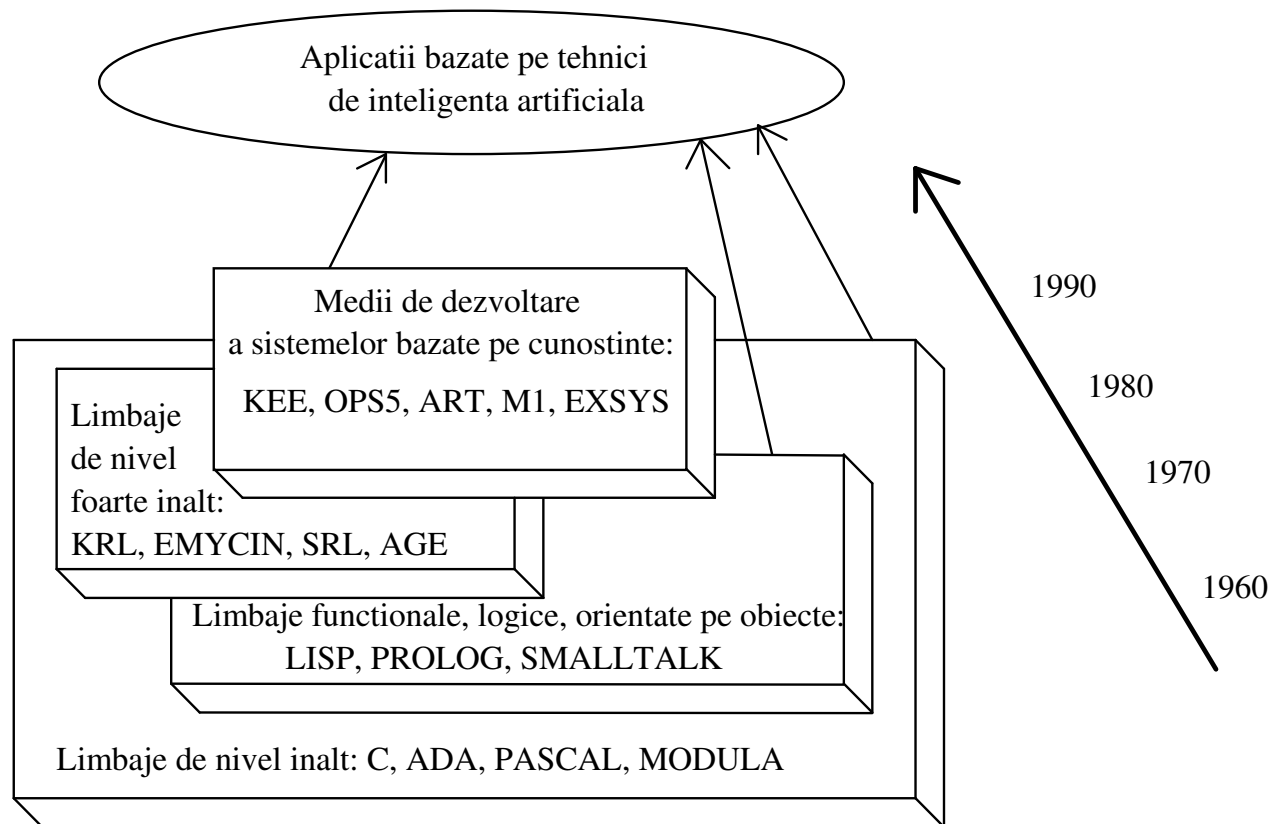


Figura 1.2 Evolutia limbajelor si sistemelor de inteligenta artificiala

Odata cu impunerea importantei cunostintelor in sistemele inteligente a aparut o noua ramura a inteligentei artificiale, *ingineria cunostintelor*. Ingineria cunostintelor se ocupa de studiul metodelor si tehnicilor de achizitie, reprezentare si organizare a cunostintelor in sistemele bazate pe cunostinte. Ingineria cunostintelor incearca sa rezolve una dintre problemele considerate cheie din punct de vedere al limitarii timpului de dezvoltare a unui sistem bazat pe cunostinte: transferul cunostintelor specializate ale expertului uman in sistemul bazat pe cunostinte. Feigenbaum [1977] defineste ingineria cunostintelor dupa cum urmeaza.

"Ingineria cunostintelor practica arta de a utiliza principiile si instrumentele cercetarilor de inteligenta artificiala in rezolvarea problemelor aplicative care necesita cunostinte experte. Aspectele tehnice ale achizitiei acestor cunostinte, ale reprezentarii acestora si ale utilizarii lor adecvate pentru construirea si explicarea liniilor de rationament, sint probleme importante in proiectarea sistemelor bazate pe cunostinte. Arta de a construi agenti inteligenti este in acelasi timp o parte din si o expresie a artei programarii. Este arta de a construi programe de calcul complexe care reprezinta si rationeaza cu cunostintele lumii inconjuratoare."

La ora actuala inteligenta artificiala a incetat sa mai fie apanajul unui numar restrins de initiati si sa fie practicata numai in universitati sau institute de cercetare. Exista numeroase sisteme comerciale de inteligenta artificiala si aplicatii functionale construite pe baza tehnicilor de inteligenta artificiala. Aceste aplicatii devin din ce in ce mai frecvent parti ale unor sisteme complexe care includ multe componente de programare clasica. Din acest motiv, mediile de

dezvoltare ale sistemelor bazate pe cunostinte existente la ora actuala au componente de interfata cu limbaje de programare de nivel inalt, sisteme de gestiune a bazelor de date, etc. In plus, anumite aplicatii de inteligenta artificiala sint dezvoltate in limbaje cum ar fi C sau Ada. Metodele de reprezentare a informatiei si tehnicile de rezolvare a problemelor descoperite de inteligenta artificiala au depasit sfera stricta a domeniului si au fost incluse in numeroase alte aplicatii complexe.

1.4 Structura sistemelor de inteligenta artificiala

Un sistem de inteligenta artificiala, sau sistem bazat pe cunostinte, este format din doua componente fundamentale:

- (1) *baza de cunostinte* care contine cunostintele specifice domeniului problemei si, eventual, cunostinte generale, si
- (2) *motorul de inferenta* care utilizeaza cunostintele pentru rezolvarea problemei.

Din punct de vedere al inteligentei artificiale cunostintele pot fi vazute ca multimea de fapte si principii acumulate de oameni sau, mai general, nivelul cunoasterii umane la un anumit moment dat. Cunostintele includ idei, concepte, teorii, abstractizari, limbaje, reguli, locuri, obiceiuri. Cunostintele dintr-un sistem inteligent, care se refera de obicei la un anumit domeniu, descriu universul problemei sau al clasei de probleme de rezolvat la nivel simbolic si formeaza continutul bazei de cunostinte.

Modul de reprezentare si organizare a cunostintelor este un element esential al oricarui sistem inteligent. Partea a II-a a textului de fata este dedicata in intregime acestui subiect. Cunostintele dintr-un sistem bazat pe cunostinte trebuie sa posede urmatoarele caracteristici:

- Cunostintele trebuie sa fie generale. Situatiile care prezinta proprietati comune trebuie sa poata fi reprezentate prin structuri simbolice comune si nu punctual, fiecare in parte. Daca cunostintele nu au aceasta proprietate cantitatea de memorie necesara descrierii acestora poate deveni imensa.
- Cunostintele unui sistem inteligent sint in continua schimbare deoarece ele reflecta schimbarile din lumea inconjuratoare. Reprezentarea cunostintelor in sistem trebuie sa poata modela usor aceste schimbari si sa permita dezvoltarea incrementala a bazei de cunostinte.
- Reprezentarea cunostintelor trebuie sa faciliteze achizitia lor. Multe din cunostintele necesare unui sistem inteligent sint cunostinte greu sau imposibil de formalizat si ele trebuie obtinute de la oamenii care le poseda. O reprezentare poate facilita sau ingreuna acest proces.
- Cunostintele trebuie sa poata fi utilizate in orice instanta a problemei de rezolvat, chiar daca sint incomplete sau partial incorecte.
- Cunostintele intr-un sistem inteligent trebuie sa fie organizate intr-o structura care sa corespunda modului in care acestea vor fi utilizate.
- Cunostintele trebuie sa fie astfel reprezentate, organizate si utilizate incit sa permita transparenta sistemului. Expertul sau utilizatorul trebuie sa poata inspecta cunostintele

utilizate în rezolvarea unei probleme și inferențele pe baza cărora problema a fost rezolvată.

Aceste caracteristici pun în evidență diferența existentă între cunoștințe și date. Feigenbaum și McCorduck [1983] ilustrează această diferență prin următorul exemplu. Un medic care tratează un pacient folosește cunoștințe și date. Datele sunt reprezentate de fișa pacientului: simptome, boli anterioare, tratament prescris, reacție la tratament, etc. Cunoștințele utilizate în tratarea pacientului reprezintă tot ceea ce medicul a învățat în facultate și în decursul întregii lui cariere prin practică, studiu, experiență, în legătură cu modul de vindecare a bolii. Aceste cunoștințe se referă la fapte, teorii, tratament și, cel mai important, la cunoștințe euristice. Astfel, cunoștințele necesită și includ utilizarea datelor dar sunt mai mult decât acestea.

Una dintre cele mai dificile probleme ale reprezentării cunoștințelor într-un sistem inteligent este reprezentarea cunoștințelor euristice. *Cunoștințele euristice* reprezintă o formă particulară de cunoștințe utilizată de oameni pentru a rezolva probleme complexe. Ele reprezintă cunoștințele utilizate pentru a judeca corect, pentru a lua o decizie, pentru a avea un comportament după o anumită strategie sau a utiliza trucuri sau reguli de bun simț. Acest tip de cunoștințe nu este nici formalizat, nici demonstrat a fi efectiv și câteodată nici corect, dar este frecvent utilizat de oameni în numeroase situații. Judea Pearl [1984], în lucrarea sa "Heuristics. Intelligent Search Strategies for Computer Problem Solving", definește cunoștințele euristice astfel:

"Euristicele sunt criterii, metode sau principii pentru a alege între diverse alternative de acțiune pe aceea care promite a fi cea mai eficientă în realizarea unui scop. Ele reprezintă compromisuri între două cerințe: necesitatea de a lucra cu criterii simple și, în același timp, dorința de a vedea ca aceste criterii fac o selecție corectă între alternativele bune și rele."

Utilizarea cunoștințelor în sistemele bazate pe cunoștințe necesită o formă de raționament, decizii și acțiuni în scopul obținerii de noi cunoștințe care în final vor reprezenta soluția problemei de rezolvat. Aceste acțiuni se fac cu ajutorul metodei de inferență.

Definiție. Se numește *metoda de inferență*, sau pe scurt inferență, procedura de obținere la un moment dat, a noi elemente (fapte) implicate în mod direct de elementele particulare reprezentării.

Fiecare model de reprezentare a cunoștințelor are metode de inferență specifice, așa cum se prezintă în Partea a II-a a acestui text. Pentru a putea ajunge la soluția unei probleme este necesar, de cele mai multe ori, o aplicare repetată a metodei de inferență.

Definiție. Se numește *strategie de control* procesul de aplicare repetată a metodei de inferență pentru a ajunge la soluție, de preferință cât mai repede.

Metoda de inferență împreună cu strategia de control formează nucleul motorului de inferență al unui sistem bazat pe cunoștințe. Capitolul 2 va fi dedicat studiului diverselor strategii de control utilizate în inteligența artificială. Datorită descoperirii și utilizării unor strategii de control adecvate, programele de inteligență artificială au reușit să rezolve probleme grele, deci NP-complete, într-un timp acceptabil pentru dimensiuni semnificative ale intrării.

În continuare se dau definițiile unor noțiuni ce vor fi frecvent utilizate pe parcursul întregii prezentări. Se reia definiția noțiunii de inferență, în forma ei riguroasă, spre deosebire de prima definiție, dată la nivel intuitiv.

Definitie. *Inferenta* este o forma de rationament prin care se trece de la un enunt la altul in mod deductiv sau inductiv direct, caz in care se numeste inferenta imediata, sau indirect, caz in care se numeste inferenta indirecta.

Definitie. *Deductia* este o forma fundamentala de rationament in planul conceptelor in care concluzia decurge cu necesitate din premise. Exemplul tipic de deductie este silogismul.

Definitie. Se numeste *silogism* un tip de rationament deductiv alcatuit din trei judecati:

- (1) *premisa majora* sau termen major care contine predicatul concluziei,
- (2) *premisa minora* sau termen minor care contine subiectul concluziei, si
- (3) *concluzia*, derivata cu necesitate din primele doua.

Legatura intre (1) si (2) este mijlocita de *termenul mediu* care figureaza in ambele premise dar nu si in concluzie.

Termenul de silogism impreuna cu definitia de mai sus au fost date de Aristotel, acesta fiind considerat fondatorul teoriei deductiei. Ulterior deductia a fost dezvoltata de Descartes, Leibniz si de logica simbolica (Capitolul 3), in care silogismul ia forma regulii de inferenta Modus Ponens.

Definitie. *Inductia* este o forma de rationament care trece de la particular la general, de la fapte la concepte. Exista doua tipuri de inductie: inductia completa, daca se enumara toate cazurile existente, si inductia incompleta, daca se enumara numai o parte din cazurile existente.

Inferenta inductiva sta la baza majoritatii proceselor de invatare. Aceasta forma de rationament va fi discutata in Capitolul 9, impreuna cu diverse sisteme de invatare automata.

1.5 Tehnici de inteligenta artificiala

Problemele de inteligenta artificiala fac parte din diverse domenii si par sa nu aiba in comun alta caracteristica decit aceea ca sint dificil de rezolvat. Exista insa tehnici specifice rezolvarii problemelor in inteligenta artificiala. Cum se pot caracteriza aceste tehnici si cum se poate decide daca aceste tehnici pot fi utilizate si in rezolvarea unor probleme care nu sint considerate a fi probleme de inteligenta artificiala?

Asa cum s-a discutat in sectiunile anterioare, rezultatul cel mai important al cercetarilor de inteligenta artificiala din ultimele decenii este punerea in evidenta a rolului fundamental al cunostintelor intr-un sistem inteligent. Caracteristicile cunostintelor impun necesitatea gasirii unor modalitati adecvate de reprezentare si prelucrare a cunostintelor in sistem. Textul de fata are ca scop tocmai prezentarea acestor modalitati de prelucrare a cunostintelor si raspunsul la intrebarea de mai sus. In aceasta sectiune insa, se discuta trei rezolvari ale unei probleme, rezolvarile fiind prezentate in ordine crescatoare a urmatoarelor caracteristici:

- generalitate
- claritatea exprimarii cunostintelor
- extensibilitatea abordarii.

In acest mod, cea de a treia rezolvare ajunge sa devina ceea ce se intelege de obicei printr-o tehnica de rezolvare a problemelor in inteligenta artificiala si reprezinta pentru cititor primul pas de atac al domeniului.

Se considera urmatorul joc, cunoscut sub numele de "Tic-Tac-Toe": pe o tabla de trei linii si trei coloane, care contine deci noua patrate, doi jucatori pot plasa X sau O. Cistiga jucatorul care reuseste sa formeze o secventa de trei simboluri (X, respectiv O) pe orizontala, verticala sau diagonala. Se cere sa se construiasca schema unui program care sa simuleze acest joc, program care sa poata juca pe postul jucatorului X sau pe postul jucatorului O si, evident, care sa incerce sa cistige jocul. Cele trei rezolvari ale problemei sint descrise in termenii reprezentarii universului problemei si ai algoritmului utilizat.

Rezolvarea 1

Se folosesc urmatoarele structuri de date:

- *Tabla* este un vector de 9 elemente care reprezinta tabla de joc, tabla fiind liniarizata pe linii. Valorile elementelor acestui vector sint 0 pentru spatiu liber (blanc), 1 pentru X si 2 pentru O.
- *Tabela_de_mutari* este un vector de 19.683 elemente (3^9), fiecare element al acestui vector fiind la rindul lui un vector de 9 elemente. Fiecarei pozitii posibile de pe tabla de joc ii corespunde o intrare in acesta tabela de mutari care contine noua pozitie obtinuta prin executarea mutarii respective.

Algoritmul de rezolvare este prezentat in continuare.

Algorithm: Problema "Tic-Tac-Toe". Rezolvarea 1

1. Initializeaza Tabla cu blanc si Tabela_de_mutari cu toate pozitile posibile

2. **cit timp** nici un jucator nu a cistigat **executa**

2.1 Considera Tabla ca un numar ternar (in baza 3) si converteste-l in baza 10 obtinind M

2.2 Utilizeaza M ca index in Tabela_de_mutari si obtine

PozNoua $\leftarrow$ Tabela_de_mutari[M]

2.3 Tabla $\leftarrow$ PozNoua

2.4 **daca** PozNoua este pozitie cistigatoare

atunci intrerupe ciclul

2.5 Citeste mutare adversar si modifica Tabla in consecinta

3. Anunta jucatorul cistigator

sfirsit.

Acest algoritm este foarte eficient din punct de vedere al timpului si, teoretic, ar putea sa joace un joc optim de "Tic-Tac-Toe." Algoritmul prezinta insa mai multe dezavantaje. In primul rind aceasta abordare necesita un spatiu foarte mare pentru memorarea tablei de mutari care specifica miscarile corecte asociate fiecarei configuratii a tablei. In al doilea rind este greu sa se stabileasca si sa se introduca valorile corecte in tabela de mutari. In final, daca se doreste extinderea jocului pentru un caz mai general, de exemplu cazul in trei dimensiuni, trebuie refacuta toata tabela de mutari de la inceput. In particular, pentru cazul jocului in trei dimensiuni aceasta abordare nu mai functioneaza de loc deoarece ar trebui memorate 3^{27} pozitii in tabela de mutari. Tehnica folosita in aceasta rezolvare nu respecta nici una din cerintele unei tehnici de inteligenta artificiala.

Rezolvarea 2

Se folosesc urmatoarele structuri de date:

- *Tabla* este un vector similar cu cel prezentat in Rezolvarea 1 dar foloseste urmatoarea codificare a simbolurilor de pe tabla: 2 pentru blanc, 3 pentru X si 5 pentru O.
- *Mutare* este o variabila cu valori intregi care indica numarul mutarii ce urmeaza a se face. Valoarea 1 indica prima mutare iar valoarea 9 indica ultima mutare.

Algoritmul de rezolvare are o strategie predefinita pentru miscarile pe care trebuie sa le execute. Algoritmul executa numai miscarile impare daca joaca pe postul jucatorului X sau miscarile pare daca joaca pe postul jucatorului O. Se folosesc trei subprograme: Muta(N) efectueaza o mutare in patratul N, PosCistig(P) evalueaza posibilitatea de cistig a jucatorului P cu urmatoarea miscare in functie de configuratia curenta a tablei si Scop implementeaza o euristica de alegere a unui patrat in cazul in care exista mai multe miscari posibile care nu pericliteaza cistigul jucatorului curent.

Subprogramul PosCistig(P) trebuie sa verifice pe rind fiecare linie, coloana si diagonala a tablei pentru a investiga posibilitatea de cistig a lui P in urmatoarea miscare. Datorita modului in care sint codificate simbolurile de pe tabla se poate testa posibilitatea de cistig a unei linii, coloane, respectiv diagonale prin inmultirea celor trei valori din secventa. Daca produsul este 27 ($3 \times 3 \times 3$) atunci jucatorul X poate sa cistige. Daca produsul este 50 ($5 \times 5 \times 2$) atunci jucatorul O poate sa cistige. In momentul in care se gaseste o linie, coloana sau diagonala din care se poate cistiga, se cauta pozitia libera din aceasta si se intoarce ca valoare a subprogramului PosCistig(P) aceasta pozitie.

Algorithm: Problema "Tic-Tac-Toe". Rezolvarea 2

1. Initializeaza Tabla cu blanc
2. Mutare $\leftarrow$ 1
3. **cit timp** nici un jucator nu a cistigat **executa**

3.1 alege Mutare dintre

Mutare = 1: Muta(1)

Mutare = 2: **daca** Tabla[5] = 2

atunci Muta(5)

altfel Muta(1)

Mutare = 3: **daca** Tabla[9] = 2

atunci Muta(9)

altfel Muta(3)

Mutare = 4: **daca** PosCistg(X) $\neq$ 0

atunci Muta(PosCistig(X)) /* blocheaza cistig adversar */

altfel Muta(Scop)

Mutare = 5: **daca** PosCistg(X) $\neq$ 0

atunci

(i) Muta(PosCistig(X)) /* deci cistiga X */

(ii) **intrerupe ciclul**

altfeldaca PosCistig(O) $\neq$ 0

atunci Muta(PosCistig(O))/* blocheaza cistig adversar */

altfeldaca Tabla[7] = 2

atunci Muta(7)

altfel Muta(3)

Mutare = 6: **daca** PosCistig(O) $\neq$ 0

atunci

(i) Muta(PosCistig(O)) /* deci cistiga O */

(ii) **intrerupe ciclul**

altfeldaca PosCistg(X) $\neq$ 0

atunci Muta(PosCistig(X))/* blocheaza cistig adversar */

altfel Muta(Scop)

Mutare = 7: **daca** PosCistg(X) $\neq$ 0

atunci

(i) Muta(PosCistig(X)) /* deci cistiga X */

(ii) **intrerupe ciclul**

altfeldaca PosCistig(O) $\neq$ 0

atunci Muta(PosCistig(O))

altfel muta in orice patrat liber

Mutare = 8: **daca** PosCistig(O) $\neq$ 0

atunci

(i) Muta(PosCistig(O)) /* deci cistiga O */

(ii) **intrerupe ciclul**

altfeldaca PosCistg(X) $\neq$ 0

atunci Muta(PosCistig(X))

altfel muta in orice patrat liber

Mutare = 9: **daca** PosCistig(O) $\neq$ 0 /* identic cu Mutare 7 */

atunci

(i) Muta(PosCistig(X)) /* deci cistiga X */

(ii) **intrerupe ciclul**

altfeldaca PosCistig(O) $\neq$ 0
atunci Muta(PosCistig(O))
altfel muta in orice patrat liber

3.2 Mutare $\leftarrow$ Mutare + 1

sfirsit.

Muta(N)

1. **daca** Mutare are valoare impara

atunci Tabla[N] $\leftarrow$ 3

2. **daca** Mutare are valoare para

atunci Tabla[N] $\leftarrow$ 5

sfirsit.

PosCistig(P)

1. **daca** jucatorul P nu poate cistiga in mutarea urmatoare

atunci intoarce 0

2. **intoarce** pozitia patratului cu miscare cistigatoare

sfirsit.

Scop

1. **daca** patratul din mijloc este gol /* patratul cu indicele 5 */

atunci intoarce 5

2. **intoarce** pozitia unui patrat care nu se afla in colturi, i.e. pozitia $\neq$ 2,4,6,8, care este liber

sfirsit.

Din punct de vedere al timpului acest algoritm nu este la fel de eficient ca primul deoarece trebuie sa verifice mai multe conditii inainte de a executa o miscare, dar este mai eficient din punct de vedere al spatiului. Algoritmul prezinta avantajul utilizarii unei strategii relativ explicite care poate fi inteleasa si modificata mult mai usor decat in cazul primei rezolvari. Cu toate acestea strategia generala este stabilita de programator apriori, este codificata direct in program si nu prezinta generalitatea necesara. Daca se doreste extinderea jocului, de exemplu la "Tic-Tac-Toe" in trei dimensiuni, intregul algoritm trebuie schimbat. Urmatoarea rezolvare va elimina aceste inconveniente.

Rezolvarea 3

Se folosesc urmatoarele structuri de date:

- *Tabla* este un vector similar cu cel prezentat in Rezolvarea 2.
- *Urm* este o lista cu toate configuratiile urmatoare posibile care se obtin din configuratia curenta a tablei prin executia mutarilor permise din configuratia curenta.
- *Merit* este un numar intreg care reprezinta o estimare a posibilitatii de cistig din configuratia curenta a tablei prin una sau mai multe mutari. Deci meritul unei configuratii reprezinta cit de promitatoare este configuratia curenta pentru cistig.

Algoritmul de rezolvare este prezentat in continuare.

Algorithm: Problema "Tic-Tac-Toe". Rezolvarea 3

1. Initializeaza Tabla cu blanc
2. **cit timp** nici un jucator nu a cistigat **executa**
 - 2.1 $Urm \leftarrow \{ \}$
 - 2.2 **pentru** fiecare mutare posibila din configuratia curenta **C executa**
 - 2.2.1 Determina configuratia urmatoare C_1 a tablei
 - 2.2.2 $Urm \leftarrow Urm \cup C_1$
 - 2.3 **pentru** fiecare configuratie C_1 din lista **Urm executa**
 - 2.3.1 $Merit \leftarrow DetMerit(C_1)$
 - 2.3.2 Retine in MeritMaxim valoarea maxima a variabilei Merit
 - 2.4 Executa mutarea asociata configuratiei cu meritul MeritMaxim si obtine PozNoua
 - 2.5 **daca** PozNoua este pozitie cistigatoare
atunci intrerupe ciclul
 - 2.6 Citeste mutare adversar si modifica Tabla in consecinta

sfirsit.

DetMerit(C_1)

1. **daca** configuratia C_1 este cistigatoare
atunci intoarce valoarea maxima predefinita a meritului
2. Considera toate mutarile posibile ale adversarului din configuratia C_1
3. Calculeaza meritul M_i al fiecarei configuratii rezultate prin aceste mutari
4. Alege configuratia cu meritul cel mai mic, M_{min} , considerind ca adversarul va face cea mai defavorabila miscare pentru opozant
5. **intoarce** M_{min}

sfirsit.

Algoritmul poate inspecta in avans mai multe secvente de miscari pentru a determina secventa care va duce la cistig si, deci, a obtine o estimare mai buna a configuratiilor urmatoare. Algoritmul incearca sa maximizeze sansele de a cistiga presupunind ca adversarul va incerca sa minimizeze aceste sanse pentru oponent. Din acest motiv algoritmul este cunoscut sub numele de strategia "MinMax" si este utilizat frecvent in implementarea problemelor de jocuri in inteligenta artificiala.

In cazul unor jocuri mai complicate decit "Tic-Tac-Toe", formularea subprogramului DetMerit(C_1) din algoritmul de mai sus poate implica un apel recursiv infinit in pasul 3. In realitate se introduce o limita a miscarilor urmatoare investigate, platind insa pretul scaderii acuratetei estimarii meritului.

Rezolvarea 3 necesita un timp de calcul mai mare decit primele doua rezolvari deoarece trebuie sa inspecteze inaintea fiecarei mutari o parte din arborele de configuratii urmatoare posibile. Pentru probleme simple, cum ar fi cea discutata, primele doua abordari sint evident mai eficiente. Ultima rezolvare este superioara insa celorlalte doua din urmatoarele motive:

- (1) poate fi extinsa la jocuri mai complicate decit "Tic-Tac-Toe", jocuri pentru care enumerarea exhaustiva a tuturor posibilitatilor de mutare este imposibila,
- (2) strategia de joc poate fi imbunatatita sau schimbata prin modul de calcul al meritului.

Aceasta ultima abordare este un exemplu tipic de tehnica de inteligenta artificiala. Ea ilustreaza trei aspecte caracteristice ale rezolvarii problemelor in inteligenta artificiala: obtinerea solutiei prin cautare, utilizarea cunostintelor specifice domeniului si abstractizarea, i.e. separarea caracteristicilor importante ale problemei de aspectele de detaliu.

Capitolul 2

Strategii de rezolvare a problemelor

Acest capitol trateaza metode de rezolvare a problemelor in inteligenta artificiala si strategiile de control posibil de utilizat. Discutia se orienteaza in special spre modul de reprezentare a solutiei problemei si a procesului de rezolvare pe baza unor algoritmi de cautare a solutiei. Metodele prezentate reprezinta modalitati general valabile de rezolvare a problemelor. Aceste metode se aplica pentru orice model particular de reprezentare a cunostintelor, deci pentru orice fel de codificare simbolica a bazei de cunostinte [Barr,s.a.,1982;Nilsson,1980;Pearl,1984]. Reprezentarea cunostintelor si modalitatile specifice de utilizare a metodelor generale in rezolvarea problemelor reprezinta subiectul Partii a II-a a acestui text.

2.1 Reprezentarea solutiei problemei

Orice activitate de rezolvare a problemelor poate fi vazuta ca un proces de identificare sau de construire a unui obiect cu anumite caracteristici, obiect ce reprezinta solutia problemei. Exista trei cerinte minimale ale oricarei activitati de rezolvare a problemelor cu ajutorul calculatorului:

- (1) O *structura simbolica* care sa poata reprezenta descrierea initiala a problemei si fiecare obiect candidat la solutie.
- (2) O multime de *instrumente computationale* capabile sa transforme descrierea unui obiect (structura simbolica) intr-o noua descriere in scopul de a investiga sistematic toti candidatii la solutie.
- (3) O *metoda de planificare* efectiva care sa indice ordinea de aplicare a transformarilor astfel incit solutia problemei sa fie gasita cit mai repede.

In terminologia inteligentei artificiale, aceste trei componente ale unui sistem de rezolvare a problemelor se numesc:

- Baza de date sau baza de cunostinte
- Operatorii de transformare sau regulile de productie
- Strategia de control

2.1.1 Spatii de cautare

Descrierea initiala a problemei si a obiectelor candidate la solutie obtinute pe parcursul rezolvarii, deci structurile simbolice care specifica universul problemei, pot fi asimilate cu o *multime de stari*. Multimea de operatori (reguli) de transformare indica modul de transformare a universului problemei dintr-o stare initiala intr-o stare finala. *Starea initiala* descrie conditiile initiale ale problemei iar *starea finala* reprezinta solutia problemei. Starea finala poate fi definita explicit, prin descrierea solutiei, sau implicit, printr-o multime de conditii pe care o stare trebuie sa le satisfaca pentru a fi stare finala, adica solutie a problemei. Rezolvarea problemei poate cere fie determinarea starii finale, fie stabilirea intregului drum de la starea initiala la starea finala.

Multimea starilor investigate pina in momentul ajungerii in starea finala formeaza *spatiul de cautare* a solutiei problemei.

De exemplu, problema celor 8 regine cere sa se gaseasca o amplasare a opt regine pe o tabla de sah astfel incit nici o regina sa nu poata ataca alta regina. Acest lucru este echivalent cu cerinta ca nici o linie, coloana sau diagonala de pe tabla de sah sa nu contina mai mult de o regina. Starea initiala a problemei este descrisa de configuratia initiala a tablei de sah in care nici o regina nu este plasata pe tabla, obiectele candidate la solutie sint reprezentate prin tabla de sah pe care s-au plasat o parte sau toate reginele, iar starea finala este plasarea tuturor reginelor pe tabla, cu respectarea restrictiilor impuse. In acest caz starea finala este descrisa (implicit) printr-un set de conditii iar solutia problemei consta in determinarea acestei stari finale. Multimea de reguli de transformare este reprezentata de actiunile de plasare a unei regine intr-un patrat al tablei de sah.

Problemele de inteligenta artificiala sint, asa cum s-a mai spus, probleme grele, deci complex computationale. De cele mai multe ori obtinerea solutiei se face printr-un *proces de cautare* si nu prin aplicarea unei secvente de transformari anterior specificata. Ideea de baza a rezolvarii unor astfel de probleme poate fi descrisa, nedeterminist, prin urmatorul algoritm.

Algoritm: Rezolvarea unei probleme prin cautare

1. Stabileste starea initiala S_i
2. $S \leftarrow S_i$
3. **repetă**
 - 3.1 Selecteaza o regula de transformare T posibil de aplicat starii curente S
 - 3.2 Aplica T asupra starii S si obtine starea S'
 - 3.3 $S \leftarrow S'$

pina S este stare finala

sfirsit.

Algoritmul de mai sus este nedeterminist deoarece pasul 3.1 nu specifica cum se selecteaza transformarea T de aplicat. Selectarea transformarilor si memorarea transformarilor efectuate constituie *strategia de control*. O strategie de control nu este doar o secventa de actiuni, ci o modalitate de descriere a selectiei unei actiuni ca raspuns la un eveniment extern, cum ar fi rezultatul unui test, rezultatul aplicarii unei proceduri complicate de calcul sau actiunea unui adversar. In plus, o strategie de control trebuie sa fie *sistematica*. Judea Pearl [1984] caracterizeaza plastic cele doua cerinte necesare unei strategii pentru a fi sistematica:

- Nu lasa nici o piatra neintoarsa.
- Nu intoarce nici o piatra de mai multe ori.

Prima cerinta, numita *completitudine*, garanteaza faptul ca strategia produce solutia, daca aceasta exista. Cea de a doua cerinta protejeaza contra ineficientei prelucrarilor repetate. Cele mai multe probleme de inteligenta artificiala necesita evaluarea unui numar mare de stari intermediare, deci obiecte candidate la solutie, pentru a determina solutia. Informatia disponibila nu permite selectia transformarii corecte in scopul rezolvarii problemei. Tocmai din acest motiv comportarea

programele de inteligență artificială poate fi caracterizată ca un proces de căutare în care diverse transformări aplicate universului problemei sunt încercate până se determină soluția problemei.

Informația euristica joacă un rol foarte important în acest proces de căutare prin reducerea numărului de stări investigate pentru obținerea soluției. Se consideră, de exemplu, jocul de șah pentru care este imposibilă evaluarea tuturor stărilor posibile ale spațiului de căutare al unei partide cistigatoare. Un maestru al șahului poate hotărî ca o anumită mișcare este mai potrivită deoarece aceasta mișcare determină o configurație a tablei de șah care "pare" mai promitătoare pentru cistig. Aceasta hotărîre se bazează pe cunoștințele lui despre jocul de șah și pe experiență, și este informație euristica.

În cazul programelor de inteligență artificială informațiile euristice trebuie înglobate în strategia de control pentru a crește eficiența procesului de rezolvare a problemei. De obicei, acest tip de informație este reprezentat printr-o funcție euristica asociată fiecărei stări, funcție care estimează cât de promitătoare este acea stare din punct de vedere al avansului spre soluție. Funcțiile euristice depind și se stabilesc pe baza cunoștințelor specifice problemei sau clasei de probleme de rezolvat și au ca scop:

- ghidarea procesului de căutare a soluției și
- evaluarea calității soluției problemei.

De exemplu, în cazul problemei celor 8 regine, o euristica care poate fi utilizată este aceea de a plasa o regină astfel încât să lase cel mai mare număr de pătrate neatacate pe tabla de șah.

Specificarea unei strategii de căutare și a informației euristice utilizate trebuie să stabilească criteriile pentru demonstrarea *validității soluției* problemei și pentru *evaluarea soluției* din punct de vedere al efortului depus în găsirea soluției sau din punct de vedere al calității soluției găsite.

2.1.2 Soluția problemei reprezentată prin spațiul stărilor

Definiție. O reprezentare a soluției problemei prin spațiul stărilor este formată dintr-un triplet (S_i, O, S_f)

cu următoarea semnificație:

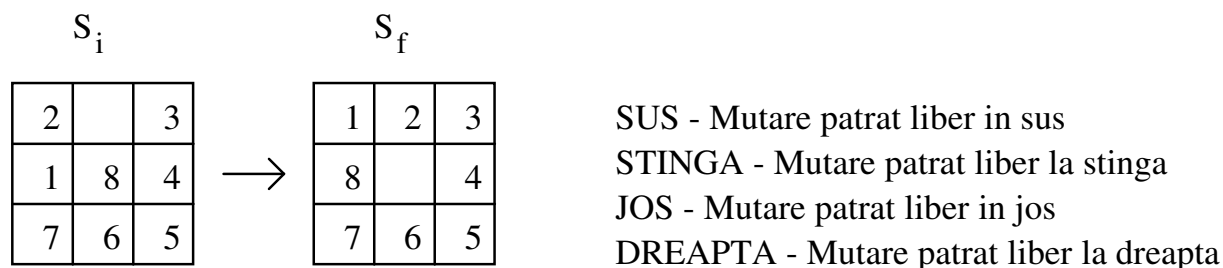
- S_i reprezintă mulțimea stărilor initiale,
- O reprezintă mulțimea de operatori posibil de aplicat asupra stărilor universului problemei pentru a ajunge în noi stări; în fiecare stare dată, numai o parte din operatori sunt legal aplicabili,
- S_f reprezintă mulțimea stărilor finale sau stări scop. Mulțimea stărilor finale poate conține și o singură stare.

În reprezentarea soluției problemei prin spațiul stărilor, spațiul de căutare are forma unui graf orientat în care nodurile sunt identificate prin stări, iar arcele reprezintă aplicarea unor operatori pentru a transforma o stare în starea următoare. O *soluție a problemei* este o secvență de operatori care transformă starea inițială în stare finală și reprezintă o cale între aceste două stări în graf. Graful spațiului de căutare este specificat implicit de reprezentare prin tripletul (S_i, O, S_f) . Pe parcursul avansului în căutare o porțiune din acest graf devine explicită, porțiunea din graful spațiului de căutare astfel construită reprezentînd partea explorată a spațiului de căutare.

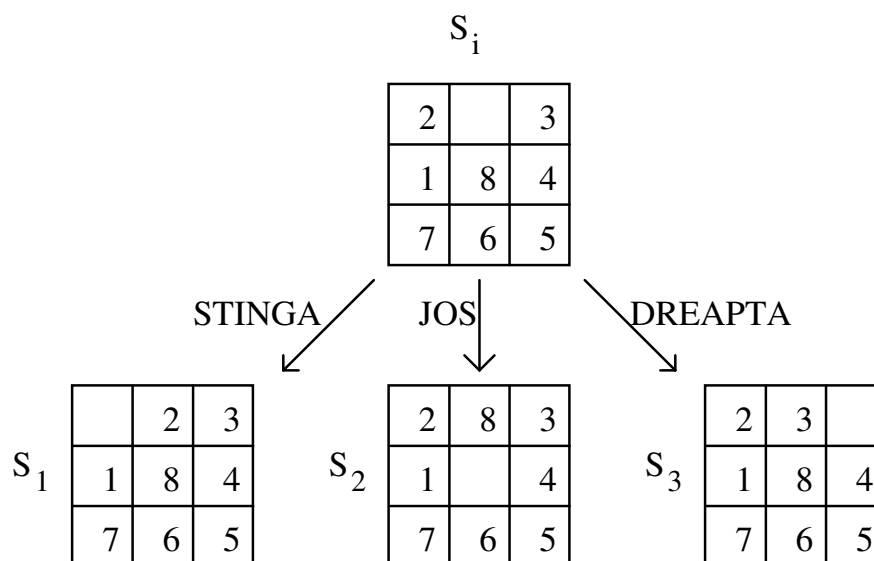
Fie jocul mozaicului de 8 numere, numit "8-puzzle", in care se cere ca, pornind de la o configuratie initiala specificata de pozitiile a opt numere si a unui patrat liber, sa se ajunga la o configuratie finala data, prin miscarea patratului liber in diverse directii, asa cum se arata in Figura 2.1. In acest caz, starea initiala este configuratia initiala (Figura 2.1(a)), starea finala, specificata explicit, este configuratia finala (Figura 2.1(b)), iar multimea de operatori este formata din urmatoarele reguli: muta patratul liber in sus cu o pozitie, muta patratul liber in jos cu o pozitie, muta patratul la dreapta cu o pozitie si muta patratul la stinga cu o pozitie (Figura 2.1(c)). Dintr-o anumita stare numai o submultime de operatori sint legal aplicabili. De exemplu, din starea initiala S_i numai trei operatori pot fi aplicati: STINGA, JOS si DREAPTA, asa cum se arata in Figura 2.1(d); operatorul SUS nu poate fi aplicat in starea S_i deoarece patratul liber este la marginea de sus a mozaicului. Prin aplicarea celor trei operatori starii initiale se pot obtine trei stari intermediare posibile: S_1, S_2, S_3 .

Pentru mozaicul de 8 numere se pot specifica diverse functii euristice care ghideaza procesul de cautare a solutiei si reduc numarul de stari generate. La un moment dat, se va alege starea care are asociata cea mai mica valoare a functiei euristice definite. Daca functia euristica este corespunzatoare se poate reduce in acest fel portiunea explicitata a grafului spatiului de cautare specificat implicit. Un exemplu de astfel de functie euristica este:

$$f(S) = \sum_{i=1}^8 t_i(S) \text{ unde } t_i(S) = \begin{cases} h_i & \text{daca patratul } i \text{ nu este in pozitie finala (in starea } S) \\ 0 & \text{in caz contrar} \end{cases}$$



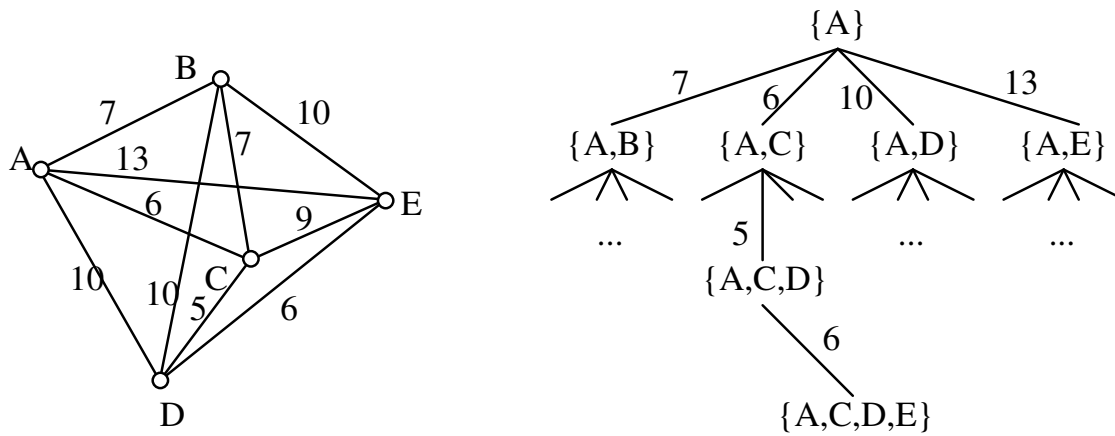
(a) Stare initiala (b) Stare finala (c) Operatori



(d) Tranzitii posibile din starea S_i

Figura 2.1 Reprezentare prin spatiul starilor a problemei mozaicului de 8 numere

O problema cunoscuta si dificila, deci NP-completa, este problema comis-voiajorului [Nilsson,1980;Pearl,1984;Sedgewick,1990]. Fiind date un numar de orase si distantele de-a lungul unor drumuri care leaga aceste orase, se cere sa se gaseasca drumul de lungime minima pe care il face un comis-voiajor care trebuie sa treaca prin toate orasele pornind dintr-un oras dat si revenind in orasul de plecare. Un posibil exemplu este cel descris in Figura 2.2(a). Starea initiala a problemei este orasul de plecare al comis-voiajorului. Starile intermediare sint orasele prin care a trecut comis-voiajorul, iar solutia problemei este secventa de stari parcurse din starea initiala pina in starea finala, secventa care trebuie sa indeplineasca conditia de cost optim, i.e. drum de lungime minima. O parte din graful spatiului de cautare este prezentat in Figura 2.2(b).



(a) Harta oraselor de parcurs

(b) O portiune din spatiul de cautare a solutiei

Figura 2.2 Reprezentarea prin spatiul starilor a problemei comis-voiajorului

Enumerarea tuturor traseelor posibile si compararea costurilor asociate pentru a gasi traseul optim este un proces neeficient computational, mai ales in cazul unui numar mare de orase. Pentru a reduce timpul de calcul se poate utiliza o functie euristica de estimare a traseului complet optim. Daca aceasta functie este optimista, adica subestimeaza intotdeauna lungimea reala a unui traseu complet, atunci primul traseu complet gasit in cautare este in acelasi timp traseul optim. O astfel de functie poate fi, de exemplu, costul distantei dintre ultimul oras selectat si orasul de plecare. O functie euristica care estimeaza mai bine traseul de cost minim este costul arborelui de acoperire minim [Sedgewick,1990] al nodurilor neparcuse la un moment dat.

2.1.3 Solutia problemei reprezentata prin grafuri SI/SAU

Exista probleme a caror rezolvare poate fi convenabil reprezentata printr-o tehnica numita *reducerea problemei la subprobleme*. Caracteristica comuna a acestei clase de probleme este aceea ca orice problema pusa de un obiect candidat la solutie poate fi vazuta ca o conjunctie de subprobleme ce pot fi rezolvate independent unele de altele. Rezolvarea problemelor din aceasta clasa poate fi abordata in urmatoorul mod: se descompune problema in subproblemele care trebuie rezolvate, subproblemele se descompun la rindul lor in alte subprobleme si asa mai departe, pina cind se obtine o descompunere a problemei initiale in subprobleme elementare, adica banal de rezolvat.

Spatiul de cautare a unei astfel de rezolvari a problemei are forma unui graf SI/SAU. Un graf SI/SAU este un caz particular al unui hipergraf. Un *hipergraf* este format dintr-o multime de noduri si o multime de hiperarce definite prin perechi ordonate in care primul element este un nod din multimea de noduri a hipergrafului si cel de al doilea element este o submultime de noduri. Un graf obisnuit este un caz particular al unui hipergraf in care cel de al doilea element al hiperarcelor este o multime formata dintr-un singur element.

Definitie. O reprezentare a solutiei problemei prin grafuri SI/SAU este formata dintr-un triplet (S_i, O, P_e)

cu urmatoarea semnificatie:

- S_i reprezinta descrierea problemei initiale,

- O reprezinta multimea de operatori de transformare (descompunere) a problemei in subprobleme,
- P_e reprezinta descrierea unei multimi de probleme elementare.

Definitie. Conform unui formalism stabilit, un *graf SI/SAU* este construit pe baza urmatoarelor reguli:

- (1) Fiecare nod reprezinta fie o singura problema fie o multime de probleme ce trebuie rezolvate.
- (2) Un nod ce reprezinta o singura problema nu are descendenti. Problema este fie o *problema elementara*, fie o *problema neelementara* care nu se mai poate descompune in subprobleme.
- (3) Nodurile ce reprezinta multimea de subprobleme in care s-a descompus o problema prin aplicarea unui operator de descompunere se numesc *noduri SI*.
- (4) Nodurile ce reprezinta descompuneri alternative ale unei probleme in subprobleme (prin aplicarea diversilor operatori de descompunere) se numesc *noduri SAU*. Aceste noduri au ca descendenti noduri SI.

Reprezentarea grafica a unui graf SI/SAU este data in Figura 2.3

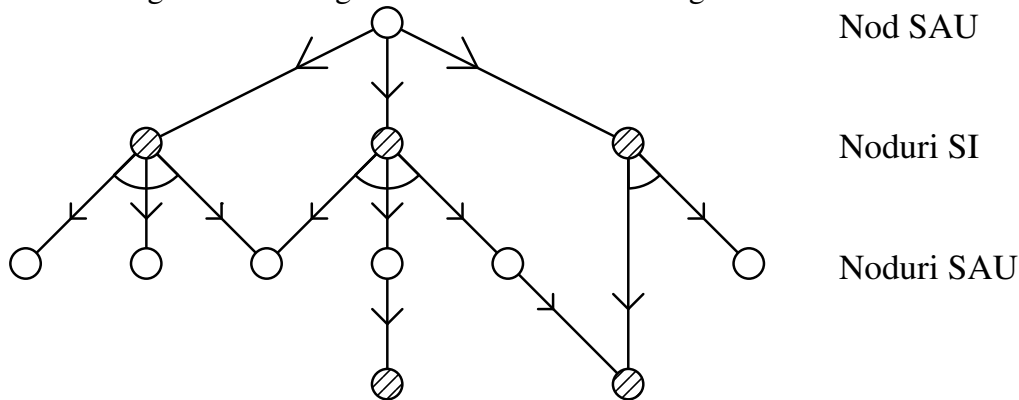


Figura 2.3 Graf SI/SAU pentru reprezentarea spatiului de cautare

O problema rezolvata astfel are solutie daca nodul initial, corespunzator descrierii initiale a problemei, este rezolvat.

Definitie. Intr-un graf SI/SAU un *nod* este *rezolvat* daca:

- (1) este un nod terminal etichetat cu o problema elementara
- (2) este un nod SI si toti succesorii lui sint noduri rezolvate
- (3) este un nod SAU si cel putin un succesori al acestuia este rezolvat

Definitie. Intr-un graf SI/SAU un *nod* este *nerezolvabil* daca:

- (1) este un nod terminal etichetat cu o problema neelementara, deci care nu se mai poate descompune in subprobleme
- (2) este un nod SI cu cel putin un succesori nerezolvabil
- (3) este un nod SAU cu toti succesorii nerezolvabili.

O *solutie a problemei* este reprezentata prin secventa de operatori de descompunere care determina ca nodul problema initiala sa devina rezolvat. Altfel spus, solutia problemei este subgraful SI/SAU care face ca nodul problema initiala sa devina rezolvat.

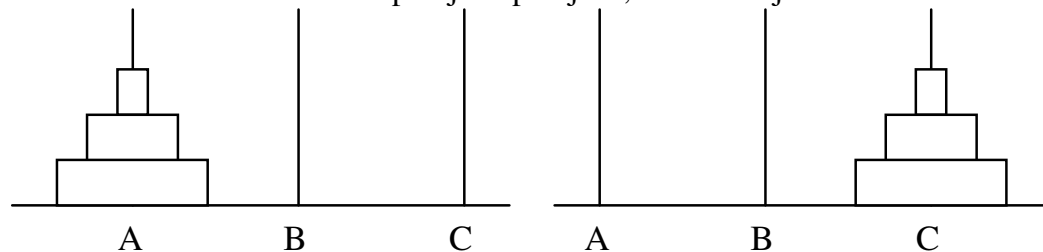
Se considera problema turnurilor din Hanoi care cere sa se mute n discuri ($n = 3$ in Figura 2.4) de pe tija A pe tija C, utilizind tija intermediara B si mentinind restrictia ca un disc sa fie asezat fie pe o tija vida, fie peste un disc de o dimensiune mai mare decit el. Starea initiala este specificata in Figura 2.4(a), starea finala in Figura 2.4(b), iar multimea de operatori de descompunere este formata dintr-un singur operator, cu trei componente:

- (1) Muta $n-1$ discuri de pe tija i pe tija j , utilizind tija k .
- (2) Muta un disc de pe tija i pe tija k .
- (3) Muta $n-1$ discuri de pe tija j pe tija k , utilizind tija i .

Singura problema elementara in acest caz este aceea de a muta un singur disc de pe o tija pe o tija vida. Descompunerea problemei in subprobleme poate fi reprezentata ca un arbore SI/SAU, prezentat in Figura 2.4(c).

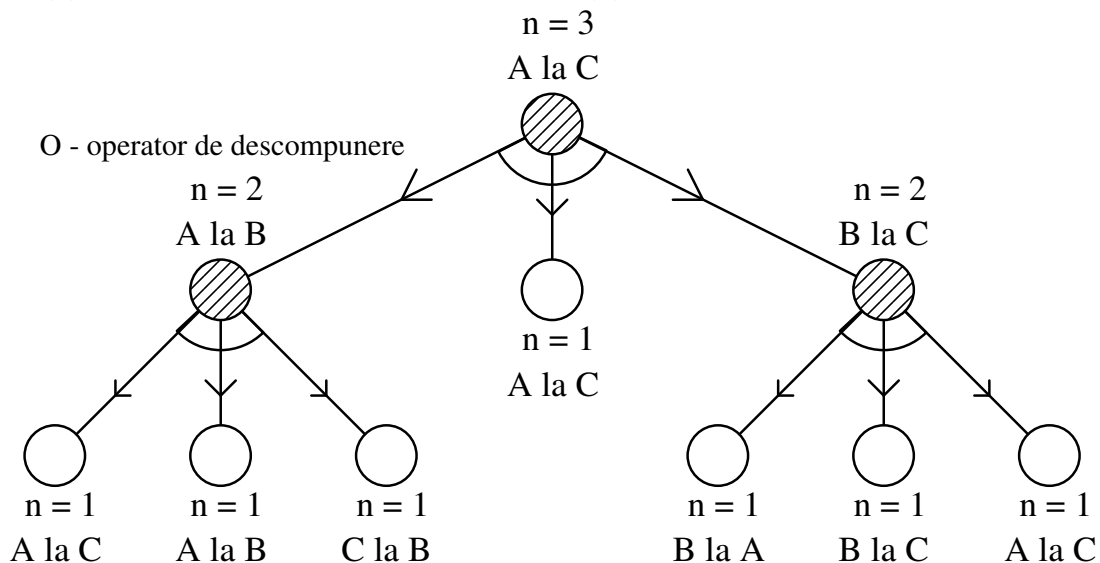
Problema turnurilor din Hanoi are un singur operator de descompunere a problemelor in subprobleme. Din acest motiv spatiul de cautare reprezentat prin graf SI/SAU nu are noduri SAU (descompuneri alternative), ci numai noduri SI si noduri probleme elementare.

PROBLEMA: Muta $n=3$ discuri de pe tija A pe tija C, utilizind tija B



(a) Stare initiala

(b) Stare finala



(c) Arborele SI/SAU de descompunere in subprobleme

Figura 2.4 Reprezentarea solutiei problemei turnurilor din Hanoi prin arbore SI/SAU

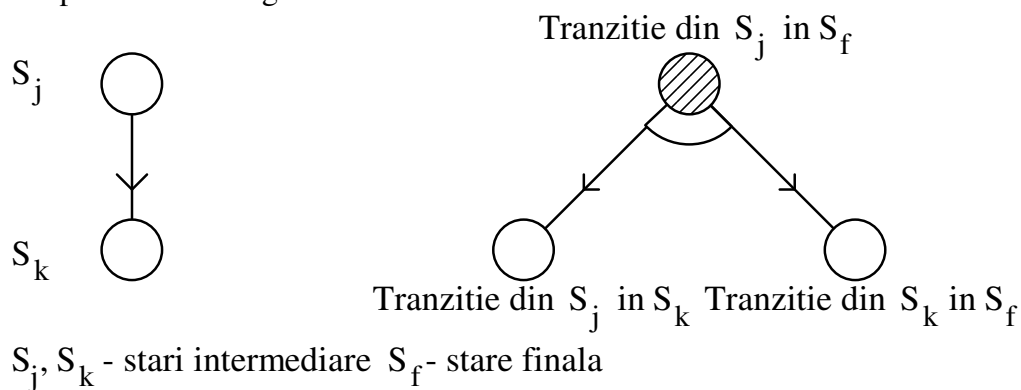
Un alt exemplu de solutie a problemei prin descompunerea in subprobleme este problema cintaririi monezilor. Fiind date N monezi dintre care una este mai usoara sau mai grea decit celelalte, se cere sa se determine moneda de greutate diferita, utilizind o balanta cu doua talere, printr-un numar minim de cintariri. Problema initiala este cintarirea a N monezi. Operatorii de

descompunere a problemei in subprobleme sint de forma: "Cintareste p monezi si rezolva toate problemele pe care aceasta cintarire le poate crea: balanta inclinata la stinga, balanta inclinata la dreapta, balanta echilibrata". Solutia problemei este arborele care include toate iesirile posibile pentru oricare din testele alese. In plus, fiecare cale in arbore trebuie sa reprezinte o secventa de teste si rezultate ale acestor teste care sa identifice neambiguu moneda diferita. Cititorul poate incerca gasirea numarului minim de cintariri pentru $N = 12$.

Modelul de rezolvare a problemelor prin descompunerea problemei in subprobleme poate fi folosit benefic numai in cazul in care rezolvarile subproblemelor sint independente intre ele. Aceasta inseamna ca procesul de cautare a solutiilor subproblemelor poate fi executat in orice ordine, solutia unei subprobleme neinfluentind solutia unei alte subprobleme. In problema cintaririi monezilor, fiecare subproblema se poate rezolva independent, si in orice ordine. Pentru alte probleme, cum ar fi de exemplu problema turnurilor din Hanoi, subproblemele de rezolvat nu sint independente si trebuie sa existe o ordine (liniara) de executie a solutiilor subproblemelor. Cu toate acestea se poate cauta solutia celei de-a treia subprobleme inaintea solutiei primei subprobleme. Aceasta proprietate nu este prezenta in problema mozaicului de 8 numere si din aceasta cauza o astfel de abordare a rezolvarii problemei nu aduce nici un beneficiu.

Rezolvarea problemelor prin descompunerea in subprobleme a stat la baza unuia dintre primele programe de inteligenta artificiala, General Problem Solver [Newell,Simon,1963] si a unor programe de planificare automata liniara cum ar fi STRIPS [Fikes,Nilsson,1971] si ABSTRIPS [Sacerdoti,1974]. Programul General Problem Solver (GPS) construiește un plan abstract pentru satisfacerea unui scop prin reducerea scopului la subscopuri si utilizeaza o metoda de rezolvare numita *analiza bazata pe modalitati*.

Se poate arata ca cele doua reprezentari ale solutiei problemei sint echivalente. De exemplu, se poate face transformarea din reprezentarea solutiei prin spatiul starilor in reprezentarea prin grafuri SI/SAU prezentata in Figura 2.5.



(a) Spatiul starilor

(b) Descompunerea problemei in subprobleme

Figura 2.5 Echivalenta reprezentarilor prin spatiul starilor si grafuri SI/SAU

Se considera un labirint si problema gasirii iesirii din labirint, fiind data o pozitie initiala in labirint. Descrierea problemei initiale este "Gaseste iesire din pozitia initiala P_1 " iar operatorii de descompunere a problemei in subprobleme sint:

- O1 Deplasare un pas la Est si gaseste iesire din noua pozitie.
- O2 Deplasare un pas la Sud si gaseste iesire din noua pozitie.

O3 Deplasare un pas la Vest si gaseste iesire din noua pozitie.

O4 Deplasare un pas la Nord si gaseste iesire din noua pozitie.

Problemele elementare sint: deplasare un pas la Est, Vest, Nord si respectiv Sud, daca labirintul permite, si iesire din labirint daca pozitia este la iesire. O portiune a grafului SI/SAU care reprezinta spatiul de cautare este prezentata in Figura 2.6.

Pentru aceasta problema se poate defini si o reprezentare prin spatiul starilor. Starea initiala este pozitia initiala in labirint iar starea finala este pozitia la iesirea din labirint. Operatorii de tranzitie dintr-o stare in alta sint miscarile la Est, Vest, Nord si respectiv Sud prin labirint.

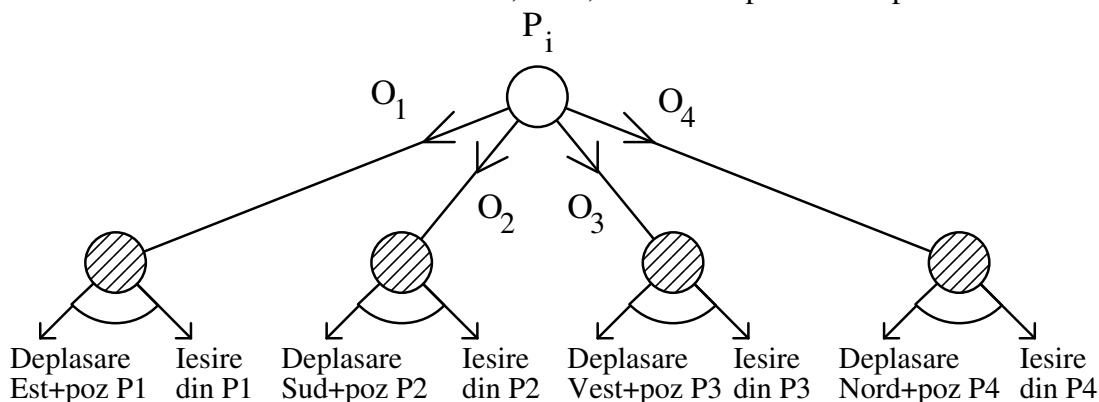


Figura 2.6 Parte a grafului de cautare SI/SAU pentru problema labirintului

Observatii:

- Orice problema poate fi solutionata prin ambele metode de reprezentare prezentate, dar pentru o anumita problema este mai usor de utilizat o reprezentare sau alta.
- Anumite probleme conduc in mod natural la o reprezentare a solutiei prin spatiul starilor, asa cum au fost, de exemplu, problema mozaicului de 8 numere, problema celor 8 regine si jocul "Tic-Tac-Toe" (Capitolul 1). O astfel de rezolvare corespunde unui *rationament direct* pentru a solutiona problema sau unui *control condus de date*.
- Alte probleme, cum ar fi problema turnurilor din Hanoi si problema cintaririi monezilor, conduc in mod natural la o rezolvare prin descompunerea problemei in subprobleme. O astfel de rezolvare corespunde unui *rationament invers* pentru rezolvarea problemei sau unui *control condus de scopuri*.

2.2 Strategii de cautare de baza

In aceasta sectiune se prezinta strategiile de cautare de baza, numite si strategii neinformate, care reprezinta un mod sistematic de investigare a spatiului de cautare a solutiei problemei. Aceste strategii stau la baza tuturor metodelor de rezolvare a problemelor in inteligenta artificiala. Ele constituie, de asemenea, suportul pentru dezvoltarea strategiilor de cautare euristica.

2.2.1 Caracterizarea strategiilor de cautare

In momentul alegerii unei strategii de cautare trebuie sa se tina cont de urmatoarele trei elemente:

- *Completitudinea strategiei* care stabileste daca strategia asigura sau nu gasirea solutiei in cazul in care problema are solutie.

- *Optimalitatea solutiei gasite* care este data de capacitatea strategiei de a obtine o solutie optimala, suboptimala sau pur si simplu o solutie.
- *Complexitatea strategiei* care se refera la complexitatea timp si spatiu a algoritmului utilizat.

Completitudinea strategiei va fi discutata in raport cu fiecare strategie in parte, atit in acest capitol cit si in capitolele urmatoare. De exemplu, in Capitolul 3 se vor pune in evidenta strategii rezolutive complete si strategii incomplete, dar utilizate datorita eficientei de calcul. Optimalitatea solutiei va fi discutata in sectiunea urmatoare, iar complexitatea strategiilor de cautare va fi prezentata pe scurt in Sectiunea 2.4.

Caracterizarea unei strategii de cautare se poate face dupa urmatoarele doua criterii [Nilsson,1980]:

(1)Capacitatea mecanismului de rezolvare de a reveni intr-o stare intermediara anterioara.

In functie de acest criteriu, strategiile de cautare se impart in:

- Strategii de cautare *irevocabile*. Un operator aplicabil este selectat, acest operator se aplica unei stari pentru a obtine o noua stare, iar starea anterioara este uitata (nu este memorata).
- Strategii de cautare *tentative*. La aplicarea unui operator intr-o stare curenta se memoreaza aceasta stare astfel incit procesul de cautare sa poata ulterior reveni in starile anterioare aplicarii operatorilor.

O strategie irevocabila este *strategia de cautare a alpinistului*, bazata pe criterii de optim local. Aceasta strategie se numeste a alpinistului deoarece, la fel ca un alpinist care doreste sa ajunga repede pe virful unui munte, alege starea urmatoare de nivel maxim pe baza unei functii de evaluare a starilor. Strategia este irevocabila deoarece pentru o stare curenta, se genereaza starile urmatoare, se alege starea de nivel maxim ca stare urmatoare si atit starea curenta cit si celelalte stari de pe nivelul starii urmatoare sint uitate. Selectia se face irevocabil, deci nu se mai poate reveni intr-una din starile anterioare starii curente sau intr-una din alternativele starii curente. Strategia alpinistului, desi simpla si putin consumatoare de memorie, prezinta o serie de limitari. De exemplu, daca problema cere determinarea starii cu o valoare maxima a functiei de evaluare, maximul global poate sa nu fie niciodata atins, cautarea blocindu-se intr-un maxim local.

Daca starea anterioara la care se poate reveni in timpul cautarii se afla numai pe calea curenta intre starea initiala si starea finala, strategia de cautare este o *strategie tentativa de tip "backtracking"*. Aceasta este, de exemplu, strategia utilizata de limbajul Prolog. Daca starea anterioara in care se poate reveni se afla pe orice cale deja parcursa in expandarea spatiului de cautare, strategia este de cautare tentativa generala pe grafuri. In sectiunea urmatoare se va discuta acest tip de strategii, ca avind cel mai mare grad de generalitate.

(2) Cantitatea de informatie folosita la gasirea solutiei

In functie de acest criteriu, strategiile de cautare se impart in:

- Strategii de cautare *neinformate*. Considerarea starilor urmatoare de inspectat se face dupa o ordine arbitrara, anterior stabilita.
- Strategii de cautare *informate*. Considerarea starilor urmatoare de inspectat se face dupa criterii euristice. Strategia foloseste o functie de evaluare a situatiei globale sau locale

care indica starea urmatoare cea mai promitatoare din punct de vedere al avansului spre solutie.

Strategiile de cautare neinformata (de baza) inspecteaza sistematic toate starile spatiului de cautare pina in momentul gasirii starii finale. Cele mai importante strategii de acest fel sint cautarea pe nivel si cautarea in adincime. Strategiile de cautare euristice incearca reducerea numarului de stari din spatiul de cautare inspectate pina la atingerea starii finale, pe baza diverselor criterii, cum ar fi functiile euristice. Strategia alpinistului descrisa anterior este un exemplu de cautare informata. Alte exemple sint strategia de cautare "best-first", algoritmul A^* si algoritmul AO^* . Algoritmii A^* si AO^* urmaresc in principal, pe lina reducerea numarului de stari inspectate, gasirea solutiei optime, asa cum se va vedea in Sectiunile 2.3.2 si 2.3.4.

Costul computational total al unui program de rezolvare a problemelor de inteligenta artificiala depinde de locul unde se situeaza strategia de control in spectrul neinformat/informat. Costul computational poate fi impartit in doua costuri: *costul aplicarii operatorilor*, sau costul parcurgerii spatiului de cautare intre starea initiala si starea finala, si *costul controlului*, sau costul evaluarii si selectiei celei mai promitatoare stari urmatoare. O strategie de cautare complet neinformata implica un cost redus al controlului datorita selectiei arbitrare a starilor urmatoare. O astfel de strategie determina insa un cost ridicat al parcurgerii spatiului de cautare deoarece, in general, necesita aplicarea unui numar mare de operatori inaintea gasirii unei solutii. O strategie de control complet informata despre domeniul problemei implica un cost al controlului ridicat atit din punct de vedere al timpului cit si al spatiului deoarece poate necesita calcule complicate pentru evaluarea meritului starilor si memorarea tuturor starilor parcurse. In schimb, o astfel de strategie determina un cost minim de parcurgere a spatiului de cautare datorita numarului redus de operatori aplicati pina la gasirea solutiei.

Costul computational total rezulta din combinarea celor doua costuri, asa cum se vede in Figura 2.7. In functie de aplicatie, proiectantul programului trebuie sa incerce determinarea celei mai bune variante de pondere a costurilor. Obtinerea unui cost computational optim este un aspect esential deoarece problemele de cautare sint probleme de complexitate timp exponentiala, asa cum se prezinta in Sectiunea 2.4.

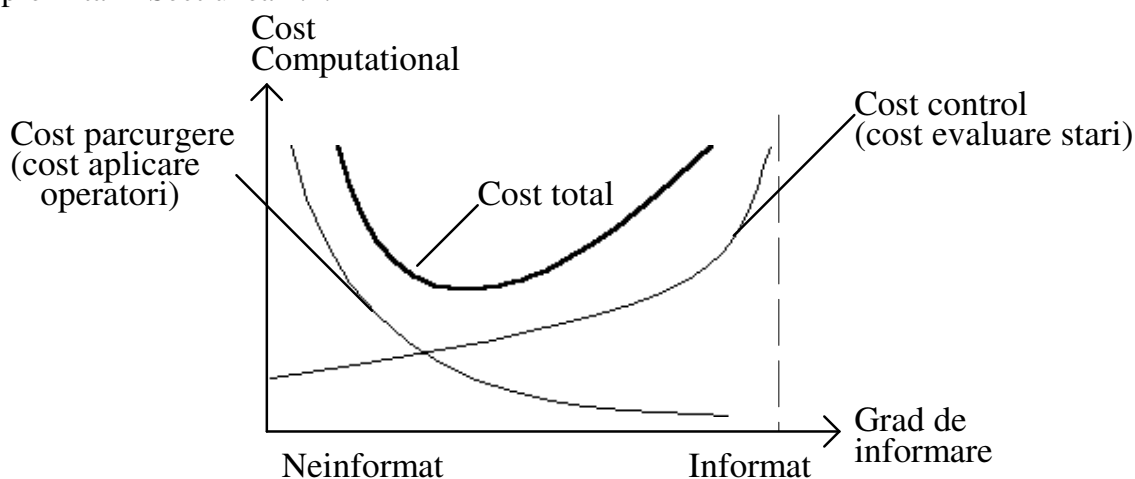


Figura 2.7 Costul total al rezolvarii unei probleme prin cautare

In continuare se va utiliza urmatoarea terminologie. In parcurgerea spatiului de cautare un nod poate fi:

- *necunoscut* - nodul apartine partii neexplorate a spatiului de cautare,
- *evaluat* - nodul este cunoscut dar fie nu se cunoaste nici un succesor al lui, fie se cunosc numai o parte din succesorii lui,
- *expandat* - nodul este cunoscut si, in plus, se cunosc toti succesorii lui.

Prin expandarea unui nod se intelege generarea tuturor succesorilor acelui nod. Aceasta inseamna obtinerea tuturor starilor urmatoare starii curente S, prin aplicarea tuturor operatorilor legali in starea S.

In procesul de cautare se vor folosi doua liste:

- FRONTIERA - lista nodurilor evaluate
- TERITORIU - lista nodurilor expandate

Lista FRONTIERA reprezinta frontiera spatiului de cautare parcurs (explicitat) spre partea necunoscuta a spatiului de cautare. Lista TERITORIU reprezinta partea cunoscuta a spatiului de cautare.

2.2.2 Cautari neinformatate in spatiul starilor

In continuare se prezinta doua strategii de cautare neinformatate: cautarea pe nivel si cautarea in adincime, strategii care difera prin ordinea de considerare, arbitrar fixata, a starilor urmatoare. Algoritmii prezentati presupun existenta a doua conditii. In primul rind, spatiul de cautare este arbore, deci exista o cale unica intre starea initiala si starea finala. Rezulta ca toate starile generate pe parcursul cautarii sint unice, deci nu au mai fost anterior generate. Extinderea si modificarile necesare pentru a generaliza algoritmii la spatii de cautare de tip graf vor fi prezentate in final. In al doilea rind, la fiecare expandare a unui nod, prin crearea tuturor nodurilor corespunzatoare starilor urmatoare, se stabileste o legatura de la fiecare nod succesor la nodul expandat. In momentul descoperirii nodului stare finala aceste legaturi permit reconstruirea caii spre solutie.

Definitie. Intr-o reprezentare a solutiei problemei prin spatiul starilor *adincimea unui nod* se defineste astfel:

- (1) $Adincime(S_i) = 0$, unde S_i este nodul stare initiala,
- (2) $Adincime(S) = Adincime(S_p) + 1$, unde S_p este nodul predecesor nodului S.

Cautarea pe nivel

Cautarea pe nivel, numita si cautare in latime, este o strategie care expandeaza starile urmatoare in ordinea apropierii fata de nodul stare initiala. Cu alte cuvinte, aceasta strategie considera intii toate secventele posibile de n operatori inaintea secventelor de n+1 operatori.

Algoritm: Strategia cautarii pe nivel in spatiul starilor

1. Creeaza listele $FRONTIERA \leftarrow \{S_i\}$ si $TERITORIU \leftarrow \{ \}$
2. **daca** $FRONTIERA = \{ \}$
atunci intoarce INSUCCES /* nu exista solutie */
3. Elimina primul nod S din FRONTIERA si insereaza-l in TERITORIU
4. Expandeaza nodul S

- 4.1. Genereaza toti succesorii directi S_j ai nodului S
- 4.2. **pentru** fiecare succesor S_j ($1 \leq j \leq m$) al lui S **executa**
 - 4.2.1. Stabileste legatura $S_j \rightarrow S$
 - 4.2.2. **daca** S_j este stare finala

atunci

 - i. Solutia este $(S_j, S, \dots, S_i)$
 - ii. **intoarce** SUCCES /* s-a gasit solutie */
 - 4.2.3. Insereaza S_j in FRONTIERA, *la sfirsit*

**5. repeta de la 2
sfirsit.**

Cautarea poate fi uneori lunga si complex computationala din punct de vedere al spatiului utilizat deoarece pentru fiecare nivel sint generate toate starile succesoare posibile. Cu toate acestea, strategia de cautare pe nivel garanteaza gasirea solutiei, in cazul in care aceasta exista si, in plus, gaseste cel mai scurt drum spre solutie in termenii numarului de tranzitii de stari executate.

Cautarea in adincime

Cautarea in adincime este o strategie care expandeaza starile cel mai recent generate, cu alte cuvinte nodurile cu adincimea cea mai mare din lista FRONTIERA. In consecinta, aceasta strategie parcurge o cale de la starea initiala pina la o stare ce poate fi stare finala sau care nu mai are nici un succesor. In acest ultim caz strategia revine pe nivelele anterioare si incearca explorarea altor cai posibile.

Strategia cautarii in adincime nu garanteaza obtinerea unei solutii a problemei, chiar in cazul in care solutia exista. O astfel de situatie poate apare, de exemplu, in cazul unui spatiu de cautare infinit in care ramura pe care s-a plecat in cautare nu contine solutia. Din acest motiv se introduce de obicei o limita a adincimii maxime de cautare, AdMax. Daca s-a atins aceasta limita fara a se gasi solutia, strategia revine si inspecteaza stari de pe nivele inferioare lui AdMax dar aflate pe cai diferite. Solutia care s-ar gasi la o adincime de $AdMax + p$, de exemplu, ar fi pierduta. Daca strategia de cautare gaseste solutia, aceasta nu este neaparat calea cea mai scurta intre starea initiala si starea finala.

Algorithm: Strategia cautarii in adincime in spatiul starilor

1. Creeaza listele $FRONTIERA \leftarrow \{S_i\}$ si $TERITORIU \leftarrow \{ \}$
2. **daca** $FRONTIERA = \{ \}$

atunci intoarce INSUCCES /* nu exista solutie sau solutia nu poate fi gasita pina la nivelul AdMax */
3. Elimina primul nod S din FRONTIERA si insereaza-l in TERITORIU
- 3'. **daca** $Adincime(S) = AdMax$

atunci repeta de la 2

4. Expandeaza nodul S

4.1. Genereaza toti succesorii directi S_j ai nodului S

4.2. **pentru** fiecare succesor S_j ($1 \leq j \leq m$) al lui S **executa**

4.2.1. Stabileste legatura $S_j \rightarrow S$

4.2.2. **daca** S_j este stare finala

atunci

i. Solutia este $(S_j, S, \dots, S_i)$

ii. **intoarce** SUCCES /* s-a gasit solutie */

4.2.3. Insereaza S_j in FRONTIERA, *la inceput*

5. repeta de la 2

sfirsit.

Algoritmul cautarii in adincime prezinta avantajul generarii unui numar de stari mult mai mic decit in cazul algoritmului de cautare pe nivel, deci consumul de spatiu este mult redus. Evident, algoritmul plateste pretul limitarilor indicate anterior.

Aceste observatii au dus la crearea *algoritmului de cautare in adincime cu nivel iterativ* [Korf,1987]. Algoritmul lui Korf elimina multe din dezavantajele cautarii pe nivel si a cautarii in adincime. Algoritmul de cautare in adincime cu nivel iterativ realizeaza la inceput o cautare in adincime in spatiul starilor cu o adincime maxima de cautare $AdMax = 1$. Daca starea finala nu a fost gasita, se reia cautarea in adincime cu $AdMax = 2$ si se continua in acest fel, crescind adincimea de cautare la fiecare iteratie. La fiecare iteratie algoritmul realizeaza o cautare in adincime completa cu adincimea de cautare egala cu valoarea curenta $AdMax$. Intre doua iteratii succesive nu se retine nici o informatie despre spatiul de cautare. Deoarece algoritmul de cautare in adincime cu nivel iterativ realizeaza de fapt o cautare pe nivel, el este garantat sa gaseasca solutia, daca aceasta exista, si, in plus, determina drumul cel mai scurt la solutie. Deoarece strategia este de adincime, numarul de stari generate la fiecare iteratie este mai mic decit cel in cazul cautarii pe nivel.

Extinderea celor doi algoritmi de cautare, pe nivel si in adincime, la cazul in care spatiul de cautare este graf se poate face tinind cont de faptul ca, in acest caz, un nod care se expandeaza poate avea ca succesor o stare ce a mai fost deja evaluata sau expandata. Pentru a evita reconsiderarea unei stari intilnite anterior, pasul de inserare a starii S_j in lista FRONTIERA (pasul 4.2.3) se modifica astfel:

4.2.3' **daca** S_j nu apartine $FRONTIERA \cup TERITORIU$

atunci insereaza S_j in FRONTIERA, *la sfirsit* /* nivel */

4.2.3' **daca** S_j nu apartine $FRONTIERA \cup TERITORIU$

atunci insereaza S_j in FRONTIERA, *la inceput* /* adincime */

In cazul algoritmului strategiei de cautare in adincime, existenta limitei de cautare AdMax protejeaza contra buclelor care s-ar putea crea prin generarea repetata a aceleiasi stari. In aceste conditii pasul 4.2.3' nu mai este absolut necesar, dar neintroducerea lui intr-un astfel de algoritm poate duce la evaluarea repetata a unor stari pina la atingerea adincimii AdMax, in cazul spatiului de cautare graf.

In cazul introducerii pasului 4.2.3', portiunea expandata a spatiului de cautare va fi intotdeauna un arbore si nu un graf, deoarece acest pas evita regenerarea unor stari anterior expandate.

Observatii:

- Cautarea pe nivel corespunde unei politici de tip FIFO de exploatare a listei FRONTIERA, in timp ce cautarea in adincime corespunde unei politici de tip LIFO.
- Ambele tipuri de cautari realizeaza un rationament direct, pornind in rezolvarea problemei de la starea initiala si generind arborele de cautare a starii finale. In anumite cazuri se poate aplica un rationament invers, executind strategiile incepind din starea finala si cautind starea initiala, cu conditia existentei unor operatori "inversi" de trecere dintr-o stare curenta in starea anterioara. Un astfel de exemplu poate fi jocul mozaicului de 8 numere, in care starea finala poate fi descrisa iar operatorii "inversi" sint usor de definit.
- In anumite probleme se poate utiliza o combinatie intre rationamentul direct si invers, adica un *rationament bidirectional* in care se cauta solutia atit din starea initiala cit si din starea finala prin construirea in paralel a doi arbori de cautare. Daca o astfel de abordare pleaca de la cautarea in adincime, exista pericolul ca sa se genereze doi arbori paraleli, unul pe calea directa si altul pe calea inversa, arbori care nu vor avea noduri intermediare comune. In acest caz avantajele cautarii bidirectionale sint pierdute.

2.2.3 Cautari neinformate in grafuri SI/SAU

Strategiile de cautare pe nivel si in adincime pot fi usor adaptate la cautarea solutiei problemei in reprezentarea cu grafuri SI/SAU. Principala diferenta consta in criteriul de determinare a conditiei de oprire. In reprezentarea prin spatiul starilor, conditia de oprire a algoritmilor este data de testarea proprietatii unui singur nod, nodul stare finala, in timp ce pentru reprezentarea prin descompunerea problemei in subprobleme trebuie sa se testeze daca o multime de noduri formeaza un arbore solutie. In consecinta, impactul fiecarui nod nou generat trebuie propagat in arborele partial construit pentru a determina daca nodul problema initiala a devenit rezolvat. Algoritmii de cautare in grafuri SI/SAU trebuie sa gestioneze, pe linga listele FRONTIERA si TERITORIU, si o structura de date care reprezinta arborele SI/SAU construit prin explicitarea spatiului de cautare definit implicit de reprezentare.

O alta diferenta a algoritmilor de cautare in grafuri SI/SAU fata de cei in spatiul starilor consta in nodurile introduse in listele FRONTIERA si TERITORIU. Nodurile SI nu se introduc in aceste liste deoarece ele nu corespund efectiv unor subprobleme, ci indica numai o multime de subprobleme care trebuie rezolvate. Aceste noduri sint insa construite si fac parte din portiunea

explicitata a spatiului de cautare. Pe baza starii de rezolvat sau nerezolvabil a acestor noduri se poate decide cind s-a obtinut arborele solutie.

De exemplu, fie o problema P_1 care poate fi redusa, alternativ, la o singura subproblema (echivalenta) P_2 prin aplicarea operatorului de descompunere O_1 si la subproblemele P_5, P_6, P_7 prin aplicarea operatorului O_2 , asa cum se arata in Figura 2.8. In acest caz expandarea nodului N_1 va genera nodurile N_2, N_3, N_5, N_6 si N_7 , fiecarui nod nou generat i se va atasa o legatura spre predecesor, dar numai nodurile N_2, N_5, N_6 si N_7 vor fi introduse in lista nodurilor explorate FRONTIERA.

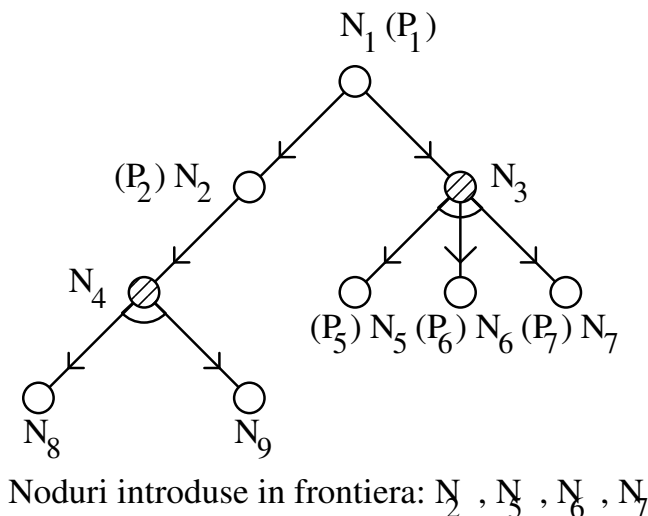


Figura 2.8 Expandarea nodurilor in grafuri SI/SAU

In aceste conditii, la calculul adincimii unui nod intr-un arbore SI/SAU nu se considera nodurile SI. Daca se considera ca nodul problema initiala are adincimea 0, adincimea oricarui nod subproblema va fi lungimea secventei de operatori care trebuie aplicati pentru a ajunge din nodul initial in acel nod.

Definitie. Intr-o reprezentare a solutiei problemei prin grafuri SI/SAU *adincimea unui nod* se defineste astfel:

- (1) $Adincime(S_i) = 0$, unde S_i este nodul problema initiala,
- (2) $Adincime(S) = Adincime(S_p) + 1$, daca S_p este nod SAU predecesor al nodului S ,
- (3) $Adincime(S) = Adincime(S_p)$, daca S_p este nod SI predecesor al nodului S .

Cautarea pe nivel

In reprezentarea prin descompunerea problemei in subprobleme strategia cautarii pe nivel foloseste aceeasi ordine de expandare a nodurilor ca in cazul reprezentarii prin spatiul starilor. Algoritmul care urmeaza presupune ca spatiul de cautare este un arbore SI/SAU si nu un graf general, deci o aceeasi stare nu poate fi generata de mai multe ori. De asemenea, se presupune ca problemele generate prin reducerea unei probleme in subprobleme pot fi rezolvate in orice ordine, deci sint independente. Nodul S_i este nodul corespunzator descrierii initiale a problemei.

Algoritm: Strategia cautarii pe nivel in arbori SI/SAU.

1. Creaza listele $FRONTIERA \leftarrow \{S_i\}$ si $TERITORIU \leftarrow \{\}$
2. Elimina primul nod S din $FRONTIERA$ si insereaza-l in $TERITORIU$

3. Expandează nodul S

3.1. Generează toți succesorii direcți S_j ai nodului S

3.2. **pentru** fiecare succesor S_j ($1 \leq j \leq m$) al lui S **execută**

3.2.1. Stabilește legătura $S_j \rightarrow S$

3.2.2. **dacă** S_j reprezintă o mulțime de cel puțin 2 subprobleme

atunci /* S_j este nod SI */

i. Generează toți succesorii subprobleme S_j^k ($1 \leq k \leq n$) ai lui S_j

ii. Stabilește legăturile între nodurile $S_j^k \rightarrow S_j$, ($1 \leq k \leq n$)

iii. Inserează nodurile S_j^k ($1 \leq k \leq n$) în FRONTIERA, *la sfîrșit*

3.2.3. **altfel** inserează S_j în FRONTIERA, *la sfîrșit*

4. **dacă** nu s-a generat nici un succesor al lui S în pasul precedent

atunci

4.1. **dacă** S este nod terminal etichetat cu o problemă neelementară

atunci

4.1.1. Etichetează S nerezolvabil

4.1.2. Etichetează cu nerezolvabil toate nodurile predecesoare lui S care devin nerezolvabile datorită lui S /* toate nodurile care îl au pe S descendent in arborele SI/SAU construit */

4.1.3. **dacă** nodul S_i este nerezolvabil

atunci întoarce INSUCCES /* problema nu are soluție */

4.1.4. Elimină din FRONTIERA toate nodurile care au predecesori nerezolvabili

4.2. **altfel** /* S este nod terminal etichetat cu o problema elementară */

4.2.1. Etichetează S rezolvat

4.2.2. Etichetează cu rezolvat toate nodurile predecesoare lui S care devin rezolvate datorită lui S

4.2.3. **dacă** nodul S_i este rezolvat

atunci

i. Construiește arborele soluție urmărind legăturile

ii. **întoarce** SUCCES /* s-a găsit soluția */

4.2.4. Elimină din FRONTIERA toate nodurile rezolvate și toate nodurile care au predecesori rezolvați

5. **repetă de la 2**

sfîrșit.

Observatii:

- Conditia din pasul 4 al algoritmului specifica faptul ca problema asociata nodului S nu mai poate fi descompusa in subprobleme. Acest lucru se intimpla fie in cazul in care problema este neelementara si ireductibila (pasul 4.1), caz in care S devine nerezolvabil, fie in cazul in care S este problema elementara, deci banal de rezolvat (pasul 4.2), caz in care S devine rezolvat.
- Legaturile care se stabilesc de la fiecare nod nou generat la predecesorul lui definesc arborele SI/SAU construit pe parcursul cautarii. Spre deosebire de reprezentarea prin spatiul starilor in care solutia problemei este calea parcursa intre nodul stare initiala si nodul stare finala, cale formata numai din noduri ce exista in lista TERITORIU, in cazul descompunerii problemei in subprobleme solutia problemei este tot arborele SI/SAU, care contine si noduri ce nu apar in TERITORIU sau FRONTIERA.

Cautarea pe nivel garanteaza gasirea solutiei, daca problema are solutie. In plus, arborele solutie construit este arborele de inaltime minima, deci cel care contine numarul minim de operatori necesari pentru a rezolva problema.

Cautarea in adincime

La fel ca in cazul cautarii in adincime in spatiul starilor, cautarea in adincime in grafuri SI/SAU considera mai intii nodurile cu cea mai mare adincime din lista FRONTIERA. Algoritmul cautarii in adincime se poate obtine din cel al cautarii pe nivel in arbori SI/SAU prin adaugarea unui pas suplimentar 2' dupa pasul 2 care limiteaza adincimea de cautare la un nivel maxim AdMax si prin inlocuirea pasilor iii si 3.2.3 cu pasii iii' si 3.2.3'.

2'. **daca** Adincime(S) = AdMax

atunci repeta de la 2

iii'. Insereaza nodurile S_j^k ($1 \leq k \leq n$) in FRONTIERA, *la inceput*

3.2.3'. **altfel** insereaza S_j in FRONTIERA, *la inceput*

Algoritmul cautarii in adincime nu garanteaza gasirea solutiei si nici construirea arborelui solutie minim, dar genereaza un numar de noduri mult mai mic decit cel din algoritmul cautarii pe nivel.

Ambii algoritmi se pot extinde pentru cazul in care spatiul de cautare este graf, printr-o tehnica similara cu cea prezentata in sectiunea anterioara, deci cu verificarea prezentei unei subprobleme nou generate in listele FRONTIERA si TERITORIU. Aceasta generalizare este inclusa in algoritmul de cautare informata in grafuri SI/SAU din sectiunea urmatoare.

2.3 Strategii de cautare euristica

In rezolvarea problemelor utilizind strategii de cautare neinformata numarul de stari investigate inainte de a gasi o solutie poate ajunge prohibitiv de mare, chiar si pentru probleme relativ simple, aparind deci explozia combinationala. Spatiul de cautare explorat poate fi redus prin aplicarea cunostintelor euristice despre problema. Acest capitol discuta modul in care informatia euristica

poate fi utilizata in cautare pornind de la strategiile de baza si obtinind strategii de cautare euristica.

Cunostintele euristice pot fi folosite pentru a creste eficienta cautarii in trei moduri:

- (1) Selectarea nodului urmator de expandat in cursul cautarii
- (2) In cursul expandarii unui nod al spatiului de cautare se poate decide pe baza informatiilor euristice care dintre succesorii lui vor fi generati si care nu.
- (3) Eliminarea din spatiul de cautare a anumitor noduri generate.

Aceasta sectiune se ocupa de prima modalitate de utilizare a cunostintelor euristice pentru a alege nodul "cel mai promitator" pentru obtinerea solutiei. Al doilea mod de utilizare a euristicilor revine de fapt la expandarea partiala a unui nod prin aplicarea numai a unui subset de operatori dintre cei posibil de aplicat. O varianta a acestei tehnici este *analiza bazata pe modalitati* utilizata in programul General Problem Solver care alege operatorul cel mai potrivit pentru a avansa spre solutie, chiar daca nu este imediat aplicabil. Ulterior, se incearca atingerea unei stari in care operatorul poate fi aplicat, deci se incearca satisfacerea unui subscop. Al treilea mod de utilizare a euristicilor incearca eliminarea anumitor noduri pe baza deciziei daca aceste noduri pot face parte din solutie sau nu. Aceasta tehnica se numeste *taierea arborelui de cautare*.

2.3.1 Cautare informata de tip "best-first"

Ideea strategiei de cautare "best-first" este aceea de a selecta spre expandare cel mai bun nod din spatiul de cautare generat pe baza cunostintelor euristice, deci pe baza unei estimari. Calitatea unui nod, din punct de vedere al gasirii solutiei, poate fi estimata in diverse moduri. Se poate atribui nodului gradul de dificultate in solutionarea problemei reprezentata de acel nod. Se poate estima calitatea unei multimi de solutii candidate care contin acel nod, deci solutii partiale care contin o cale ce duce la acel nod. O a treia alternativa este aceea de a evalua cantitatea de informatie care poate fi obtinuta prin expandarea acelui nod si importanta acestei informatii in ghidarea procesului de cautare. In toate aceste cazuri calitatea unui nod este estimata de *functia de evaluare euristica*, notata $w(n)$ pentru nodul n , care poate depinde de descrierea lui n , de descrierea scopului si de cunostinte suplimentare despre problema.

Una dintre cele mai simple strategii informate pentru modelul reprezentarii prin spatiul starilor, bazata pe un criteriu de optim local, este strategia de cautare a alpinistului, amintita anterior. Ideea acestei strategii este expandarea unui nod, inspectarea succesorilor acestuia si calculul valorilor functiei euristice pentru acesti succesorii, apoi alegerea celui mai bun nod in functie de aceste valori. Toate celelalte noduri sint uitate, inclusiv nodul stare curenta, deci strategia este irevocabila. Simplitatea acestei strategii este platita de dezavantajele strategiei: posibilitatea de a pierde solutia, blocarea in maxime locale si inspectarea repetata a aceleiasi stari. Strategia este evident incompleta.

In cazul strategiilor tentative informate generale, selectia nodului cel mai promitator se face evaluind toate nodurile generate pina la un moment dat, indiferent de calea in arborele de cautare pe care se afla un nod. In continuare se prezinta un algoritm de cautare de tip "best-first" pentru reprezentarea solutiei problemei prin spatiul starilor. Se presupune ca spatiul de cautare este un

graf si ca nodul selectat pentru expandare este cel care are cea mai mica valoare a functiei euristice $w(n)$; S_i este starea initiala.

Algorithm: Strategia de cautare "best-first" in spatiul starilor

1. Creeaza listele $FRONTIERA \leftarrow \{S_i\}$ si $TERITORIU \leftarrow \{ \}$
 2. Calculeaza $w(S_i)$ si asociaza aceasta valoare nodului S_i
 3. **daca** $FRONTIERA = \{ \}$
 atunci intoarce INSUCCES /* nu exista solutie */
 4. Alege din $FRONTIERA$ un nod S pentru care $w(S)$ este minima
 5. Elimina nodul S din $FRONTIERA$ si insereaza-l in $TERITORIU$
 6. **daca** S este starea finala
 atunci
 - 6.1. Construiesc solutia $(S, \dots, S_i)$ prin urmarirea legaturilor
 - 6.2. **intoarce** SUCCES /* s-a gasit solutia */
 7. Expandeaza nodul S
 - 7.1. Genereaza toti succesorii directi S_j ai nodului S
 - 7.2. **pentru** fiecare succesor S_j ($1 \leq j \leq m$) al lui S **executa**
 - 7.2.1. Calculeaza $w(S_j)$ si asociaza valoarea lui S_j
 - 7.2.2. Stabileste legatura $S_j \rightarrow S$
 - 7.2.3. **daca** $S_j \notin FRONTIERA \cup TERITORIU$
 atunci introduce S_j in $FRONTIERA$
 - 7.2.4. **altfel**
 - i. Fie S'_j copia lui S_j din $FRONTIERA$ sau $TERITORIU$
 - ii. **daca** $w(S_j) < w(S'_j)$
 atunci
 - Modifica legatura $S'_j \rightarrow \text{predecesor}(S'_j)$ in legatura $S'_j \rightarrow S$
 - Inlocuieste $w(S'_j)$ asociata lui S'_j cu $w(S_j)$
 - **daca** $S'_j \in TERITORIU$
 atunci elimina S'_j din $TERITORIU$ si insereaza-l in $FRONTIERA$
 - iii. Ignora nodul S_j
 8. **repetă de la 3**
- sfîrsit.**

Observatii:

- Pasii 7.2.3 si 7.2.4 sint necesari pentru a trata cazul in care spatiul de cautare este graf. In timpul cautarii se genereaza graful spatiului de cautare si, in acelasi timp, arborele de cautare definit de legaturile intre noduri stabilite la pasul 7.2.2.
- Daca pe parcursul cautarii o stare a fost redescoperita iar drumul pina la noua aparitie a starii este mai scurt, algoritmul trebuie sa considere noul drum gasit. Nodul pina la care s-a descoperit un drum mai scurt devine din nod expandat nod explorat prin reintroducerea lui in FRONTIERA, cu noua valoare mai mica a functiei w gasita (pasul ii). Acest proces este prezentat intuitiv in Figura 2.9.

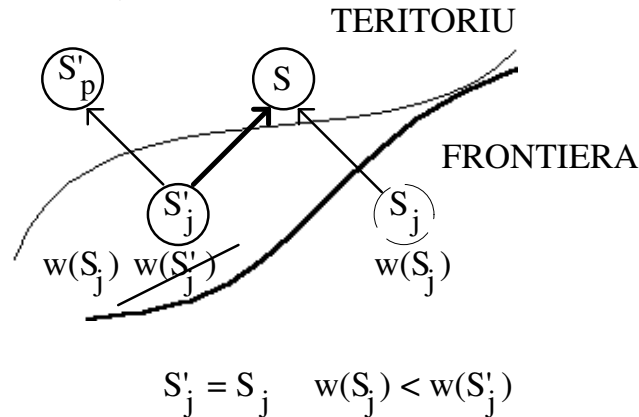


Figura 2.9 Reexplorarea unui nod la gasirea unei valori euristice inferioare.

Strategia de cautare "best-first" este o generalizare a strategiilor de cautare neinformate prezentate anterior. Prin particularizarea functiei w se pot obtine ambele strategii de baza astfel:

- $w(S) = \text{adincimea nodului } S$, conduce la strategia de cautare pe nivel
- $w(S) = -\text{adincimea nodului } S$, conduce la strategia de cautare in adincime

Criteriul nodului celui mai promitator si stabilirea functiei de evaluare euristica depinde de problema si de proprietatile solutiei cautate.

Daca spatiul de cautare contine mai multe cai spre solutie si se asociaza un cost fiecarui arc intre doua noduri S_k si S_{k+1} , cost determinat de costul aplicarii operatorului pentru a trece din starea S_k in starea S_{k+1} , atunci fiecare cale spre starea finala are asociat un cost. Daca se doreste gasirea *caii de cost minim* se stabileste urmatoarea formula pentru calculul functiei de evaluare:

$$w(S_j) = \sum_{k=i}^{j-1} \text{cost_arc}(S_k, S_{k+1}), \text{ unde } i \text{ este indicele starii initiale.}$$

In acest caz se obtine o *strategie de cautare de cost uniform*, numita si strategie de tip "branch and bound". Aceasta strategie garanteaza gasirea solutiei optime, daca exista solutie. Daca nu intereseaza optimalitatea solutiei si se urmareste numai *minimizarea efortului de cautare*, atunci se alege o functie euristica w care estimeaza, pentru fiecare nod, cit de aproape sau cit de departe este acel nod fata de nodul stare finala. Daca intereseaza ambele aspecte, deci atat calitatea solutiei cit si minimizarea efortului de cautare, se utilizeaza strategia de cautare A^* care va fi prezentata in sectiunea urmatoare.

Principiul strategiei de cautare "best-first" poate fi aplicat si pentru cazul reprezentarii solutiei problemei prin descompunerea in subprobleme. De aceasta data insa functia de evaluare trebuie sa se refere la un arbore solutie partial si nu la un singur nod, asa cum se face pentru

reprezentarea prin spatiul starilor. Varianta AO^* a strategiei "best-first" pentru grafuri SI/SAU care clarifica aceste aspecte va fi discutata in Sectiunea 2.3.4.

Observatie. Algoritmul de cautare "best-first" este o strategie completa daca reuseste sa elimine intotdeauna caile ciclice. Acest lucru este evident realizat daca costul unei cai fara cicluri este intotdeauna egal sau mai mic decit costul unei cai care contine cicluri.

2.3.2 Cautarea solutiei optime in spatiul starilor. Algoritmul A^*

Algoritmul A^* urmareste determinarea caii de cost minim intre nodul asociat starii initiale si nodul asociat unei stari finale. Acest obiectiv include si problema gasirii caii spre solutie care contine un numar minim de arce, i.e. calea care implica aplicarea unui numar minim de operatori. Problema gasirii celei mai scurte cai este o particularizare a cazului general, particularizare in care costul arcelor este considerat unitar.

Algoritmul A^* este o strategie de cautare informata de tip "best-first" pentru reprezentarea solutiei folosind spatiul starilor. Caracteristica distinctiva a algoritmului consta in modul de definire a functiei de evaluare $w(S)$ care este notata in acest caz cu $f(S)$. Nodul ales pentru expandare este nodul cu valoarea minima a functiei f [Nilsson,1980;Pearl,1984]. Deoarece f evalueaza nodurile din punct de vedere al gasirii solutiei de cost minim, $f(S)$ include doua componente:

- $g(S)$, o functie care estimeaza costul real $g^*(S)$ al caii de cautare intre starea initiala S_i si starea S ,
- $h(S)$, o functie care estimeaza costul real $h^*(S)$ al caii de cautare intre starea curenta S si starea finala S_f .

In aceste conditii functia de evaluare se defineste astfel:

$$f(S) = g(S) + h(S)$$

si reprezinta o estimare a costului real $f^*(S) = g^*(S) + h^*(S)$ al unei solutii a problemei care trece prin starea S (Figura 2.10).

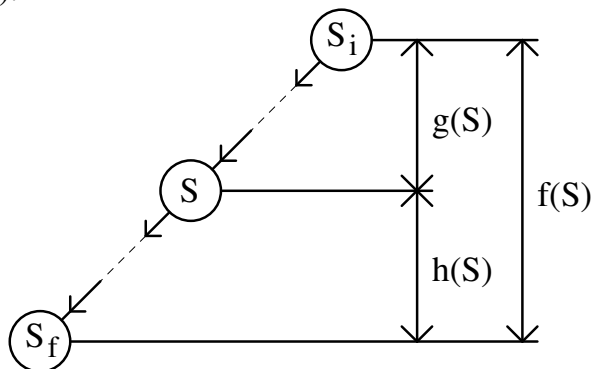


Figura 2.10 Componentele functiei euristice a algoritmului A^*

Functia $g(S)$ este calculata ca fiind costul actual al drumului parcurs in cautare intre starea initiala S_i si starea S , deci

$$g(S) = \sum_{k=i}^n \text{cost_arc}(S_k, S_{k+1}), \text{ cu } S_n = S \text{ si } S_i \text{ starea initiala}$$

Daca spatiul starilor este un arbore, $g(S)$ este o estimare perfecta a costului real $g^*(S)$. Daca spatiul de cautare este graf, $g(S)$ poate numai sa supraestimeze costul real $g^*(S)$. In consecinta $g(S) \geq g^*(S)$ pentru orice stare S . Algoritmul A^* se obtine din algoritmul "best-first" prin utilizarea functiei $f(S)$ in locul lui $w(S)$ si inlocuirea conditiei din pasul 7.2.4. ii cu conditia $g(S_j) < g(S'_j)$.

Functia $h(S)$ este functia purtatoare de informatie euristica si trebuie definita in raport cu caracteristicile problemei particulare de rezolvat. Pentru ca algoritmul A^* sa gaseasca solutia optima, functia h trebuie sa fie nenegativa si sa subestimeze intotdeauna costul real $h^*(S)$ al caii care a mai ramas de parcurs pina in starea finala.

Definitie. O functie euristica h se numeste *admisibila* daca $0 \leq h(S) \leq h^*(S)$ pentru orice stare S . Definitia stabileste *conditia de admisibilitate* a functiei h si este folosita pentru a defini *proprietatea de admisibilitate* a unui algoritm A^* .

Teorema. Fie un algoritm A^* care utilizeaza cele doua componente g si h ale functiei de evaluare f . Daca

- (1) functia h satisface conditia de admisibilitate
- (2) $\text{cost_arc}(S, S') \geq c$, pentru orice doua stari S, S' , unde $c > 0$ este o constanta si costul c este finit

atunci algoritmul A^* este *admisibil*, adica este garantat sa gaseasca calea de cost minim spre solutie.

Observatie. Se poate demonstra ca algoritmul A^* este o strategie completa, chiar si pentru spatii de cautare grafuri infinite [Pearl,1984].

2.3.3 Caracteristicile euristicii algoritmului A^*

Conditia de admisibilitate a functiei euristice indica faptul ca $h(S)$ trebuie sa fie o subestimare a costului real $h^*(S)$, adica sa fie optimista, pentru ca algoritmul A^* sa gaseasca intotdeauna solutia optima. Daca $h(S)$ este cu mult mai mic decit $h^*(S)$ atunci algoritmul A^* isi pierde din performantele timpului de cautare. Pentru ca numarul de stari inspectate in cautare sa fie cit mai mic, este de dorit ca $h(S)$ sa fie cit mai apropiat de $h^*(S)$. Ideal, daca $h(S)$ ar fi egal cu $h^*(S)$ pentru orice stare S parcursa in cautare, atunci algoritmul A^* gaseste drumul optim spre starea finala fara a expanda niciodata vreun nod care nu se afla pe calea optima spre solutie. Daca $h(S) = 0$ atunci algoritmul A^* se reduce la o strategie de cautare de cost uniform. Deci algoritmul A^* este cu atit mai informat cu cit $h(S)$ este mai apropiat de $h^*(S)$.

Gradul de informare al unui algoritm A^*

Definitie. Fie doi algoritmi A^* , A_1 si A_2 , cu functiile de evaluare

$$f_1(S) = g_1(S) + h_1(S)$$

$$f_2(S) = g_2(S) + h_2(S)$$

Se spune ca algoritmul A_2 este *mai informat* decit algoritmul A_1 daca $h_2(S) > h_1(S)$ pentru orice stare S , $S \neq S_f$, starea finala.

Se poate demonstra ca daca A_2 este mai informat decit A_1 atunci A_2 nu expandeaza niciodata mai multe stari decit A_1 . Se poate demonstra de asemenea ca daca componenta h a

functiei f a unui algoritm A^* are proprietatea de a fi *monoton crescatoare*, i.e. $h(S) \leq h(S') + \text{cost_arc}(S, S')$ pentru orice doua stari S, S' cu $S' = \text{succ}(S)$ atunci un nod, odata introdus in lista TERITORIU, nu va mai fi niciodata eliminat din TERITORIU si reinserat in FRONTIERA.

Determinarea functiei de evaluare f

Pentru stabilirea functiei f trebuie definite functiile g si h . De obicei, componenta $g(S)$ poate fi calculata ca suma costurilor arcelor parcurse din starea initiala S_i pina in starea S sau, daca costurile arcelor sint unitare sau problema nu are definite costuri pentru operatori, ca numarul de arce parcurse din S_i pina in S .

Determinarea functiei $h(S)$, purtatoarea informatiei euristice, este mai dificila deoarece functia h depinde de problema specifica de rezolvat. Este sarcina celui care construiește algoritmul A^* sa descopere o functie euristica adecvata.

Fie problema mozaicului de 8 numere definita in Sectiunea 2.1.2. Un algoritm A^* de rezolvare a acestei probleme ar putea utiliza o definitie a functiei $h(S)$ cum este cea care a fost indicata la specificarea problemei:

$$h_1(S) = \sum_{i=1}^8 t_i(S) \text{ unde } t_i(S) = \begin{cases} \text{daca patratul nevid } t_i \text{ nu este in pozitie finala (in starea } S) \\ \text{\$} \\ \text{in caz contrar } \text{P} \end{cases}$$

Functia de evaluare este $f_1(S) = g(S) + h_1(S)$, unde $g(S)$ este numarul de miscari parcurse din starea initiala S_i pina in starea S . Se poate defini insa o functie euristica mai buna decit h_1 , adica mai informata,

$$h_2(S) = \sum_{i=1}^8 \text{Distanța}(t_i)$$

unde $\text{Distanța}(t_i)$ este distanta (pe orizontala si verticala) a patratului nevid t_i in starea S fata de pozitia lui in starea finala S_f . Aceasta distanta se mai numeste si "distanta Manhattan". Pentru exemplul din Figura 2.1 se obtin urmatoarele valori ale functiilor euristice h_1 si h_2 :

$$\begin{aligned} h_1(S_1) &= 2 & h_1(S_2) &= 3 & h_1(S_3) &= 4 \\ h_2(S_1) &= 2 & h_2(S_2) &= 4 & h_2(S_3) &= 4 \end{aligned}$$

Se poate arata ca un algoritm A^* care utilizeaza functia $f_2(S) = g(S) + h_2(S)$ are un grad de informare mai mare decit un algoritm A^* care utilizeaza functia $f_1(S)$ definita mai sus. Acest lucru se poate justifica informal prin faptul ca $h_1(S)$ nu ofera o estimare foarte buna a dificultatii unei configuratii. Functia $h_2(S)$ ofera o mai buna estimare din punct de vedere al numarului de pasi necesari pina la atingerea starii finale. Atit h_1 cit si h_2 sint functii admisibile.

Tot in Sectiunea 2.1.2 s-a prezentat problema comis-voiajorului. Un algoritm A^* care rezolva aceasta problema poate utiliza o functie de evaluare $f_1(S) = g(S) + h_1(S)$, unde $g(S)$ reprezinta lungimea drumului (suma distantelor) parcurs de comis-voiajor din orasul de plecare pina in orasul asociat starii S , iar h_1 este definita astfel $h_1(S) = \text{cost_arc}(S_i, S)$, unde S_i este starea initiala asociata orasului de plecare iar S este starea orasului in care se afla comis-voiajorul. Se reaminteste ca in problema comis-voiajorului oricare doua orase sint legate intre ele printr-un drum. Se poate folosi si functia euristica $h_2(S)$ egala cu costul arborelui de acoperire minim al oraselor neparcurse pina in starea S . Atit h_1 cit si h_2 sint functii admisibile. Se poate demonstra ca

un algoritm A^* care foloseste functia $f_2(S) = g(S) + h_2(S)$ este mai informat decit un algoritm care foloseste $f_1(S)$.

O problema asemanatoare ca natura cu problema comis-voiajorului este problema gasirii drumului minim intre doua orase pe o harta. Harta este definita printr-un numar de orase si prin legaturile dintre aceste orase, cu distantele asociate. Un algoritm A^* pentru aflarea drumului minim intre doua orase poate utiliza o functie euristica $f(S) = g(S) + h(S)$, cu $g(S)$ definita ca lungimea drumului deja parcurs intre orasul de plecare si starea S si $h(S)$ definita ca distanta directa pe harta (zbor de pasare) intre orasul asociat starii S si orasul de destinatie. Functia h astfel definita este admisibila.

In final se considera problema misionarilor si canibalilor. Trei misionari si trei canibali ajung la un riu. Exista o barca de doua locuri cu care se poate traversa riul. Daca numarul canibalilor este mai mare decit numarul misionarilor pe unul din malurile riului, misionarii vor fi mincati de canibali. Cum pot trece toti riul fara ca misionarii sa fie mincati? Starea initiala si starea finala a acestei probleme sint descrise in Figura 2.11.

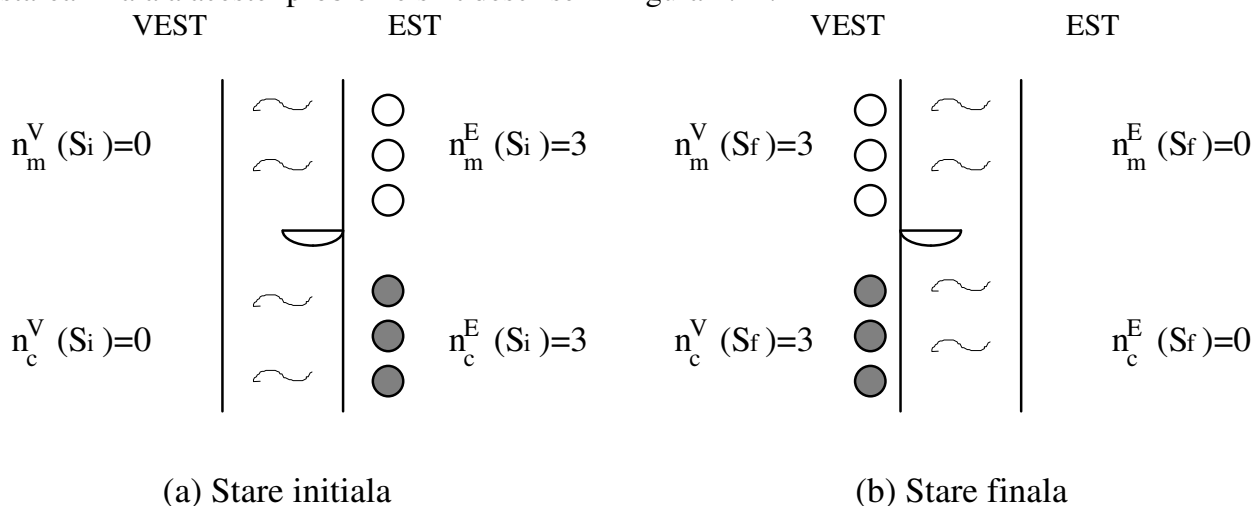


Figura 2.11 Problema misionarilor si canibalilor

Incercind rezolvarea aceste probleme cu ajutorul unui algoritm A^* se propun urmatoarele trei functii de evaluare (Giumale,1989):

$$f_1(S) = g(S) + h_1(S)$$

$$f_2(S) = g(S) + h_2(S)$$

$$f_3(S) = g(S) + h_3(S)$$

Functia $g(S)$ este definita ca fiind egala cu numarul de tranzitii de stari efectuate din starea initiala pina in starea curenta S . Pentru definitia functiilor h_1 , h_2 , si h_3 se fac urmatoarele conventii:

$n_m^E(S)$ - numar de misionari pe malul de EST in starea S

$n_m^V(S)$ - numar de misionari pe malul de VEST in starea S

$n_c^E(S)$ - numar de canibali pe malul de EST in starea S

$n_c^V(S)$ - numar de canibali pe malul de VEST in starea S

$n^E(S) = n_m^E(S) + n_c^E(S)$ - numar de persoane pe malul de EST in starea S

$n^V(S) = n_m^V(S) + n_c^V(S)$ - numar de persoane pe malul de VEST in starea S

si se definesc cele trei functii euristice propuse astfel:

- $h_1(S) = n^E(S)$
- $h_2(S) = n^E(S) / 2$
- $h_3(S) = \begin{cases} n^E(S) + 1 & \text{daca barca este pe malul de VEST si } n^E(S) \neq 0 \\ n^E(S) - 1 & \text{daca barca este pe malul de EST si } n^E(S) \neq 0 \\ 0 & \text{daca } n^E(S) = 0 \end{cases}$

Funcția h_1 nu este admisibilă deoarece pentru starea S_k definită prin: 1 misionar și 1 canibal pe malul de EST și 2 misionari și doi canibali pe malul de VEST, cele două persoane de pe malul estic pot fi transportate imediat pe malul vestic, deci cu un cost unitar. În consecință $h_1(S_k) = 2$ este mai mare decât costul real $h^*(S_k) = 1$, deci h_1 nu este admisibilă. Funcțiile h_2 și h_3 sunt admisibile și, în plus, sunt monotone. Se poate demonstra că funcția h_3 este mai informată decât h_2 , deci un algoritm A^* care utilizează h_3 va face mai puține treceri ale riului decât un algoritm A^* care utilizează h_2 .

Relaxarea condiției de optimalitate a algoritmului A^*

S-a arătat că algoritmul A^* găsește soluția optimă dacă componenta euristica h este admisibilă. În multe cazuri însă, algoritmul A^* consumă o cantitate de timp semnificativă cu încercările de a alege între diverse cai al căror cost nu diferă semnificativ. Proprietatea de admisibilitate poate deveni uneori o limitare serioasă și poate duce la creșterea timpului de rezolvare a problemei. În funcție de cerințele problemei, se poate relaxa proprietatea de admisibilitate a algoritmului A^* , chiar cu riscul obținerii unei soluții suboptimale dar cu avantajul reducerii timpului de căutare. În general, pot apărea cele trei situații prezentate în continuare [Barr, 1982].

Există probleme pentru care s-ar putea să intereseze mai puțin obținerea unei soluții de cost minim și scopul urmărit să fie mai ales *minimizarea efortului de căutare*. Motivul includerii în funcția de evaluare f a funcției g este acela de a adăuga căutării o componentă de căutare pe nivel și de a ghida astfel căutarea spre descoperirea soluției optime. Fără funcția g , $f(S)$ ar estima tot timpul, pentru fiecare nod S , distanța ramasă pînă la starea finală. Dacă scopul este acela de a minimiza efortul de căutare și nu costul soluției, atunci ponderea funcției h trebuie să fie cît mai mare. Pentru a ajusta ponderile între aceste două tendințe, cost optim și avans rapid spre soluție, Pohl [1970] a propus următoarea definiție ponderată a funcției f :

$$f(S) = (1 - p) \cdot g(S) + p \cdot h(S),$$

unde p este o constantă pozitivă. Dacă $p = 0$ atunci algoritmul de căutare devine o strategie de căutare de cost uniform, dacă $p = 1/2$ atunci se obține varianta standard a algoritmului A^* , iar dacă $p = 1$ atunci se obține o căutare de tip "best-first" care urmărește numai minimizarea efortului de căutare. Dacă h este admisibilă, este ușor de arătat că algoritmul este admisibil în domeniul $p \in [0, 1/2]$ dar își poate pierde admisibilitatea pentru domeniul $p \in (1/2, 1)$ în funcție de distanța funcției h față de h^* .

Pentru o altă categorie de probleme este posibil să intereseze soluția de cost minim dar problema să fie atît de grea încît un algoritm A^* admisibil să fie practic imposibil de executat pînă

la sfîrşit din criterii de eficienţă. Într-o astfel de situaţie se urmăreşte găsirea unei *soluţii suficient de apropiate de soluţia de cost minim* într-un interval de timp acceptabil. Funcţia f poate fi definită printr-o ponderare dinamică a componentei h

$$f(S) = g(S) + c(S) \cdot h(S),$$

unde $c(S)$ este o funcţie de ponderare a cărei valoare depinde de nodul S . În 1973, Pohl propune următoarea variantă de funcţie ponderată dinamic:

$$f(S) = g(S) + h(S) + \varepsilon \left(1 - \frac{d(S)}{N}\right) \cdot h(S),$$

unde $d(S)$ este adîncimea nodului asociat stării S şi N este adîncimea estimată a nodului stare finală. Dacă funcţia h este admisibilă atunci un algoritm A^* care utilizează această definiţie a funcţiei f va găsi o soluţie suboptimală al cărei cost diferă cu cel mult ε de costul soluţiei optime.

Utilizînd această abordare s-a încercat rezolvarea problemei comis-voiajorului pentru cazul a 20 de oraşe, problema cunoscută sub numele de problema Croes şi pentru care se ştie că soluţia de cost optim este 246. Un algoritm A^* admisibil nu a produs soluţia optimă nici după expandarea a 500 de noduri. Cu definiţia funcţiei de evaluare dată mai sus s-a găsit o soluţie de cost 260 după expandarea a numai 53 de noduri.

O a treia situaţie întîlnită este aceea în care determinarea unei funcţii euristice h admisibile suficient de bune, adică suficient de apropiate de funcţia reală h^* , este foarte dificilă sau chiar imposibilă. O funcţie h admisibilă dar cu valori mult mai mici decît cele ale funcţiei h^* face ca algoritmul A^* să degenereze într-o strategie de căutare neinformată. Dacă nu se poate găsi nici o funcţie euristica h suficient de bună, se poate utiliza o funcţie h ε -admisibilă.

Definiţie. O funcţie euristica h se numeşte ε -admisibilă dacă $h(S) \leq h^*(S) + \varepsilon$, $\varepsilon > 0$, constantă.

S-a demonstrat [Pearl, 1984] că algoritmul A^* care utilizează o funcţie de evaluare f cu o componentă h ε -admisibilă găseşte întotdeauna o soluţie al cărei cost depăşeşte costul soluţiei optime cu cel mult ε . Un astfel de algoritm se numeşte *algoritm A^* ε -admisibil* iar soluţia găsită se numeşte *soluţie ε -optimală*.

De exemplu, în cazul problemei mozaicului de 8 numere se poate utiliza funcţia de evaluare $f_3(S) = g(S) + h_3(S)$, cu funcţia euristica h_3 definită astfel:

$$h_3(S) = h_2(S) + 3 \cdot T(S)$$

$$T(S) = \sum_{i=1}^8 \text{Scor}[t_i(S)], \text{ unde}$$

$$\text{Scor}[t_i(S)] = \begin{cases} 2 & \text{dacă patrulul } t_i \text{ în starea } S \text{ nu este urmat de} \\ & \text{succesorul corect din starea finală} \\ 1 & \text{pentru orice poziţie a lui } t_i \text{ diferită de centru} \\ 0 & \text{pentru } t_i \text{ aflat la centrul mozaicului} \end{cases}$$

Deşi funcţia h_3 nu este admisibilă, s-a constatat experimental că algoritmul A^* care utilizează această funcţie are performanţe foarte bune, mai ales pentru cazuri generalizate ale problemei mozaicului, de exemplu pentru mozaicul de 15 numere.

2.3.4 Căutarea soluţiei optime în grafuri SI/SAU. Algoritmul AO^*

Obţinerea soluţiei optime pentru reprezentarea prin descompunerea problemei în subprobleme se poate realiza cu un algoritm similar ca idee cu algoritmul A^* . Diferenţa între cei doi algoritmi

consta in natura solutiei problemei, respectiv prezenta nodurilor SI care indica o multime de subprobleme ce trebuie rezolvate. Aspectele specifice care trebuie considerate in cazul unei solutii arbore SI/SAU sint:

- Cum poate fi utilizata informatia euristica in cautarea solutiei optime
- Cum se defineste o solutie optima

La executia unui algoritm de cautare de tip "best-first" in spatiul starilor exista o corespondenta de unu la unu intre nodurile candidate la expandare si solutiile parțiale construite. La un moment dat, toate solutiile parțiale potentiale sint reprezentate prin caile descoperite de la starea initiala la starile din FRONTIERA, fiecare dintre aceste cai fiind reprezentate printr-un nod unic in FRONTIERA. In cazul cautarii solutiei intr-un graf SI/SAU aceasta corespondenta de unu la unu intre nodul ales spre expandare si solutia potentiala de extins nu se mai pastreaza. Fiecare solutie partiala poate contine mai multe noduri candidate la expansiune si un nod dat poate face parte din mai multi arbori solutie potentiali. De exemplu, expandarea nodului SI S din Figura 2.12(a) inseamna generarea a doi arbori solutie potentiali, cel din Figura 2.12(b), respectiv cel din Figura 2.12(c).

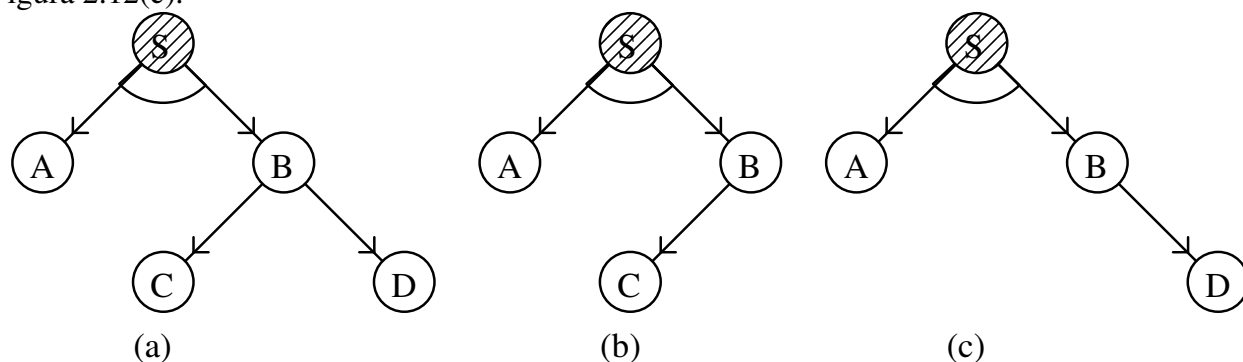


Figura 2.12 Extinderea solutiei potentiale intr-un arbore SI/SAU

In aceste conditii, informatia euristica poate fi utilizata in doua etape ale cautarii. In primul rind se identifica *solutia cea mai promitatoare* prin utilizarea unei *functii de evaluare a grafului* f . In al doilea rind se selecteaza din aceasta solutie partiala nodul urmator de expandat pe baza unei *functii de evaluare a nodurilor* f_n . Aceste doua functii, cu doua roluri diferite, ofera doua tipuri de estimari: f estimeaza proprietatile arborilor solutie care pot fi generati dintr-un arbore candidat curent, in timp ce f_n estimeaza cantitatea de informatie pe care o poate oferi expandarea unui nod cu privire la superioritatea grafului ce contine acel nod. Functia f este cea care stabileste optimalitatea solutiei pe baza unor costuri asociate procesului de descompunere a problemei in subprobleme. Cele mai multe cercetari in domeniul cautarii s-au orientat spre gasirea unor modalitati de calcul a functiei f . Functia de evaluare a nodurilor s-a bucurat de mai putina atentie si se alege intr-o maniera ad hoc.

Determinarea arborelui solutie cel mai promitator se face pe baza costului asociat arborilor generati in cautare. Costul unui arbore solutie SI/SAU poate fi definit in doua moduri: cost suma si cost maxim, pe baza costurilor asociate arcelor din graf. *Costul suma* al unui arbore solutie este suma costurilor tuturor arcelor din arbore. *Costul maxim* al unui arbore solutie este suma costurilor de-a lungul caii celei mai costisitoare intre radacina si un nod terminal. Daca fiecare arc

al arborelui solutie are costul unitar, costul suma este numarul de arce din arbore si costul maxim este adincimea nodului celui mai departat de radacina.

Daca intreg spatiul de cautare ar putea fi explorat, atunci s-ar putea stabili cu usurinta arborele solutie optima conform definitiei urmatoare.

Definitie. *Costul unui arbore solutie optima*, notat cu c , intr-un spatiu de cautare graf SI/SAU se calculeaza astfel:

- (1) Daca S este nod terminal etichetat cu o problema elementara atunci $c(S) = 0$
- (2) Daca S este nod SAU cu succesorii $S_1, S_2, \dots, S_k$ atunci

$$c(S) = \min_{j=1, k} (\text{cost_arc}(S, S_j) + c(S_j))$$
- (3) Daca S este nod SI cu succesorii $S_1, S_2, \dots, S_m$ si se foloseste *costul suma* atunci

$$c(S) = \sum_{j=1}^m (\text{cost_arc}(S, S_j) + c(S_j))$$
- (4) Daca S este nod SI cu succesorii $S_1, S_2, \dots, S_m$ si se foloseste *costul maxim* atunci

$$c(S) = \max_{j=1, m} (\text{cost_arc}(S, S_j) + c(S_j))$$
- (5) Daca S este nod terminal etichetat cu o problema neelementara (care nu se mai poate descompune) atunci $c(S) = \inf$ (infinit)

Conform acestei definitii, costul nodului problema initiala $c(P_i)$ este finit daca si numai daca problema reprezentata prin nodul initial P_i este rezolvata. Pentru fiecare nod S , $c(S)$ calculeaza costul arborelui solutie optimal a problemei reprezentata prin nodul S .

Exemplu. Fie arborele SI/SAU din Figura 2.13(a), unde cu e_i s-au notat nodurile terminale etichetate cu probleme elementare si cu n_i nodurile terminale etichetate cu probleme neelementare. Nodurile terminale e_1, e_2, e_3, e_4, e_5 si e_6 au asociat un cost zero deoarece corespund unor probleme elementare (banal de rezolvat), iar nodurile n_1, n_2 au asociat costul infinit (inf) deoarece corespund unor probleme neelementare care nu se mai pot descompune in subprobleme, deci sint imposibil de rezolvat. Atit arcele corespunzatoare nodurilor SAU, cit si cele corespunzatoare nodurilor SI au asociate costurile specificate in figura, costuri corespunzatoare tranzitiilor sau descompunerilor. Daca se utilizeaza costul suma, valorile functiei cost c sint prezentate in Figura 2.13(b) iar arborele solutie optima este format din nodurile S, B, D, E, e_5 si e_6 . Daca se utilizeaza costul maxim atunci valorile functiei c sint cele prezentate in Figura 2.13(c) si arborele solutie optima este format din nodurile S, A, e_1, e_2, e_3 .

Functia $c(S)$ asociata unui arbore solutie optima este costul real, similar functiei $f^*(S)$ din cautarea informata in spatiul starilor. Functia de evaluare a grafului f este o estimare a functiei de cost reale $c(S)$. Functia de evaluare a nodurilor f_n este o estimare a meritului real f_n^* al unui nod.

Algoritmul AO^* prezentat in continuare determina solutia optima prin construirea arborelui celui mai promitator pe baza functiei euristice f . *Costul estimat* al acestui arbore este $f(P_i)$ unde P_i este problema initiala asociata nodului radacina al arborelui.

Exemple de stabilire si utilizare a acestor costuri pentru probleme particulare vor fi prezentate in continuare.

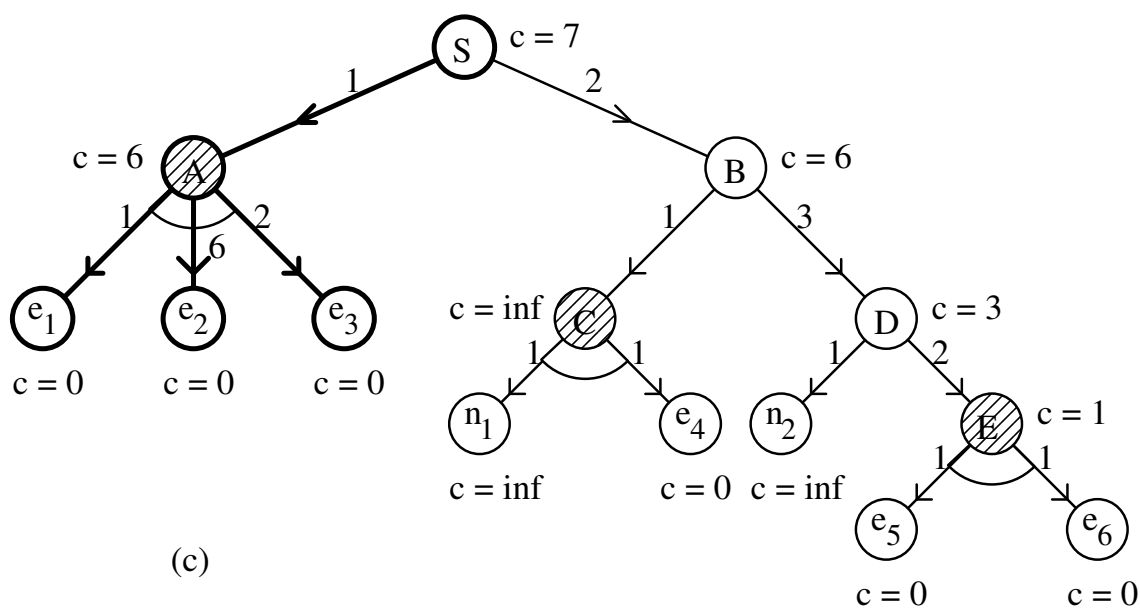
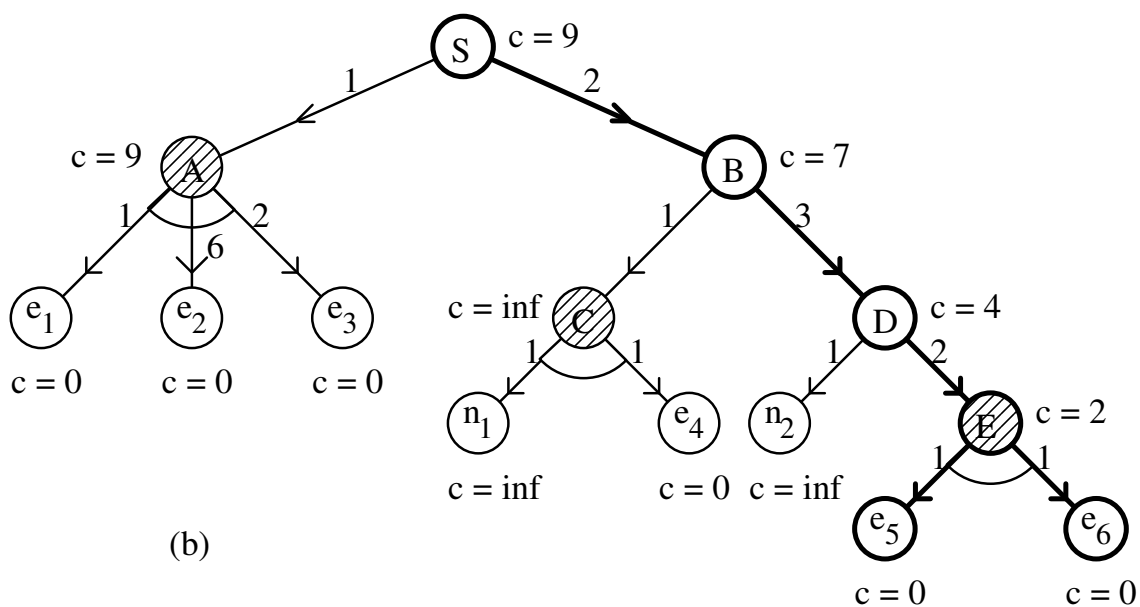
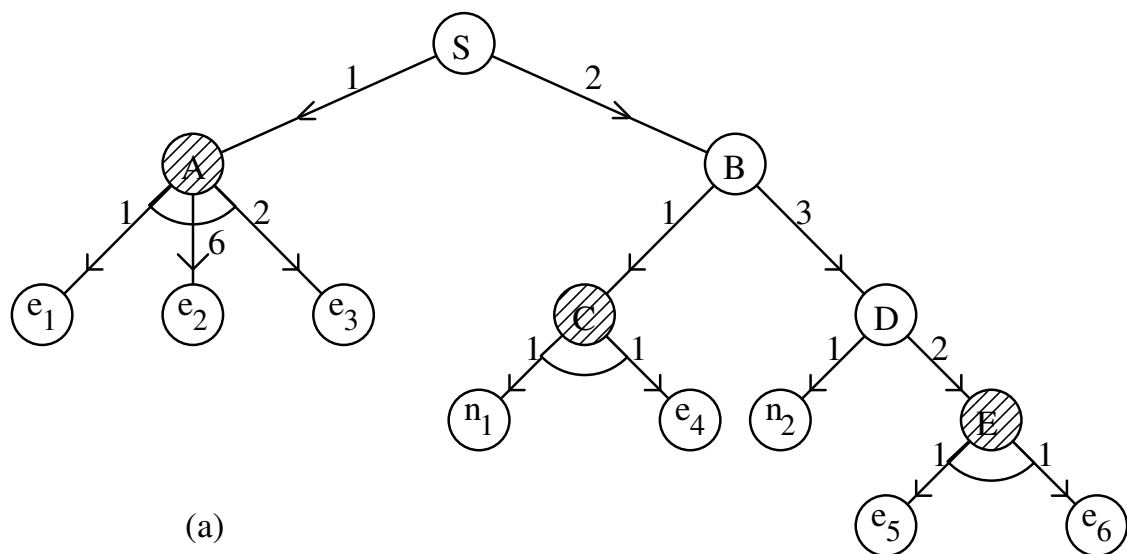


Figura 2.13 Costul suma si costul maxim al unui arbore solutie optima.

Definitie. *Arborele solutie cel mai promitator* T intr-un graf SI/SAU ponderat, i.e. cu costuri asociate arcelor, se defineste astfel:

- (1) Nodul problema initiala S_i este in T
- (2) Daca arborele SI/SAU de cautare contine un nod SI atunci toti succesorii acestui nod sint in T .
- (3) Daca arborele de cautare contine un nod SAU cu succesorii $S_1, S_2, \dots, S_k$ atunci nodul S_j , $j = 1, k$ pentru care suma $\text{cost_arc}(S, S_j) + f(S_j)$ este minima apartine lui T .

In timpul algoritmului de cautare costul arborelui solutie cel mai promitator $f(S_i)$ se calculeaza de la frunze la radacina. Deoarece la un moment dat arborele este doar partial construit, functia $f(S_j)$ trebuie sa estimeze euristic costul nodurilor S_j neexpandate inca. La o expandare a unui astfel de nod se face o reevaluare a costului total al arborelui $f(S_i)$ pe baza noului cost obtinut pentru nodul S_j .

Algoritmul AO^* este *admisibil*, deci gaseste arborele solutie optima, daca se indeplinesc urmatoarele doua conditii:

- $f(S) \leq c(S)$ pentru orice nod S
- $\text{cost}(S_k, S_{k+1}) > 0$ si este finit, pentru orice S_k, S_{k+1} cu S_{k+1} succesorul direct al lui S_k

In algoritmul prezentat in continuare se utilizeaza numai functia de evaluare euristica a grafului f . Starea problema initiala este notata cu S_i .

Algoritm: Algoritmul AO^*

1. Construiesc listele $FRONTIERA \leftarrow \{S_i\}$ si $TERITORIU \leftarrow \{ \}$
2. Initializeaza arborele solutie $T \leftarrow \{S_i\}$ si calculeaza $f(S_i)$
3. **daca** nodul S_i este rezolvat */* nodul problema initiala este rezolvat */*

atunci

3.1. Solutia este arborele T

3.2. **intoarce** SUCCES

4. Construiesc din T arborele solutie cel mai promitator T'

/ conform definitiei */*

5. $T \leftarrow T'$

6. Selecteaza un nod $S \in FRONTIERA \cap T$

7. **daca** S este un nod terminal etichetat cu problema elementara

atunci

7.1. Eticheteaza nodul S rezolvat

7.2. Asociaza nodului S costul $f(S) = c(S)$ */* costul real al unui nod problema elementara poate fi estimat */*

7.3. Eticheteaza cu rezolvat toate nodurile predecesoare lui S din $FRONTIERA$ care devin rezolvate datorita lui S

7.4. Recalculeaza $f(S_i)$ pe baza noului cost al lui S

7.5. repeta de la 3

8. **daca** S este un nod terminal etichetat cu problema neelementara
atunci

8.1. Eticheteaza nodul S nerezolvabil

8.2. Asociaza nodului S costul $f(S) = \inf$

8.3. Recalculeaza $f(S_i)$ pe baza noului cost al lui S

8.4. **daca** $f(S_i) = \inf$

atunci

8.4.1. Problema nu are solutie

8.4.2. **intoarce** INSUCCES

8.5. **altfel**

8.5.1. Elimina din FRONTIERA toate nodurile cu un predecesor nerezolvabil

8.5.2. **repeta de la 3**

9. Expandeaza nodul S

9.1. Genereaza toti succesorii directi S_j ai nodului S

9.2. **pentru** fiecare succesor S_j ($1 \leq j \leq m$) al lui S **executa**

9.2.1. Stabileste legatura $S_j \rightarrow S$

9.2.2. **daca** S_j reprezinta o multime de cel putin 2 subprobleme

atunci /* S_j este nod SI */

i. Genereaza toti succesorii subprobleme S_j^k ($1 \leq k \leq n$) ai lui S_j

ii. Stabileste legaturile intre nodurile $S_j^k \rightarrow S_j$, ($1 \leq k \leq n$)

iii. Insereaza nodurile S_j^k ($1 \leq k \leq n$) in FRONTIERA

9.2.3. **altfel** insereaza S_j in FRONTIERA

9.3. Recalculeaza $f(S)$ si $f(S_k)$ pentru toate nodurile predecesoare S_k ale nodului S

10. **repeta de la 3**

sfirsit.

Observatii:

- Daca se utilizeaza costul suma, la recalcularea valorii $f(S_i)$ nu este necesara reparcurgerea intregului arbore generat. Citiva parametri auxiliari asociati predecesorului nodului S vor permite calculul functiei $f(S_i)$ local si transmiterea acestei valori de la tata la fiu pentru fiecare expandare de nod.
- Eficienta algoritmului depinde atit de gradul de informare a functiei f cit si de implementarea pasului 6 in care, gasind cel mai promitator arbore solutie, trebuie sa se

decida care nod din acest arbore va fi expandat. Daca arborele partial T construit este intr-adevar o parte din solutia optima, atunci alegerea nodului nu conteaza. In caz contrar, cel mai bun nod ales va fi acela care va demonstra cit mai repede ca T nu este arborele solutie optima.

- Daca se utilizeaza si functia de evaluare a nodurilor f_n , atunci in pasul 6 selectia urmatorului nod de expandat se face pe baza valorilor acestei functii.

Algoritmul prezentat determina arborele solutie optima tinind cont de criteriul de cost minim. Daca evaluarea solutiei se face in termenii calitatii unei strategii de rezolvare a problemei, arborele solutie optima trebuie redefinit astfel incit pentru un nod SAU selectia succesorului preferat sa se faca pe baza valorii maxime a functiei de evaluare.

Modul de calcul al functiei euristice f depinde de problema de rezolvat. De exemplu, pentru problema cintaririi monezilor prezentata in Sectiunea 2.2.3 se poate alege functia f asociata unui nod subproblema ca fiind estimarea efortului necesar pentru rezolvarea acestei subprobleme. Functia f se defineste ca mai jos.

$$f(S) = \begin{cases} 0 & \text{daca } S \text{ este nod terminal etichetat cu problema elementara} \\ \min_{j=1,k} [c(S, S_j) + f(S_j)] & \text{daca } S \text{ este un nod SAU cu succesorii } S_j, j = 1, k \\ \sum_{j=1}^3 p_j \cdot f(S_j) & \text{daca } S \text{ este un nod SI cu succesorii } S_j, j = 1, 3 \end{cases}$$

In formula, $c(S, S_j)$ este costul unui test reprezentat de legatura intre nodul S si nodul S_j iar p_1, p_2, p_3 sint probabilitatile asociate celor trei rezultate posibile ale unui test de cintarire. Pentru nodurile S_j neevaluate, $f(S_j)$ va fi o valoare estimata care se va recalcula in functie de expandarea nodului S_j . In acest caz se aplica criteriul de cost minim, deci definitia anterioara a arborelui solutie cel mai promitator.

In cadrul teoriei jocurilor, reprezentarea solutiei problemei prin subprobleme este utilizata pentru a descrie arborele de miscari posibile a doi adversari care joaca un joc. Daca se noteaza cu $v(S)$ cistigul obtinut de jucatorul 1 care a atins nodul terminal (cistigator) S si se presupune ca jucatorul 2 actioneaza astfel incit sa minimizeze cistigul primului jucator, functia de evaluare se poate defini astfel:

$$f(S) = \begin{cases} v(S) & \text{daca } S \text{ este nod terminal cistigator pentru jucatorul 1} \\ \max_j [f(S_j)] & \text{daca din nodul } S \text{ jucatorul 1 este cel care muta} \\ \min_j [f(S_j)] & \text{daca din nodul } S \text{ jucatorul 2 este cel care muta} \end{cases}$$

In acest caz definitia arborelui solutie cel mai promitator trebuie modificata astfel incit in conditia (3) sa se aleaga succesorul pentru care $f(S)$ are valoarea maxima. Se obtine astfel strategia de cautare MinMax, strategie utilizata in teoria jocurilor si prezentata pe scurt in Sectiunea 1.5, in care functia f apreciaza calitatea strategiei utilizata de jucatorul 1.

2.4 Consideratii de complexitate a strategiilor de cautare

Toate problemele care implica o rezolvare prin cautare sint supuse pericolului exploziei combinationale. Este greu de imaginat dimensiunea cresterii combinationale a unei rezolvari, deci complexitatea exponentiala. De exemplu, s-a estimat ca numarul de stari al unui spatiu de cautare complet in jocul de sah, in termenii numarului de miscari posibile, este de 10^{120} . Evident, oricit de performante ar fi sau ar putea deveni calculatoarele, este imposibil de investigat exhaustiv un astfel de spatiu. Valoarea 10^{120} este comparabila cu numarul de molecule din univers! In problema comis-voiajorului complexitatea unei cautari exhaustive a traseului optim pentru N orase este de $(N-1)!$. Pentru problema Croes cu 20 de orase ar trebui investigate $20!$ trasee pentru a usura viata comis-voiajorului.

Toate strategiile de cautare au o complexitate timp exponentiala pentru cazul cel mai defavorabil. In cazul strategiilor de cautare informata posibilitatea atingerii cazului celui mai defavorabil este mult redusa, mai ales daca se alege o functie euristica bine informata. S-au propus mai multe variante de calcul a complexitatii strategiilor de cautare. De cele mai multe ori complexitatea algoritmilor de cautare este evaluata in functie de factorul de ramificare. *Factorul de ramificare* al unui spatiu de cautare, notat cu B, este definit ca numarul mediu de succesori directi ai unei stari in acest spatiu. Numarul de stari posibil de generat pe un nivel de cautare d, notat cu NS, se poate calcula in functie de factorul de ramificare si este $NS_d = B^d$. Odata calculat factorul de ramificare, este posibil sa se estimeze costul cautarii pentru a genera o cale de lungime L. Notind cu T numarul total de stari generate intr-un proces de cautare, exista relatia

$$T = B + B^2 + \dots + B^L = B(B^L - 1) / (B - 1)$$

Complexitatea timp a unei strategii de cautare pe nivel este $O(B^d)$ deoarece se genereaza toate starile de pe fiecare nivel si numarul total de stari generate este cel din formula de mai sus. Complexitatea spatiu este de asemenea $O(B^d)$ deoarece toate nodurile de pe un anumit nivel trebuie memorate, deci B^{d-1} noduri sint memorate la nivelul d-1 pentru a genera nodurile de pe nivelul d. Aceasta complexitate exponentiala atit a timpului cit si a spatiului este dezavantajul principal al strategiei de cautare pe nivel.

Complexitatea timp a unei strategii de cautare in adincime este tot exponentiala, deci $O(B^d)$. Spatiul utilizat de aceasta cautare este, in schimb, dependent liniar de lungimea caii de cautare curenta. Pentru fiecare nivel d-1 se memoreaza numai succesorii directi ai unei singure stari, deci este nevoie de $B \cdot d$ stari memorate pentru a cauta pina la un nivel d. In consecinta complexitatea spatiu a cautarii in adincime este $O(d)$.

Strategia de cautare in adincime cu nivel iterativ are o complexitate spatiu de $O(d)$ deoarece la fiecare iteratie se aplica politica de cautare in adincime. Desi s-ar parea ca este mai putin eficienta din punct de vedere al timpului de cautare decit cautarea pe nivel sau cautarea in adincime obisnuita, complexitatea timp a cautarii in adincime cu nivel iterativ este de acelasi ordin de marime, i.e. $O(B^d)$. Deci aceasta strategie reduce complexitatea spatiu la o dependenta liniara, dar garanteaza gasirea solutiei optime, spre deosebire de cautarea in adincime care, desi de aceeasi complexitate spatiala, poate pierde solutia. Cu toate ca ordinul de complexitate temporală

este acelasi, strategia de cautare in adincime cu nivel iterativ pierde totusi ceva mai mult timp facind calcule repetate pentru regenerarea starilor.

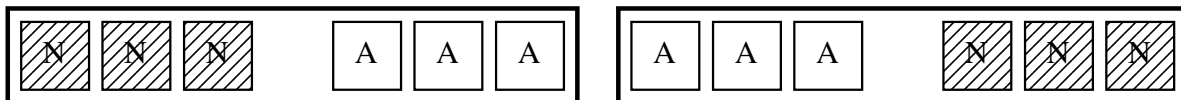
Determinarea factorului de ramificare al unui spatiu de cautare pentru a evalua performantele unei strategii nu se poate face riguros. De exemplu, se considera din nou problema mozaicului de 8 numere. Numarul total de miscari este: 2 miscari din fiecare colt genereaza 8 miscari, 3 miscari din centrul fiecărei laturi genereaza 12 miscari, si mai exista 4 miscari posibile din pozitia centrala, in total 24 de miscari posibile. Factorul de ramificare se obtine prin impartirea valorii 24 la 9, numarul de pozitii diferite posibile ale patratului liber, deci 2.67. Acest factor de ramificare conduce la rezultate destul de proaste. Daca se elimina miscarile care duc direct dintr-o stare in starea ei anterioara, atunci exista o miscare mai putin pentru fiecare stare, ceea ce determina un factor de ramificare de 1.67. Aceasta valoare este considerabil mai buna. Factorul de ramificare poate fi semnificativ redus prin utilizarea strategiilor euristice. Cu cit o functie euristica este mai informata, cu atit numarul de stari generate in cautare va fi mai mic.

2.5 Exerciții și probleme

1. Care dintre strategiile de cautare neinformata ar fi mai potrivita pentru rezolvarea urmatoarelor probleme:
 - un program de jucat sah
 - un program de diagnosticare medicala
 - un program de planificare automata care conduce miscarile unui robot printr-o camera
 - un program de recunoastere a obiectelor ca apartinand sau nu unor clase de obiecte date
2. Se considera urmatoarea problema: un fermier trebuie sa transporte de pe un mal pe malul opus al unui riu un lup, o capra si o varza utilizind o barca care poate contine fermierul si inca un element de transportat. Din criterii de siguranta, pe acelasi mal nu trebuie sa ramina lupul si capra sau capra si varza nesupravegheate.
 - (a) Sa se defineasca o reprezentare a solutiei problemei prin spatiul starilor si sa se indice solutia.
 - (b) Este potrivita o reprezentare prin grafuri SI/SAU pentru aceasta problema? Daca da, sa se indice una, daca nu sa se justifice de ce nu.
3. Sa se defineasca o functie euristica h admisibila pe care o poate utiliza un algoritm A^* care rezolva problema cu fermierul, lupul, capra si varza.
4. Fie urmatoarea problema a mozaicului de 6 piese. Mozaicul este format din trei piese negre la stinga, trei piese albe la dreapta si un spatiu liber intre aceste doua grupuri de piese, dupa cum

se vede in Figura 2.14. Problema cere sa se ajunga din configuratia initiala prezentata in Figura 2.14(a) in configuratia finala prezentata in Figura 2.14(b). Exista doua miscari permise:

- O piesa poate fi mutata intr-un spatiu liber alaturat. Aceasta miscare are asociat costul 1.
- O piesa poate fi deplasata peste una sau doua piese alaturate intr-un spatiu liber. Aceasta miscare are asociat un cost egal cu numarul de piese peste care s-a deplasat.



(a) Configuratia initiala

(b) Configuratia finala

Figura 2.14 Problema mozaicului de 6 piese

- Sa se specifice o reprezentare adecvata a solutiei problemei.
 - Sa se deseneze portiunea din spatiul de cautare care contine solutia problemei.
 - Sa se propuna o functie euristica care ar putea fi utilizata de un algoritm de cautare informata. Este aceasta functie admisibila?
- Sa se specifice care este reprezentarea prin descompunerea problemei in subprobleme pentru o problema de integrare simbolica prin parti.
 - Se considera problema celor opt regine definita in Sectiunea 2.1.1.
 - Sa se indice o reprezentare prin spatiul starilor a solutiei.
 - Sa se defineasca o functie de evaluare a starii celei mai promitatoare care va fi folosita de un algoritm de cautare de tip "best-first" pentru rezolvarea acestei probleme.
 - Dindu-se arborele de cautare din Figura 2.15, sa se indice continutul listelor FRONTIERA si TERITORIU in momentul in care un algoritm de cautare de tip "best-first" a construit acest arbore. Numerele din dreptul fiecarui nod reprezinta estimarea distantei pina la starea finala.

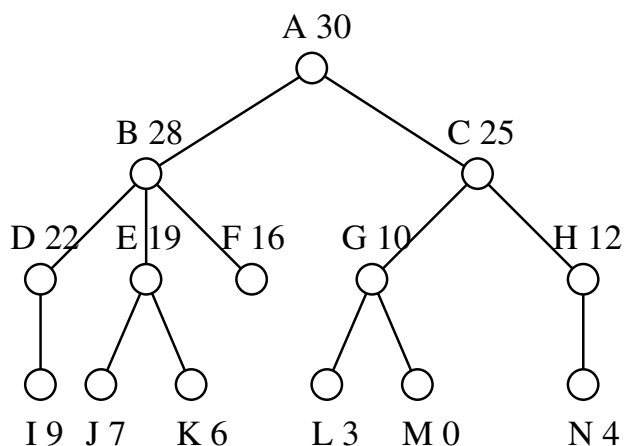
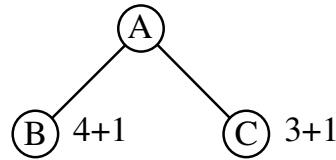
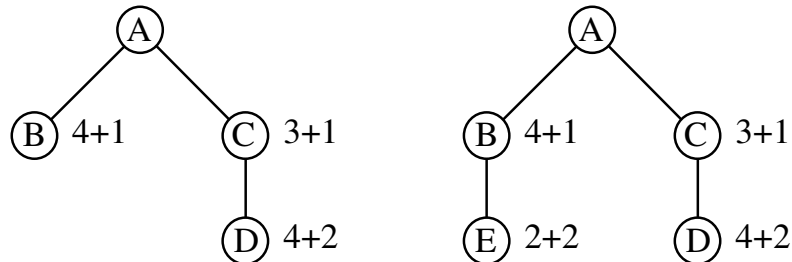


Figura 2.15 Arbore de cautare cu estimarea distantelor pina la starea finala

8. Sa se dea doua exemple de probleme in care intereseaza mai mult minimizarea efortului de cautare decit optimalitatea solutiei.
9. Se presupune ca prin expandarea nodului stare initiala A de un algoritmul A^* rezulta urmatorul arbore, in care valorile functiei f sint notate sub forma $h + g$.



A doua si a treia expandare de noduri facuta de algoritm rezulta in urmatorii doi arbori:



- (a) Care va fi nodul urmator expandat?
- (b) Este algoritmul admisibil?
10. Sa se scrie un algoritm pentru implementarea strategiei de cautare in adincime cu nivel iterativ.
11. Sa se scrie algoritmul de cautare "best-first" pentru reprezentarea solutiei problemei prin descompunere in subprobleme. Ce modificari trebuie facute pentru a determina calea de cost minim de la problema initiala P_i la o multime de probleme elementare date $\{P_{e_j}\}$?
12. Sa se modifice algoritmul AO^* astfel incit la selectia unui nod sa se aleaga nodul cu cea mai mare valoare a functiei de evaluare a nodurilor f_n . In pasul 6 al algoritmului AO^* (Sectiunea 2.3.4) s-a ales la intimplare un nod pentru a fi expandat. Sa se modifice algoritmul astfel incit sa se aleaga nodul al carui cost curent este cel mai mic. Argumentul in favoarea acestei modificari consta in faptul ca pentru acel nod mai sint necesari foarte putini pasi pina cind fie s-a gasit o solutie, fie se produce o revizuire a costului estimat. Pentru noduri care au costul curent estimat foarte mare, pe de alta parte, sint necesari inca mai multi pasi pina cind se obtin noi informatii. Cum trebuie modificat algoritmul astfel incit sa implementeze aceasta euristica de alegere a nodurilor pentru expandare?
13. Spatiul de cautare a solutiei unei probleme reprezentata prin descompunerea problemei in subprobleme este definit astfel:

- N_1 este nod problema initiala, nod SAU avind ca succesorii trei noduri SI: N_2 , N_3 si N_4 .
- N_2 se descompune in trei subprobleme: N_5 (elementara), N_6 (elementara) si N_7 .
- N_3 se descompune in trei subprobleme: N_7 , N_8 si N_9 (elementara).
- N_4 se descompune in doua subprobleme: N_3 si N_{10} (elementara).
- N_7 este nod SAU ca succesorii doua noduri SI: N_8 si N_{11} .
- N_8 se descompune in doua subprobleme: N_{12} (elementara) si N_{13} (elementara).
- N_{11} se descompune in subproblema N_{14} (elementara).

Se cere:

- (a) Sa se construiasca graful SI/SAU asociat.
 - (b) Sa se indice toti arborii solutie.
 - (c) Sa se adauge costuri arcelor si sa se calculeze arborele solutie optim utilizind aceste costuri.
 - (d) Daca toate costurile sint unitare care este arborele solutie optim?
14. Care este factorul de ramificare intr-un arbore de cautare pentru problema mozaicului de 15 numere?. Problema mozaicului de 15 numere este similara cu cea a celor 8 numere cu diferenta ca mozaicul are o dimensiunea de 4×4 a tablei.
15. Sa se aleaga un limbaj de programare si sa se implementeze in acest limbaj toti algoritmi de cautare prezentati.

Partea a II-a
Reprezentarea cunostintelor

Capitolul 3

Modelul logicii simbolice

Partea a II-a a textului de fata este dedicata metodelor de reprezentare a cunostintelor. Acest capitol si urmatoarele trei prezinta cele mai importante modele de reprezentare a cunostintelor utilizate in programele de inteligenta artificiala. In finalul Capitolului 6 se vor pune in evidenta modalitati de combinare a modelelor discutate, combinarea paradigmelor de reprezentare fiind tendinta actuala in sistemele bazate pe cunostinte.

Unul dintre primele si cele mai importante modele de reprezentare a cunostintelor in inteligenta artificiala este logica simbolica. Logica simbolica a fost dezvoltata de logicieni ca o metoda formala de rationament, in principal in domeniul matematicii, cel mai raspindit model logic fiind logica cu predicate de ordinul I.

Logica cu predicate de ordinul I a fost folosita pentru prima data ca metoda de reprezentare a cunostintelor in inteligenta artificiala in programul de demonstrare a teoremelor al lui Gilmore si in programul "The Logic Theorist" al lui Newell, Shaw si Simon [1963]. Programul lui Gilmore pentru demonstrarea teoremelor in logica cu predicate de ordinul I s-a bazat pe rezultatele teoretice importante ale lui Herbrand [Chang, Lee, 1973] care au stabilit conditiile in care o multime de formule (clauze) este inconsistentă. In 1965 Robinson, plecind de la rezultatele lui Herbrand, a propus o metoda mult mai eficienta de stabilire a inconsistentei unei formule: rezolutia. Aceasta metoda si diversele ei rafinari ulterioare au devenit abordarea preferentiala a celor mai multe demonstratoare de teoreme dezvoltate pina in prezent.

Utilizarea logicii simbolice ca model de reprezentare a cunostintelor in inteligenta artificiala este importanta deoarece ofera o abordare formala a rationamentului, cu fundamente teoretice riguroase. Formalismul logic permite derivarea unor cunostinte noi, plecind de la cele existente, pe baza deductiei si a demonstrarii teoremelor. Acest lucru faciliteaza automatizarea proceselor de rationament si executia inferentelor corecte si logic valide. Pe de alta parte, logica simbolica este suficient de expresiva si flexibila pentru a permite reprezentarea cu acuratete a cunostintelor problemei de rezolvat.

Rezolvarea problemelor in cadrul formalismului logic se bazeaza in special pe *demonstrarea teoremelor*. Exista doua abordari importante in demonstrarea teoremelor: metodele sintactice si metodele semantice.

Metodele sintactice de demonstrare a teoremelor, cum ar fi de exemplu cele propuse de Herbrand, Gilmore si Robinson, se bazeaza pe procedee mecanice de aplicare a regulilor de inferenta si sint independente de domeniul de interpretare al formulei. Aceste metode pot fi usor automatizate si constituie baza tuturor programelor de demonstrare a teoremelor.

Metodele semantice de demonstrare a teoremelor se bazeaza pe utilizarea sistematica a valorilor de adevar ale formulelor si depind de domeniul de interpretare fixat. Deoarece domeniul de interpretare al unei formule este de multe ori infinit, aceste metode sint greu algoritimizabile. O

discutie a metodelor semantice de demonstrare poate fi gasita in Malita [1987] si Popa [1992]. O prezentare detaliata a principiilor teoretice si a metodelor sintactice de demonstrare a teoremelor poate fi gasita in Chang si Lee [1973] si Gabbay s.a. [1993].

Reprezentarea cunostintelor in logica se bazeaza in esenta pe urmatoarele doua componente:

- Crearea structurilor formale care reprezinta fapte de baza, inferente si alte tipuri de cunostinte, numite *structuri presupuse*.
- Aplicarea regulilor de inferenta ale sistemului logic pentru a compara, combina si obtine din aceste structuri presupuse (date) noi structuri, numite *structuri deduse*.

De exemplu, enuntul "Toti studentii de la calculatoare stiu sa programeze" poate fi exprimat in logica cu predicate de ordinul I astfel:

$$(\forall x) (\text{StudentLaCalculatoare}(x) \rightarrow \text{Programeaza}(x))$$

Daca se stie in plus ca Radu este student la calculatoare, deci

$$\text{StudentLaCalculatoare}(\text{Radu})$$

se poate concluziona pe baza celor doua structuri presupuse noua structura

$$\text{Programeaza}(\text{Radu})$$

3.1 Logica propozitionala

Logica propozitionala este un caz particular al logicii cu predicate de ordinul I. Elementele de baza ale logicii propozitionale sint *propozitiile*, numite si *atomi* sau *propozitii simple*. Urmatoarele enunturi sint propozitii:

Masina este alba P

Oamenii traiesc pe luna Q

Notatiile din dreapta reprezinta simbolic propozitiile enuntate. O propozitie poate fi adevarata sau falsa, deci poate avea doua valori de adevar. De exemplu, propozitia P este adevarata si propozitia Q este falsa. Atita timp cit se admit doar doua valori de adevar pentru o propozitie, adevarat si fals, logica se numeste *logica clasica* sau *logica bivalenta*. Acceptarea unui numar mai mare de valori conduce la *logici polivalente* (neclasice). Prezentarea care urmeaza se situeaza numai in cadrul logicii bivalente.

3.1.1 Sintaxa logicii propozitionale

Propozitiile simple sau atomii sint compozabile. Ele se pot combina, dind nastere la noi propozitii care sint, la rindul lor, adevarate sau false. Propozitiile combinate se formeaza din atomi folosind conectorii logici. Conectorii logici indica operatiile de asociere sau combinare care sint cele mai frecvente in vorbire sau rationament. De exemplu, din doua propozitii simple

Mihaela este frumoasa P₁

Mihaela este buna P₂

se poate forma propozitia compusa

Mihaela este frumoasa si Mihaela este buna

notata cu P₁ ∧ P₂. In continuare se vor folosi urmatoarele simboluri pentru conectorii logici:

~ negatie

- $\wedge$ conjunctie
- $\vee$ disjunctie
- $\rightarrow$ implicatie simpla
- $\leftrightarrow$ implicatie dubla

In consecinta, alfabetul logicii propozitionale este format din simbolurile care desemneaza atomii, conectorii logici si alte simboluri cum ar fi parantezele. In plus exista doua simboluri speciale pentru desemnarea valorilor logice de adevarat si fals, notate prin conventie cu **a**, respectiv **f**.

Limbajul logicii propozitionale se construiește pornind de la definitia alfabetului si definind regulile corecte de formare a cuvintelor limbajului cu simboluri din alfabet. Un cuvint in acest limbaj se numeste formula bine formata.

Definitie. O formula bine formata in calculul propozitional se defineste recursiv astfel:

- (1) Un atom este o formula bine formata
- (2) Daca P este formula bine formata, atunci $\sim P$ este formula bine formata.
- (3) Daca P si Q sint formule bine formate atunci $P \wedge Q$, $P \vee Q$, $P \rightarrow Q$ si $P \leftrightarrow Q$ sint formule bine formate.
- (4) multimea formulelor bine formate este generata prin aplicarea repetata a regulilor (1)÷(3) de un numar finit de ori.

O formula bine formata se scrie riguros utilizind paranteze. De obicei se omit aceste paranteze ori de cite ori absentia lor nu da nastere la confuzii, tinind cont de precedenta conectorilor logici. Precedenta conectorilor logici, in ordine descrescatoare, este: $\sim$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$.

Example:

1. $(P \rightarrow)$ nu este o formula bine formata.
2. $(P \rightarrow (Q \wedge (\sim R)))$ este o formula bine formata si poate fi scrisa $P \rightarrow (Q \wedge \sim R)$.
3. $((P \wedge (\sim Q \vee R) \rightarrow (Q \rightarrow S)))$ este o formula bine formata si poate fi de asemenea scrisa $P \wedge (\sim Q \vee R) \rightarrow (Q \rightarrow S)$.

3.1.2 Semantica logicii propozitionale

Semantica sau intelesul unei propozitii este valoarea de adevarat sau fals a acesteia, i.e. atribuirea unei valori de adevar acelei propozitii pe baza unei functii de evaluare a propozitiei.

Definitie. Se numeste *interpretare* a unei formule bine formate atribuirea de valori de adevar fiecarui atom din formula. Altfel spus, o interpretare specifica functiile de evaluare ale tuturor atomilor componentii ai formulei.

Exemplu. Fie formula $(P \wedge \sim Q)$ si o interpretare I_1 care asigneaza valorile de adevar **a** lui P si **f** lui Q. In aceasta interpretare formula este adevarata. O interpretare diferita I_2 asigneaza valorile de adevar **a** lui P si **a** lui Q, formula fiind falsa in aceasta interpretare. Evident, exista patru interpretari distincte pentru aceasta propozitie.

Functia de evaluare a unei formule asociaza formulei o unica valoare de adevar, pe baza valorilor de adevar ale atomilor componentii ai formulei, utilizind regulile de evaluare ale conectorilor logici, reguli specificate de obicei prin tabele de adevar. Deoarece se refera la semantica formulei, aceste reguli se mai numesc si *reguli semantice*. In continuare se va nota

valoarea de adevar a unei formule P cu P^V . Odata ce s-a stabilit o interpretare unei formule, valoarea ei de adevar poate fi determinata. Aceasta se poate face prin aplicarea repetata a regulilor semantice asupra unor portiuni din ce in ce mai mari ale formulei, pina cind se determina o singura valoare de adevar pentru formula. Regulile semantice sint prezentate in Figura 3.1.

Regula	Propozitie adevarata	Propozitie falsa
1	a	f
2	$\sim \mathbf{f}$	$\sim \mathbf{a}$
3	$A \wedge A'$	$F \wedge P$
4	$A \vee P$	$P \wedge F$
5	$P \vee A$	$F \vee F'$
6	$P \rightarrow A$	$A \rightarrow F$
7	$F \rightarrow P$	$A \rightarrow F$
8	$A \leftrightarrow A'$	$F \leftrightarrow A$
9	$F \leftrightarrow F'$	

A si A' reprezinta propozitii adevarate.

F si F' reprezinta propozitii false.

P reprezinta orice propozitie.

Figura 3.1 Regulile semantice ale formulelor bine formate in calculul propozitional

Se observa ca Figura 3.1 reprezinta concis regulile de evaluare ale conectorilor logici. De exemplu, semnificatia tabelor de adevar ale conectorilor logici $\wedge$ si $\leftrightarrow$

P	Q	a	f
a	a	a	f
f	f	f	f

$P \wedge Q$

P	Q	a	f
a	a	a	f
f	f	f	a

$P \leftrightarrow Q$

este surprinsa de regulile 3, 4, 8 si 9 din Figura 3.1. In aceste conditii se poate gasi semnificatia unei propozitii fiind data o interpretare I pentru acea propozitie.

Exemplu. Fie formula $((P \wedge \sim Q) \rightarrow R) \vee Q$ si interpretarea I: P adevarat, Q fals si R fals. Atunci:

prin aplicarea regulii 2 $\sim Q$ adevarat

prin aplicarea regulii 3 $P \wedge \sim Q$ adevarat

prin aplicarea regulii 6 $(P \wedge \sim Q) \rightarrow R$ fals

prin aplicarea regulii 5 $((P \wedge \sim Q) \rightarrow R) \vee Q$ fals

Deci valoarea de adevar P^V a formulei in interpretarea I este fals.

3.1.3 Proprietatile propozitiilor

Pe baza functiei de evaluare si a domeniului de interpretare se pot specifica urmatoarele proprietati ale formulelor bine formate:

- O formula bine formata este *valida* sau *tautologie* daca formula are valoarea adevarat in orice interpretare.
- O formula bine formata este *inconsistenta* (*contradictie*, *nerealizabila*) daca formula are valoarea fals in orice interpretare.
- O formula bine formata este *realizabila* sau *consistenta* daca exista cel putin o interpretare in care formula are valoarea adevarat.
- Doua formule sint *echivalente* daca au aceeasi valoare de adevar in orice interpretare.

Figura 3.2 prezinta intuitiv aceste proprietati ale formulelor bine formate.

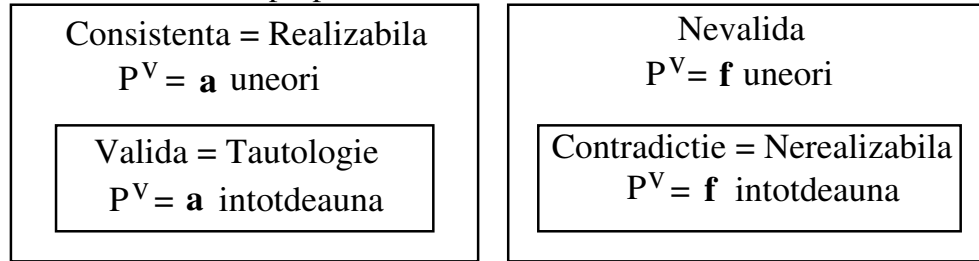


Figura 3.2 Realizabilitatea formulelor bine formate

Realizarea rationamentului intr-un sistem logic implica existenta unui mecanism de obtinere a noi formule pe baza formulelor existente, deci de extindere consistenta a cunostintelor universului problemei.

Definitie. O formula F este o *consecinta logica* a unei formule P daca F are valoarea adevarat in toate interpretarile in care P are valoarea adevarat. Definitia se poate extinde si in cazul a n formule. O formula F este *consecinta logica* a unei multimi de formule $P_1, P_2, \dots, P_n$ daca formula F este adevarata in toate interpretarile in care $P_1, P_2, \dots, P_n$ sint adevarate.

Consecinta logica se noteaza $P_1, P_2, \dots, P_n \Rightarrow F$.

Example:

1. P este o formula realizabila dar nu este valida deoarece o interpretare care atribuie fals lui P atribuie fals si formulei P .
2. $P \vee \sim P$ este o formula valida deoarece pentru orice interpretare, formula $(P \vee \sim P)$ este adevarata.
3. $P \wedge \sim P$ este o contradictie deoarece pentru orice interpretare, formula $(P \wedge \sim P)$ este falsa.
4. P si $\sim(\sim P)$ sint formule echivalente deoarece fiecare are aceeasi valoare de adevar pentru orice interpretare.
5. P este o consecinta logica a formulei $(P \wedge Q)$ deoarece pentru orice interpretare in care $(P \wedge Q)$ este adevarata, P este totdeauna adevarata.

Notiunea de consecinta logica ofera o modalitate de a realiza inferenta valide in logica propozitionala. Urmatoarele doua teoreme stabilesc criteriile in functie de care o formula este o consecinta logica a unui set de formule.

Teorema. Formula F este consecinta logica a unei multimi de formule $P_1, P_2, \dots, P_n$ daca formula $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow F$ este valida.

Demonstratie.

($\Rightarrow$) Daca F este o consecinta logica a multimii de formule $P_1, P_2, \dots, P_n$, atunci pentru orice interpretare I in care $P_1, P_2, \dots, P_n$ sint adevarate, deci formula $P_1 \wedge P_2 \wedge \dots \wedge P_n$ este adevarata, F este de asemenea adevarata prin definitie. Deci $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow F$ este adevarata pentru astfel de interpretari. Pentru interpretarile in care $P_1 \wedge P_2 \wedge \dots \wedge P_n$ este falsa, $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow F$ este adevarata conform regulii 7 din Figura 3.1. Rezulta de aici ca $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow F$ este adevarata in orice interpretare, deci este formula valida.

($\Leftarrow$) Daca $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow F$ este valida, atunci pentru orice interpretare I pentru care $P_1 \wedge P_2 \wedge \dots \wedge P_n$ este adevarata, deci in care si $P_1, P_2, \dots, P_n$ sint adevarate, F este adevarata, deci F este o consecinta logica a multimii de formule $P_1, P_2, \dots, P_n$.

Teorema. Formula F este consecinta logica a unei multimii de formule $P_1, P_2, \dots, P_n$ daca formula $P_1 \wedge P_2 \wedge \dots \wedge P_n \wedge \sim F$ este inconsistentă.

Demonstratie.

Demonstratia acestei teoreme se face pe baza primei teoreme. Conform acesteia, F este o consecinta logica a multimii de formule $P_1, P_2, \dots, P_n$ daca si numai daca $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow F$ este valida, i.e. daca si numai daca $\sim (P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow F)$ este inconsistentă.

$$\sim (P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow F) \equiv \sim (\sim (P_1 \wedge P_2 \wedge \dots \wedge P_n) \vee F) \equiv (\sim (P_1 \wedge P_2 \wedge \dots \wedge P_n)) \wedge \sim F \equiv P_1 \wedge P_2 \wedge \dots \wedge P_n \wedge \sim F, \text{ deci teorema este demonstrata.}$$

Aceste doua teoreme prezinta o importanta deosebita in rationamentul logic, deoarece problema stabilirii consecintelor logice se reduce la problema demonstrarii validitatii sau a inconsistentei unei formule bine formate. Aceasta idee este importanta si reprezinta esenta tuturor metodelor de demonstrare automata a teoremelor in logica simbolica, asa cum se va vedea in Sectiunea 3.3.

O posibilitate de a determina echivalenta a doua formule este utilizarea tabelelor de adevar. Se observa ca demonstratia celei de a doua teoreme s-a facut utilizind echivalenta formulelor logice. De exemplu, pentru a arata ca $P \rightarrow Q$ este o formula echivalenta cu $\sim P \vee Q$, se poate construi urmatoarea tabela de adevar.

P	Q	$P \rightarrow Q$	$\sim P$	$\sim P \vee Q$
a	a	a	f	a
a	f	f	f	f
f	a	a	a	a
f	f	a	a	a

In tabelul urimator sint prezentate citeva legi importante de echivalenta in logica propozitionala.

Idempotentă	$P \vee P \equiv P$	$P \wedge P \equiv P$	
Asociativitate	$(P \vee Q) \vee R \equiv P \vee (Q \vee R)$	$(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$	
Comutativitate	$P \vee Q \equiv Q \vee P$	$P \wedge Q \equiv Q \wedge P$	$P \leftrightarrow Q \equiv Q \leftrightarrow P$
Distributivitate	$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$	$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$	
De Morgan	$\sim (P \vee Q) \equiv \sim P \wedge \sim Q$	$\sim (P \wedge Q) \equiv \sim P \vee \sim Q$	
Eliminarea implicatiei	$P \rightarrow Q \equiv \sim P \vee Q$		
Eliminarea implicatiei duble	$P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$		

3.1.4 Discutie despre conectori

Definitie. O multime de conectori logici se numeste *multime adecvata* daca orice formula logica poate fi exprimata folosind numai conectorii acestei multimi.

Multimea $\{\sim, \wedge, \vee\}$ este o multime adecvata de conectori. Deasemenea, perechile $\{\sim, \wedge\}$, $\{\sim, \vee\}$ si $\{\sim, \rightarrow\}$ sint si ele multimi adecvate de conectori. Nici o alta pereche de conectori din multimea $\{\sim, \wedge, \vee, \rightarrow, \leftrightarrow\}$ nu este o multime adecvata de conectori; de exemplu $\{\sim, \leftrightarrow\}$ nu este o multime adecvata.

Se pune problema minimalitatii unei multimi de conectori, respectiv se pune intrebarea "Exista multimi adecvate formate dintr-un singur conector?". Exista doi conectori, care, fiecare in parte, formeaza o multime adecvata de conectori.

Conectorul lui Nicod, numit si Nor si notat cu $\downarrow$, permite definirea urmatoarelor formule de echivalenta: $\sim P = P \downarrow P$ si $P \wedge Q = (P \downarrow P) \downarrow (Q \downarrow Q)$, ceea ce face ca $\{\downarrow\}$ sa fie o multime adecvata. Conectorul are tabela de adevar:

P	Q	$P \downarrow Q$
a	a	f
a	f	f
f	a	f
f	f	a

Conectorul lui Sheffer, numit incompatibilitate, si notat cu $|$, este definit prin urmatoarea formula: $P | Q = \sim P \vee \sim Q$, ceea ce inseamna ca $P | Q$ este adevarat cind cel putin un atom este fals. Sint adevarate formulele: $\sim P = P | P$ si $P \wedge Q = (P | P) | (Q | Q)$, ceea ce face ca $\{| \}$ sa fie o multime adecvata. Conectorul are tabela de adevar:

P	Q	$P Q$
a	a	f
a	f	a
f	a	a
f	f	a

Semnificatia anumitor conectori a fost mult discutata in logica. De exemplu, disjunctia introduce ambiguitatea. Exista doua tipuri de disjunctie, disjunctia simpla si disjunctia exclusiva, notata de obicei cu $\oplus$. Urmatoarele formule definesc disjunctia exclusiva:

$$(P \oplus Q) = (P \wedge \sim Q) \vee (Q \wedge \sim P)$$

$$(P \oplus Q) = (P \vee Q) \wedge (\sim P \vee \sim Q)$$

Replicatia, notata $\xrightarrow{*}$, este folosita pentru a elimina argumentatiile care nu sint de acord cu semnificatia implicatiei logice conform careia falsul implica orice si adevarul este implicat de orice. Tabela de adevar a replicatiei este prezentata mai jos, comparativ cu cea a implicatiei.

P	Q	$P \rightarrow Q$	$P \xrightarrow{*} Q$
a	a	a	a
a	f	f	a
f	a	a	f
f	f	a	a

3.1.5 Reguli de inferenta in logica propozitionala

Regulile de inferenta in logica propozitionala ofera o modalitate de a realiza demonstratii logice sau deductii. Fiind data o multime de formule $P = \{P_1, P_2, \dots, P_n\}$, problema este de a demonstra adevarul unei formule F , numita *concluzie* sau *teorema*, pe baza formulelor din P , numite *axiome* si utilizind regulile de inferenta.

Metodele sintactice de demonstrare a teoremelor, care nu se bazeaza pe atribuirea de valori de adevar atomilor din formula, utilizeaza reguli de inferenta sintactice. Aceste reguli de inferenta, logic valide, permit obtinerea de noi formule din multimea de formule initiale numai pe baza unor operatii sintactice. Cele mai importante reguli de inferenta (deductie) in logica propozitionala sint:

$$(1) \text{ Modus Ponens } \frac{\begin{array}{c} P \\ P \rightarrow Q \end{array}}{Q}$$

(2) *Substitutia*. Daca P este o formula valida, formula P' obtinuta din P prin substitutia consistenta a atomilor din P este de asemenea valida.

Exista doua tipuri de substitutie: *substitutia uniforma*, in care o variabila se inlocuieste peste tot cu aceeasi formula (echivalenta cu ea sau nu) si *substitutia prin echivalenta*, in care se poate inlocui fiecare aparitie a unei variabile cu o alta formula, dar aceste formule trebuie sa fie echivalente cu variabila substituita.

Exemplu. Formula $P \vee \sim P$ este valida; atunci formula $Q \vee \sim Q$ este de asemenea valida prin aplicare regulii de substitutie uniforma.

$$(3) \text{ Regula inlantuirii } \frac{\begin{array}{c} P \rightarrow Q \\ Q \rightarrow R \end{array}}{P \rightarrow R}$$

$$(4) \text{ Regula conjunctiei } \frac{\begin{array}{c} P \\ Q \end{array}}{P \wedge Q}$$

$$(5) \text{ Regula transpozitiei } \frac{P \rightarrow Q}{\sim Q \rightarrow \sim P}$$

Observatie. Aceste reguli de inferenta pastreaza caracterul de tautologie al formulelor obtinute prin aplicarea lor, daca formulele de plecare sint tautologii.

3.2 Logica cu predicate de ordinul I

Modelul de reprezentare a cunostintelor in programele de inteligenta artificiala trebuie sa posede un grad mare de flexibilitate pentru a putea reprezenta adecvat domeniul discursului. Logica propozitionala nu are aceasta proprietate deoarece nu permite exprimarea proprietatilor obiectelor si a relatiilor existente intre obiecte, si nici generalizarea enunturilor la clase de obiecte cu caracteristici similare.

Logica cu predicate de ordinul I a fost dezvoltata tocmai pentru a da posibilitatea exprimarii rationamentelor despre obiecte complexe sau clase de obiecte si despre relatiile existente intre ele. Aceasta generalizare se face pe baza introducerii predicatelor in locul propozitiilor, a utilizarii functiilor, a variabilelor si a cuantificatorilor de variabile. Aceste concepte sint riguros definite in continuare.

3.2.1 Sintaxa logicii cu predicate de ordinul I

Alfabetul logicii cu predicate de ordinul I contine simboluri pentru reprezentarea constantelor, notate prin conventie cu litere mici de la inceputul alfabetului ($a, b, c, \dots$), variabilelor, notate prin conventie cu litere mici de la sfirsitul alfabetului ($x, y, z, \dots$), functiilor, notate cu $f, g, \dots$, predicatelor, notate cu $P, Q, R, \dots$, a conectorilor si a cuantificatorilor logici. Conectorii logici folositi in logica cu predicate de ordinul I sint: $\sim, \wedge, \vee, \rightarrow$ si $\leftrightarrow$, iar cuantificatorii sint cuantificatorul existential ($\exists$) si cuantificatorul universal ($\forall$).

In cazul logicii cu predicate de ordinul I, predicatele sint functii logice de mai multe argumente, argumentele predicatelor numindu-se termeni.

Definitie. Fie D un domeniu de valori. Un *termen* se defineste astfel:

- (1) O constanta este un termen cu valoare fixa apartinind domeniului D .
- (2) O variabila este un termen ce poate primi valori diferite din domeniul D .
- (3) Daca f este o functie de n argumente ($f: D^n \rightarrow D$) si $t_1, \dots, t_n$ sint termeni, atunci $f(t_1, \dots, t_n)$ este termen.
- (4) Toti termenii sint generati prin aplicarea regulilor (1)-(3).

Definitie. Se numeste *predicat de aritate n* o functie P de n argumente cu valori adevarat sau fals, $P: D^n \rightarrow \{a, f\}$. Un predikat de aritate 0 este o propozitie, numita si *predicat constant*.

Definitie. Daca P este un predikat de aritate n si $t_1, \dots, t_n$ sint termeni, atunci $P(t_1, \dots, t_n)$ se numeste *atom* sau *formula atomica*. Nici o alta expresie nu poate fi atom.

Definitie. Se numeste *literal* un atom sau un atom negat.

Definitie. O *formula bine formata* in logica cu predicate de ordinul I se defineste astfel:

- (1) Un atom este o formula bine formata.
- (2) Daca P este o formula bine formata atunci: $\sim P$, $(\exists x)P(x)$, $(\forall x)P(x)$ sint formule bine formate.
- (3) Daca P si Q sint formule bine formate atunci: $P \wedge Q$, $P \vee Q$, $P \rightarrow Q$, $P \leftrightarrow Q$ sint formule bine formate.
- (4) Orice formula bine formata este generata prin aplicarea de un numar finit de ori a regulilor (1)-(3).

O reprezentare intuitiva a modului de construire a cuvintelor, deci a formelor bine formate in limbajul logicii cu predicate de ordinul I, este prezentata in Figura 3.3. In continuare se vor omite parantezele din formule ori de cate ori aceasta nu creeaza ambiguitate. De asemenea, in anumite conditii, se va folosi $\forall x$ si $\exists y$ in loc de $(\forall x)$ si $(\exists y)$.

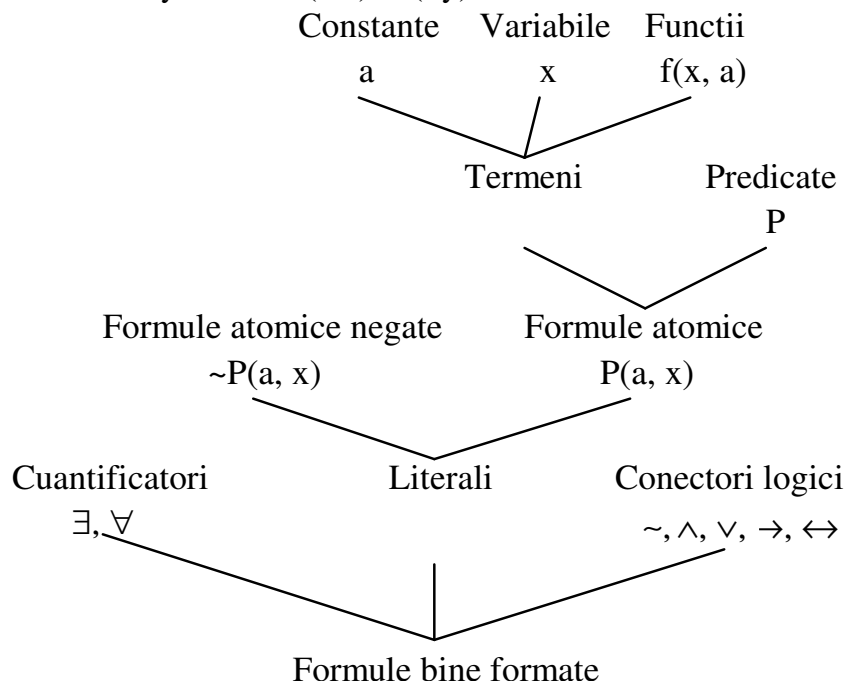


Figura 3.3 Constructia formulelor bine formate in logica cu predicate de ordinul I

Exemple:

1. $(\exists x)(\exists y)(\exists z)(Tata(x, y) \wedge Tata(y, z) \rightarrow Bunic(x, z))$ este o formula bine formata. $Tata(x, y)$, $Tata(y, z)$ si $Bunic(x, z)$ sint literali, in acest caz literali pozitivi deci nenegati. x, y, z sint variabile.
2. $(\forall x)(\exists y)(Egal(y, f(x)) \vee (\forall z)(Egal(z, f(x)) \rightarrow Egal(y, z)))$ este o formula bine formata. x, y si z sint variabile, $f(x)$ functie, toate fiind considerate termeni. $Egal(y, f(x))$ este un literal.
3. $(\exists x)(Vinzator(x) \rightarrow \sim Onest(x))$ este o formula bine formata. $Vinzator(x)$ si $\sim Onest(x)$ sint literali, primul pozitiv si cel de al doilea negativ.
4. $(\forall P)(P(x) \rightarrow Q(x))$ nu este o formula bine formata deoarece cuantificatorii nu pot fi aplicati predicatelor. Acest lucru este posibil numai in logicile de ordin superior (logici de ordinul II).
5. $Om(\sim aristotel)$ nu este o formula bine formata deoarece negatia nu poate fi aplicata unei constante si, in general, nici unui termen.
6. $(\exists x)(\exists y)Casatorit(Barbat(x), Femeie(y))$ nu este o formula bine formata deoarece argumentele predicatelor nu pot fi predicate.

Definitie. O formula bine formata este in *forma normala conjunctiva*, pe scurt FNC, daca formula are forma $F_1 \wedge F_2 \wedge \dots \wedge F_n$, unde $F_i, i = 1, n$ este o formula formata dintr-o disjunctie de literali.

Definitie. O formula bine formata este in *forma normala disjunctiva*, pe scurt FND, daca formula are forma $F_1 \vee F_2 \vee \dots \vee F_n$, unde $F_i, i = 1, n$ este o formula formata dintr-o conjunctie de literali.

De exemplu, $(\sim P \vee Q \vee R) \wedge (\sim P \vee \sim Q)$ este o formula in forma normal conjunctiva, iar $(P \wedge Q \wedge R) \vee (\sim P \wedge \sim Q)$ este o formula in forma normal disjunctiva.

3.2.2 Semantica logicii cu predicate de ordinul I

Similar logicii propozitiilor, semantica unei formule bine formate in logica cu predicate de ordinul I se poate stabili pe baza domeniului de interpretare al formulei. Domeniul de interpretare este multimea tuturor obiectelor din care se selecteaza constantele, variabilele si care stabileste domeniul de definitie si domeniul de valori ale functiilor din formule. Daca nu este exprimat explicit, domeniul de interpretare se deduce din context, putind fi si infinit. Odata stabilit domeniul de interpretare, se poate specifica interpretarea unei formule bine formate, deci se poate afla valoarea ei de adevar in acea interpretare.

Definitie. Interpretarea unei formule F in logica cu predicate de ordinul I consta in fixarea unui domeniu de valori nevid D si a unei asignari de valori pentru fiecare constanta, functie si predicat ce apar in F astfel:

- (1) Fiecarei constante i se asociaza un element din D .
- (2) Fiecarei functii f , de aritate n , i se asociaza o corespondenta $D^n \rightarrow D$, unde $D^n = \{(x_1, \dots, x_n) | x_1 \in D, \dots, x_n \in D\}$.
- (3) Fiecarui predicat de aritate n , i se asociaza o corespondenta $P: D^n \rightarrow \{\mathbf{a}, \mathbf{f}\}$.

Exemplu. Fie urmatoarea formula bine formata $(\forall x)((A(a, x) \vee B(f(x))) \wedge C(x)) \rightarrow D(x)$ cu domeniul de interpretare $D = \{1, 2\}$ si urmatoarea interpretare I :

a	f(1)	f(2)	A(2,1)	A(2,2)	B(1)	B(2)	C(1)	C(2)	D(1)	D(2)
2	2	1	a	f	a	f	a	f	f	a

Pentru a stabili valoarea de adevar a formulei se considera:

- $x = 1$ $((\mathbf{a} \vee \mathbf{f}) \wedge \mathbf{a}) \rightarrow \mathbf{f}$ fals
- $x = 2$ $((\mathbf{f} \vee \mathbf{a}) \wedge \mathbf{f}) \rightarrow \mathbf{a}$ adevarat

In consecinta, deoarece formula nu este adevarata pentru orice x din domeniul de interpretare D , expresia are valoarea de adevar fals.

Odata stabilite sintaxa si semantica logicii cu predicate de ordinul I, aceasta poate fi utilizata pentru a reprezenta cunostinte. In continuare se dau exemple de transformare a unor enunturi din limbaj natural in logica cu predicate de ordinul I.

Fie urmatoarele patru enunturi, dintre care primele trei sint axiome si cel de-al patrulea este teorema de demonstrat, sau concluzia.

- (1) Oricine poate citi este literat.
- (2) Delfinii nu sint literati.
- (3) Anumiti delfini sint inteligenti.
- (4) Exista inteligenti care nu pot citi.

Exprimind cele patru propozitii in logica cu predicate de ordinul I se obtine:

- (A1) $(\forall x)(\text{Citeste}(x) \rightarrow \text{Literat}(x))$
- (A2) $(\forall x)(\text{Delfin}(x) \rightarrow \sim \text{Literat}(x))$
- (A3) $(\exists x)(\text{Delfin}(x) \wedge \text{Inteligent}(x))$
- (A4) $(\exists x)(\text{Inteligent}(x) \wedge \sim \text{Citeste}(x))$

unde cu $Citeste(x)$ s-a notat asertiunea "x poate citi", cu $Delfin(x)$ "x este delfin", cu $Literat(x)$ "x este literat" si cu $Inteligent(x)$ "x este inteligent". Domeniul de interpretare al acestor formule este considerat implicit multimea tuturor fiintelor.

Fie axiomele de baza ale numerelor naturale:

- (1) Pentru fiecare numar natural exista un unic succesori imediat.
- (2) Nu exista nici un numar natural pentru care 0 este succesori imediat.
- (3) Pentru orice numar natural diferit de zero, exista un unic predecesori imediat.

Utilizind functia $s(x)$ pentru a desemna succesori imediat al lui x , functia $p(x)$ pentru predecesori imediat al lui x si predicatul $Egal(x,y)$ pentru a exprima asertiunea "x este egal cu y", se obtine urmatoarea exprimare a axiomelor numerelor naturale in logica cu predicate de ordinul I.

$$(A1) (\forall x)(\exists y)(Egal(y,s(x)) \wedge (\forall z)(Egal(z,s(x)) \rightarrow Egal(y,z)))$$

$$(A2) \sim ((\exists x)(Egal(0,s(x))))$$

$$(A3) (\forall x)(\sim Egal(x,0) \rightarrow ((\exists y)(Egal(y,p(x)) \wedge (\forall z)(Egal(z,p(x)) \rightarrow Egal(y,z))))))$$

Domeniul de interpretare al acestor formule este evident multimea numerelor naturale, iar functiile $s(x)$ si $p(x)$ sint definite in consecinta.

Intr-o formula variabilele pot fi variabile libere sau legate. O variabila este legata intr-o formula daca exista un cuantificator ce o refera. In caz contrar, variabila este libera. O formula care contine variabile libere nu poate fi evaluata. De exemplu, formula $(\forall x)(P(x,y) \rightarrow Q(a,z))$ nu poate fi evaluata deoarece nu se cunoaste cuantificarea lui y si nici cea a lui z . Variabila x este legata, iar variabilele y si z sint libere.

3.2.3 Proprietatile formulelor bine formate

Proprietatile formulelor bine formate in logica cu predicate de ordinul I sint aceleasi ca si in logica propozitiilor: validitate, inconsistentta, realizabilitate, echivalenta formulelor, consecinta logica. Definititiile sint similare dar, de aceasta data, trebuie sa se considere interpretarile pentru toate domeniile de interpretare posibile ale formulelor. De exemplu, formula:

$$(\forall x)((A(a,x) \vee B(f(x))) \wedge C(x)) \rightarrow D(x)$$

din sectiunea anterioara nu este o tautologie, deci nu este valida, deoarece exista cel putin o interpretare pentru care ea este falsa. Formula $(\forall x)(P(x) \wedge \sim P(x))$ este inconsistentta sau contradictie deoarece nu exista nici o interpretare pentru care aceasta formula sa fie adevarata. Formula $(\forall x)(P(x) \vee \sim P(x))$ este insa valida, deoarece ea este adevarata indiferent de interpretare.

Fie urmatoarele doua formule:

$$(A1) Bun(roco)$$

$$(A2) (\forall x)(Bun(x) \rightarrow Iubit(x))$$

Se poate arata ca formula $Iubit(roco)$, unde $roco$ este o constanta, este o consecinta logica a formulelor (A1) si (A2). Sa presupunem ca atat (A1) cit si (A2) sint adevarate intr-o interpretare I. Atunci formula $(Bun(roco) \rightarrow Iubit(roco))$ este adevarata deoarece (A2) specifica pentru "orice x" din domeniu. Dar se stie ca $Bun(roco)$ este adevarata in interpretarea I pe baza lui (A1), deci rezulta ca $Iubit(roco)$ este adevarata deoarece adevarat nu poate implica fals. S-a demonstrat ca

pentru orice interpretare in care (A1) si (A2) sint formule adevarate si formula Iubit(roco) este adevarata, deci Iubit(roco) este o consecinta logica a formulelor (A1) si (A2).

Echivalenta formulelor poate fi stabilita utilizind legile de echivalenta din logica cu predicate de ordinul I prezentate in Figura 3.4. In figura s-au notat cu Q si Q_i , $i = 1, 2$ cuantificatorii universal si existential.

Echivalenta conectorilor logici

Negarea negatiei	$\sim (\sim P) \equiv P$		
Idempotentia	$P \vee P \equiv P$	$P \wedge P \equiv P$	
Asociativitate	$(P \vee Q) \vee R \equiv P \vee (Q \vee R)$	$(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$	
Comutativitate	$P \vee Q \equiv Q \vee P$	$P \wedge Q \equiv Q \wedge P$	$P \leftrightarrow Q \equiv Q \leftrightarrow P$
Distributivitate	$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$	$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$	
De Morgan	$\sim (P \vee Q) \equiv \sim P \wedge \sim Q$	$\sim (P \wedge Q) \equiv \sim P \vee \sim Q$	
Eliminarea implicatiei	$P \rightarrow Q \equiv \sim P \vee Q$		
Eliminarea implicatiei duble	$P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$		

Figura 3.4 Legi de echivalenta a formulelor in logica cu predicate de ordinul I
Echivalenta cuantificatorilor

$(Qx)F[x] \vee G \equiv (Qx)(F[x] \vee G)$	$(Qx)F[x] \wedge G \equiv (Qx)(F[x] \wedge G)$
$\sim ((\forall x)F[x]) \equiv (\exists x)(\sim F[x])$	$\sim ((\exists x)F[x]) \equiv (\forall x)(\sim F[x])$
$(\forall x)F[x] \wedge (\forall x)H[x] \equiv (\forall x)(F[x] \wedge H[x])$	$(\exists x)F[x] \vee (\exists x)H[x] \equiv (\exists x)(F[x] \vee H[x])$
$(Q_1x)F[x] \wedge (Q_2x)H[x] \equiv (Q_1x)(Q_2z)(F[x] \wedge H[z])$	$(Q_1x)F[x] \vee (Q_2x)H[x] \equiv (Q_1x)(Q_2z)(F[x] \vee H[z])$

Figura 3.4 (continuare) Legi de echivalenta a formulelor in logica cu predicate de ordinul I

In toate exemplele mentionate mai sus s-au folosit metode semantice pentru stabilirea caracterului formulelor. In sectiunea urmatoare se vor discuta metode semantice pentru realizarea deductiilor si stabilirea caracterului unei multimi de formule. Dupa cum se observa din exemplele de mai sus, inspectarea tuturor interpretarilor unei formule peste toate domeniile posibile poate fi deosebit de dificila, daca nu imposibila in anumite cazuri, deci o astfel de abordare este greu de automatizat.

3.2.4 Reguli de inferenta in logica cu predicate de ordinul I

Regulile de inferenta sintactice din logica propozitionala prezentate in Sectiunea 3.1.5, Modus Ponens, substitutia, inlantuirea, conjunctia si transpozitia, pot fi generalizate in cazul logicii cu predicate de ordinul I. De exemplu, regula Modus Ponens are urmatoarea forma:

$$\text{Modus Ponens} \quad \frac{P(a) \quad (\forall x)(P(x) \rightarrow Q(x))}{Q(a)}$$

Se observa ca s-a facut substitutia lui a cu x, ceea ce a fost posibil deoarece $P(x) \rightarrow Q(x)$ este adevarata pentru orice interpretare.

Regula substitutiei poate avea forme mai sofisticate in cazul logicii cu predicate de ordinul I. Aceste forme vor fi discutate in sectiunea urmatoare. Tot in aceeasi sectiune se prezinta in detaliu si rezolutia, regula de inferenta sintactica importanta.

Regulile de inferenta prezentate sint reguli deductive, deci valide. Ele pastreaza caracterul de tautologie al formulei. In programele de inteligenta artificiala se folosesc insa si *reguli de inferenta nedeductive*, numite si *invalide* deoarece rezultatele obtinute pe baza acestor reguli nu sint intotdeauna adevarate. Aceste reguli pot fi insa utile in numeroase cazuri, asa cum se va vedea mai tirziu, desi nu garanteaza corectitudinea rezultatului obtinut. In continuare se prezinta citeva astfel de reguli de inferenta invalide, dar utilizate:

- (1) *Inferenta abductiva*. Inferenta abductiva se bazeaza pe utilizarea cunostintelor cauzale pentru a explica sau a justifica o concluzie, posibil invalida. Inferenta abductiva are urmatoarea forma:

$$\frac{Q(a) \quad (\forall x)(P(x) \rightarrow Q(x))}{P(a)}$$

De exemplu, din formulele:

SeLeagana(radu)

$(\forall x)(\text{Beat}(x) \rightarrow \text{SeLeagana}(x))$

se poate infera $\text{Beat}(\text{radu})$ desi s-ar putea ca Radu sa se lege datorita unui cutremur de pamint.

- (2) *Inferenta inductiva*. Inferenta inductiva se bazeaza pe ideea ca o proprietate adevarata pentru o submultime de obiecte dintr-o clasa este adevarata pentru toate exemplele din acea clasa. Inferenta inductiva are forma:

$$\frac{P(a_1), P(a_2), \dots, P(a_n)}{(\forall x)P(x)}$$

De exemplu, dupa ce se constata ca cele mai multe lebede sint albe, se poate infera prin inductie ca toate lebedele sint albe, desi exista si lebede negre, cum ar fi unele dintre cele care cresc in Australia.

- (3) *Inferenta analogica*. Inferenta analogica este o forma de inferenta bazata pe experienta si se bazeaza pe ideea conform careia situatii sau entitati care tind sa fie asemanatoare sub anumite aspecte sint asemanatoare in general. Inferenta analogica este de fapt o combinatie a celorlalte forme de inferenta: abductive, deductive si inductive. Inferenta analogica are forma:

$$\frac{P(x) \xrightarrow{r} Q(x)}{P'(x) \xrightarrow{r} Q'(x)}$$

Toate aceste trei forme de inferenta sint invalide, dar reprezinta modalitati de simulare a rationamentului de bun simt. Ele sint folosite in inteligenta artificiala, mai ales in cazul programelor de invatare automata, asa cum se va vedea in Capitolul 9.

3.2.5 Rezolvarea problemelor in cadrul formalismului logic

Pentru a putea investiga cum se rezolva problemele in logica cu predicate de ordinul I si puterea expresiva a acestui model de reprezentare a cunostintelor, se considera urmatorul exemplu. Fie multimea de enunturi:

- (1) Marcus era om.
- (2) Marcus era pompeian.
- (3) Toti pompeenii erau romani.
- (4) Cezar era dictator.
- (5) Toti romanii fie erau devotati lui Cezar, fie il urau.
- (6) Fiecare om este devotat cuiva.
- (7) Oamenii incearca sa asasineze dictatorii fata de care nu sint devotati.
- (8) Marcus a incercat sa-l asasineze pe Cezar.

Faptele descrise de aceste propozitii pot fi reprezentate sub forma de formule bine formate in calculul cu predicate de ordinul I astfel:

(1) *Marcus era om* se exprima sub forma:

(A1) $\text{Om}(\text{marcus})$

Aceasta reprezentare surprinde elementul esential al propozitiei, si anume faptul ca Marcus era om, dar nu exprima informatia continuta in limbaj natural despre timpul trecut utilizat. Pentru exemplul considerat aceasta informatie este nerelevanta, dar in alte cazuri ea ar putea sa conteze, deci ar trebui extinsa reprezentarea in consecinta.

(2) *Marcus era pompeian* se exprima sub forma:

(A2) $\text{Pompeian}(\text{marcus})$

(3) *Toti pompeenii erau romani* se exprima sub forma:

(A3) $(\forall x)(\text{Pompeian}(x) \rightarrow \text{Roman}(x))$

Conform conventiei facute x este o variabila, in timp ce simbolul marcus utilizat in (A1) si (A2) este o constanta.

(4) *Cezar era dictator* se exprima sub forma:

(A4) $\text{Dictator}(\text{cezar})$

Tot conform conventiei facute, simbolul cez ar este considerat o constanta. In plus, se face presupunerea implicita ca exista un unic individ care se numeste Cez ar in universul problemei de rezolvat.

(5) *Toti romanii erau fie devotati lui Cez ar fie il urau*. Pentru a exprima aceasta propozitie, tinind cont de semantica ei, nu se poate folosi conectorul logic $\vee$ care are semnificatia de "sau inclusiv", ci trebuie folosit un "sau exclusiv". In aceste conditii propozitia se exprima sub forma:

(A5) $(\forall x)(\text{Roman}(x) \rightarrow ((\text{Devotat}(x, \text{cezar}) \vee \text{Ura}(x, \text{cezar})) \wedge \sim (\text{Devotat}(x, \text{cezar}) \wedge \text{Ura}(x, \text{cezar}))))$

(6) *Fiecare om este devotat cuiva* se exprima sub forma:

$$(A6) (\forall x)(\exists y)\text{Devotat}(x,y)$$

Aici apare o noua problema de traducere a limbajului natural in forma logica, si anume ordinea cuantificatorilor. Utilizind formula logica de mai sus s-a presupus ca pentru orice persoana x, exista o persoana y fata de care x este devotata. Dar aceeasi fraza ar fi putut fi interpretata, eventual, si ca exista o persoana y fata de care toate celelalte persoane sint devotate, ceea ce s-ar fi exprimat in logica sub forma:

$$(\exists y)(\forall x)\text{Devotat}(x,y)$$

(7) *Oamenii incearca sa asasineze dictatorii fata de care nu sint devotati* poate fi exprimata sub forma:

$$(A7) (\forall x)(\forall y)((\text{Persoana}(x) \wedge \text{Dictator}(y) \wedge \text{IncearcaAsasinat}(x,y)) \rightarrow \sim \text{Devotat}(x,y))$$

Evident ca si in acest caz ar fi putut sa existe o exprimare logica diferita. Cititorul poate incerca sa o gaseasca.

(8) *Marcus a incercat sa-l asasineze pe Cezar* se exprima sub forma:

$$(A8) \text{IncearcaAsasinat}(\text{marcus}, \text{cezar})$$

Presupunind ca faptele (1)÷(8) sint adevarate, deci sint axiome, cum se poate stabili daca Marcus nu era devotat lui Cezar, deci cum se poate demonstra teorema (concluzia):

$$(c) \sim \text{Devotat}(\text{marcus}, \text{cezar})?$$

Aceasta demonstratie se poate face aplicind regulile de inferenta ale logicii cu predicate de ordinul I prezentate in sectiunea anterioara. Inspectind axiomele (A1)÷(A8), se observa ca demonstratia s-ar putea face numai pe baza axiomelor (A1), (A4), (A7) si (A8). Dar apare urmatoarea problema: desi oricine stie ca daca Marcus este om el este in acelasi timp persoana, acest lucru nu este explicit indicat in enunt. Se observa de aici una din dificultatile fundamentale in rezolvarea problemelor de inteligenta artificiala si anume reprezentarea cunostintelor de bun simt. Pentru a putea rezolva problema, trebuie adaugat un nou enunt care sa statueze ca toti oamenii sint persoane.

(9) *Toti oamenii sint persoane* se exprima sub forma:

$$(A9) (\forall x)(\text{Om}(x) \rightarrow \text{Persoana}(x))$$

In acest moment se poate demonstra concluzia (c) pe baza axiomelor (A1), (A4), (A7), (A8) si (A9). Aplicind regula substitutiei in axioma (A7) si substituind uniform variabila x cu constanta marcus si variabila y cu constanta cezar se obtine o noua axioma

$$(A10) \text{Persoana}(\text{marcus}) \wedge \text{Dictator}(\text{cezar}) \wedge \text{IncearcaAsasinat}(\text{marcus}, \text{cezar}) \rightarrow \sim \text{Devotat}(\text{marcus}, \text{cezar})$$

Aplicind regula Modus Ponens asupra axiomelor (A1) si (A9) se obtine:

$$(A11) \text{Persoana}(\text{marcus})$$

si aplicind din nou regula Modus Ponens asupra axiomelor (A11), (A4), (A8) si (A10) se obtine concluzia cautata:

$$\sim \text{Devotat}(\text{marcus}, \text{cezar})$$

Se observa ca aceasta demonstratie a fost facuta prin selectia intuitiva a axiomelor ce trebuie combinate si a diverselor reguli de inferenta ce trebuie utilizate. O asemenea abordare este evident nepractica intr-un program de demonstrare automata a teoremelor. Pentru a putea automatiza procesul de demonstrare este preferabil sa existe o singura regula de inferenta. Aceasta este

rezolutia. In plus, trebuie stabilita o strategie de aplicare a regulii de inferenta pentru a ajunge cit mai repede la solutie, daca exista solutie. Aceste probleme vor fi abordate in sectiunea urmatoare.

Un alt aspect important ce trebuie considerat in momentul in care se discuta rezolvarea problemelor in cadrul formalismului logic este posibilitatea existentei unei solutii. Este orice teorema demonstrabila? In cazul logicii propozitionale exista intotdeauna proceduri efective care permit atit stabilirea faptului ca o formula este teorema, cit si a faptului ca nu este teorema. In consecinta problema demonstrarii teoremelor in logica propozitionala este *decidabila*. In cazul logicii cu predicate de ordinul I se garanteaza existenta unei proceduri care sa demonstreze ca o formula este teorema daca acea formula este intr-adevar teorema. Dar aceasta procedura nu este garantata sa se opreasca daca formula de demonstrat nu este teorema. In consecinta demonstrarea teoremelor in logica cu predicate de ordinul I nu este decidabila, ci este o problema *semidecidabila*.

In pofida acestui rezultat trist, formalismul logic este utilizat intens ca metoda de rezolvare a problemelor in inteligenta artificiala deoarece, in cele mai multe cazuri, produce rezultatele dorite. De asemenea trebuie tinut cont de faptul ca utilizarea unei strategii particulare sau a unei combinatii de strategii in demonstrarea teoremelor poate genera situatii in care o teorema nu se poate demonstra chiar daca acea formula este intr-adevar teorema. Acesta este cazul *strategiilor incomplete*. Si in aceasta situatie s-au facut compromisuri, in sensul ca s-au acceptat strategii incomplete de demonstrare a teoremelor datorita avantajului de eficienta pe care il aduc. O *strategie completa* va reusi intotdeauna sa demonstreze ca o formula este teorema daca formula este cu adevarat teorema, dar aceasta completitudine poate fi penalizata de necesitati crescute ale resurselor de timp si spatiu utilizate de programul care o implementeaza.

3.3 Demonstrarea teoremelor utilizind rezolutia

In 1965, Robinson propune principiul rezolutiei ca metoda eficienta de demonstrare a teoremelor, principiu care reprezinta baza tuturor demonstratoarelor automate de teoreme actuale. Rezolutia este o metoda de inferenta sintactica care, aplicata repetat unei multimi de formule in forma standard, determina daca multimea de formule este inconsistentă. Pentru a demonstra ca formula C este o consecinta logica a formulelor $P_1, P_2, \dots, P_n$, se demonstreaza ca $P_1 \wedge P_2 \wedge \dots \wedge P_n \wedge \sim C$ este o formula nerealizabila prin deducerea unei contradictii.

Principiul rezolutiei este o metoda de demonstrare prin respingere, care corespunde in general unei demonstrari prin reducere la absurd. De aceea, utilizarea principiului rezolutiei in demonstrarea teoremelor se mai numeste si *metoda respingerii prin rezolutie* sau *respingere rezolutiva*. Metoda rezolutiei se aplica insa unei forme standard a formulelor, numita forma clauzala, forma introdusa de Davis si Putnam.

3.3.1 Transformarea formulelor in forma clauzala

Definitie. Se numeste *clauza* o disjunctie de literali. Se numeste *clauza de baza* o clauza fara variabile. Se numeste *clauza Horn* o clauza care contine cel mult un literal pozitiv.

Definitie. Se numeste *clauza vida* o clauza fara nici un literal; clauza vida se noteaza, prin conventie, cu $\square$. Se numeste *clauza unitara* o clauza ce contine un singur literal.

O *clauza Horn* poate avea una din urmatoarele patru forme: o clauza unitara pozitiva ce consta intr-un singur literal pozitiv; o clauza negativa formata numai din literalii negati; o clauza formata dintr-un literal pozitiv si cel putin un literal negativ (*clauza Horn mixta*) sau clauza vida. Se numeste *clauza (Horn) distincta* o clauza ce are exact un literal pozitiv, ea fiind fie o clauza unitara pozitiva, fie o clauza Horn mixta.

Exemple:

1. $P(x,y) \vee \sim Q(x,f(y)) \vee R(z)$ este o clauza. Intr-o clauza toate variabilele sunt implicit cuantificate universal.
2. $\sim P(a,y) \vee \sim Q(x,z) \vee P(x,y)$ este o clauza Horn, in particular o clauza Horn distincta.
3. $P(a,b) \vee Q(c,d) \vee R(a)$ este o clauza de baza deoarece nu contine variabile.

Transformarea unei formule bine formate in forma clauzala se face pe baza regulilor prezentate in continuare.

Pasul 1. Se elimina toti conectorii logici de implicatie si echivalenta folosind legile de eliminare a implicatiei si a implicatiei duble prezentate in Figura 3.4.

Pasul 2. Se muta toate negatiile din formula astfel incit sa preceada atomii folosind legea negarii negatiei, legile lui De Morgan si legile de echivalenta a cuantificatorilor prezentate in Figura 3.4.

Exemplu. Formula $\sim((\forall x)P(x) \rightarrow (\exists y)Q(y))$ se transforma in $\sim(\sim((\forall x)P(x)) \vee (\exists y)Q(y))$ apoi in $(\forall x)P(x) \wedge \sim((\exists y)Q(y))$ din care se obtine in final formula $(\forall x)P(x) \wedge (\forall y) \sim Q(y)$.

Pasul 3. Se redenumesc variabilele, daca este cazul, astfel incit toti cuantificatorii sa se refere la variabile diferite, i.e. se redenumesc variabilele astfel incit variabilele referite de un cuantificator sa nu aiba acelasi nume cu variabilele referite de alt cuantificator.

Exemplu. In formula $(\forall x)(P(x) \rightarrow (\exists x)Q(x))$ se redenumeste cea de a doua variabila x referita de cuantificatorul existential $(\exists x)$ si se obtine formula $(\forall x)(P(x) \rightarrow (\exists y)Q(y))$.

Pasul 4. Se elimina toti cuantificatorii existentiali din formula printr-un proces de substitutie numit *skolemizare*. Acest proces necesita ca toate variabilele definite de un cuantificator existential sa fie inlocuite prin functii Skolem, adica functii arbitrare care pot lua intotdeauna valoarea ceruta de cuantificatorul existential. Skolemizarea se executa dupa urmatoarele reguli:

4.1. Daca primul (cel mai din stinga) cuantificator este un cuantificator existential, se inlocuiesc toate aparitiile variabilei pe care o cuantifica cu o constanta arbitrara care nu apare nicaieri in expresie si se elimina cuantificatorul. Acest proces se aplica pentru toti cuantificatorii existentiali care nu sunt precedati de cuantificatori universali, folosind constante diferite in substitutie.

4.2. Pentru fiecare cuantificator existential care este precedat de unul sau mai multi cuantificatori universali, se inlocuiesc toate aparitiile variabilei cuantificate printr-o functie care nu mai apare in expresie si care are ca argumente toate variabilele cuantificate universal ce preced cuantificatorul existential. Cuantificatorul existential se elimina. Procesul se repeta pentru fiecare cuantificator existential folosind un simbol de functie diferit si alegind ca variabile ale functiei argumentele care corespund tuturor variabilelor cuantificate universal ce preced cuantificatorul existential.

Exemplu. Expresia $(\exists u)(\forall v)(\forall x)(\exists y)(P(f(u), v, x, y) \vee Q(u, v, y))$ se transforma prin substitutii de skolemizare in $(\forall v)(\forall x)(P(f(a), v, x, g(v, x)) \vee Q(a, v, g(v, x)))$. Inlocuirea variabilei y cu o functie arbitrara de argumente v si x se justifica pe baza faptului ca variabila y, urmind dupa variabilele v si x, poate fi dependenta functional de acestea iar in acest caz, functia arbitrara g poate reproduce aceasta dependenta.

Pasul 5. Se muta toti cuantificatorii univiersali la stinga expresiei si se transforma expresia in forma normal conjunctiva.

Pasul 6. Se elimina toti cuantificatorii universali deoarece ei sint retinuti implicit in forma clauzala si se elimina conjunctiile din forma normal conjunctiva. In acest fel se obtine o multime de formule numite clauze.

Observatie. Multimea de clauze obtinute prin procesul de mai sus nu este echivalenta cu formula originala dar realizabilitatea formulei este pastrata. Multimea de clauze este realizabila respectiv inconsistenta, daca si numai daca formula originala este realizabila, respectiv inconsistenta.

Exemplu. Se considera urmatoarea formula

$$(\exists x)(\forall y)((\forall z)P(f(x), y, z) \rightarrow ((\exists u)Q(x, u) \wedge (\exists v)R(y, v)))$$

Pentru a transforma aceasta formula in forma clauzala se aplica procedeul descris anterior. Prin executia procedurii pas cu pas se obtine

Pasul 1. Se elimina conectorul de implicatie logica si se obtine

$$(\exists x)(\forall y)(\sim (\forall z)P(f(x), y, z) \vee ((\exists u)Q(x, u) \wedge (\exists v)R(y, v)))$$

Pasul 2. Se aduc negatiile in fata atomilor si se obtine

$$(\exists x)(\forall y)((\exists z) \sim P(f(x), y, z) \vee ((\exists u)Q(x, u) \wedge (\exists v)R(y, v)))$$

Pasul 3. Acest pas nu este necesar deoarece toate variabilele cuantificate au nume distincte.

Pasul 4. Se aplica skolemizarea si se elimina astfel cuantificatorii existentiali prin introducerea functiilor g(y), h(y) si l(y) si a constantei a. Se obtine

$$(\forall y)(\sim P(f(a), y, g(y)) \vee (Q(a, h(y)) \wedge R(y, l(y))))$$

Pasul 5. Se transforma formula in forma normal conjunctiva si se obtine

$$(\forall y)((\sim P(f(a), y, g(y)) \vee Q(a, h(y))) \wedge (\sim P(f(a), y, g(y)) \vee R(y, l(y))))$$

Pasul 6. Se elimina cuantificatorul universal si conjunctia, obtinindu-se multimea de doua clauze

$$\sim P(f(a), y, g(y)) \vee Q(a, h(y))$$

$$\sim P(f(a), y, g(y)) \vee R(y, l(y))$$

Aceasta multime de clauze reprezinta transformarea formulei initiale in forma clauzala.

3.3.2 Rezolutia in logica propozitionala

Pentru a explica principiul rezolutiei, in aceasta sectiune se prezinta rezolutia pentru cazul particular al demonstrarii teoremelor in logica propozitionala. Principiul rezolutiei in logica propozitionala este urmatorul. Pentru orice doua clauze C_1 si C_2 , daca exista un literal L_1 in C_1 care este complementar cu un literal L_2 in C_2 ($L_1 = \sim L_2$) atunci disjunctia intre C_1 din care s-a eliminat L_1 si C_2 din care s-a eliminat L_2 este *rezolventul* clauzelor C_1 si C_2 . Se mai spune ca cele doua clauze, C_1 si C_2 , *rezolva*.

Definitie. Fie clauzele:

$$(C_1) P_1 \vee P_2 \vee \dots \vee P_i \vee \dots \vee P_n$$

$$(C_2) \quad Q_1 \vee Q_2 \vee \dots \vee \sim Q_j \vee \dots \vee Q_m$$

cu $P_i = Q_j = L$. *Rezolventul* clauzelor C_1 si C_2 este $C = \text{rez}(C_1, C_2) = (C_1 - \{L\}) \vee (C_2 - \{\sim L\})$
deci

$$(C) \quad P_1 \vee P_2 \vee \dots \vee P_{i-1} \vee P_{i+1} \vee \dots \vee P_n \vee Q_1 \vee \dots \vee Q_{j-1} \vee Q_{j+1} \vee \dots \vee Q_m$$

Teorema. Fiind date doua clauze, C_1 si C_2 , un rezolvent C al clauzelor C_1 si C_2 este o consecinta logica a clauzelor C_1 si C_2 .

Pentru a demonstra ca o formula S este o teorema derivata dintr-un set de axiome A utilizind principiul rezolutiei, se aplica algoritmul prezentat in continuare. Ideea algoritmului este aceea de a porni de la o multime de clauze care se presupune a fi realizabila si a genera noi clauze care reprezinta restrictii asupra modului in care clauzele originale pot fi facute adevarate. Apare o contradictie in momentul in care o clauza devine atat de restrictionata incit nu mai poate fi facuta adevarata. Acest lucru este indicat de generarea clauzei vide.

Algoritm: Respingeria prin rezolutie in logica propozitionala.

1. Converteste setul de axiome A in forma clauzala si obtine multimea de clauze S_0
2. Neaga teorema, transforma teorema negata in forma clauzala si adauga rezultatul la S_0

$$S \leftarrow S_0$$

3. repeta

3.1. Selecteaza o pereche de clauze C_1 si C_2 din S

3.2. Determina $C = \text{rez}(C_1, C_2)$

3.3. **daca** $C \neq \square$

$$\text{atunci } S \leftarrow S \cup \{C\}$$

pina s-a obtinut clauza vida ($\square$) **sau**

nu mai exista nici o pereche de clauze care rezolva

4. **daca** s-a obtinut clauza vida

atunci teorema este adevarata (este demonstrata)

5. **altfel** teorema este falsa

sfirsit.

Se considera urmatoarele enunturi:

- (1) Am timp liber.
- (2) Daca am timp liber si ma plimb atunci cunosc orasul.
- (3) Daca este soare sau este cald atunci ma plimb.
- (4) Este cald.

Se cere sa se demonstreze utilizind metoda respingerii prin rezolutie enuntul:

- (5) Cunosc orasul.

Pentru aceasta se exprima primele patru enunturi si enuntul de demonstrat in logica propozitionala obtinandu-se urmatorul set de axiome

(A1) T

(A2) $T \wedge P \rightarrow O$

(A3) $S \vee C \rightarrow P$

(A4) C

si concluzia de demonstrat

(C) O

Se transforma axiomele in forma clauzala, se neaga teorema (C) si se adauga la multimea de clauze obtinute din axiome. In urma acestui proces se obtine urmatoarea multime de clauze:

(C1) T

(C2) $\sim T \vee \sim P \vee O$

(C3) $\sim S \vee P$

(C3') $\sim C \vee P$

(C4) C

(C5) $\sim O$

Deducerea clauzei vide din setul de axiome, deci demonstrarea prin respingere a teoremei "Cunosc orasul", este prezentata in Figura 3.5.

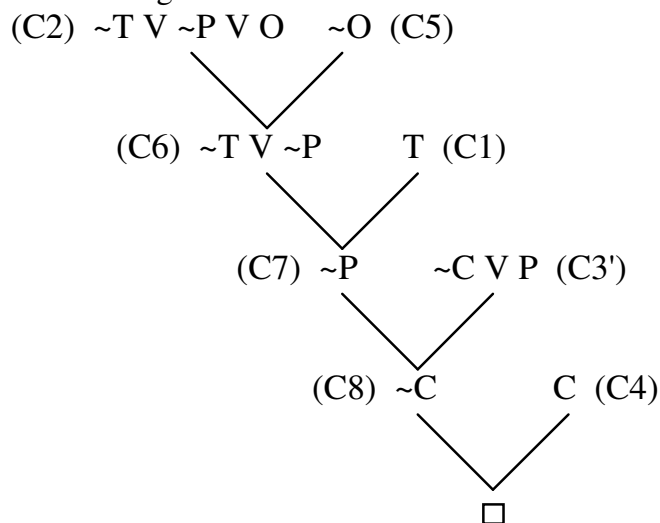


Figura 3.5 Respingerea prin rezolutie in logica propozitionala

3.3.3 Unificarea expresiilor

In logica propozitionala este usor sa se identifice perechile de literali complementari, L si $\sim L$, din doua clauze pentru a aplica rezolutia. In logica cu predicate de ordinul I acest lucru este mai dificil deoarece in procesul de identificare trebuie tinut cont de argumentele predicatelor. De exemplu, literalii $P(x)$ si $P(a)$ pot unifica cu conditia ca sa se aplice o regula de substitutie in primul literal prin care variabila x sa fie inlocuita cu constanta a . Gasirea unei substitutii pentru variabilele din expresii sau subexpresii astfel incit expresiile, respectiv subexpresiile, sa devina identice, se numeste unificare si este un proces esential in demonstrarea teoremelor in general si, in particular, prin metoda rezolutiei.

Definitie. O *substitutie* este o multime de perechi (t_i, v_i) , $i = 1, n$, in care v_i sint variabile distincte si t_i sint termeni care nu contin v_i . Termenii t_i inlocuiesc variabilele v_i , $i = 1, n$ in orice expresie in care se aplica substitutia. O substitutie se noteaza $\alpha = \{t_1 / v_1, t_2 / v_2, \dots, t_n / v_n\}$, $n \geq 1$.

In continuare se vor folosi litere grecesti pentru reprezentarea substitutiilor. Rezultatul aplicarii unei substitutii α asupra unei expresii E este notat $E\alpha$ si este expresia obtinuta prin

inlocuirea tuturor aparitiilor variabilei v_i cu termenul t_i in expresia E , pentru toate perechile t_i/v_i din substitutia α . O expresie poate fi un termen, un literal, un atom sau o formula bine formata sau o multime de termeni, literali, atomi sau formule bine formate.

Example:

1. Se considera expresia $E = P(x, f(y), b)$ si substitutiile $\alpha_1 = \{z/x, w/y\}$, $\alpha_2 = \{a/y\}$, $\alpha_3 = \{g(z)/x, a/y\}$, $\alpha_4 = \{c/x, a/y\}$. Prin aplicarea, pe rind, a acestor substitutii expresiei E se obtine:
 $E\alpha_1 = P(z, f(w), b)$
 $E\alpha_2 = P(x, f(a), b)$
 $E\alpha_3 = P(g(z), f(a), b)$
 $E\alpha_4 = P(c, f(a), b)$
 Se observa ca $E\alpha_4 = P(c, f(a), b)$ este o clauza de baza.
2. Fie expresia $E = P(x, y) \vee Q(x, f(y))$ si aplicind substitutia $\beta = \{a/x, g(b)/y\}$ se obtine expresia $E' = E\beta = P(a, g(b)) \vee Q(a, f(g(b)))$.

Definitie. Se numeste *unificator* al unei multimi de expresii $\{E_1, E_2, \dots, E_n\}$, o substitutie α care face ca expresiile sa devina identice, adica $E_1\alpha = E_2\alpha = \dots = E_n\alpha$. Multimea $\{E_1, E_2, \dots, E_n\}$ se numeste *multime de expresii unificabila*, daca exista un unificator pentru aceasta multime. Se mai spune ca multimea de expresii unifica.

Definitie. Un unificator β al unei multimi de expresii $\{E_1, E_2, \dots, E_n\}$ este *cel mai general unificator*, pe scurt mgu, daca si numai daca pentru orice alt unificator α al multimii exista o substitutie α' astfel incit $E_i\alpha = E_i\beta\alpha'$, $\forall i = 1, n$. Altfel spus, orice unificator α al multimii $\{E_1, E_2, \dots, E_n\}$ este o instanta a lui β .

Observatie. Daca doua expresii unifica, atunci exista un *unic* cel mai general unificator.

Example:

1. Fie expresiile $E_1 = P(x, f(y), b)$ si $E_2 = P(x, f(b), b)$. Cele doua expresii unifica aplicind substitutia $\alpha = \{a/x, b/y\}$, $E_1\alpha = E_2\alpha = P(a, f(b), b)$. Aplicind substitutia $\beta = \{b/y\}$ se obtine $E_1\beta = E_2\beta = P(x, f(b), b)$. Se observa ca $\beta = \{b/y\}$ este cel mai general unificator al celor doua expresii.
2. Fie expresiile $E_1 = Q(x, f(x, y), z)$ si $E_2 = Q(y, z, g(t))$. Cel mai general unificator al celor doua expresii este $\beta = \{x/y', f(x, y)/z', g(t)/z\}$ unde y' si z' sint aparitiile variabilelor y si z in E_2 . Rezultatul unificarii este $E_1\beta = E_2\beta = Q(x, f(x, y), g(t))$.
3. Fie expresiile $E_1 = P(x, f(x), y)$ si $E_2 = P(z, z, t)$. Aceste doua expresii *nu* unifica deoarece o posibila incercare de substitutie de tipul $\{x/z, f(x, y)/z\}$ este ilegala. Daca z este substituit cu x si cu $f(x, y)$, de fapt x este substituit cu $f(x, y)$ ceea ce contrazice definitia substitutiei.

Observatie. Unificarea se poate aplica si literalilor dintr-o aceeaasi clauza. Daca exista un cel mai general unificator astfel incit doi sau mai multi literali dintr-o clauza unifica, clauza care ramine prin eliminarea tuturor literalilor cu exceptia unuia din literalii unificati este numita *factor* al clauzei originale. De exemplu, fie clauza $C = P(x) \vee Q(x, y) \vee P(f(z))$ si cel mai general unificator $\beta = \{f(z)/x\}$. Atunci clauza $C' = C\beta = P(f(z)) \vee Q(f(z), y)$ este un factor al clauzei initiale C .

In continuare se prezinta algoritmul de unificare a literalilor sau a expresiilor in general in logica cu predicate de ordinul I.

Algoritm: Unificarea expresiilor

Unifica(E_1, E_2)

1. **daca** E_1 si E_2 sint constante

atunci

1.1. **daca** $E_1 = E_2$

atunci intoarce { }

1.2. **intoarce** INSUCCES

2. **daca** E_1 este variabila **sau** E_2 este variabila

atunci

2.1. Schimba E_1 cu E_2 astfel incit E_1 sa fie variabila

2.2. **daca** $E_1 = E_2$

atunci intoarce { }

2.3. **daca** E_1 apare in E_2

atunci intoarce INSUCCES

2.4. **intoarce** { E_2 / E_1 }

3. **daca** $E_1 = P(t_{11}, \dots, t_{1n})$ **si** $E_2 = P(t_{21}, \dots, t_{2n})$ **sau**

$E_1 = f(t_{11}, \dots, t_{1n})$ **si** $E_2 = f(t_{21}, \dots, t_{2n})$ /* aceleasi simboluri predicative sau functionale cu aceeasi aritate */

atunci

3.1. $x \leftarrow t_{11}$

3.2. $y \leftarrow t_{21}$

3.3. $Rest_1 \leftarrow t_{12}, \dots, t_{1n}$

3.4. $Rest_2 \leftarrow t_{22}, \dots, t_{2n}$

3.5. $\alpha_1 \leftarrow \text{Unifica}(x, y)$

3.6. **daca** $\alpha_1 = \text{INSUCCES}$

atunci intoarce INSUCCES

3.7. $G_1 \leftarrow$ rezultatul aplicarii substitutiei α_1 asupra lui $Rest_1$

3.8. $G_2 \leftarrow$ rezultatul aplicarii substitutiei α_2 asupra lui $Rest_2$

3.9. $\alpha_2 \leftarrow \text{Unifica}(G_1, G_2)$

3.10. **daca** $\alpha_2 = \text{INSUCCES}$

atunci intoarce INSUCCES

3.11. intoarce concatenare(α_1, α_2)

4. intoarce INSUCCES

sfirsit.

Observatii:

1. Algoritmul intoarce lista de substitutii care formeaza cel mai general unificator al celor doua expresii (literali) E_1 si E_2 . Algoritmul este garantat sa produca cel mai general unificator, daca acesta exista.
2. In cazul in care cele doua expresii nu unifica, algoritmul intoarce valoarea speciala INSUCCES pentru a marca esecul unificarii.
3. Pasul 2.3 verifica daca o expresie care contine o anumita variabila nu este unificata cu acea variabila.

3.3.4 Rezolutia in logica cu predicate de ordinul I

Aplicarea principiului rezolutiei in logica cu predicate de ordinul I implica construirea rezolventului a doi literali complementari, care fie sint identici, fie au fost facuti identici prin aplicarea substitutiei definite de cel mai general unificator al celor doi literali asupra clauzelor ce contin acesti doi literali.

Definitie. Fie clauzele:

$$(C_1) \quad P_1 \vee P_2 \vee \dots \vee P_i \vee \dots \vee P_n$$

$$(C_2) \quad Q_1 \vee Q_2 \vee \dots \vee \sim Q_j \vee \dots \vee Q_m$$

numite clauze parinte si β cel mai general unificator al literalilor P_i si Q_j , cu $P_i\beta = Q_j\beta$. Atunci $C = \text{rez}(C_1, C_2) = (C_1\beta - \{P_i\beta\}) \cup (C_2\beta - \{\sim Q_j\beta\})$ este un *rezolvent binar* al clauzelor C_1 si C_2 .

Observatie. Rezolventul a doua clauze nu este unic. Aplicarea rezolutiei intre doua clauze care rezolva poate genera diversi rezolventi in cazul in care in cele doua clauze exista mai multi literali complementari care, prin unificare, pot fi facuti identici.

Example:

1. Fie clauzele $C_1 = \sim \text{Citeste}(x) \vee \text{Literat}(x)$ si $C_2 = \sim \text{Delfin}(y) \vee \sim \text{Literat}(y)$. Cel mai general unificator al celor doua clauze este $\beta = \{x / y\}$ si rezolventul celor doua clauze este $C = \text{rez}(C_1, C_2) = \sim \text{Citeste}(x) \vee \sim \text{Delfin}(x)$, literalii complementari care au rezolvat fiind $\text{Literat}(x)$ si $\sim \text{Literat}(y)$.
2. Fie clauzele $C_1 = P(x, f(a)) \vee P(x, f(y)) \vee Q(y)$ si $C_2 = \sim P(z, f(a)) \vee \sim Q(z)$. Aceste doua clauze pot rezolva si pot produce diversi rezolventi. Daca se selecteaza $\beta = \{x / z\}$ atunci $C_1\beta = P(x, f(a)) \vee P(x, f(y)) \vee Q(y)$ si $C_2\beta = \sim P(x, f(a)) \vee \sim Q(x)$ si $C = \text{rez}(C_1, C_2) = P(x, f(y)) \vee Q(y) \vee \sim Q(x)$ prin unificarea literalilor $P(x, f(a))$ si $\sim P(z, f(a))$. Aplicind o noua substitutie pentru aceasta clauza, $\beta = \{y / x\}$, se obtine $C' = C\beta = P(y, f(y))$.

Daca se selecteaza cel mai general unificator al literalilor $Q(y)$ si $\sim Q(z)$, $\beta = \{y / z\}$ atunci se obtine un alt rezolvent $C = \text{rez}(C_1, C_2) = P(x, f(a)) \vee P(x, f(y)) \vee \sim P(y, f(a))$.

Daca se selecteaza $\beta = \{a / y, z / x\}$ ca cel mai general unificator al literalilor $P(x, f(y))$ si $\sim P(z, f(a))$ atunci $C = \text{rez}(C_1, C_2) = P(z, f(a)) \vee Q(a) \vee \sim Q(z)$.

Demonstrarea teoremelor aplicind metoda respingerii prin rezolutie poate fi descrisa de algoritmul urmat. Enunturile care descriu problema trebuie exprimate in modelul logic si formeaza multimea de axiome A. Concluzia care trebuie obtinuta, deci rezolvarea problemei, este teorema de demonstrat.

Algoritm: Respingerea prin rezolutie in logica cu predicate de ordinul I

1. Converteste setul de axiome A in forma clauzala si obtine multimea de clauze S_0
2. Neaga teorema de demonstrat, transforma teorema negata in forma clauzala si adauga rezultatul obtinut la S_0

$S \leftarrow S_0$

3. **repetă**

- 3.1. Selecteaza o pereche de clauze C_1, C_2
- 3.2. Fie literalii $L_1 \in C_1$ si $\sim L_2 \in C_2$
- 3.3. Aplica unificarea si calculeaza $\beta = \text{mgu}(L_1, L_2)$
- 3.4. **daca** $\beta \neq \phi$

atunci

- 3.4.1. Determina $C = \text{rez}(C_1\beta, C_2\beta)$

- 3.4.2. **daca** $C \neq \square$

atunci $S \leftarrow S \cup \{C\}$

pina s-a obtinut clauza vida ($\square$) **sau**

nu mai exista nici o pereche de clauze care rezolva **sau**
o cantitate predefinita de efort a fost epuizata

4. **daca** s-a obtinut clauza vida

atunci teorema este adevarata (este demonstrata)

5. **altfel**

- 5.1. **daca** nu mai exista nici o pereche de clauze care rezolva

atunci teorema este falsa

- 5.2. **altfel** nu se poate spune nimic despre adevarul teoremei

sfirsit.

Observatii:

- In cazul in care s-a obtinut clauza vida, metoda respingerii prin rezolutie garanteaza faptul ca teorema este adevarata, deci este demonstrabila pe baza setului de axiome A.
- Reciproc, daca teorema este adevarata, se poate obtine clauza vida dupa un numar finit de executii a pasului 3, cu conditia ca strategia de rezolutie sa fie completa.
- Conditia de oprire a ciclului, "o cantitate predefinita de efort a fost epuizata", absenta in cazul algoritmului respingerii prin rezolutie in calculul cu propozitii, a fost introdusa in acest caz deoarece metoda demonstrarii teoremelor prin respingere rezolutiva este

semidecidabila in logica cu predicate de ordinul I. In cazul in care concluzia T de demonstrat este falsa, deci nu este teorema, este posibil sa se ajunga in situatia in care, daca avem noroc, "nu mai exista nici o pereche de clauze care rezolva". Atunci se poate concluziona ca teorema este falsa. Dar este de asemenea posibil ca pasul 3 sa se execute la infinit daca T nu este teorema. Din acest motiv se introduce o cantitate predefinita de efort (resurse de timp sau spatiu) la epuizarea careia algoritmul se opreste. In acest caz s-ar putea ca teorema sa fie adevarata, dar efortul predefinit impus sa fie prea mic, sau se poate ca T sa nu fie teorema. Rezulta deci ca nu se poate spune nimic despre adevarul teoremei.

Se prezinta in continuare doua exemple de demonstrare a teoremelor utilizind metoda respingerii prin rezolutie. Primul exemplu considera din nou problema delfinilor inteligenti, prezentata in Sectiunea 3.2.2, exprimata prin urmatoarele trei enunturi:

- (1) Oricine poate citi este literat.
- (2) Delfinii nu sint literati.
- (2) Anumiti delfini sint inteligenti.

si cere sa se demonstreze ca

- (4) Exista inteligenti care nu pot citi.

Se exprima setul de propozitii in logica cu predicate si se obtin urmatoarele trei axiome si concluzia de demonstrat:

- (A1) $(\forall x)(\text{Citeste}(x) \rightarrow \text{Literat}(x))$
- (A2) $(\forall y)(\text{Delfin}(y) \rightarrow \sim \text{Literat}(y))$
- (A3) $(\exists z)(\text{Delfin}(z) \wedge \text{Intelligent}(z))$
- (C) $(\exists u)(\text{Intelligent}(u) \wedge \sim \text{Citeste}(u))$

Se transforma axiomele in forma clauzala si se obtine:

- (C1) $\sim \text{Citeste}(x) \vee \text{Literat}(x)$
- (C2) $\sim \text{Delfin}(y) \vee \sim \text{Literat}(y)$
- (C3) $\text{Delfin}(a)$
- (C3') $\text{Intelligent}(a)$

Se neaga teorema, obtinindu-se $\sim ((\exists u)(\text{Intelligent}(u) \wedge \sim \text{Citeste}(u)))$ si se transforma teorma negata in forma clauzala, rezultatul adaugindu-se la multimea de clauze de mai sus.

- (C4) $\sim \text{Intelligent}(u) \vee \text{Citeste}(u)$

Deducerea clauzei vide, deci demonstratia teoremei, este prezentata in Figura 3.6.

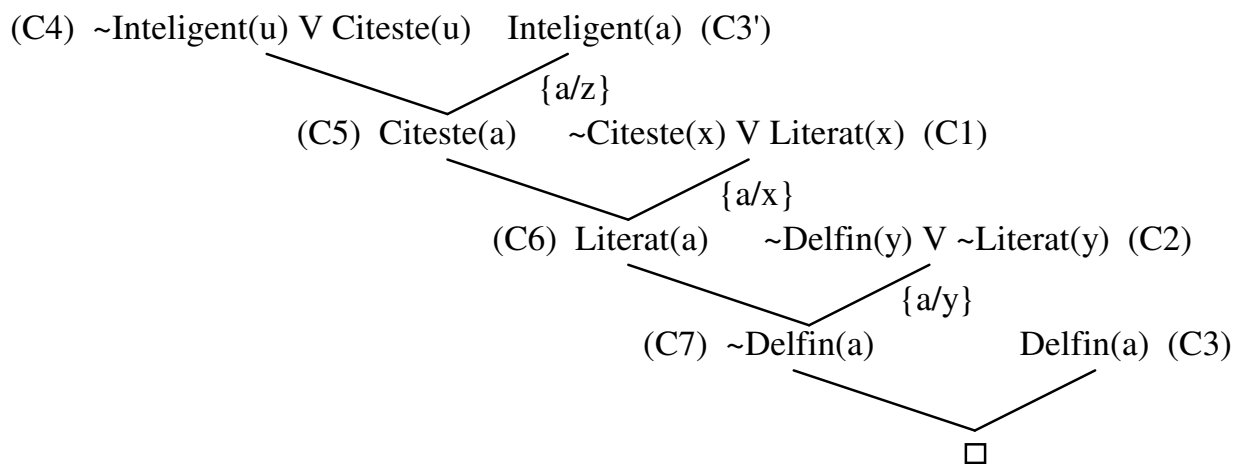


Figura 3.6 Respingerea prin rezolutie in logica cu predicate de ordinul I

Se observa ca o demonstratie prin respingere prin rezolutie poate fi reprezentata convenabil printr-un *arbore de respingere* sau *arbore de deductie* care are ca radacina clauza vida. In acest arbore trebuie marcate clauzele care rezolva si puse in evidenta substitutiile efectuate pentru unificarea literalilor complementari.

Se considera in continuare urmatoarea problema de transport.

- (1) Daca orasul x este legat de orasul y prin drumul z si pot circula biciclete pe drumul z, atunci se poate merge de la x la y.
- (2) Daca orasul x este legat de orasul y prin drumul z, atunci orasul y este legat de orasul x prin drumul z.
- (3) Daca se poate merge de la x la y si de la y la z atunci se poate merge de la x la z.
- (4) Orasul a este legat de orasul b prin drumul d1.
- (5) Orasul b este legat de orasul c prin drumul d2.
- (6) Orasul a este legat de orasul c prin drumul d3.
- (7) Pot circula biciclete pe d1.
- (8) Pot circula biciclete pe d2.

Se cere sa se demonstreze ca se poate merge de la orasul a la orasul c.

Exprimarea in logica cu predicate a problemei date conduce la urmatoarea multime de formule, din care primele opt sint axiomele problemei, ultima fiind concluzia de demonstrat.

- (A1) $(\forall x)(\forall y)(\forall z)(\text{Legat}(x, y, z) \wedge \text{Bic}(z) \rightarrow \text{Merg}(x, y))$
- (A2) $(\forall x)(\forall y)(\forall z)(\text{Legat}(x, y, z) \rightarrow \text{Legat}(y, x, z))$
- (A3) $(\forall x)(\forall y)(\forall z)(\text{Merg}(x, y) \wedge \text{Merg}(y, z) \rightarrow \text{Merg}(x, z))$
- (A4) $\text{Legat}(a, b, d1)$
- (A5) $\text{Legat}(b, c, d2)$
- (A6) $\text{Legat}(a, c, d3)$
- (A7) $\text{Bic}(d1)$
- (A8) $\text{Bic}(d2)$
- (C) $\text{Merg}(a, c)$

Se transforma axiomele in forma clauzala, se neaga teorema si se transforma teorema negata in forma clauzala, obtinindu-se urmatoarea multime de clauze:

- (C1) $\sim \text{Legat}(x, y, z) \vee \sim \text{Bic}(z) \vee \text{Merg}(x, y)$

pentru a rezolva problema este stabilit de strategia de control utilizata. Strategia rezolutiva trebuie sa dea criteriile de selectie a perechilor de clauze care rezolva, in cazul in care exista mai multe astfel de clauze. Eventual, strategia de control poate sa stabileasca si care literalii din cele doua clauze care rezolva sunt selectati pentru a produce rezolventul.

Se reaminteste faptul ca o strategie rezolutiva este completa daca, prin aplicarea ei, se poate demonstra teorema (se produce clauza vida) ori de cate ori formula de demonstrat este teorema.

Cele mai importante strategii rezolutive sunt prezentate in continuare.

- *Strategia dezvoltarii pe latime* sau pe nivel, numita si metoda saturarii nivelului, are la baza urmatoarea idee: se calculeaza toti rezolventii posibili de pe un nivel, acesti rezolventi se adauga la acest nivel pentru a forma nivelul urmator si se reia procesul pentru nivelul urmator. Aceasta strategie este o strategie completa dar prezinta dezavantajul unui consum mare de resurse spatiu si timp.
- *Strategia multimii suport* are la baza urmatoarea idee: se imparte multimea de clauze in doua submultimi de clauze S_1 si S_2 , de preferinta astfel incit, pentru orice interpretare I , clauzele din S_1 sunt adevarate in I , iar clauzele din S_2 sunt false in I . Se aplica rezolutia numai intre perechi de rezolventi din multimi diferite, deci $C_1 \in S_1$ si $C_2 \in S_2$. Aceasta strategie este completa si poate fi asimilata cu o cautare pe nivel in spatiul starilor.
- *Strategia rezolutiei semantice* combina strategia multimii suport cu rezolvarea in simultan a mai multor clauze. In acest fel se incearca eliminarea clauzelor inutile prin rezolvarea simultana a unui grup de clauze, ordonarea predicatelor si ordonarea clauzelor. Strategia rezolutiei semantice este o strategie completa.
- *Strategia rezolutiei liniare* are la baza urmatoarea idee: orice rezolvent C_i obtinut in rezolutie este utilizat ca unul din cei doi rezolventi pe baza carora se obtine urmatorul rezolvent C_{i+1} , $i = 1, 2, \dots, n-1$. Aceasta strategie este completa si, in plus, simplu si eficient de implementat.
- *Strategia rezolutiei de intrare liniara* este un caz particular al strategiei rezolutiei liniare in care una din clauzele care rezolva apartine intotdeauna setului initial de axiome. Este o strategie foarte eficienta dar nu este completa. Aceasta strategie de control sta la baza functionarii mecanismului de demonstrare a teoremelor din limbajul Prolog, asa cum se va explica in detaliu in Capitolul 11.
- *Strategia rezolutiei unitare*, numita si strategia preferintei unitare, este un alt caz particular al strategiei rezolutiei liniare, in care una din clauzele ce rezolva este o clauza unitara, deci o clauza care contine un singur literal. Aceasta strategie nu este completa.

Cele doua exemple de demonstrare a teoremelor prezentate in sectiunea anterioara, i.e. problema delfinilor inteligenti si problema de transport, au folosit o strategie rezolutiva liniara, in particular rezolutia de intrare liniara. In acelasi timp se poate considera ca s-a aplicat si o strategie a multimii suport in care multimea de clauze s-a impartit in multimea S_1 care contine toate clauzele provenite din setul initial de axiome si multimea S_2 care contine clauzele provenite din negarea teoremei de demonstrat.

Aplicind strategia dezvoltarii pe latime in cazul problemei delfinilor inteligenti se vor obtine pentru primele doua nivele rezolventii (distincti) prezentati in Figura 3.8. Fiind o strategie de

cautare pe nivel, strategia dezvoltării pe latime, dacă poate deduce clauza vida, va găsi automat și drumul cel mai scurt spre soluție. De multe ori, în demonstrarea teoremelor interesează mai puțin drumul cel mai scurt spre clauza vida și mai mult numărul de rezolvenți generați. Din această cauză strategii care generează mai puțini rezolvenți, cum ar fi strategiile liniare, sunt preferate.

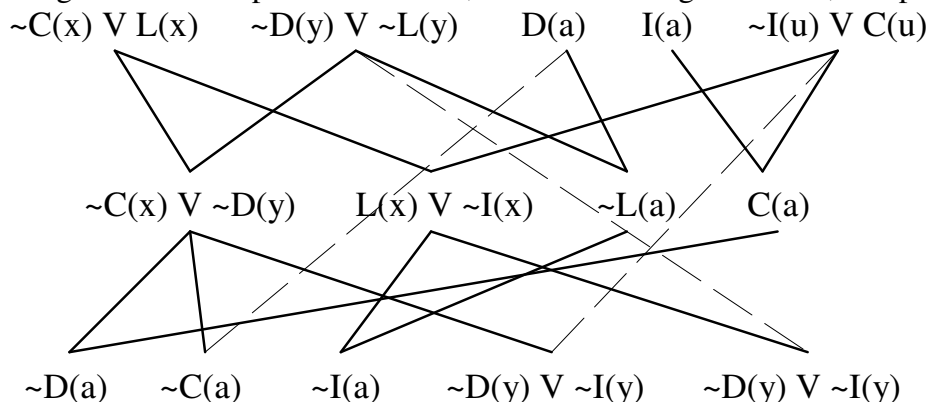


Figura 3.8 Strategia dezvoltării pe latime

Strategiile prezentate oferă criterii de selecție a clauzelor utilizate în producerea rezolventului, deci o modalitate sistematică de generare a rezolvenților, dar de multe ori, nu indică selecția unei perechi unice de clauze dintre clauzele ce pot rezolva la un moment dat. În plus, nu se spune nimic despre perechea de literală complementari care trebuie selectată în cazul în care două clauze pot rezolva în mai multe feluri. Pentru rezolvarea acestor probleme și construirea unui program performant de demonstrare a teoremelor trebuie utilizate și criterii euristice [Chang, Lee, 1973].

3.3.6 Obținerea răspunsurilor utilizând respingerea prin rezoluții

Tehnica demonstrării teoremelor poate fi utilizată și pentru a obține răspunsuri la întrebări despre universul problemei descris de axiome [Nilsson, 1980]. Multe teoreme sunt reprezentate prin formule care conțin variabile cuantificate existențial, de forma $(\exists x)P(x)$. În aceste cazuri este dorit să se poată răspunde la întrebări de tipul "Ce valoare are x dacă formula $(\exists x)P(x)$ este adevărată?" Pentru a putea răspunde la aceste întrebări este nevoie de o metodă de demonstrație constructivă. Această metodă se obține printr-o extindere a metodei respingerii rezoluției.

Se consideră următorul enunț: "Dacă Grivei merge oriunde merge Mihai și Mihai este la școală, unde este Grivei?". Pentru a rezolva această problemă, se exprimă cunoștințele în modelul logic

$$(A1) (\forall x)(\text{EsteLa}(\text{mihai}, x) \rightarrow \text{EsteLa}(\text{grivei}, x))$$

$$(A2) \text{EsteLa}(\text{mihai}, \text{scoala})$$

$$(C) (\exists y)\text{EsteLa}(\text{grivei}, y)$$

Forma clauzala echivalentă este:

$$(C1) \sim \text{EsteLa}(\text{mihai}, x) \vee \text{EsteLa}(\text{grivei}, x)$$

$$(C2) \text{EsteLa}(\text{mihai}, \text{scoala})$$

$$(C3) \sim \text{EsteLa}(\text{grivei}, y)$$

Demonstratia teoremei $(\exists y)\text{EsteLa}(\text{grivei}, y)$ este banală, arborele de deducție fiind prezentat în Figura 3.9(a). Pentru a putea obține însă și o instanță a variabilei y , instanța care va

indica unde este Grivei, se adauga clauzei care a rezultat din negarea teoremei chiar negarea ei, astfel incit aceasta sa devina o tautologie: $\sim \text{EsteLa}(\text{grivei}, y) \vee \text{EsteLa}(\text{grivei}, y)$. Apoi, urmarind structura arborelui de deductie generat anterior, se executa aceleasi rezolutii care s-au executat pentru demonstrarea teoremei, asa cum se prezinta in Figura 3.9(b). Instanta obtinuta in radacina acestui nou arbore, deci formula care inlocuieste clauza vida a arborelui initial, contine raspunsul la intrebare. Deci locul in care se afla Grivei este scoala.

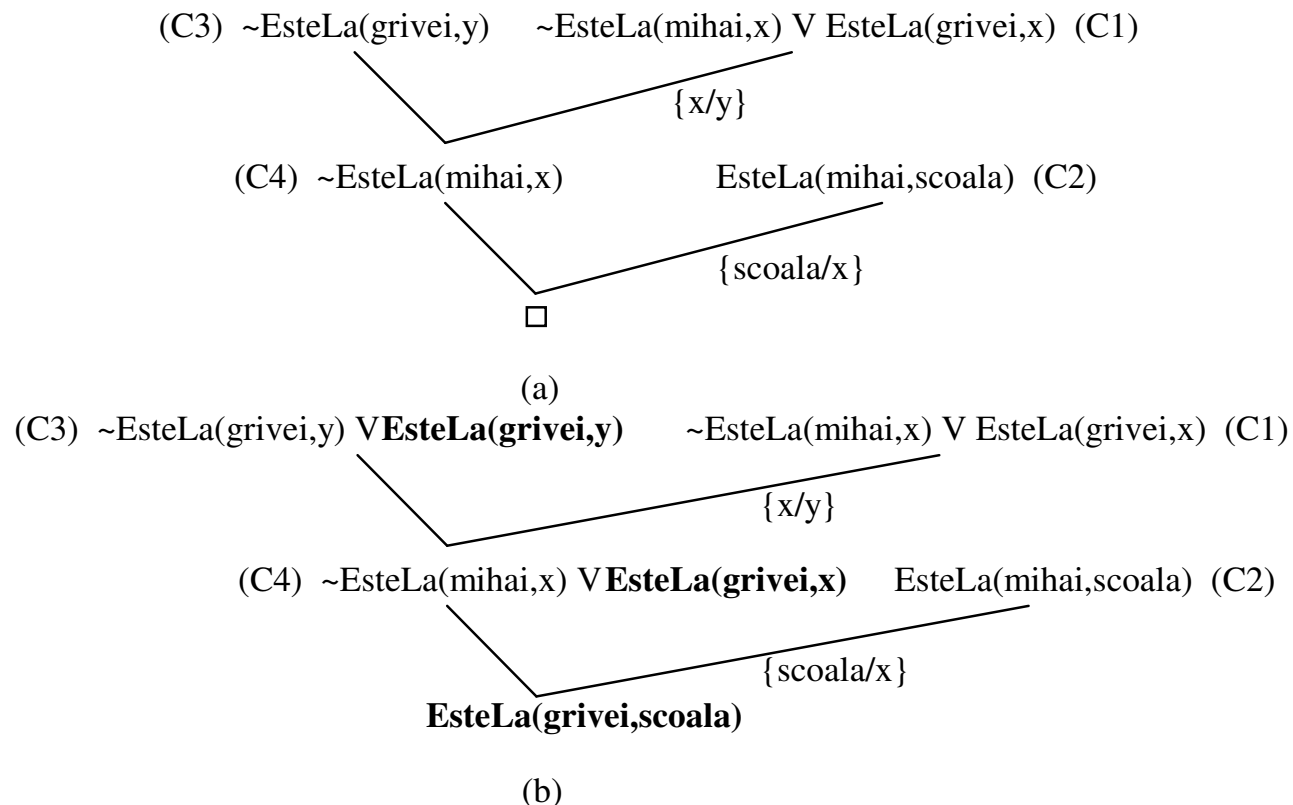


Figura 3.9 Obținerea răspunsurilor la întrebări utilizând rezoluția

Procesul descris implica transformarea fiecărei clauze care apare din negarea teoremei într-o tautologie. Arborele de deductie modificat este deci o demonstratie prin rezoluție a faptului ca formula din radacina se deduce logic din axiome si tautologii, ceea ce inseamna de fapt numai din axiome. Acest lucru justifica faptul ca procesul descris pentru a obtine raspunsuri este corect.

Sintetizand, obtinerea raspunsurilor la intrebări utilizand metoda respingerii prin rezoluție este un proces format din urmatoarii pasi:

- (1) Se construiește arborele de respingere prin rezoluție care demonstrează teorema pe baza multimii initiale de axiome.
- (2) Se substituie functiile Skolem, (daca exista), din clauzele care rezulta din negarea teoremei cu noi variabile.
- (3) Clauzele care rezulta din negarea teoremei sunt transformate in tautologii prin adaugarea literalilor potriviti.
- (4) Se construiește un arbore de deductie modificat cu aceeasi structura cu cea a arborelui initial. Fiecare rezoluție in arborele modificat utilizeaza aceleasi substitutii ca cele folosite in arborele de respingere initial.
- (5) Clauza obtinuta in radacina arborelui de deductie modificat contine raspunsul cautat.

Logica cu predicate de ordinul I se inscrie in domeniul logicii clasice. Studiile de logica si abordarile logice ale problemelor de inteligenta artificiala au investigat insa si alte tipuri de logici, cum ar fi logicile multivalente, in special logicile vagi, logicile modale si temporale si logicile nemonotone, caz particular al logicilor modale. Logicile nemonotone au un rol important in inteligenta artificiala deoarece sint capabile sa elimine o parte din limitarile logicii clasice in domeniul reprezentarii si rationamentului, utilizind cunostinte de bun simt.

3.4 Exerciții și probleme

1. Sa se construiasca tabelele de adevar asociate urmatoarelor formule:

- $P \wedge (P \vee Q)$
- $P \rightarrow (Q \rightarrow P)$
- $\sim P \wedge \sim (P \rightarrow Q)$
- $P \wedge (Q \rightarrow P) \rightarrow P$
- $(P \vee \sim Q) \wedge (\sim P \vee Q)$

Sa se discute caracterul (tautologie, contradictie, realizabila) fiecarei formule.

2. Sa se transforme urmatoarele formule in forma normal conjunctiva:

- $(P \rightarrow Q) \rightarrow R$
- $P \vee (\sim P \wedge Q \wedge R)$
- $(\sim P \wedge Q) \vee (P \wedge \sim Q) \wedge S$

3. Se considera urmatoarea formula $(\exists x)(P(x) \rightarrow (\forall x)P(x))$.

- (a) Sa se demonstreze ca aceasta formula este intotdeauna valida daca domeniul ei de interpretare contine un singur element.
- (b) Fie domeniul $D = \{a, b\}$. Sa se gaseasca o interpretare a formulei peste D , interpretare pentru care formula este falsa.

4. Se considera domeniul $D = \{1, 2\}$ si urmatoarea interpretare:

c	d	f(1)	f(2)	P(1,1)	P(1,2)	P(2,1)	P(2,2)
1	2	2	1	a	a	f	f

Sa se evalueze valorile de adevar ale urmatoarelor formule in interpretarea de mai sus:

- $P(c, f(c)) \wedge P(d, f(d))$
- $(\forall x)(\exists y)P(y, x)$
- $(\forall x)(\forall y)(P(x, y) \rightarrow P(f(x), f(y)))$

5. Sa se transforme in forma clauzala formula:

$$(\forall x)(\sim P(x,a) \rightarrow ((\exists y)(P(y,g(x)) \wedge (\forall z)(P(z,g(x)) \rightarrow P(y,z))))))$$

6. Se considera exemplul asasinului roman din Sectiunea 3.2.5 si se cere:

- (a) Sa se transforme axiomele in forma clauzala.
- (b) Sa se demonstreze utilizind metoda respingerii prin rezolutie teorema $\text{Devotat}(\text{marcus}, \text{cezar})$.
- (c) Sa se demonstreze prin metoda respingerii prin rezolutie teorema $\text{Ura}(\text{marcus}, \text{cezar})$.
- (d) Ce se poate spune despre adevarul teoremei $\sim \text{Devotat}(\text{marcus}, \text{cezar})$.

7. Se considera problema de transport prezentata in Sectiunea 3.3.4 in care se modifica enuntul (8) in urmatorul fel:

(8') Pot circula biciclete fie pe drumul d2 fie pe drumul d3, dar niciodata pe ambele drumuri in acelasi timp.

Se cere:

- (a) Sa se exprime enunturile in logica cu predicate de ordinul I;
- (b) Sa se demonstreze, in noile conditii, utilizind metoda respingerii prin rezolutie, teorema $\text{Merg}(a,c)$.

8. Se stie ca un grup G satisface urmatoarele 4 axiome:

A1. Pentru orice $x, y \in G$, $x \cdot y \in G$

A2. Pentru orice $x, y, z \in G$, $x \cdot (y \cdot z) = (x \cdot y) \cdot z$

A3. $x \cdot e = e \cdot x = x$ pentru orice $x \in G$, unde e este elementul identic al grupului.

A4. Pentru fiecare $x \in G$ exista un element $x^{-1} \in G$ astfel incit $x \cdot x^{-1} = x^{-1} \cdot x = e$

Se cere:

- (a) Sa se exprime axiomele A1÷A4 in logica cu predicate de ordinul I utilizind predicatele $P(x,y,z)$ pentru $x \cdot y = z$ si $I(x)$ pentru x^{-1} .
- (b) Sa se exprime in logica cu predicate teorema:
Daca $x \cdot x = e$ pentru orice $x \in G$ atunci pentru orice $u, v \in G$, $u \cdot v = v \cdot u$, deci G este comutativ.
- (c) Sa se demonstreze teorema pe baza axiomelor date.

9. Se considera urmatorul enunt: "Pentru orice persoane x, y, z , daca x este parintele lui y si y este parintele lui z , atunci x este bunicul lui z ".

- (a) Se cere sa se demonstreze utilizind rezolutia ca "Exista persoane x si y astfel incit x este bunicul lui y ".
- (b) Sa se utilizeze metoda de obtinere a raspunsurilor pe baza respingerii prin rezolutie pentru a afla instantele variabilelor x si y din enuntul de la punctul (a).

Capitolul 4

Modelul regulilor de productie

Modelul regulilor de productie este deosebit de important in inteligenta artificiala deoarece acest model a jucat un rol semnificativ in evolutia sistemelor bazate pe cunostinte, de la stadiul de produse ale laboratoarelor de cercetare la acela al produselor comerciale, cu aplicabilitate directa. Regulile de productie isi au originea in sistemele de productie propuse de E. Post, in 1943, ca mecanism computational general. In modelul regulilor de productie cunostintele sint reprezentate sub forma unor instructiuni conditionale exprimate intr-un limbaj intuitiv, apropiat de cel natural. Desi din punct de vedere formal, regulile de productie provin din modelul computational propus de Post, forma de exprimare a cunostintelor utilizind reguli precede cu mult aparitia calculatoarelor. Jaynes [1976] descrie o colectie de 20 000÷30 000 de tablite babiloniene, dintre care aproximativ 20% contin o multime de reguli de productie, numite *omenuri*, pentru indrumarea activitatii de zi cu zi. Aceste reguli au fost catalogate inca din anul 650 I.C. si aveau o forma asemanatoare celei din sistemele bazate pe reguli din inteligenta artificiala, de exemplu:

"Daca un cal intra in casa unui om si musca acel om, atunci proprietarul casei va muri si casa lui se va prabusi."

"Daca un om calca, din neatenție, o sopirla si o omoara, atunci el va triumfa asupra unui adversar al sau."

Modelul regulilor de productie a fost utilizat in inteligenta artificiala pentru prima oara in sistemele DENDRAL [Buchanan,Feigenbaum,1978;Lindsay,s.a.,1980] si MYCIN [Buchanan, Shortliffe,1984]. Aceste sisteme au fost prezentate pe scurt in Capitolul 1, iar sistemul MYCIN va fi discutat pe larg in Capitolul 7. Sistemele bazate pe reguli de productie au stat la baza dezvoltarii unui numar de sisteme dedicate bazate pe cunostinte, in care sintaxa regulilor era construita astfel incit sa permita exprimarea usoara a cunostintelor unui domeniu particular. Ulterior, din aceste sisteme s-au dezvoltat sisteme bazate pe reguli independente de domeniu, deci sisteme cadru de dezvoltare a sistemelor expert, cum ar fi OPS5 [Cooper,Wogrin,1988], EMYCIN [van Melle,s.a.,1984], sistemele M1 si K1 ale firmei Tecknowledge, sistemul PCPlus al firmei Texas Instruments, si altele.

4.1 Reprezentarea cunostintelor sub forma regulilor de productie

Pina in acest moment, discutia despre modelele de reprezentare a cunostintelor s-a concentrat asupra formalismului logic. Din acest motiv, ca punct de plecare al modelului bazat pe reguli se va folosi logica cu predicate.

In capitolul anterior reprezentarea logica a fost prezentata ca o forma declarativa de reprezentare a cunostintelor. O *reprezentare declarativa* este o reprezentare in care cunostintele sint specificate prin descrierea lor, fara a indica modul in care ele vor fi utilizate. Pentru a utiliza o

reprezentare declarativa, aceasta trebuie completata cu metode care specifica cum vor fi utilizate cunostintele reprezentarii. De exemplu, pentru a rezolva o problema particulara, o multime de asertiuni logice poate fi combinata cu un demonstrator automat de teoreme bazat pe strategia rezolutiei. Exista insa si un alt mod in care asertiunile logice (de o anumita forma) pot fi interpretate, respectiv ca un program si nu ca date ale unui program. In aceasta perspectiva, implicatiile logice definesc cai de rationament, iar formulele atomice reprezinta punctele de oprire ale rationamentului. Aceste cai de rationament definesc posibilele cai de executie ale unui program intr-o maniera apropiata de structura de control traditionala "if-then-else". Cu alte cuvinte, asertiunile logice de o anumita forma pot fi vazute ca o modalitate de reprezentare procedurala. O *reprezentare procedurala* este o reprezentare in care cunostintele de control necesare rezolvării problemei sint inglobate in insasi reprezentarea data. Pentru a utiliza reprezentarea procedurala in rezolvarea problemei, trebuie sa existe un interpretor care urmareste specificatiile de control incluse in reprezentare. Limbajul Prolog, prezentat in Capitolul 11, este un exemplu de astfel de abordare, in care forma particulara a asertiunilor este aceea a clauzelor Horn. Limbajele bazate pe reguli de productie sint un alt exemplu de reprezentare procedurala a cunostintelor, cu o forma particulara a asertiunilor logice care va fi prezentata in continuare.

Multe dintre cunostintele problemelor ce trebuie rezolvate cu ajutorul programelor de inteligenta artificiala sint exprimate sub forma de implicatie. Exprimarea acestor cunostinte in logica cu predicate de ordinul I si transformarea formulelor logice in forma clauzala duce uneori la pierderea unor informatii de control pretioase continute in forma directa a implicatiei. De exemplu, forma clauzala $(A \vee B \vee C)$ este logic echivalenta cu oricare din urmatoarele implicatii: $(\sim A \wedge \sim B) \rightarrow C$, $(\sim A \wedge \sim C) \rightarrow B$, $(\sim B \wedge \sim C) \rightarrow A$, $\sim A \rightarrow (B \vee C)$, $\sim C \rightarrow (A \vee B)$, etc. Fiecare din aceste implicatii, insa, contine si informatie de control extralogica, specifica implicatiei, informatie care nu apare in forma clauzala. De multe ori este preferabil sa se utilizeze in rezolvarea problemei implicatiile in forma lor originala, sub forma regulilor de productie. Utilizarea implicatiilor sub forma de reguli de productie intr-un sistem de rezolvare a problemelor poate creste eficienta sistemului prin eliminarea multiplicarilor introduse de transformarea implicatiilor in forma clauzala.

Intr-un model de reprezentare bazat pe reguli de productie, cunostintele despre problema sint reprezentate prin doua tipuri de entitati: reguli si fapte. *Regulile* sint cunostintele reprezentate de implicatii si exprima cunostinte generale despre domeniul problemei de rezolvat. *Faptele* sint asertiuni unitare si reprezinta cunostintele specifice care descriu un caz particular, i.e. o instanta a problemei de rezolvat.

Regulile de productie sint formate din doua componente: *partea stinga a regulii*, prescurtat din limba engleza LHS, numita si antecedent, premisa, conditie sau situatie, si *partea dreapta a regulii*, prescurtat din limba engleza RHS, numita si consecinta, concluzie, actiune sau raspuns. Legatura logica intre partea stinga si partea dreapta a regulii este implicatia, in sensul ca adevarul partii stingi determina adevarul partii drepte a regulii, deci $LHS \rightarrow RHS$. Pornind de la structura de control decizionala "if-then" comuna limbajelor de programare, cele mai multe limbaje bazate pe reguli folosesc cuvintele cheie **if then** (uneori **when do**), in romaneste **daca atunci**, pentru a marca partea stinga, respectiv partea dreapta a regulii.

Exemple:

- R1: **daca** Coco zboara
si Coco are pene
atunci Coco este pasare.
- R2: **daca** pacientul are temperatura mare
si tipul organismului este gram-pozitiv
si pacientul are gitul uscat
atunci organismul este streptococ
- R3: **daca** masina nu porneste
si farurile nu se aprind
atunci bateria este consumata
sau bornele bateriei nu fac contact
- R4: **daca** temperatura $> 95^{\circ} \text{C}$
atunci deschide valva de protectie

Semnificatia unei reguli de productie este urmatoarea: daca premisa poate fi satisfacuta intr-un context dat, atunci consecinta poate fi satisfacuta in acel context. Daca partea dreapta a regulii defineste o concluzie, efectul satisfacerii premisei este inferarea acelei concluzii (exemplele R1, R2 si R3), iar daca partea dreapta a regulii defineste o actiune, efectul satisfacerii premisei este executia acelei actiuni (exemplul R4). O inferenta in reprezentarea bazata pe reguli consta in aplicarea unei astfel de reguli. Exista diverse forme sintactice pentru exprimarea regulilor de productie, in functie de limbajul de reprezentare a cunostintelor ales. In general, partea stinga a regulii este o conjunctie de conditii, iar partea dreapta a regulii este o conjunctie sau disjunctie de actiuni sau concluzii. De exemplu:

daca I1 **si** I2 **si** I3
atunci C1 **si** C2
daca I1 **si** I2 **si** I3 **si** I4
atunci C1 **sau** C2 **si** C3

Fiecare regula reprezinta o unitate de cunostinte despre un anumit domeniu de expertiza. O multime de reguli poate corespunde unui lant de inferente care duc de la faptele cunoscute initial la concluzii necunoscute, concluzii care reprezinta rezolvarea unei probleme.

Faptele, cealalta forma de reprezentare a cunostintelor in modelul regulilor de productie, au o forma care permite identificarea ipotezelor de satisfacut din reguli. Spre deosebire de reguli, care sunt imperative si dinamice, faptele sunt statice si inactive. Urmatoarele enunturi sunt exemple de fapte posibile intr-o baza de cunostinte:

Coco zboara.
Ilie are temperatura mare.
Tipul organismului o1 este gram-pozitiv.
Masina1 nu porneste.

Se poate observa legatura existenta intre modelul regulilor de productie si logica cu predicate de ordinul I. Regulile R1÷R3 din exemplele enuntate anterior pot avea urmatoarea formulare logica.

$Zboara(coco) \wedge ArePene(coco) \rightarrow Pasare(coco)$

$(\forall p)(\forall o)(Temperatura(p, mare) \wedge Tip(o, gram - pozitiv) \wedge GitUscat(p)) \rightarrow$
 $Identitate(o, streptococ)$

$(\forall m)(\exists b)(NuPorneste(m) \wedge NuAprinde(m, far)) \rightarrow$
 $(Stare(b, consumata) \vee Borne(b, fara_contact))$

Regula R4 este mai dificil de exprimat in calculul cu predicate deoarece concluzia ei implica o actiune. Din punct de vedere logic, faptele pot fi vazute ca formule atomice; de exemplu faptele anterioare pot fi exprimate in formalismul logic astfel:

$Zboara(coco)$

$Temperatura(ilie, mare)$

$Tip(o1, gram - pozitiv)$

$NuPorneste(masina1)$

Formulara regulilor si a faptelor din exemplele date s-a facut in limbaj natural. Cele mai multe limbaje bazate pe reguli de productie au o sintaxa fixa care, desi apropiata de limbajul natural, defineste un limbaj independent de context. Utilizind o astfel de sintaxa, exemplele de reguli R2÷R4 vor fi de fapt exprimate intr-un posibil limbaj de reprezentare astfel:

R2': **daca** Temperatura-Pacient = mare
 si Tip-Organism = gram-pozitiv
 si GitUscat-Pacient
 atunci Identitate-Organism = streptococ

R3': **daca** NuPorneste-Masina
 si NuAprinde-Masina = far
 atunci Stare-Baterie = consumata
 sau Borne-Baterie = fara-contact

R4': **daca** Temperatura > 95
 atunci DeschideValva

Se observa ca atit conditiile din partea stinga a regulilor, cit si concluzia (sau actiunile) din partea dreapta a regulilor pot contine entitati echivalente cu variabilele din calculul cu predicate. Unele dintre sistemele care folosesc reprezentarea bazata pe reguli de productie, cum ar fi sistemul MYCIN, permit existenta a doua forme alternative pentru reguli si fapte: o forma apropiata de limbajul natural pentru interfata cu utilizatorul si o forma interna, in care regulile sint exprimate intr-un limbaj apropiat unui limbaj de programare, deci avind o sintaxa fixa.

Multe din sistemele bazate pe reguli extind reprezentarea stricta sub forma de reguli si fapte cu o structurare a cunostintelor referite de acestea. Universul problemei este descris prin obiecte si attributele asociate care le definesc, iar regulile de productie si faptele refera aceste obiecte, attribute si valorile lor. In exemplele de reguli R2'÷R3' se poate presupune, de exemplu, ca exista obiectul Pacient, cu attributele Temperatura si GitUscat, obiectul Organism cu attributele Tip1 si Identitate, obiectul Masina cu attributele NuPorneste, NuAprinde si obiectul Baterie cu attributele Stare si Borne. Aceasta extindere reprezinta de fapt introducerea in modelul pur al regulilor de productie a unor elemente particulare reprezentarii structurate a cunostintelor, model ce va fi descris in Capitolul 6.

Sintaxa deschisa a limbajelor bazate pe reguli permite proiectantilor mult mai multe libertati decat calculul cu predicate de ordinul I. In plus, pot apare actiuni asociate consecintelor regulilor, actiuni care pot consta fie in executia unor proceduri sau functii externe, fie in comanda sau executia unor actiuni. Modelul regulilor permite, de asemenea, o mare flexibilitate la nivelul structurii de control, asa cum se va vedea in Sectiunea 4.3. Pe langa aspectele mentionate, utilizarea regulilor de productie aduce urmatoarele avantaje din punct de vedere al modelarii cunostintelor in sistem [Hayes-Roth,s.a.,1983]:

- separarea cunostintelor generale despre problema de datele specifice unei instante a problemei de rezolvat
- partitionarea cunostintelor in unitati de cunostinte independente, facilitind astfel dezvoltarea incrementala a bazei de cunostinte
- posibilitatea mentinerii a doua forme de expresie a regulilor: o forma interna sistemului, adecvata procesului de rezolvare si o forma externa, apropiata limbajului natural, pentru interfata utilizatorului cu sistemul.

4.2 Sisteme bazate pe reguli de productie

Pentru prezentarea proprietatilor reprezentarii si utilizarea cunostintelor exprimate sub forma de reguli de productie, in continuare se descriu structura si functionarea sistemelor de rezolvare a problemelor care utilizeaza acest formalism. Sectiunile 4.3.1 si 4.3.2 vor prezenta apoi doua exemple tipice de astfel de sisteme.

4.2.1 Structura unui sistem bazat pe reguli

Un sistem care utilizeaza modelul regulilor de productie pentru reprezentarea cunostintelor se numeste *sistem bazat pe reguli*, pe scurt SBR [Hayes-Roth,1985]. In general, un sistem bazat pe reguli de productie este format din urmatoarele trei componente principale:

- Baza de cunostinte (BC)
- Memoria de lucru (ML)
- Interpretorul de reguli sau Motorul de inferenta (MI)

Structura generala a unui sistem bazat pe reguli este prezentata in Figura 4.1.

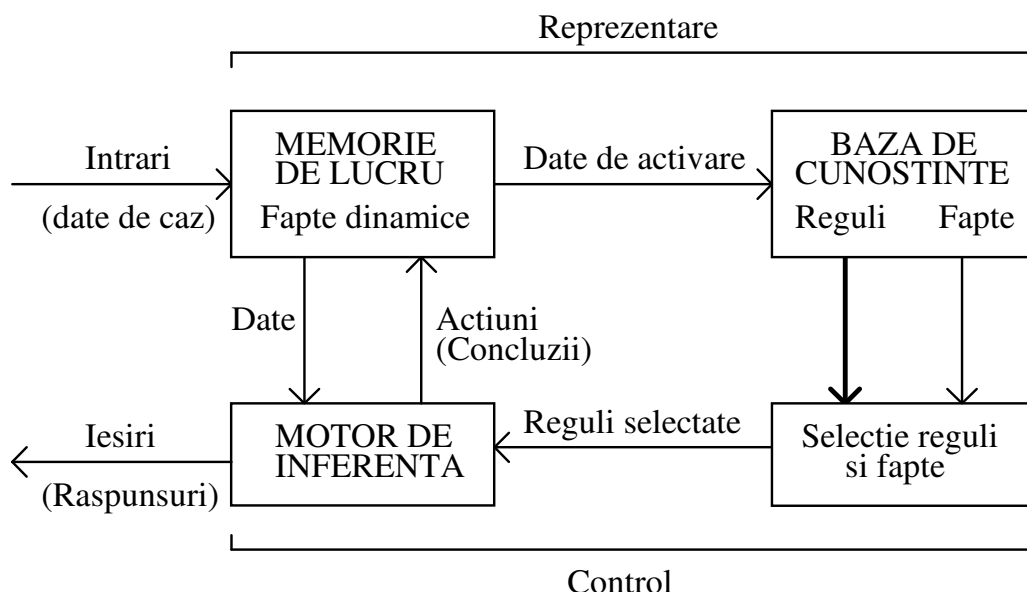


Figura 4.1 Organizarea unui sistem bazat pe reguli

Baza de cunostinte contine cunostintele domeniului problemei de rezolvat, exprimate sub forma de reguli, si cunostinte specifice instantelor problemei, exprimate sub forma de fapte. Rezolvarea problemelor complexe implica existenta unor baze de cunostinte de dimensiuni considerabile, necesitind din acest motiv mecanisme de organizare si indexare a regulilor.

Memoria de lucru contine informatia de stare a rezolvarii problemei, informatie reprezentata prin asertiuni temporare. Aceste asertiuni temporare sint atit datele initiale ale problemei de rezolvat, cit si toate faptele inferate pe parcursul rezolvarii problemei. De obicei, datele din memoria de lucru respecta conventiile sintactice ale faptelor din baza de cunostinte, ceea ce face ca aceste asertiuni temporare sa se numeasca si *fapte dinamice*, iar memoria de lucru sa se numeasca *memorie dinamica*.

Motorul de inferenta reprezinta componenta de control si executie a unui sistem bazat pe reguli. Motorul de inferenta inspecteaza partea stinga a fiecarei reguli din baza de cunostinte pina cind gaseste o regula care identifica continutul memoriei de lucru, dupa care executa (aplica) aceasta regula. Executia regulii de productie determina schimbarea continutului memoriei de lucru pe baza partii drepte a regulii care a identificat. Acest proces continua pina cind in memoria de lucru se acumuleaza faptele care reprezinta solutia problemei sau pina cind nu se mai poate aplica nici o regula. Rezolvarea unei probleme folosind un sistem bazat pe reguli consta deci in realizarea unei serii de inferente printr-un proces de inlantuire recursiva a regulilor, pina la gasirea solutiei sau pina la intilnirea unei situatii de blocaje (nu intotdeauna se ajunge intr-una din aceste doua stari, sistemul putind cicla la infinit in anumite cazuri). Intr-un astfel de sistem, procesul de inferenta se executa, tipic, intr-un mod interactiv, utilizatorul putind furniza noi date pe parcursul executiei, daca aceste date sint cerute de sistem.

In general, motorul de inferenta accepta intrebari de la utilizator si raspunde folosind informatia dinamica din memoria de lucru (datele de caz) si cunostintele statice din baza de cunostinte (reguli). Cunostintele din baza de cunostinte sint folosite pentru a deriva concluzii despre cazul sau situatia curenta prezentata de utilizator.

4.2.2 Ciclul de inferenta al unui sistem bazat pe reguli

Functionarea unui sistem bazat pe reguli este formata din executia repetata a unui ciclu de operatii care realizeaza, in esenta, aplicarea unei reguli, deci un pas de inferenta al reprezentarii. Acest ciclu de operatii se numeste *ciclu de inferenta* al sistemului expert bazat pe reguli de productie si se compune din urmatoarele trei etape: identificare, selectie si executie.

- (1) *Identificare*. In timpul etapei de identificare se compara continutul memoriei de lucru cu baza de cunostinte si regulile care identifica sint grupate de motorul de inferenta in multimea de conflicte. *Multimea de conflicte* reprezinta multimea regulilor aplicabile intr-un anumit context, context specificat de memoria de lucru.
- (2) *Selectia*. Etapa de selectie consta in selectarea unei reguli din multimea de conflicte, pe baza unui criteriu de selectie. Aceasta etapa se mai numeste si *rezolvarea conflictelor*.
- (3) *Executie*. In timpul etapei de executie se aplica regula prin executia partii drepte a regulii. Daca partea dreapta a regulii este o concluzie, se adauga faptele din concluzie in memoria de lucru, iar daca partea dreapta este o actiune, se executa aceasta actiune. Actiunea poate indica diverse operatii, cum ar fi: adauga fapte in memoria de lucru, sterge fapte, modifica fapte existente, tipareste mesaje, pune intrebari, opreste procesul de inferenta.

Ciclul de inferenta al unui sistem bazat pe reguli poate fi exprimat, la nivel general, prin urmatorul algoritm.

Algoritm: Functionarea unui sistem bazat pe reguli

1. $ML \leftarrow$ date de caz

2. **repeta**

2.1. Executa identificare intre ML si BC (partea stinga sau partea dreapta a regulilor si fapte) si creeaza multimea de conflicte a regulilor aplicabile

2.2. Selecteaza o regula dupa un criteriu de selectie

2.3. Aplica regula prin executia partii drepte a regulii

pina nu mai sint reguli aplicabile **sau**

memoria de lucru satisface conditia de stare scop **sau**

o cantitate predefinita de efort a fost epuizata

sfirsit.

Observatii:

- Forma specifica a algoritmului pentru un sistem sau altul difera in functie de directia de aplicare a regulilor, asa cum se prezinta in Sectiunea 4.2.4.
- Conditia de stare scop a memoriei de lucru este atinsa in momentul in care memoria de lucru contine solutia problemei.

Etapa de identificare are ca scop determinarea acelor reguli care pot fi satisfacute pe baza continutului curent al memoriei de lucru, prin identificarea partii stingi a regulilor cu faptele din

memoria de lucru. In cazul in care regulile contin variabile, procesul de identificare este similar cu cel al unificarii expresiilor din calculul cu predicate de ordinul I (Sectiunea 3.3.3), expresia care unifica fiind partea stinga a regulii. Legarile de variabile produse de aceasta unificare influenteaza si variabilele cu aceleasi nume din partea dreapta a regulii deoarece, la fel ca in logica cu predicate de ordinul I, contextul unei variabile este toata regula in care apare variabila. Din cauza prezentei variabilelor in regula, etapa de identificare poate genera pentru o aceeași regula din baza de cunostinte mai multe reguli in multimea de conflicte, reguli corespunzatoare diverselor instantieri posibile ale variabilelor. Baza de cunostinte poate avea dimensiuni impresionante in cazul rezolvarii unor probleme complexe, fapt ce influenteaza semnificativ atat timpul de identificare, cit si dimensiunea multimii de conflicte. Timpul identificarii regulilor aplicabile poate fi redus prin diverse tehnici cum ar fi partitionarea regulilor sau aplicarea unor algoritmi destepti de identificare, asa cum se va discuta mai tirziu.

Etapa de selectie este cea care stabileste de fapt forma de cautare utilizata de sistemul bazat pe reguli, deci strategia de control. Diversele criterii de selectie a regulilor vor fi prezentate in sectiunea urmatoare. *Strategia de control* este un element important al ciclului de inferenta al unui sistem bazat pe reguli. Intr-un sistem bazat pe reguli strategia de control are doua componente: *stabilirea criteriului de selectie a regulilor* din multimea de conflicte si *directia de aplicare a regulilor*: inlantuirea inainte sau inlantuirea inapoi a regulilor.

Rezolvarea unei probleme de catre un sistem bazat pe reguli este de fapt un proces de cautare a solutiei in care operatorii sint reprezentati de regulile sistemului. In consecinta, toate tehnicile de cautare prezentate in Capitolul 2 pot fi aplicate in cazul sistemelor bazate pe reguli. Din punct de vedere al directiei de aplicare a regulilor, inlantuirea inainte a regulilor corespunde unei reprezentari a solutiei problemei prin spatiul starilor, iar inlantuirea inapoi a regulilor corespunde unei reprezentari prin grafuri SI/SAU a solutiei problemei. Rezolvarea conflictelor se refera la ordinea de selectie si preferarea unui operator fata de alti operatori aplicabili intr-un context dat.

La fel ca in orice problema de cautare, strategia unui sistem bazat pe reguli poate fi irevocabila, i.e. fara posibilitatea revenirii in stari anterioare, sau tentativa, i.e. se aplica regula dar se mentine informatia necesara unei posibile reveniri in punctul anterior aplicarii regulii. De asemenea, strategia sistemului poate avea un grad de informare mai mare sau mai mic, daca exista cunostinte suficiente pentru a alege regulile potrivite, sau poate fi complet neinformatata, caz in care selectia se face conform unei ordini stabilite a priori.

Costul computational al rezolvarii unei probleme utilizind reguli de productie implica, pe linga costul controlului si cel al aplicarii regulilor, si costul procesului de identificare. In urma studiilor efectuate, s-a observat ca identificarea este etapa cea mai consumatoare de timp din ciclul de inferenta al unui sistem bazat pe reguli. Ponderea costului controlului (selectiei) regulilor si a costului aplicarii (executiei) regulilor in costul total este similara celei descrise de graficul din Figura 2.7 (Capitolul 2).

4.2.3 Criterii de selectie a regulilor

Rezultatul etapei de identificare este multimea de conflicte, deci multimea tuturor regulilor care au identificat cu descrierea starii curente a rezolvarii problemei, descriere continuta in memoria de lucru. Rezolvarea conflictelor din etapa de selectie are rolul alegerii uneia sau mai multor reguli care vor fi aplicate. Exista diverse criterii de selectie, prezentate in continuare.

(a) *Selectia primei reguli aplicabile*

Considerind ordinea fizica a regulilor din baza de cunostinte, se alege prima regula aplicabila, deci prima care a identificat, si se aplica acea regula. In acest caz, nu se creeaza de fapt o multime de conflicte, regimul de control fiind un control numit *focalizarea atentiei*. Aceasta strategie este aplicata de exemplu, de sistemul Prolog si corespunde unei strategii de tip "backtracking".

(b) *Alegerea unei reguli din multimea de conflicte*

In acest caz se foloseste o tehnica explicita de rezolvare a conflictelor pentru a decide asupra regulii de aplicat. Preferintele in alegerea unei reguli se pot baza pe natura regulilor, pe natura faptelor (obiectelor) identificate, pe starile urmatoare generate sau pe utilizarea unor cunostinte de control exterioare cunostintelor specifice domeniului, metaregulile. Aceasta strategie este de fapt cunoscuta sub numele de *rezolvarea conflictelor*. Preferintele de alegere a regulilor din multimea de conflicte pot fi:

(b.1) *Preferinte bazate pe natura regulilor*

Unul din cazurile cele mai intilnite este acela al preferintei regulilor cu specificitate mai mare. O regula R1 are o *specificitate mai mare* decat o alta regula R2 daca:

- multimea de conditii ale regulii R1 include multimea de conditii ale regulii R2, deci este un superset al acesteia;
- conditiile regulii R1 sint aceleasi cu cele ale regulii R2, dar conditiile regulii R1 refera valori constante, in timp ce acelea ale lui R2 refera variabile.

Utilizarea acestui criteriu este justificata deoarece se considera ca o regula cu specificitate mai mare poate descrie cu o mai mare acuratete caracteristicile starii curente, deci are mai multe sanse din punct de vedere al avansului spre solutie. Un alt criteriu de preferinta este acela al *momentului folosirii anterioare* a regulii, in sensul ca se prefera regula cea mai recent folosita sau regula folosita cel mai de demult. Aceste alegeri sint asemanatoare criteriilor MRU si LRU din mecanismele de paginare a memoriei folosite in sistemele de operare.

Reducerea multimii de conflicte poate fi realizata prin partitionarea regulilor in diverse clase, fiecare clasa fiind definita prin contextul de aplicare al regulii. Aceasta partitionare a bazei de reguli se realizeaza de obicei prin introducerea unei prime conditii in fiecare regula care specifica contextul (starea memoriei de lucru) de aplicare a regulii, asa cum se va explica in Sectiunea 4.3.1.

De multe ori, rezolvarea conflictelor se face prin aplicarea mai multor criterii in scopul eliminarii, pe cit posibil, a ambiguitatilor. De exemplu, in sistemul OPS5 [Cooper, Wogrin, 1988] rezolvarea conflictelor se face pe baza partitionarii regulilor, a specificitatii regulilor, a eliminarii instantelor de reguli care au fost deja aplicate, plus un criteriu de preferinta bazata pe obiecte, acela al regulii care a identificat faptul cel mai recent introdus in memoria de lucru.

(b.2) *Preferinte bazate pe obiectele identificate*

Se pot atasa factori de merit faptelor din memoria de lucru si a celor din baza de cunostinte si se vor prefera regulile care identifica faptele cu factori de merit maxim. O alta posibilitate este preferarea celui mai recent fapt identificat, criteriu amintit anterior si folosit in OPS5.

(b.3) *Preferinte bazate pe stari*

Daca exista mai multe reguli in multimea de conflicte, se pot aplica temporar toate aceste reguli, generindu-se astfel in memoria de lucru diverse stari urmatoare. Apoi, pe baza unei functii euristice de evaluare a starilor rezultate, se alege ca regula preferata acea regula care a generat o stare cu valoarea maxima (minima) a functiei euristice. Aceasta abordare ar trebui sa fie deja familiara cititorului constiincios deoarece este identica cu strategia de control euristica de tip "best-first", strategie prezentata in Capitolul 2.

(b.4) *Utilizarea metaregulilor*

Anumite criterii de selectie a regulilor din multimea de conflicte pot fi exprimate explicit tot sub forma de reguli. Cunostintele inglobate in acest tip de reguli se numesc *cunostinte de control* si fac parte din categoria metacunostintelor. Ele se numesc *metacunostinte* deoarece nu sint cunostinte care caracterizeaza domeniul problemei de rezolvat ci sint cunostinte despre cunostinte. Modelul regulilor de productie ofera avantajul posibilitatii exprimarii acestor metacunostinte in acelasi fel ca si cunostintele domeniului, i.e. sub forma de reguli. Acest lucru este considerat un avantaj deoarece interpretorul de reguli poate fi folosit atit pentru aplicarea regulilor, cit si a metaregulilor. Cunostintele de control pot lua diverse forme, de exemplu:

- cunostinte despre ce stari sint preferate fata de altele
- cunostinte despre ce reguli trebuie preferate in anumite situatii
- cunostinte despre ordinea in care trebuie realizate scopurile
- cunostinte despre secvente utile de reguli care pot fi aplicate.

Urmatoarea schema generala de regula prezinta cititorului exemplele unor metareguli de preferare a regulilor in anumite situatii [Davis,1980].

daca o regula are conditiile A si B

si regula refera {nu refera} X

{ de loc/

numai in partea stinga/

numai in partea dreapta }

atunci regula va fi in special utila

{ probabil utila/

probabil inutila/

sigur inutila }

Unul dintre cele mai cunoscute sisteme care include metacunostinte despre secvente de reguli util de aplicat este sistemul SOAR [Laird,s.a.,1987].

(c) *Aplicarea tuturor regulilor din multimea de conflicte*

O astfel de strategie aplica toate regulile din multimea de conflicte si produce mai multe stari care vor fi memorate si prelucrate independent. Ea se numeste strategie de tipul "*incearca toate regulile*" si poate fi aplicata in cazul in care se poate evita un cost ridicat al exploziei

combinationale. Criteriul "incearca toate regulile" se aplica, in general, in cazul sistemelor bazate pe reguli cu rationament incert. In acest tip de sisteme, toate datele (faptele) au asociat un coeficient de certitudine care indica increderea sistemului in acele valori, iar sistemul calculeaza noi coeficienti de certitudine pentru datele nou inferate. Executia unor secvente de reguli diferite, pornind de la aceeasi stare, poate duce la deductia unor date diferite, fiecare avind inasa asociat un alt coeficient de certitudine. Un astfel de sistem bazat pe reguli este sistemul MYCIN despre care se va vorbi in Sectiunea 4.3.2, Capitolul 5 si Capitolul 7.

4.2.4 Directia de aplicare a regulilor

A doua componenta a strategiei de control a unui sistem bazat pe reguli este directia de aplicare a regulilor. Regulele pot fi aplicate utilizind *inlantuirea inainte* prin identificarea partii stingi a regulii cu memoria de lucru, sau utilizind *inlantuirea inapoi* prin identificarea partii drepte a regulii cu memoria de lucru si incercarea satisfacerii partii stingi a regulii. Cele doua abordari sint prezentate sintetic in Figura 4.2.

INLANTUIRE INAINTE INLANTUIRE INAPOI

daca A atunci B	determina C
daca B atunci C	daca B atunci C
<u>A (data)</u>	<u>daca A atunci B</u>
C (concluzie)	(daca A atunci C , implicit)
	Este A adevarata? (data)

Figura 4.2 Inlantuirea executiei regulilor de productie

Ideea functionarii sistemelor cu inlantuire inainte, respectiv inlantuire inapoi, poate fi explicata si facind o paralela cu teoria limbajelor formale. Considerind urmatoarea gramatica, data sub forma de reguli de productie:

$$S \rightarrow ABA$$

$$A \rightarrow A1, A \rightarrow 1, B \rightarrow B0, B \rightarrow 0$$

identificarea partii stingi a unei multimi de reguli cu o baza de date (memorie de lucru) care contine simbolul de start S, produce un generator de propozitii din limbajul specificat de gramatica, iar identificarea partii drepte a aceleiasi multimi de reguli cu baza de date produce o metoda de recunoastere (un acceptor) pentru acest limbaj.

Exemplu. Fie urmatorul set de reguli de productie:

- R1: **daca** A
 si B
 atunci C
- R2: **daca** C
 atunci D
- R3: **daca** E
 atunci B
- R4: **daca** A
 si E
 atunci C

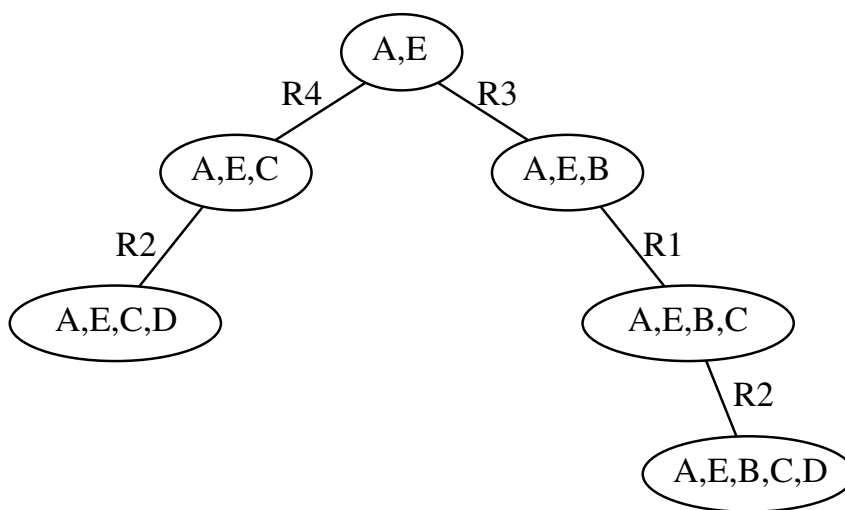
Memoria de lucru contine initial faptele A si E care reprezinta datele de caz ale instantei problemei de rezolvat, si starea scop D, D reprezentind solutia cautata.

Aplicind inlantuirea inainte a regulilor se poate gasi secventa de reguli R3, R1, R2, prezentata in Figura 4.3 (a), care produce in memoria de lucru stare scop D cautata. Considerind toate regulile aplicabile la un moment dat, exemplul genereaza arborele de cautare in spatiul starilor din Figura 4.3 (b). Se observa ca starea initiala, definita prin continutul A, E al memoriei de lucru, creeaza multimea de conflicte {R4, R3}. Fiecare aplicare de regula, pe o cale de cautare, va adauga noi fapte la memoria de lucru astfel incit, in final, aceasta contine fie secventa A, E, C, D fie A, E, B, C, D. Din spatiul de cautare al solutiei s-au eliminat aplicari de reguli care nu modifica continutul memoriei de lucru, de exemplu nu s-a figurat regula R4 care s-ar fi putut aplica secventei A, E, C dar care nu ar fi produs nici o schimbare.

Continut initial al memoriei de lucru: A,E
Stare scop: D

$$A,E \xrightarrow{R3} A,E,B \xrightarrow{R1} A,E,B,C \xrightarrow{R2} A,E,B,C,D$$

(a)



(b)

Figura 4.3 Inlantuirea inainte a regulilor de productie

Aplicind inlantuirea inapoi a regulilor de productie, se incearca satisfacerea scopului D prin aplicarea unei reguli care mentioneaza in concluzie D. O astfel de regula este R2. Pentru ca R2 sa poata fi executata trebuie ca memoria de lucru sa contina faptele care satisfac premisa regulii. Pentru aceasta trebuie indeplinite scopurile din premisa, deci cautate regulile care refera aceste scopuri in concluzie. In cazul in care nu exista astfel de reguli, fie faptele sint deja in memoria de lucru, si regula poate fi direct aplicata, fie, in caz contrar, se solicita utilizatorului informatii despre adevarul acestor fapte. O posibila secventa de satisfacere a scopului D prin aplicarea

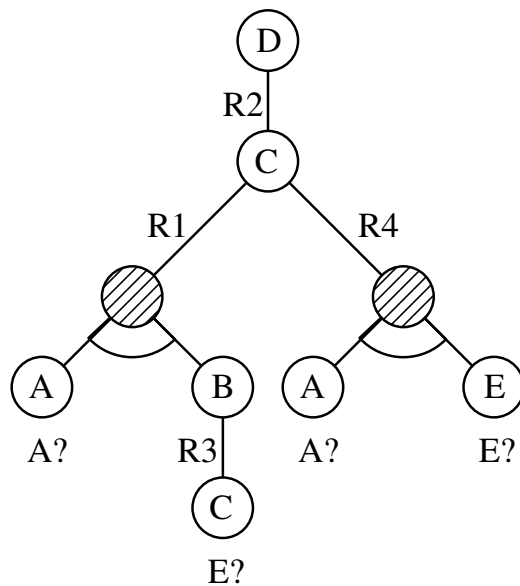
inlantuiri inapoi a regulilor este prezentata in Figura 4.4 (a). Daca se considera toate posibilitatile de satisfacere a scopului D se obtine arborele SI/SAU din Figura 4.4 (b).

Continut initial al memoriei de lucru: A,E

Stare scop: D

$$D? \xrightarrow{R2} C? \xrightarrow{R1} A?, B? \xrightarrow{R3} A?, E?$$

(a)



(b)

Figura 4.4 Inlantuirea inapoi a regulilor de productie

Revenind la exemplul mozaicului de opt numere prezentat in Capitolul 2, problema poate fi reformulata prin definirea unor reguli de productie. Presupunind ca patratele mozaicului sint numerotate de la 1 la 9, de-a lungul liniilor, se pot defini reguli de tipul

- R1: **daca** Patrutul 1 este vid
si Patrutul 2 contine numarul N
atunci Patrutul 2 devine vid
si Patrutul 1 contine numarul N
- R2: **daca** Patrutul 1 este vid
si Patrutul 4 contine numarul N
atunci Patrutul 4 devine vid
si Patrutul 1 contine numarul N

...

Regula R1 corespunde operatorului de deplasare a patratului liber DREAPTA, iar regula R2 operatorului JOS. Rezolvind problema prin aplicarea inlantuiri inainte a regulilor, memoria de lucru contine la inceput descrierea starii initiale in termenii continutului fiecarui patrat de la 1 la 9: numar sau vid. Se aplica regulile care identifica memoria de lucru, N fiind o variabila ce poate fi instantiala la orice numar de la 1 la 9. In acest caz, partea dreapta a regulilor are efectul de a

modifica continutul memoriei de lucru prin eliminarea unor fapte si adaugarea de noi fapte. In momentul in care memoria de lucru satisface conditia de stare finala, i.e. contine configuratia corespunzatoare starii finale, problema este rezolvata.

Rezolvind problema prin aplicarea inlantuirii inapoi a regulilor, memoria de lucru este initializata la starea finala. Se cauta acele reguli ale caror concluzii produc descrierea starii finale si se genereaza arborele SI/SAU corespunzator pina in momentul in care se ajunge la o multime de ipoteze de reguli care sint satisfacute de descrierea starii initiale. Regulile de tipul celor indicate mai sus reprezinta o formulare ineficienta a tranzitiilor posibile in problema deoarece trebuie specificata cite o regula pentru fiecare posibila pereche de patrate. Inlocuind constantele Patratul1, Patratul2, etc., cu variabilele PatratulX, PatratulY si adaugind in partea stinga a regulii o conditie suplimentara referitor la pozitia relativa a celor doua patrate, se obtine un set de reguli redus, deci o reprezentare mai eficienta. Se propune cititorului formularea acestei noi multimi de reguli.

Observatii:

- Asa cum s-a spus si in Sectiunea 4.2.2, inlantuirea inainte a regulilor pentru rezolvarea problemei corespunde unei cautari in spatiul starilor, in timp ce inlantuirea inapoi a regulilor corespunde unei cautari in grafuri SI/SAU.
- Spre deosebire de modelele de cautare prezentate in Capitolul 2, aceleasi reguli pot fi aplicate atit folosind inlantuirea inainte, cit si inlantuirea inapoi, asa cum se vede din exemplul anterior. Alegerea unei directii de aplicare a regulilor sau a alteia este importanta. In functie de topologia spatiului de cautare, cautarea intr-o directie sau alta poate fi semnificativ mai eficienta decit cautarea in cealalta directie.
- Etapa de identificare a ciclului de inferenta al unui sistem bazat pe reguli este mult mai complexa in cazul inlantuirii inainte decit in cazul inlantuirii inapoi. Au fost dezvoltate diverse tehnici pentru reducerea timpului necesar acestei etape, cum ar fi de exemplu algoritmul RETE, de identificare rapida a obiectelor multiple cu sabloane multiple [Foggy,1982], algoritmul folosit in sistemul OPS5.

Exista sisteme bazate pe reguli care utilizeaza ambele directii de aplicare a regulilor. In general, in astfel de sisteme, anumite reguli se aplica folosind inlantuirea inainte, iar alte reguli se aplica folosind inlantuirea inapoi, regulile fiind marcate fie ca "reguli inainte", fie ca "reguli inapoi". Strategia de control a unui astfel de sistem trebuie sa poata comuta intre o directie si alta, in functie de tipul regulii care a identificat.

4.3 Paradigme de sisteme bazate pe reguli

In aceasta sectiune se prezinta citeva exemple de sisteme bazate pe reguli impreuna cu algoritmi de baza ai structurilor de control asociate. Exemplele prezentate sint didactice si foarte simple, avind ca principal scop explicarea functionarii acestor sisteme in termenii unei complexitati de prezentare rezonabile. Cu toate acestea, exemplele sint inspirate din arhitecturile functionale ale unor sisteme reale foarte cunoscute si aplicate cu succes in rezolvarea unor probleme dificile, care necesita expertiza umana.

4.3.1 Sisteme cu inlantuire inainte a regulilor

Functionarea unui sistem cu inlantuire inainte a regulilor urmareste algoritmul general al ciclului de inferenta prezentat in Sectiunea 4.2.2. Particularitatea rezolvarii problemelor intr-un astfel de sistem poate fi descrisa prin urmatorul algoritm de obtinere a valorii unui obiect sau a unui atribut al unui obiect. In algoritm se noteaza cu A obiectul sau atributul obiectului pentru care se incearca determinarea valorii.

Algoritm: Determinarea unei valori cu inlantuire inainte a regulilor

DetValoareD(A)

1. **daca** A are valoare

atunci intoarce SUCCES

2. Construiesc multimea de conflicte, MC

3. **intoarce** Determina(A,MC)

sfirsit.

Determina(A,MC)

1. **daca** MC = { }

atunci intoarce INSUCCES

2. Alege o regula $R \in MC$ dupa un criteriu de selectie

3. **pentru** fiecare ipoteza $I_j, j = 1, N$ a premisei regulii R **executa**

 3.1. Determina adevarul lui I_j

4. **daca** toate ipotezele $I_j, j = 1, N$ sunt satisfacute

atunci

 4.1. Adauga faptul din concluzia regulii R la ML

 4.2. **daca** A are valoare

atunci intoarce SUCCES

5. $MC \leftarrow MC - R$

6. **intoarce** Determina(A,MC)

sfirsit.

Observatie. Algoritmul dat nu specifica criteriul de selectie a regulilor de executat din multimea de conflicte. Se poate folosi unul sau mai multe dintre criteriile prezentate in Sectiunea 4.2.3. Diversele detalii si forme particulare ale algoritmului difera de la un sistem la altul.

Exemplul prezentat in continuare va crea o imagine sugestiva a functionarii unui sistem bazat pe reguli cu inlantuire inainte. Se considera problema ambalarii unor obiecte pentru a fi trimise la organizarea unei expozitii. Fiecare obiect este descris printr-o serie de attribute care sunt specificate in baza de cunostinte ca fapte initiale. Un obiect care trebuie ambalat are urmatoarele attribute: nume, greutate, dimensiune si fragilitate, care sunt attribute cu valori predefinite pentru

fiecare obiect. In plus, un obiect are un atribut numit impachetat cu valorile nu, respectiv da, corespunzator starii obiect impachetat intr-o cutie sau nu. Baza de cunostinte contine urmatoarele tipuri de reguli de rezolvare a problemei:

- reguli care se refera la criteriile de ambalare a obiectelor, de exemplu: nu se pune un obiect greu peste unul usor, se incearca intii ambalarea obiectelor mai mari, etc.
- reguli pentru considerarea unei noi cutii de ambalaj cind cea precedenta este completa
- reguli de verificare a completitudinii setului de obiecte trimise in expozitie.

In rezolvarea problemei se pot distinge mai multi pasi sau etape de rezolvare independente. De exemplu, exista o etapa in care se verifica completitudinea decorului, o etapa in care se ambaleaza obiectele mari, o etapa pentru ambalarea obiectelor de dimensiune medie si o ultima etapa pentru ambalarea celor de dimensiune mica. Problemele care prezinta aceasta caracteristica au avantajul posibilitatii partitionarii spatiului de cautare in subspatii de cautare distincte, deci reducerea efortului de cautare prin reducerea multimei de conflicte a regulilor aplicabile la un moment dat. Regulile se partitioneaza in submultimi distincte, fiecare submultime de reguli referindu-se la o anumita etapa de rezolvare a problemei si fiecare regula continind in premiza o ipoteza ce verifica etapa din care face parte regula. Aceasta abordare este de fapt o modalitate de indexare a regulilor care permite reducerea efortului necesar etapei de identificare.

La inceperea executiei, memoria de lucru a sistemului este initializata cu datele de caz ale problemei particulare de rezolvat, respectiv obiectele de impachetat descrise prin valorile de atribute. Tot in memoria de lucru se depune informatia de stare asupra rezolvarii problemei: etapa initiala, ambalajul curent si lista obiectelor de impachetat. Continutul memoriei de lucru este prezentat in Figura 4.5.

Nume	Greutate	Dimensiune	Fragil	Impachetat
Statuie	mediu	mare	nu	nu
Tablou	usor	medie	nu	nu
Vaza	usor	mica	da	nu
Soclu	greu	mare	nu	nu

Etapa : Verifica - Decor
Cutie1 : { }
Obiecte– Neimpachetate: Statuie, Tablou, Vaza, Soclu

Figura 4.5 Continutul initial al memoriei de lucru la ambalarea obiectelor

Fiecare regula din baza de cunostinte testeaza numele etapei. Exemple de reguli din prima etapa sint:

- R11: **daca** Etapa este Verifica-Decor
si exista o statuie
si nu exista soclu
atunci adauga soclu la Obiecte-Neimpachetate
- R17: **daca** Etapa este Verifica-Decor
atunci termina Etapa Verifica-Decor
si incepe Etapa Obiecte-Mari

La prima vedere regula R17 pare periculoasa deoarece ar putea fi apelata oricind. Acest lucru nu se intimpla deoarece sistemul functioneaza ordonind regulile din multimea de conflicte

dupa *criteriul de specificitate* al regulilor, deci regula R17 va fi ultima regula aplicabila in etapa Verifica-Decor. Regula R17 va determina o schimbare adecvata in memoria de lucru: continutul atributului Etapa se modifica la Obiecte-Mari facindu-se astfel comutarea starii sistemului la o noua etapa. Exemple de reguli din etapa Obiecte-Mari sint:

R21: **daca** Etapa este Obiecte-Mari
 si exista un obiect mare de ambalat
 si exista un obiect greu de ambalat
 si exista o cutie cu mai putin de trei obiecte mari
 atunci pune obiectul greu in cutie

R22: **daca** Etapa este Obiecte-Mari
 si exista un obiect mare de ambalat
 si exista o cutie cu mai putin de trei obiecte mari
 atunci pune obiectul mare in cutie

Obiectele mari sint ambalate primele in cutii dar, daca exista un obiect greu, acesta este ambalat primul. Tot pe baza criteriului de specificitate, regula R21 se executa inaintea regulii R22, desi ambele reguli sint aplicabile intr-un context dat. Tot in aceasta Etapa trebuie sa mai existe o regula pentru alegerea unei noi cutii in cazul in care cutia curenta s-a umplut si o regula pentru parasirea etapei Obiecte-Mari si inceperea etapei Obiecte-Medii. Aceste reguli sint:

R28: **daca** Etapa este Obiecte-Mari
 si exista un obiect mare de pus
 atunci foloseste o cutie noua
 R29: **daca** Etapa este Obiecte-Mari
 atunci termina Etapa Obiecte-Mari
 si incepe Etapa Obiecte-Medii

Memoria de lucru in acest moment va inregistra urmatoarele modificari:

Etapa :	Obiecte - Medii
Cutie 1 :	Soclu, Statuie
Obiecte– Neimpachetate:	Tablou, Vaza

Observatii:

- Regulile din acest exemplu sint formulate in limbaj natural fara a tine cont de o sintaxa specifica a unui limbaj bazat pe reguli.
- Se observa ca regulile se refera la obiecte generice (variabile) care vor fi instantiate in faza de identificare. De exemplu, "obiect mare de pus" din regula R22 sta pe postul unei variabile si se instantiaza la obiectul particular Statuie.
- Partea dreapta a regulilor poate adauga elemente la memoria de lucru, poate sterge sau modifica valori din memoria de lucru sau poate comuta intr-o noua Etapa.
- Un posibil criteriu de oprire al sistemului, deci detectarea starii scop, ar fi momentul in care toate obiectele au atributul impachetat egal cu da.

Modelul de rezolvare al acestei probleme este inspirat din functionarea unuia dintre cele mai de succes sisteme expert, sistemul R1/XCON [McDermott,1982] realizat pentru configurarea comenzilor de calculatoare VAX ale firmei DEC. Sistemul a fost implementat in limbajul OPS5,

si va fi prezentat in Capitolul 7. Un exemplu de problema similara poate fi gasit in Winston [1984].

4.3.2 Sisteme cu inlantuire inapoi a regulilor

Intr-un sistem cu inlantuire inapoi a regulilor de productie functionarea se porneste de la scopul de demonstrat sau de aflat. De obicei acest scop este aflarea valorii unui obiect sau a unui atribut al unui obiect. Particularitatea rezolvarii problemelor intr-un astfel de sistem poate fi descrisa prin algoritmul urmator.

Algoritm: Determinarea unei valori cu inlantuirea inapoi a regulilor

DetValoareI(A)

1. Construiesc multimea regulilor care refera A in concluzie, MC

2. **daca** MC = { } /* nu exista nici o regula care sa deduca valoarea lui A */

atunci

2.1. Intreaba utilizatorul valoarea lui A

2.2. **daca** se cunoaste valoarea lui A

atunci depune in ML aceasta valoare

3. **altfel**

3.1. Alege o regula $R \in MC$ dupa un criteriu de selectie

3.2. **pentru** fiecare ipoteza I_j , $j = 1, N$, a premisei lui R **executa**

3.2.1. Fie A_j atributul referit de ipoteza I_j

3.2.2. **daca** A_j are valoare

atunci atunci evalueaza adevarul ipotezei I_j

3.2.3. **altfel**

i. **daca** $\text{DetValoareI}(A_j) = \text{SUCCES}$

atunci evalueaza adevarul ipotezei I_j

ii. **altfel** considera ipoteza I_j nesatisfacuta

3.3. **daca** toate ipotezele I_j , $j = 1, N$ sint satisfacute

atunci depune in ML valoarea lui A dedusa pe baza concluziei R

4. **daca** A are valoare

atunci intoarce SUCCES

5. **intoarce** INSUCCES

sfirsit.

In anumite sisteme se marcheaza attributele a caror valoare poate fi obtinuta intrebnd utilizatorul, acestea avind statut de date primare. In cazul existentei atributelor primare, intii se solicita utilizatorului valoarea atributului si numai daca aceasta valoare nu este cunoscuta se

incearca aplicarea regulilor pentru determinarea ei. Pentru a reflecta aceasta modificare, se introduce in algoritm pasul 1' inaintea pasului 1

1'. **daca** A este data primara

atunci

1'.1. Intreaba utilizatorul valoarea lui A

1'.2. **daca** se cunoaste valoarea lui A

atunci intoarce SUCCES

In anumite cazuri, incercarea de a determina valoarea unui atribut data primara cu ajutorul regulilor sistemului este complet abandonata, algoritmul intorcind INSUCCES daca utilizatorul nu cunoaste valoarea unui astfel de atribut. Problema de decizie gastronomică prezentata in continuare va utiliza o astfel de abordare.

In pasul 3.1 al algoritmului se va folosi, de la caz la caz, un criteriu de selectie. In cazul in care se aplica strategia de tipul "incearca toate regulile", pasul 3.1 al algoritmului trebuie inlocuit cu pasul

3.1'. **pentru** toate regulile $R_i \in MC$ **executa**

In cazul in care se obtin mai multe valori distincte pentru atributul A, acestea se vor cumula in memoria de lucru, asa cum se va vedea in exemplul deciziei gastronomice.

Se considera un sistem bazat pe reguli care functioneaza cu inlantuire inapoi a regulilor. Baza de cunostinte contine urmatoarele reguli:

R1: **daca** X are par

atunci X este mamifer

R2: **daca** X hraneste puii cu lapte

atunci X este mamifer

R3: **daca** X este mamifer

si X are dinti ascutiti

si X are falci

atunci X este carnivor

R4: **daca** X este carnivor

si X este maroniu

si X are pete

atunci X este leopard

R5: **daca** X este carnivor

si X este maroniu

si X are dungi

atunci X este tigru

Scopul de demonstrat este "Ce este X?". Pentru acesta se cauta regulile care refera in concluzie tipul lui X; acestea sint R5 si R4. Se incearca aplicarea regulii R5, pentru care se incearca aplicarea regulii R3, pentru care se incearca aplicarea regulilor R1 si R2. Deoarece nu mai exista reguli care sa refere attributele premiselor regulilor R1 si R2, se intreaba daca X are par si la raspunsul afirmativ al utilizatorului, se indeplineste regula R1 si se revine la validarea

ipotezelor regulii R3. Presupunind ca utilizatorul raspunde in consecinta, se incearca satisfacerea restului conditiilor regulii R4. Daca utilizatorul raspunde ca X este maroniu si are dungi, regula R4 nu este satisfacuta, dar se indeplineste regula R5, deci tipul lui X este tigru. Se observa ca in cazul in care nu exista nici o regula care sa refere in concluzie un atribut din ipoteza regulii curente de verificat, se intreaba utilizatorul. Sistemul ar fi putut functiona si cu introducerea a o parte (sau toate) din datele initiale ale problemei in memoria de lucru; in acest caz utilizatorul ar fi fost intrebat numai despre valorile de atribut care nu sint prezente in memoria de lucru.

In continuare se prezinta un exemplu de program bazat pe reguli cu inlantuire inapoi care foloseste strategia de tip "incearca toate regulile". Un sistem care foloseste o astfel de strategie se mai numeste si *sistem cu acumulare de probe*. Se pune problema unei decizii gastronomice in care se cere alegerea vinului potrivit pentru un meniu cu componente specificate de utilizator. Aceasta problema prezinta o caracteristica intilnita si in alte probleme de decizie sau diagnosticare, cum ar fi diagnosticul medical, si anume existenta unei incertitudini asupra valorilor corecte de atribute. De exemplu, daca meniul contine sos alb, se poate combina atit un vin sec cit si un vin demisec. Pentru a modela aceasta incertitudine, se asociaza valorilor din sistem o masura a increderei, numita *factor de certitudine* sau *coeficient de certitudine*, notat cu CF.

Cunostintele sint reprezentate sub forma de obiect-atribut, si valorile asociate atributelor, si sub forma de reguli care refera obiectele si atributele din baza de cunostinte. Aceasta inseamna ca, pe linga reguli, baza de cunostinte contine fapte reprezentate sub forma de triplete atribut-obiect-valoare. Atributele obiectelor pot fi de doua tipuri: monovaloare si multivaloare. Un atribut monovaloare este un atribut care, in final, va avea o singura valoare asociata, de exemplu culoarea vinului. Este evident ca un vin nu poate avea mai multe culori in acelasi timp. Atributele multivaloare sint atributele care in final pot avea mai multe valori, toate admisibile. De exemplu, atributul vin poate avea mai multe valori (chardonnay, riesling), interpretarea acestor valori fiind aceea ca sistemul a indicat mai multe posibilitati de vinuri care se potrivesc la meniul indicat.

Exemplul prezentat este un model tipic de rationament incert sau statistic. Rationamentul incert implementat foloseste coeficientii de certitudine asociati faptelor in doua moduri:

- Valoarea fiecarui atribut este memorata impreuna cu coeficientul de certitudine asociat, coeficientul de certitudine indicind increderea sistemului in acea valoare. Coeficientii de certitudine sint valori pozitive in intervalul $[0,1]$. De exemplu, (vin chardonnay 0.8 riesling 0.6) indica faptul ca vinul potrivit este Chardonnay cu increderea 0.8 si Riesling cu increderea 0.6.
- O regula poate avea asociat un coeficient de certitudine care indica increderea sistemului in concluzia regulii, in cazul in care premisa este adevarata. De exemplu, regula:

daca sos-meniu = sos-alb

atunci culoare-vin = alba 0.6

indica increderea de 0.6 in faptul ca un vin alb se potriveste unui meniu cu sos alb. Daca regulile nu au un coeficient de certitudine asociat, acesta se considera implicit 1, indicind incredere totala in concluzie.

Continutul bazei de cunostinte a problemei deciziei gastronomice este prezentat in continuare, utilizind o sintaxa asemanatoare celei din sistemului bazat pe reguli M1.

R11: **daca** componenta-meniu = curcan

atunci culoare-vin = rosie 0.7

si culoare-vin = alba 0.2

R12: **daca** componenta-meniu = peste

atunci culoare-vin = alba

R13: **daca** sos-meniu = sos-alb

atunci culoare-vin = alba 0.6

R14: **daca** componenta-meniu = porc

atunci culoare-vin = rosie

R21: **daca** sos-meniu = sos-alb

atunci tip-vin = sec 0.8

si tip-vin = demisec 0.6

R22: **daca** sos-meniu = sos-tomat

atunci tip-vin = dulce 0.8

si tip-vin = demisec 0.5

R23: **daca** sos-meniu = necunoscut

atunci tip-vin = demisec

R24: **daca** componenta-meniu = curcan

atunci tip-vin = dulce 0.6

si tip-vin = demisec 0.4

R31: **daca** culoare-vin = rosie

si tip-vin = dulce

atunci vin = gamay

R32: **daca** culoare-vin = rosie

si tip-vin = sec

atunci vin = cabernet-sauvignon

R33: **daca** culoare-vin = rosie

si tip-vin = demisec

atunci vin = pinot-noir

R34: **daca** culoare-vin = alba

si tip-vin = dulce

atunci vin = chenin-blanc

R35: **daca** culoare-vin = alba

si tip-vin = sec

atunci vin = chardonnay

R36: **daca** culoare-vin = alba

si tip-vin = demisec

atunci vin = riesling

scop(vin) /* se indica atributul a carei valoare trebuie aflata */

monovaloare(componenta-meniu)

monovaloare(culoare-vin)

monovaloare(sos-meniu)

multivaloare(tip-vin)

multivaloare(vin)

valori-legale(componenta-meniu) = [curcan, peste, porc] /* domeniul de valori */

valori-legale(sos-meniu) = [sos-alb, sos-tomat]

valori-legale(tip-vin) = [sec, demisec, dulce]

valori-legale(vin) = [gamay, cabernet-sauvignon, pinot-noir,
chenin-blanc, chardonnay, riesling]

valori-legale(culoare-vin) = [rosie, alba]

Se considera existenta unui interpretor de reguli pentru aceasta baza de cunostinte, interpretor care lucreaza cu inlantuirea inapoi a regulilor si acumulare de probe. Pe parcursul rezolvarii problemei un atribut monovaloare poate avea mai multe valori competitive, cu diversi coeficienti de certitudine asociati, dar in final se va selecta o singura valoare pentru acesta. Daca se deduce o valoare cu coeficientul 1 (sigura) pentru un atribut monovaloare se abandoneaza acumularea de valori pentru acel atribut si nu se mai incearca aplicarea altor reguli alternative care il refera.

Atributele multivaloare pot avea mai multe valori posibile asociate, atat pe parcursul rezolvarii, cit si in final deci in solutia problemei. Se presupune ca interpretorul de reguli intreaba utilizatorul ori de cate ori intilneste un atribut care nu este referit de nici o regula. Deci orice atribut care nu apare in concluzia unei reguli este considerat data primara.

Pe langa acestea, motorul de inferenta realizeaza o forma de rationament incert pe baza coeficientilor de certitudine asociati valorilor de attribute din memoria de lucru si regulilor din sistem. Regulile de inferenta incerta sint urmatoarele:

- (1) Coeficientul de certitudine asociat valorii de atribut dedusa de o regula R este egal cu produsul dintre coeficientul de certitudine al premisei regulii ($CF_{\text{premise } R}$) si coeficientul de certitudine asociat regulii (CF_R):

$$CF_{\text{atr dedus } R} = CF_{\text{premise } R} \cdot CF_R$$

- (2) Coeficientul de certitudine al premisei unei reguli R este valoarea minima a coeficientilor de certitudine asociati fiecarei ipoteze I_j din premisa acelei reguli. Coeficientul de certitudine al unei ipoteze este stabilit de procesul de identificare a regulii cu memoria de lucru si este egal cu coeficientul valorii de atribut care a satisfacut ipoteza. Factorul de corectitudine al intregii premise se calculeaza astfel:

$$CF_{\text{premise } R} = \min_j \{CF_{\text{ipoteza}_j R}\}$$

- (3) Daca un atribut A are deja o valoare V cu un coeficient de certitudine CF_1 in memoria de lucru, si pe o ramura de deductie alternativa, se obtine aceeasi valoare V pentru atributul A cu un coeficient de certitudine CF_2 , memoria de lucru este actualizata, valoarea V avind un coeficient de certitudine asociat CF pe baza formulei:

$$CF = CF_1 + CF_2 \cdot (1 - CF_1)$$

În continuare se urmareste functionarea exemplului utilizind interpretorul de reguli descris, pentru cazul în care utilizatorul doreste aflarea vinului recomandat unui meniu care contine peste si sos-alb. Scopul de satisfacut este vin. Regulile care refera vin în concluzie sint R31÷R36. Pentru a satisface aceste reguli trebuie satisfacute subscopurile culoare-vin si tip-vin. Regulile care refera culoare-vin în concluzie sint R11÷R14. Pentru a satisface R11 sistemul întreaba utilizatorul valoarea atributului componenta-meniu, întrebare la care utilizatorul raspunde peste. În acest moment R11 nu este satisfacuta, dar R12 reuseste si se adauga în memoria de lucru faptul (culoare-vin alb 1)

Deoarece culoare-vin este un atribut monovaloare pentru care s-a dedus o valoare cu factor de certitudine 1, se opreste acumularea de valori pentru acest atribut. Se continua cu satisfacerea scopului tip-vin care apare în concluziile regulilor R21÷R24. Pentru a satisface regula R21 se întreaba utilizatorul valoarea atributului sos-meniu si se primeste raspunsul sos-alb. Regula R21 reuseste si adauga la memoria de lucru faptul

(tip-vin sec 0.8 demisec 0.6)

Se observa aplicarea regulii (1) de inferenta incerta care asociaza coeficientul de certitudine valorilor de atribut deduse. Se încearca aplicarea celorlalte reguli care refera tip-vin în concluzie, respectiv R22, R23 si R24, dar nici una nu reuseste. Se revine apoi la satisfacerea scopului vin, deci a regulilor R31÷R36. Prima regula dintre acestea care reuseste este R35, regula care ar trebui sa adauge în memorie faptul (vin chardonnay). Tinind cont de regulile de inferenta (1)÷(2) faptul adaugat este

(vin chardonnay 0.8)

Deoarece vin este un atribut monovaloare, se continua acumularea de valori si se încearca si ultima regula care refera vin în concluzie, R36. Pe baza acestei reguli, care reuseste, si utilizind regulile de inferenta (1)÷(2), se actualizeaza memoria de lucru cu faptul

(vin chardonnay 0.8 riesling 0.6)

Deoarece nu mai exista reguli aplicabile, sistemul se opreste si raspunsul este

vin =chardonnay 0.8

riesling 0.6

Observatii:

- Exemplul prezentat are o functionare relativ similara cu modul de rezolvare a problemelor din sistemul MYCIN, sistem care va fi prezentat în Capitolul 7. Diversele simplificari facute fata de modelul MYCIN sint simplificari prezente în sistemul bazat pe reguli independent de domeniu M1.
- Rationamentul incert utilizat în exemplu este de asemenea inspirat din modelul de rationament incert al sistemului MYCIN. Capitolul urmator este în întregime dedicat problematiei rationamentului incert în inteligenta artificiala.

4.3.4 Sisteme bazate pe agenda

Structura de control a anumitor sisteme bazate pe reguli nu foloseste nici înlantuirea înainte nici înlantuirea înapoi a regulilor, ci o *strategie de control de tip agenda*. Aceasta strategie este în

special utila in cazul in care se foloseste un criteriu de selectie a regulilor bazat pe preferinta starilor, deci o functie euristica de evaluare. Un exemplu de astfel de sistem este programul AM [Lenat,1983;Lenat,Brown,1984] care descopera concepte in matematica elementara si in teoria multimilor.

O *agenda* este o lista de sarcini pe care sistemul trebuie sa le execute. O sarcina poate fi executia unei reguli, dar si executia altor actiuni, cum ar fi executia unui pachet de reguli, a instantierii unui obiect sau a unei multimi de obiecte, executia unei metareguli.

Intr-o agenda, fiecare sarcina are asociate cel putin doua informatii: prioritatea sarcinii, care reprezinta estimarea meritului ei pe baza unei functii euristice si o lista de motive (justificari) pentru care acea sarcina trebuie executata. Strategia de control de tip agenda selcteaza la fiecare ciclu de executie sarcina cea mai interesanta, adica cu prioritatea cea mai mare. Aceasta este executata si, ca urmare, noi sarcini pot fi generate si introduse in agenda. Acest mecanism corespunde selectiei nodului celui mai promitator intr-o strategie de control de tip "best-first". Spre deosebire de strategia "best-first", o strategie de tip agenda permite evaluarea meritului combinat al diverselor cai catre un nod. De exemplu, in sistemul AM faptul ca mai multe cai diferite indica executia aceleiasi sarcini este important. Fiecare cale aduce un motiv in plus pentru cresterea prioritatii acelei sarcini. Agenda folosita de sistemul AM permite inregistrarea sarcinilor propuse impreuna cu motivele pentru care acestea au fost propuse. In plus, o strategie de tip agenda poate permite modificarea dinamica a modului de selectie a sarcinilor din agenda, deci utilizarea cunostintelor de control, exprimate pentru un sistem bazat pe reguli sub forma de metareguli. Un sistem bazat pe agenda urmareste, in mare, algoritmul urmator:

Algoritm: Strategia de control de tip "agenda"

1. Initializeaza agenda cu sarcina de executat

2. **repetă**

2.1. Selecteaza sarcina cu prioritate maxima, T

2.2. Executa sarcina T in limitele unor resurse de timp si spatiu determinate de importanta lui T

2.3. **pentru** fiecare noua sarcina T_i , $i = 1, N$, generata de T **executa**

2.3.1. **daca** T_i este deja in agenda

atunci

i. Fie T'_i copia lui T_i din agenda

ii. **daca** justificarile lui T'_i nu includ justificarea lui T_i

atunci adauga justificarea lui T_i justificarilor lui T'_i

iii. $T_i \leftarrow T'_i$

2.3.2. **altfel** adauga T_i si justificarea asociata in agenda

2.3.3. Calculeaza prioritatea sarcinii T_i pe baza justificarilor asociate

pina agenda satisface conditia de stare finala **sau**

agenda este vida

sfîrsit.

Observatii:

- O sarcina in agenda poate fi reprezentata in diverse forme. O sarcina poate fi o indicatie explicita a ceea ce trebuie sa se execute in continuare sau o indicatie a starii urmatoare de expandat.
- Fiecarei sarcini din agenda i se asociaza o cantitate limitata de resurse de timp si/sau spatiu, in functie de prioritatea sarcinii.
- Nu toate justificarile sarcinilor au pondere egala. Uneori este necesar sa se asocieze fiecărei justificari o masura a importantei ei. Aceste masuri sint combinate in pasul 2.3.3 pentru a produce prioritatea asociata sarcinii.

O problema importanta care apare in sistemele bazate pe agenda este aceea a determinarii sarcinii cu prioritate maxima in fiecare ciclu de executie. O posibilitate este aceea de a mentine tot timpul agenda sortata dupa prioritatea sarcinilor si de a introduce fiecare noua sarcina la locul potrivit. Daca prioritatea unei sarcini se schimba, agenda trebuie sortata din nou. Aceasta abordare poate determina insa un consum mare de timp pentru mentinerea agendei sortate in permanenta. O strategie putin modificata poate determina, din cind in cind, omiterea executiei unor sarcini cu prioritate mare, dar reduce timpul de gestiune a agendei. In aceasta abordare, la generarea unei noi sarcini se calculeaza prioritatea acesteia si se compara aceasta prioritate numai cu prioritatile primelor citeva sarcini din agenda, de obicei un numar de 5÷10. Daca prioritatea noii sarcini are o valoare cuprinsa in limitele prioritatilor acestor citeva sarcini, se introduce sarcina noua la locul potrivit din agenda. In caz contrar, se introduce noua sarcina la sfîrsitul agendei sau, daca ea exista deja in agenda, pozitia ei nu se modifica. Din timp in timp se executa o sortare a intregii agende.

Se observa ca mecanismul de agenda ofera o modalitate avantajoasa de focalizare a atentiei pentru sistemele in care functia de evaluare a meritului este complexa. Gestiunea sarcinilor in agenda, insa, este consumatoare de timp. Acest lucru ridica problema granularitatii impartirii problemei in sarcini. Daca dimensiunea sarcinilor este mica, atunci o sarcina (actiune) executata fie va fi cea mai buna, fie se va descoperi repede ca nu este cea mai buna. In acest caz, penalizarea este timpul foarte mare de gestiune a sarcinilor. Daca dimensiunea sarcinilor este mare, se poate consuma timp pentru executia unor sarcini care aparent sint cele mai bune dar ulterior se dovedesc a nu fi asa. In schimb, timpul de gestiune al agendei este mult mai mic. Alegerea dimensiunii optime a sarcinilor depinde de problema.

Exista domenii in care modelul agenda este recomandat. Acest model ofera posibilitatea integrarii intr-un program a unor surse de cunostinte diverse deoarece fiecare sursa de cunostinte nu face decit sa adauge o sarcina in agenda. Din aceasta cauza modelul tabla este foarte potrivit pentru implementarea sistemelor de cunostinte distribuite, asa cum se va prezenta in Capitolul 8. Exista insa probleme pentru care un mecanism de agenda nu este potrivit, de exemplu probleme in care este inacceptabila o comutare a atentiei intre o actiune si alta. Mai precis, modelul agenda este potrivit pentru sistemele de rationament monoton si mai putin potrivit pentru cele de rationament nemonoton.

4.4 Exerciții și probleme

1. Fie următoarea bază de cunoștințe:

R1: **dacă** A și F R2: **dacă** C
 atunci M **atunci** B
R3: **dacă** M R4: **dacă** A și E
 atunci N **atunci** C
R5: **dacă** A și E și F R6: **dacă** A și B
 atunci C **atunci** N

- (a) Să se indice funcționarea unui sistem cu înlănțuire înapoi a regulilor pentru această bază de cunoștințe în cazul determinării scopului N cu faptele inițiale A,E,F. Se vor considera numai stările distincte ale memoriei de lucru.
 - (b) Să se indice funcționarea unui sistem cu înlănțuire înapoi a regulilor pentru această bază de cunoștințe în cazul determinării scopului N.
 - (c) Care din direcțiile de aplicare a regulilor este mai potrivită pentru această bază de cunoștințe? Justificare.
2. Să se scrie setul complet de reguli de producție care descrie problema împachetării obiectelor pentru expoziție (Secțiunea 4.3.1) și să se traseze conținutul memoriei de lucru pînă la sfîrșitul rezolvării problemei.
3. Se considera problema cu lupul, capra și verza definită în Secțiunea 2.4. Să se definească un set de reguli de producție pentru această problemă. Care este direcția de aplicare a regulilor recomandată în rezolvarea acestei probleme?
4. Se considera următoarea problemă a robotului Robo. Robo cumpără diverse obiecte într-o băcanie și trebuie să le transporte acasă în pungi de plastic. Fără a fi interesat într-o soluție optimă, Robo dorește să respecte anumite reguli de introducere a obiectelor în pungi: sticlele de Coca Cola trebuie puse primele în pungi, deoarece sînt cele mai grele; o pungă nu poate conține mai mult de patru sticle de Coca Cola; dacă cumpără înghețată trebuie cumpărată și o pungă suplimentară pentru a ambala înghețată separat în acea pungă înainte de a o introduce în celelalte pungi; ouăle și pizza trebuie puse peste sticlele de Coca Cola. În plus, dacă lista de cumpărături specifică pizza dar nu specifică Coca Cola, Robo trebuie să adauge Coca Cola la lista de cumpărături, cite o sticlă pentru fiecare pizza.
- Să se descrie bază de cunoștințe pentru rezolvarea acestei probleme într-un sistem bazat pe reguli de producție. Să se specifice strategia utilizată de sistem și să se traseze funcționarea sistemului pentru o listă de obiecte de băcanie dată.

5. Considerind exemplul deciziei gastronomice prezentat in Sectiunea 4.3.2 sa se explice functionarea sistemului pentru cazul unui meniu in care

componenta-meniu = curcan

sos-meniu = sos-tomat

si sa se indice care este vinul (vinurile) recomandate de sistem pentru acest meniu.

6. Sa se scrie un program care implementeaza un interpretor de reguli de productie pentru reguli care nu contin variabile. Interpretorul utilizeaza inlantuirea inapoi a regulilor si alege intotdeauna prima regula posibil de aplicat pentru a fi executata. Sa se extinda programul astfel incit acesta sa detecteze attributele de tip date primare, attribute pentru care se solicita utilizatorului o valoare; apoi, daca acesta nu cunoaste raspunsul, se incearca aplicarea regulilor din sistem pentru deducerea valorii atributului.

Capitolul 5

Cunostinte incerte si rationament statistic

Modelele de reprezentare a cunostintelor prezentate pina acum se refera la cunostinte sigure, complete si consistente. In activitatea lor, oamenii sint insa capabili sa rezolve probleme si pe baza cunostintelor incerte sau contradictorii. Din acest motiv, cercetatorii in inteligenta artificiala au propus metode de reprezentare a ignorantei in sistemele bazate pe cunostinte. Aceste metode se pot imparti in doua mari categorii:

- Reprezentarea cunostintelor este extinsa prin asocierea unei masuri numerice a certitudinii (incertitudinii) diverselor entitati din baza de cunostinte. Sistemul trebuie sa fie capabil sa rationeze cu aceasta reprezentare, tipul de rationament efectuat numindu-se *rationament incert* sau *rationament statistic*.
- Axiomele si/sau regulile de inferenta din sistemul bazat pe cunostinte sint extinse astfel incit sa permita rationamentul bazat pe cunostinte incomplete si contradictorii. Acest tip de rationament se numeste *rationament nemonoton* si este o generalizare a rationamentului monoton din logica cu predicate de ordinul I.

Rationamentul statistic este subiectul acestui capitol. Reprezentarea cunostintelor incerte are asociate metode de inferenta specifice care modeleaza un rationament ce propaga incertitudinea de la date si ipoteze la concluzii. Aceste metode de reprezentare a cunostintelor pot fi folosite in rezolvarea problemelor ce implica date nesigure, vagi, incomplete sau chiar inconsistente. O astfel de categorie de probleme este, de exemplu, domeniul diagnosticarii medicale. Pentru o perspectiva ampla asupra rationamentului incert se poate consulta Kruse s.a.[1991]. O prezentare a rationamentului nemonoton poate fi gasita in Patterson [1990] si in Rich si Knight [1991].

5.1 Modelul probabilistic Bayesian

Metoda Bayesiană de calcul probabilistic a fost introdusa de preotul Thomas Bayes in secolul al XVIII-lea. Aceasta forma de rationament se bazeaza pe utilizarea probabilitatilor conditionate ale unor evenimente specifice in prezenta producerii unor alte evenimente. In teoria probabilitatilor, notiunea de *eveniment* este o notiune primara; evenimentele se considera numai din punctul de vedere al producerii sau al neproducerii lor in decursul unui experiment. *Evenimentul contrar* unui eveniment A, notat cu $\sim A$, este evenimentul care se produce atunci si numai atunci cind nu se produce evenimentul A.

Definitie. *Probabilitatea unui eveniment incert A este masura gradului de plauzibilitate al producerii acelui eveniment. Multimea tuturor evenimentelor posibile se numeste cimp de evenimente sau spatiu de esantioane, notat in continuare cu S.*

Definitie. *O masura a probabilitatii unui eveniment A este o functie $P : S \rightarrow R$ care pune in corespondenta orice eveniment $e_1, e_2, \dots, e_n$ din S cu numere reale si care satisface urmatoarele axiome ale teoriei probabilitatii:*

- (1) $0 \leq P(A) \leq 1$ pentru orice eveniment $A \subseteq S$
- (2) $P(S) = 1$
- (3) Daca $e_i \cap e_j = \emptyset$, pentru $\forall i \neq j, i, j = 1, n$, i.e. $e_i, i = 1, n$ sint evenimente mutual exclusive, atunci $P(e_1 \cup e_2 \cup \dots \cup e_n) = P(e_1) + P(e_2) + \dots + P(e_n)$

Definitie. Pentru doua evenimente h si e , cu probabilitatea $P(e) > 0$, *probabilitatea conditionata* a evenimentului h in conditiile producerii evenimentului e , este definita prin urmatoarea formula

$$P(h|e) = \frac{P(h \& e)}{P(e)} \quad (1)$$

Probabilitatea conditionata de producere a evenimentului e in conditiile producerii evenimentului h se defineste simetric prin formula

$$P(e|h) = \frac{P(h \& e)}{P(h)} \quad (2)$$

Din ecuatiile (1) si (2) rezulta una dintre regulile modelului Bayesian, si anume

$$P(h|e) = \frac{P(e|h) \cdot P(h)}{P(e)} \quad (3)$$

Considerind doua evenimente A si $\sim A$ care sint mutual exclusive, i.e. $A \cap \sim A = \emptyset$, si exhaustive, i.e. $A \cup \sim A = S$, probabilitatea de aparitie a unui eveniment B se poate exprima astfel:

$$P(B) = P(B \& A) + P(B \& \sim A) = P(B|A) \cdot P(A) + P(B|\sim A) \cdot P(\sim A) \quad (4)$$

Utilizind aceasta formula, ecuatia (3) poate fi rescrisa obtinindu-se urmatoarea formula pentru probabilitatea conditionata de aparitie a evenimentului h in conditiile producerii evenimentului e .

$$P(h|e) = \frac{P(e|h) \cdot P(h)}{P(e|h) \cdot P(h) + P(e|\sim h) \cdot P(\sim h)} \quad (5)$$

Ecuatia (5) poate fi generalizata pentru un numar arbitrar de evenimente $h_i, i = 1, k$, independente si mutual exclusive, in conditiile producerii evenimentului e , astfel:

$$P(e) = \sum_{i=1}^k P(e \& h_i) = \sum_{i=1}^k P(e|h_i) \cdot P(h_i) \quad (6)$$

si deci

$$P(h_i|e) = \frac{P(e|h_i) \cdot P(h_i)}{\sum_{j=1}^k P(e|h_j) \cdot P(h_j)}, \quad i = 1, k \quad (7)$$

Evenimentele h_i pot fi vazute ca *ipoteze probabile*, numite si *ipoteze statistice*, in conditiile existentei probei e . Probabilitatile conditionate ale ipotezelor h_i in conditiile existentei probei e pot fi utilizate in modelarea rationamentului incert pentru a selecta ipoteza cea mai probabila in conditiile unei probe observate. In cazul in care exista surse multiple de probe, deci $e = \{e_1, e_2, \dots, e_n\}$, formula (7) se defineste ca mai jos, obtinindu-se *teorema lui Bayes*:

$$P(h_i|e_1, e_2, \dots, e_n) = \frac{P(e_1, e_2, \dots, e_n|h_i) \cdot P(h_i)}{\sum_{j=1}^k P(e_1, e_2, \dots, e_n|h_j) \cdot P(h_j)}, \quad i = 1, k \quad (8)$$

Considerind exemplul diagnosticarii medicale, selectarea unei ipoteze h_i dintr-o multime de ipoteze pe baza unei multimi de probe observate $e = \{e_1, e_2, \dots, e_n\}$ poate fi vazuta ca selectarea

unui diagnostic h_i pe baza probelor clinice $e_1, e_2, \dots, e_n$. In aceasta interpretare, evenimentele si probabilitatile lor conditionate au urmatoarea semnificatie:

- $e = \{e_1, e_2, \dots, e_n\}$ este multimea probelor clinice considerate
- h_i este al i -lea diagnostic considerat ($i = 1, k$)
- $P(h_i)$ este probabilitatea ca pacientului sa i potriveasca diagnosticul h_i
- $P(h_i|e)$ este probabilitatea ca pacientul sa aiba diagnosticul h_i pe baza probelor clinice e
- $P(e|h_i)$ este probabilitatea ca sa existe toate probele clinice e daca diagnosticul h_i este adevarat, deci probabilitatea ca pacientul sa aiba totalitatea simptomelor e (simptomatologie completa) daca i se pune diagnosticul h_i .

Teorema lui Bayes data de formula (8) ofera o modalitate de calcul al diagnosticului probabil al unui pacient in conditiile cunoasterii probelor clinice e . In cazul in care exista mai multe ipoteze plauzibile si mai multe surse de probe, formula (8) poate duce la calcule extrem de complicate. Daca se presupune ca $e = \{e_1, e_2, \dots, e_n\}$ sint probe independente, calculul probabilitatii $P(e_1, e_2, \dots, e_n|h_j)$ se poate face ca mai jos, ducind la o simplificare a formulei (8).

$$P(e|h_j) = P(e_1, e_2, \dots, e_n|h_j) = P(e_1|h_j) \cdot P(e_2|h_j) \cdot \dots \cdot P(e_n|h_j), j = 1, k \quad (9)$$

In general, in multe probleme reale probele sint acumulate pe rind. De exemplu, in diagnosticarea medicala este posibil ca probele clinice sa apara la diverse momente de timp. Din aceasta cauza sistemele care folosesc modelul Bayesian utilizeaza o varianta modificata de calcul al probabilitatii care reflecta obtinerea incrementala de probe. Daca $e_1 = \{e_{11}, e_{12}, \dots, e_{1k}\}$ sint probele deja observate, s_1 este o noua proba si $e = e_1 \cup s_1$ atunci probabilitatea ipotezei h_i in conditiile existentei probelor e se poate calcula pe baza probabilitatii aceleasi ipoteze (presupus a fi deja calculata) in conditiile existentei probelor e_1 , prin cumularea efectului lui s_1 , astfel:

$$P(h_i|e) = \frac{P(s_1|h_i \& e_1) \cdot P(h_i|e_1)}{\sum_{j=1}^k P(s_1|h_j \& e_1) \cdot P(h_j|e_1)}, i = 1, k, e_1 = \{e_{11}, e_{12}, \dots, e_{1n}\} \text{ si } e = e_1 \cup s_1 \quad (10)$$

Modelul probabilistic Bayesian a fost aplicat in diverse domenii cum ar fi diagnosticarea medicala si cercetarile geologice. Sistemul PROSPECTOR [Duda, s.a., 1979], sistem expert in domeniul geologiei, este unul din marile succese ale sistemelor bazate pe cunostinte aplicate. Sistemul utilizeaza modelul Bayesian si a fost folosit cu succes in localizarea unor zacaminte de minerale, cum ar fi cupru si uraniu. Ideea de baza a abordarii probabilistice in sistemul PROSPECTOR este urmatoarea. Se doreste examinarea probelor geologice ale unui anumit loc pentru a determina daca in acest loc este posibil sa se gaseasca mineralul dorit. Daca se cunosc probabilitatile *a priori* de gasire a diverselor minerale si probabilitatile de gasire a unui mineral in functie de anumite caracteristici fizice, atunci teorema lui Bayes poate calcula probabilitatea de descoperire a unui zacamint intr-un anumit loc, pe baza probelor geologice acumulate.

Modelul probabilistic Bayesian are o serie de dezavantaje, atit din punct de vedere al eficientei de calcul cit si din punct de vedere al puterii de expresivitate a reprezentarii cunostintelor incerte. Dezavantajele si limitarile semnificative ale abordarii bayesiene sint:

- Programele care folosesc un astfel de model necesita o cantitate mare de date statistice care sint greu de adunat si calculat. Complexitatea timp este exponentiala in raport cu numarul de probe si ipoteze. Modelul presupune independenta ipotezelor, pentru ca

formula sa fie practic aplicabila pe cazuri reale ce contin foarte multe date. De multe ori, independenta ipotezelor este greu sau imposibil de realizat.

- Probabilitatile sint descrise printr-o valoare numerica unica. Acest lucru poate fi o simplificare a modelului de gindire umana. De cele mai multe ori, expertii au dificultati in a estima cu precizie probabilitatea unei ipoteze printr-o singura valoare, avind tendinta de a specifica un interval de probabilitate.
- Modelul Bayesian nu poate discerne intre ignoranta si incertitudine. De exemplu, fie trei organizatii teroriste A, B si C care sint suspecte de un atac asupra unei institutii publice. Exista anumite probe care sustin ipoteza vinovatiei organizatiei C cu probabilitatea 0.8. Cu toate acestea, fara alte probe asupra vinovatiei organizatiilor A si B, nu se poate spune ca A si B sint vinovate, fiecare cu probabilitatea 0.1.
- Modelul Bayesian considera increderea intr-o ipoteza si neincrederea in negarea ei ca doua functii opuse, i.e.

$$P(A) = X \rightarrow P(\sim A) = 1 - X$$

Abordarea conform careia probele in favoarea unei ipoteze trebuie considerate probe in favoarea negarii acelei ipoteze este in multe cazuri falsa.

In plus, interpretarea probabilitatii unei ipoteze in conditiile existentei unei probe ca o forma de confirmare a ipotezei pe baza acestei probe poate duce la rezultate surprinzatoare. In acest sens se poate cita *paradoxul lui Carl Hempel* care consta in urmatorul exemplu. Fie:

- (a) $C[h|e]$ - confirmarea ipotezei h pe baza probei e
- (b) h_1 = ipoteza "Toti corbii sint negrii"
- (c) h_2 = ipoteza "Orice obiect care nu este negru nu este corb"
- (d) e = proba "Vaza este verde"

Evident, h_1 este logic echivalent cu h_2 . Daca s-ar face o analogie a confirmarii unei ipoteze pe baza unei probe cu probabilitatile conditionate s-ar putea stabili egalitatea $C[h_1|e] = C[h_2|e]$, pentru orice proba e . Cu toate acestea, este total neintuitiv sa se spuna ca observarea probei e , "Vaza este verde", confirma ipoteza h_1 , "Toti corbii sint negrii". In anumite domenii, cum ar fi medicina, in care semnificatia certitudinii unei ipoteze pe baza probelor este mai mult o confirmare a ipotezei decit o probabilitate de aparitie, este necesar sa se introduca o diferenta intre increderea si neincrederea intr-o ipoteza. Modelele prezentate in continuare incearca sa elimine limitările modelului probabilistic Bayesian.

5.2 Modelul factorilor de certitudine din sistemul MYCIN

Modelul factorilor de certitudine reprezinta o abordare practica si eficienta a rationamentului incert. El a fost dezvoltat in sistemul expert bazat pe reguli de productie MYCIN [Buchanan, Shortliffe, 1984], sistem de diagnosticare si recomandare a terapiei in infectiile bacteriene ale singelui. Factorii de certitudine asociati cunostintelor (reguli si fapte) sint considerati o abordare euristica a reprezentarii cunostintelor incerte deoarece nu se bazeaza pe o teorie perfect riguroasa. In schimb, ei elimina o parte din limitările modelului Bayesian cum ar fi complexitatea calculului

probabilitatilor, nediscernarea intre incertitudine si ignoranta, si inconsistentele determinate de interpretarea probabilitatilor drept confirmari ale ipotezelor.

5.2.1 Masurile incertitudinii

In sistemul MYCIN se folosesc doua functii probabilistice pentru a modela increderea si neincrederea intr-o ipoteza: *functia de masura a increderii*, notata MB, si *functia de masura a neincrederii*, notata MD. Fiind data ipoteza h si proba e , interpretarea acestor functii este:

- $MB[h,e]$ reprezinta masura cresterii increderii in ipoteza h pe baza probei e ,
- $MD[h,e]$ reprezinta masura cresterii neincrederii in ipoteza h pe baza probei e .

Proba e poate fi o proba observata dar si o alta ipoteza care a fost sau trebuie confirmata. Astfel, se poate scrie $MB[h_1, h_2]$ pentru a indica masura cresterii increderii in ipoteza h_1 in conditiile in care ipoteza h_2 este adevarata. Pentru a ilustra semnificatia acestor functii in contextul sistemului MYCIN, se considera $e = \text{"organismul este coc gram-pozitiv care creste in lanturi"}$ si $h = \text{"organismul este streptococ"}$. Daca expertul indica $MB[h,e] = 0.7$, acest lucru semnifica faptul ca numarul 0.7 reflecta cresterea increderii expertului in adevarul ipotezei h stiind ca proba e este adevarata.

Facind legatura cu teoria probabilitatilor, functiile de incredere si neincredere pot fi definite dupa cum urmeaza. Fie:

e - o data observata, o proba sau ipoteza (inferata),

$P(h)$ - probabilitatea *a priori* ca ipoteza h sa fie adevarata,

$P(h|e)$ - probabilitatea ca ipoteza h sa fie adevarata pe baza probei e ,

$1 - P(h)$ - estimarea neincrederii in adevarul ipotezei h .

Daca $P(h|e) > P(h)$ atunci observarea probei e creste increderea in ipoteza h , iar daca $P(h|e) < P(h)$, atunci observarea probei e scade increderea in ipoteza h si creste neincrederea in adevarul lui h . Aceste conditii pot fi exprimate sub urmatoarea forma:

$$\frac{P(h|e) - P(h)}{1 - P(h)} \quad (11)$$

$$\frac{P(h) - P(h|e)}{P(h)} \quad (12)$$

Formulele (11) si (12) reprezinta masura cresterii increderii, respectiv a neincrederii, in ipoteza h pe baza probei e , deci $MB[h,e]$ si $MD[h,e]$. In acest context, functiile MB si MD se definesc in functie de probabilitatile conditionate si probabilitatile *a priori*, astfel:

$$MB[h,e] = \begin{cases} 1 & \text{daca } P(h) = 1 \\ \frac{\max(P(h|e), P(h)) - P(h)}{\max(0, 1) - P(h)} & \text{in caz contrar} \end{cases} \quad (13)$$

$$MD[h,e] = \begin{cases} 0 & \text{daca } P(h) = 0 \\ \frac{\min(P(h|e), P(h)) - P(h)}{\min(0, 1) - P(h)} & \text{in caz contrar} \end{cases} \quad (14)$$

Observatie. In formulele de mai sus se foloseste $\max(0,1)$ in loc de 1 si $\min(0,1)$ in loc de 0 pentru a pune in evidenta simetria relatiilor. Formulele (13) si (14) sint, evident, echivalente cu formulele (11) si (12).

Se introduce si o a treia masura a incertitudinii, numita *factorul* (coeficientul) *de certitudine*, notat CF si definit astfel

$$CF[h,e] = MB[h,e] - MD[h,e] \quad (15)$$

Factorul de certitudine este o modalitate de a combina gradele de incredere si neincredere intr-o singura masura. Un astfel de numar este util pentru a putea compara puterea de semnificatie a diverselor ipoteze competitive. Urmatoarele caracteristici ale celor trei masuri de certitudine ajuta la clarificarea semnificatiei lor.

(a) *Domeniul de valori*

$$0 \leq MB[h,e] \leq 1$$

$$0 \leq MD[h,e] \leq 1$$

$$-1 \leq CF[h,e] \leq 1$$

(b) *Ipoteze mutual exclusive*

Daca se stie ca h este o ipoteza sigura, i.e. $P(h|e) = 1$, atunci

$$MB[h,e] = \frac{1 - P(h)}{1 - P(h)} = 1$$

$$MD[h,e] = 0$$

$$CF[h,e] = 1$$

Daca se stie ca negatia lui h este sigura, i.e. $P(\sim h|e) = 1$, atunci

$$MB[h,e] = 0$$

$$MD[h,e] = \frac{0 - P(h)}{0 - P(h)} = 1$$

$$CF[h,e] = -1$$

(c) *Lipsa probelor*

$MB[h,e] = 0$ daca h nu este confirmat de e, i.e. e si h sint independente sau e infirma h.

$MD[h,e] = 0$ daca h nu este infirmat de e, i.e. e si h sint independente sau e confirma h.

$CF[h,e] = 0$ daca e nici nu confirma nici nu infirma h, i.e. e si h sint independente.

In sistemul MYCIN, functiile de masura a increderii si a neincrederii sint asociate faptelor, reprezentate sub forma de triplete atribut-obiect-valoare, iar factorii de certitudine sint asociati regulilor. Factorul de certitudine asociat unei reguli reprezinta increderea in concluzia acelei reguli presupunind premisa cunoscuta cu certitudine, i.e. $MB = 1$ si $MD = 0$ pentru premisa.

Exemplu. O regula in sistemul MYCIN, exprimata intr-un limbaj asemanator celui din MYCIN, este

- daca**
- (1) tipul organismului este gram-pozitiv, si
 - (2) morfologia organismului este coc, si
 - (3) conformatia cresterii organismului este lant

atunci exista o incredere puternica (0.7) ca identitatea organismului este streptococ.

Exemple de fapte in sistemul MYCIN sint urmatoarele:

(identitate organism-1 pseudomonas 0.8)

(identitate organism-2 e.coli 0.15)

(loc cultura-2 git 1.0)

5.2.2 Functii de combinare a incertitudinii

Odata asociate masuri ale incertitudinii cunostintelor din sistem, realizarea rationamentului incert pentru rezolvarea problemei necesita stabilirea unor inferente incerte, deci modalitati de combinare a increderii, respectiv neincrederii. In sistemul MYCIN s-au definit o serie de functii pentru combinarea celor doua functii de baza, MB si MD, functii de combinare care servesc la calculul increderii si neincrederii in diversele ipoteze pe parcursul stabilirii diagnosticului. Aceste functii sint prezentate in continuare.

(1) *Probe adunate incremental.*

Aceeasi valoare de atribut, h , este obtinuta pe doua cai de deductie distincte, cu doua perechi diferite de valori pentru functiile MB si MD: $MB[h, s_1]$ si $MB[h, s_2]$, respectiv $MD[h, s_1]$ si $MD[h, s_2]$. Cele doua cai de deductie distincte, corespunzatoare probelor sau ipotezelor s_1 si s_2 pot fi ramuri diferite ale arborelui de cautare generat prin aplicarea regulilor sau probe indicate explicit sistemului de medic. Masurile increderii si neincrederii rezultate prin cumulara acestor valori, pentru valoarea de atribut h , sint:

$$MB[h, s_1 \& s_2] = \begin{cases} 1 & \text{daca } MD[h, s_1 \& s_2] = 1 \\ MB[h, s_1] + MB[h, s_2] - MB[h, s_1] \cdot MB[h, s_2] & \text{in caz contrar} \end{cases} \quad (16)$$

$$MD[h, s_1 \& s_2] = \begin{cases} 1 & \text{daca } MB[h, s_1 \& s_2] = 1 \\ MD[h, s_1] + MD[h, s_2] - MD[h, s_1] \cdot MD[h, s_2] & \text{in caz contrar} \end{cases} \quad (17)$$

Pe baza valorilor MB si MD se poate calcula factorul de certitudine asociat unei ipoteze. Se observa ca daca mai multe probe sustin o aceeași ipoteza, valoarea absoluta a factorului de certitudine va crește. Daca probele sustin ipoteze diferite, valoarea absoluta a factorului de certitudine asociat unei ipoteze va scădea.

Exemplu. Se presupune ca pe baza observatiilor initiale, fie acestea s_1 , s-a obtinut o confirmare a increderii in ipoteza h cu $MB[h, s_1] = 0.3$. Atunci $MD[h, s_1] = 0$ si $CF[h, s_1] = 0.3$. Se face apoi o a doua observatie s_2 , care confirma deasemenea h , cu $MB[h, s_2] = 0.2$. In acest caz:

$$MB[h, s_1 \& s_2] = 0.3 + 0.2 - 0.2 \cdot 0.7 = 0.36$$

$$MD[h, s_1 \& s_2] = 0.0$$

$$CF[h, s_1 \& s_2] = 0.36$$

Se observa din acest exemplu cum probe acumulate incremental in favoarea unei ipoteze pot duce la cresterea factorului de certitudine al ipotezei.

(2) *Conjunctie de ipoteze.*

Aceasta functie se aplica pentru calculul masurilor increderii si a neincrederii asociate unei premise de regula care contine mai multe conditii. Se considera modul de calcul pentru cazul a doua conditii in premisa regulii, extinderea la mai multe ipoteze in premisa facindu-se foarte simplu. Fie regula

daca conditie₁

si conditie₂

atunci concluzie

unde $conditie_1$ are asociati $MB[h_1,e]$ si $MD[h_1,e]$, si $conditie_2$ are asociati $MB[h_2,e]$ si $MD[h_2,e]$. Valorile MB si MD asociate unei conditii din premisa regulii se obtin pe baza valorilor corespunzatoare tripletelor atribut-obiect-valoare cu care a identificat conditia. In acest caz masurile increderii si neincrederii asociate intregii premise, $MB[h_1 \& h_2, e]$ si $MD[h_1 \& h_2, e]$, se calculeaza astfel:

$$MB[h_1 \& h_2, e] = \min(MB[h_1, e], MB[h_2, e]) \quad (18)$$

$$MD[h_1 \& h_2, e] = \max(MD[h_1, e], MD[h_2, e]) \quad (19)$$

(3) *Combinarea increderii.*

Aceasta functie se foloseste in cazul aplicarii uneia sau a mai multor reguli. Printr-o astfel de inlantuire o valoare incerta este dedusa pe baza unei reguli care are drept conditie de intrare alte valori incerte, deduse eventual prin aplicarea altor reguli. Functia permite calculul factorului de certitudine asociat valorii deduse pe baza aplicarii unei reguli care refera valoarea in concluzie, tinind cont de masura increderii si masura neincrederii asociate premisei regulii. Daca increderea intr-o ipoteza s este data de un coeficient de certitudine CF pe baza unor probe anterioare e si daca $MB'[h,s]$ si $MD'[h,s]$ sint masurile increderii, respectiv neincrederii in h in cazul in care s este sigura, atunci valorile increderii si neincrederii in h sint date de relatiile:

$$MB[h,s] = MB'[h,s] \cdot \max(0, CF[s,e]) \quad (20)$$

$$MD[h,s] = MD'[h,s] \cdot \max(0, CF[s,e]) \quad (21)$$

La nivelul unei reguli, interpretarea acestei functii este urmatoarea. Fie o regula de forma

daca premisa

atunci concluzie

pentru care premisa are asociate valorile MB' si MD' (calculate eventual anterior prin aplicarea functiilor (2) si (1)) si reprezinta ipoteza s care s-a calculat pe baza probelor e. Concluzia regulii refera ipoteza h ca valoare a atributului din concluzie. Daca corelatia concluzie-premisa a regulii ar fi sigura, deci $CF[s,e] = 1$, atunci $MB[h,s] = MB'[h,s]$ si $MD[h,s] = MD'[h,s]$. In cazul in care $CF[s,e] < 1$ se aplica formulele definite mai sus.

In variantele mai noi ale sistemului expert MYCIN si in sistemul independent de domeniu EMYCIN [Bennett,Engelmore,1984;vanMelle,s.a.,1984] provenit din MYCIN, pe baza observarilor statistice, s-a modificat formula de calcul al factorului de certitudine astfel:

$$CF = \frac{MB - MD}{1 - \min(MB, MD)} \quad (22)$$

Modelul de rationament incert utilizat de sistemul MYCIN s-a dovedit destul de util in rezolvarea problemelor practice ale diagnosticarii medicale (stabilirea tipului de infectie bacteriana pe baza simptomelor si probelor de laborator ale unui pacient) dar a fost deseori criticat pentru lipsa de rigurozitate a modelului matematic si pentru anumite neconcordanțe pe care le introduce.

Modelul coeficientilor de certitudine din MYCIN presupune ca ipotezele sustinute de probe sint independente. In continuare se considera un exemplu care arata ce se intimpla in cazul in care aceasta conditie este violata.

Fie urmatoarele fapte:

A: Aspensorul a functionat noaptea trecuta.

U: Iarba este uda dimineata.

P: Noaptea trecuta a plouat.

si urmatoarele doua reguli care leaga intre ele aceste fapte:

R1: **daca** aspersorul a functionat noaptea trecuta

atunci exista o incredere puternica (0.9) ca iarba este uda dimineata

R2: **daca** iarba este uda dimineata

atunci exista o incredere puternica (0.8) ca noaptea trecuta a plouat

Fiecare dintre aceste reguli, luata in parte, descrie o corelatie corecta. Sa analizam insa ce se intimpla daca regulile sint considerate impreuna. Folosind modelul sistemului MYCIN se obtine:

$MB[U, A] = 0.9$, deci aspersorul sugereaza iarba uda

$MB[P, U] = 0.8$

$MB[P, U] = 0.8 \cdot 0.9 = 0.72$, deci iarba uda sugereaza ploaie

Cu alte cuvinte, sistemul considera ca noaptea trecuta a plouat deoarece aspersorul a fost in functiune. Acest lucru se obtine desi, daca aspersorul a functionat, el este cel care a udat iarba si nu exista nici o proba in favoarea ipotezei "noaptea trecuta a plouat". Desi unul din marile avantaje ale sistemului MYCIN si a sistemelor bazate pe reguli in general, este acela de a permite modularitatea si tratarea relatiilor premisa-concluzie independent unele de altele, acest exemplu pune in evidenta un pericol al avantajului modularitatii unor astfel de sisteme. Cele doua reguli nu sint la fel din punct de vedere conceptual. Prima regula descrie o relatie cauzala (cauza→efect), pe cind cea de a doua descrie o relatie cauzala inversa (efect→cauza). Desi se pot deduce manifestari ale unui simptom pornind de la cauzele lui si se poate deduce o cauza pe baza simptomelor ei asociate, este important ca probele sa fie deduse numai intr-un fel, sau numai in altul. Pentru a ocoli aceasta problema, multe dintre sistemele bazate pe reguli fie utilizeaza numai un singur fel de reguli, fie partitioneaza regulile in doua clase si nu permit inferenta intre cele doua clase. In Sectiunea 5.4 se discuta retelele Bayesiene care reprezinta o solutie sistematica a acestei probleme.

5.3 Teoria Dempster-Shafer

Limitarile modelului probabilistic Bayesian si a modelului factorilor de certitudine din MYCIN au condus la investigarea unor abordari alternative a rationamentului statistic. Una dintre acestea este teoria matematica a probelor propusa de Arthur Dempster in anii '60 si extinsa de studentul lui, Glenn Shafer in 1976 [Gordon, Shortliffe, 1984; Kruse, s.a., 1991]. Teoria Dempster-Shafer modeleaza reducerea unei multimi de ipoteze competitive pe baza acumularii de probe, proces care caracterizeaza rationamentul medical si rationamentul incert in general. Teoria se bazeaza pe ideea asocierii de probabilitati tuturor submultimilor de ipoteze din universul problemei si nu numai ipotezelor individuale. In acest fel se poate reprezenta adecvat modul de rationament al unui expert care foloseste probele initiale pentru a forma o multime de ipoteze. Ulterior, pe baza aparitiei a noi probe, expertul reduce treptat aceasta multime de ipoteze pina la una sau mai multe ipoteze preferentiale. De exemplu, in procesul stabilirii identitatii unui organism care a produs infectia, o proba care indica prezenta unui organism gram-negativ reduce multimea de ipoteze a

tuturor organismelor de infectare posibile. Aceasta submultime redusa de ipoteze poate fi vazuta ca o noua ipoteza: organismul este unul din organismele gram-negative. O proba ca cea amintita mai sus nu aduce nici o informatie referitor la probabilitatea organismelor individuale din submultimea de organisme gram-negative. Modelul Bayesian ar atribui probabilitati egale tuturor organismelor din aceasta submultime dar, in acest fel, nu s-ar mai face distinctia intre incertitudine (lipsa cunostintelor) si probabilitatea egala a ipotezelor. Aceasta limitare este eliminata de teoria Dempster-Shafer prin asocierea de functii de incredere atit ipotezelor individuale cit si submultimilor de ipoteze, oferind astfel un model mai bun al procesului acumularii de probe si al rationamentului incert.

Mai multe probe acumulate pot reduce multimea initiala de ipoteze si pot modifica increderea in ipoteze prin combinarea functiilor de incredere conform regulilor stabilite de teoria Dempster-Shafer. La fel ca in modelul Bayesian si cel al coeficientilor de certitudine din MYCIN, regulile de combinare a increderii sint independente de ordinea de acumulare a probelor dar presupun ca ipotezele sustinute de probe sint mutual exclusive si exhaustive. De fapt, functiile de combinare a increderii din teoria Dempster-Shafer includ, drept cazuri particulare, functiile de combinare probabilistica din modelul Bayesian si modelul MYCIN.

O alta consecinta a functiilor de incredere din aceasta teorie este eliminarea restrictiei conform careia o probabilitate P asociata unei ipoteze implica asocierea probabilitatii $1-P$ negarii acestei ipoteze, i.e. $P(h) = 1 - P(\sim h)$. La fel ca si in modelul factorilor de certitudine din MYCIN, teoria Dempster-Shafer elimina aceasta restrictie. Increderea in fiecare din ipotezele multimii initiale nu trebuie sa aiba suma unitara deoarece se asociaza increderi si submultimilor de ipoteze.

O ipoteza are asociata, pe linga functia de incredere, si o plauzibilitate. Fiecare ipoteza este caracterizata de un interval de incredere, definit prin $[Incredere, Plauzibilitate]$. Increderea masoara taria cu care probele sustin o ipoteza (sau o multime de ipoteze) iar plauzibilitatea masoara cit de mult contribuie o proba in favoarea ipotezei contrare la aprecierea increderii in ipoteza. Astfel, daca se noteaza cu $Bel(h)$ increderea intr-o ipoteza, plauzibilitatea ipotezei h este definita de $Pl(h) = 1 - Bel(\sim h)$. Intervalul $[Incredere, Plauzibilitate]$ masoara nu numai increderea intr-o ipoteza sau intr-o multime de ipoteze, ci si cantitatea de informatie existenta.

Teoria Dempster-Shafer a fost utilizata in multe sisteme, printre care si o rescriere a sistemului MYCIN prin inlocuirea factorilor de certitudine cu intervale de incredere. S-au pus in evidenta multe caracteristici comune ale celor doua modele, teoria Dempster-Shafer avind insa avantajul unei fundamentari matematice riguroase. In continuare, se va descrie in detaliu modelul de rationament statistic propus de aceasta teorie.

5.3.1 Un exemplu de rationament incert

Fie patru organizatii teroriste suspecte de organizarea unui atac terorist: doua organizatii irakiene, D1 si D2, si doua organizatii fasciste, S1 si S2, pentru care exista o multime de ipoteze de vinovatie. Se presupune ca nu exista alta organizatie suspecta in afara celor patru organizatii. Diagnosticul de vinovatie a uneia din cele patru organizatii va fi reprezentat chiar de numele organizatiilor, S1, S2, D1 sau D2. In teoria Dempster-Shafer multimea de ipoteze posibile se numeste *cadru de selectare* si se noteaza cu θ . Ipotezele din θ trebuie sa fie mutual exclusive si

exhaustive. Pentru exemplul considerat cadrul de selectare este $\theta = \{S1, S2, D1, D2\}$ si ipotezele din multime satisfac conditiile enuntate anterior.

Se stie ca exista o proba care poate conduce la ipoteza de vinovatie a organizatiilor teroriste fasciste, corespunzatoare multimii $\{S1, S2\}$ din θ . O alta proba poate sa indice excluderea ipotezei de vinovatie a lui S1 intr-o anumita masura, ceea ce este echivalent cu o proba care confirma negarea ipotezei S1, i.e. $\sim S1$. Aceasta a doua proba corespunde ipotezei $S2 \vee D1 \vee D2$, adica submultimii $\{S1, D2, D2\}$ din θ . Aceasta submultime poate fi considerata la rindul ei o ipoteza, deci o submultime a lui θ poate da nastere la o noua ipoteza.

Fie 2^θ multimea partilor lui θ . Daca θ are n elemente atunci multimea 2^θ are 2^n elemente. Multimea vida apartine multimii 2^θ , $\emptyset \in 2^\theta$, si corespunde unei ipoteze despre care se stie ca este falsa, deoarece s-au presupus ipotezele exhaustive. O reprezentare grafica a multimii partilor lui $\theta = \{S1, S2, D1, D2\}$ este cea indicata in Figura 5.1.

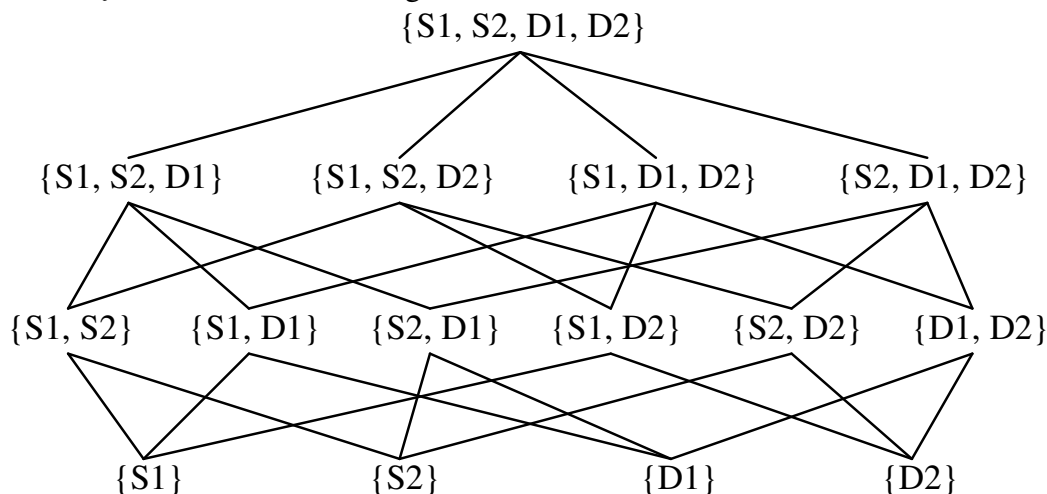


Figura 5.1 Cadrul de selectare si submultimile de ipoteze in problema atacului terorist

Pentru un anumit domeniu, numai un subset al multimii 2^θ este de interes in luarea deciziilor, deci graful submultimilor de ipoteze poate fi redus la o *ierarhie de relatii* care prezinta interes din punct de vedere al caracteristicilor domeniului problemei. In exemplul considerat, daca se stie ca probele existente pot indica fie numai organizatii fasciste, fie numai organizatii irakiene, graful din Figura 5.1 se reduce la ierarhia de relatii din Figura 5.2. In general, multimea de submultimi de ipoteze are mai putine elemente de interes daca se tine cont de caracteristicile problemei.

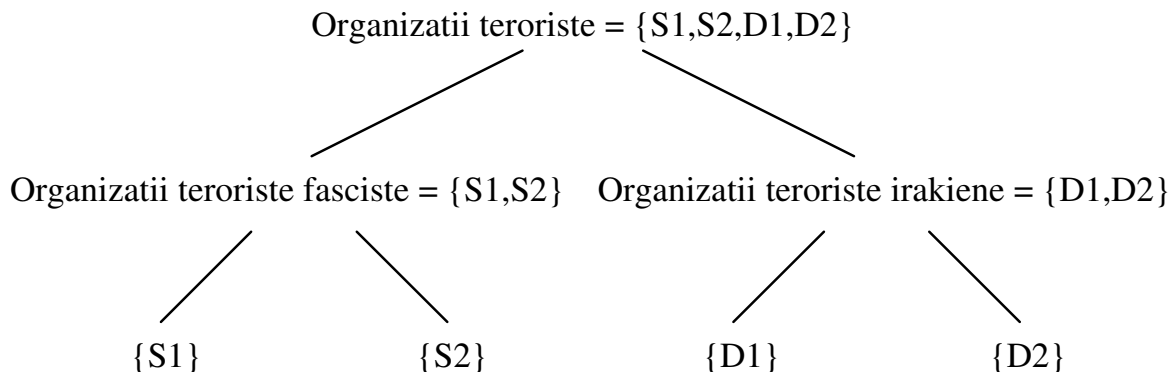


Figura 5.2 Multimile de ipoteze de interes in problema atacului terorist

5.3.2 Functii de incredere

Teoria Dempster-Shafer foloseste o valoare reala in intervalul $[0,1]$ pentru a indica increderea intr-o ipoteza sau intr-o multime de ipoteze pe baza unei probe date, i.e. gradul in care proba sustine ipoteza. O proba impotriva ipotezei este prezentata ca o proba in favoarea negarii ipotezei. In acest fel modelul Dempster-Shafer evita calculul cu numere negative necesar, de exemplu, in modelul MYCIN.

Increderea intr-o ipoteza pe baza unor probe date este reprezentata printr-o functie numita atribuire probabilistica de baza. O atribuire probabilistica de baza este o generalizare a functiei de densitate de probabilitate clasica. Functia probabilistica in modelul Bayesian asociaza o valoare reala in intervalul $[0,1]$ fiecarei multimii cu un singur element din θ astfel incit suma acestor valori sa fie 1, iar multimii vide ϕ i se asociaza valoarea 0. Spre deosebire de aceasta functie, atribuirea probabilistica de baza se defineste dupa cum urmeaza.

Definitie. Functia de *atribuire probabilistica de baza* in teoria Dempster-Shafer, notata cu m , se defineste astfel:

- (1) $m(A): 2^\theta \rightarrow [0,1]$ pentru orice $A \subseteq \theta$,
- (2) $\sum_{A \subseteq \theta} m(A) = 1$,
- (3) $m(\phi) = 0$

Atribuirea probabilistica de baza m defineste o distributie de probabilitate pe multimea 2^θ . Functia $m(A)$ reprezinta masura increderii asociata submultimii de ipoteze $A \in 2^\theta$ si nu poate fi impartita intre elementele lui A , adica intre ipotezele din A . Daca exista o proba care sustine o submultime de ipoteze A si nici o proba pentru o alta submultime din θ , deci daca $m(A) = s$ si $m(X) = 0$ pentru orice $X \subseteq \theta$ cu $X \neq A$, atunci $m(\theta) = 1 - s$. Astfel valoarea $1 - s$ se asociaza multimii totale de ipoteze θ si nu negarii ipotezei, $\sim A$, ca in modelul Bayesian.

Exemple:

1. Considerind problema atacului terorist, se presupune ca nu exista probe pentru vinovatia nici unei organizatii. In acest caz, atribuirea probabilistica de baza se reprezinta astfel:

$$m(\theta) = m(\{S1, S2, D1, D2\}) = 1$$

$$m(X) = 0 \text{ pentru orice } X \subset \theta$$

In modelul Bayesian aceasta situatie s-ar fi exprimat asociind o valoare de probabilitate egala cu 0.25 fiecarei ipoteze din θ .

2. Se presupune existenta unei probe care sustine ipoteza vinovatiei organizatiilor teroriste irakiene cu gradul de incredere 0.6. Atunci reprezentarea atribuirii probabilistice de baza este:

$$m(\{S1, S2\}) = 0.6$$

$$m(\theta) = m(\{S1, S2, D1, D2\}) = 0.4$$

$$m(X) = 0 \text{ pentru orice alt } X \subset \theta, X \neq \{S1, S2\}$$

Modelul Bayesian ar fi asociat probabilitatea 0.4 ipotezei $\{D1, D2\}$, ipoteza echivalenta negarii ipotezei $\{S1, S2\}$.

3. Se presupune existenta unei probe care infirma ipoteza vinovatiei organizatiei S1 cu probabilitatea 0.7. Aceasta este echivalent cu a spune ca exista o proba care confirma

negarea ipotezei S1 cu probabilitatea 0.7. In acest caz, atribuirea probabilistica de baza se reprezinta astfel:

$$m(\{S2, D1, D2\}) = 0.7$$

$$m(\theta) = 0.3$$

$$m(X) = 0 \text{ pentru orice alt } X \subset \theta, X \neq \{S2, D1, D2\}$$

Deoarece increderea intr-o submultime de ipoteze A asigura si increderea in submultimile care contin A, i.e. noduri superioare in graful de parti ale multimii θ , este interesant de gasit o functie care sa calculeze cantitatea totala de incredere in submultimea de ipoteze A. Aceasta functie va include nu numai increderea in A dar si increderea in toate submultimile multimii A.

Definitie. Se numeste *functie de incredere*, notata cu Bel, corespunzatoare unei functii de atribuire probabilistica de baza m, functia care asociaza pentru orice submultime de ipoteze A din 2^θ suma increderilor fiecarei submultimi din A pe baza lui m, conform urmatoarei formule:

$$Bel(A) = \sum_{B \subseteq A} m(B) \quad (23)$$

Functia de incredere reprezinta masura increderii totale in submultimea de ipoteze A pe baza probelor care au generat m si are urmatoarele proprietati:

- $Bel(A) = m(A)$ daca A este ipoteza individuala
- $Bel(\theta) = 1$
- $Bel(A) + Bel(\sim A) \leq 1$

Exemplu.

$$Bel(\{S2, D1, D2\}) = m(\{S2, D1, D2\}) + m(\{S2, D1\}) + m(\{S2, D2\}) + m(\{D1, D2\}) + m(\{S2\}) + m(\{D1\}) + m(\{D2\})$$

Daca se considera valorile atribuirii probabilistice de baza din exemplul 3 anterior, atunci $Bel(\{S2, D1, D2\}) = 0.7$.

5.3.3 Combinarea functiilor de incredere

Modelul MYCIN prezentat in sectiunea anterioara ofera o serie de reguli de combinare a incertitudinii intr-o ipoteza sau in mai multe ipoteze pe baza diverselor probe, pentru realizarea rationamentului incert. Teoria Dempster-Shafer realizeaza acest lucru printr-o singura regula care permite combinarea functiilor de incredere atat in cazul in care ele reprezinta probe multiple in favoarea aceleiasi ipoteze, cit si in cazul in care diverse probe sustin ipoteze diferite.

Fie doua probe avind asociate functiile de atribuire probabilistica de baza m_1 si m_2 , si functiile de incredere Bel_1 si Bel_2 . Functiile m_1 si m_2 vor asocia diverse valori probabilistice submultimilor de ipoteze X_i si respectiv Y_j , din cadrul de selectare θ ($X_i \subseteq \theta$, $Y_j \subseteq \theta$). Functia de incredere $Bel_1 \oplus Bel_2$ rezultata din combinarea increderii aduse de cele doua probe se calculeaza, conform formulei (23), astfel:

$$Bel_1 \oplus Bel_2(A) = \sum_{B \subseteq A} m_1 \oplus m_2(B) \quad (24)$$

In consecinta, pentru a calcula $Bel_1 \oplus Bel_2$, trebuie calculata atribuirea probabilistica de baza combinata $m_1 \oplus m_2$, pe baza functiilor m_1 si m_2 . Regula de combinare a doua atribuirii probabilistice da baza este

$$m_1 \oplus m_2(A) = \sum_{\substack{X_i, Y_j \subseteq \theta \\ X_i \cap Y_j = A}} m_1(X_i) \cdot m_2(Y_j) \text{ pentru orice } A \subseteq \theta \quad (25)$$

Se observa ca, pe baza regulilor de algebra elementara si a definitiei atribuirii probabilistice de baza, exista relatia

$$\sum_{X_i, Y_j \subseteq \theta} m_1(X_i) \cdot m_2(Y_j) = \sum_{X_i \subseteq \theta} m_1(X_i) \cdot \sum_{Y_j \subseteq \theta} m_2(Y_j) = 1 \cdot 1 = 1 \quad (26)$$

deci formula (25) indeplineste conditiile impuse functiei de atribuire probabilistica de baza.

Observatie. Comutativitatea inmultirii asigura independenta rezultatului calculat cu regula de combinare din formula (25) de ordinea in care se face combinarea functiilor de atribuire probabilistica de baza, deci de ordinea de considerare a probelor.

Se poate da o reprezentare grafica pentru combinarea a doua functii de atribuire probabilistica de baza, asa cum se prezinta in Figura 5.3. Fiecare dreptunghi reprezinta compozitia atribuirii probabilistice de baza din doua surse de probe, cu incredерile Bel_1 si Bel_2 , si poate fi privit ca intersectia dintre X_i si Y_j , cu masura asociata $m_1(X_i) \cdot m_2(Y_j)$. Dreptunghiul total reprezinta increderea totala asociata de functiile m_1 si m_2 submultimilor lor comune.

	$m_1(X_i)$	L	$m_1(X_i)$	L	
M		L		L	
$m_2(Y_j)$	$m_{X_i \cap Y_j}$	L	$m_{X_i \cap Y_j}$	L	unde $m_{X_i \cap Y_j} = m_1(X_i) \cdot m_2(Y_j)$
M		L		L	
$m_2(Y_i)$	$m_{X_i \cap Y_i}$	L	$m_{X_i \cap Y_i}$	L	

Figura 5.3 Combinarea incredерii in ipoteze pe baza a doua probe

Exemplu. Se presupune existenta unei probe care indica ipoteza vinovatiei organizatiilor teroriste fasciste, i.e. S_1 si S_2 , cu atribuirea probabilistica de baza $m_1 = 0.6$, si o alta proba care infirma vinovatia organizatiei teroriste S_1 cu atribuirea probabilistica de baza $m_2 = 0.7$. Aceasta proba este echivalenta cu o proba care confirma vinovatia submultimii de organizatii $\{S_2, D1, D2\}$ cu $m_2 = 0.7$. Increderea combinata bazata pe ambele probe este data de $m_1 \oplus m_2$ asa cum se arata in continuare.

		m_2			
		$\{S_2, D1, D2\}$	(0.7)	θ	(0.3)
$\{S1, S2\}$	(0.6)	$\{S2\}$	(0.42)	$\{S1, S2\}$	(0.18)
θ	(0.4)	$\{S2, D1, D2\}$	(0.28)	θ	(0.12)
		m_1			

$$m_1 \oplus m_2(\{S2\}) = 0.42$$

$$m_1 \oplus m_2(\{S1, S2\}) = 0.18$$

$$m_1 \oplus m_2(\{S2, D1, D2\}) = 0.28$$

$$m_1 \oplus m_2(\theta) = 0.12$$

$$m_1 \oplus m_2(A) = 0 \text{ pentru orice alta submultime } A \subset \theta$$

Astfel se poate calcula increderea combinata $Bel_1 \oplus Bel_2$ pe baza celor doua probe. Deoarece calculul functiei $Bel_1 \oplus Bel_2$ pentru fiecare submultime este simplu dar laborios, se dau in continuare exemple numai pentru citeva cazuri.

$$Bel_1 \oplus Bel_2(\{S1, S2\}) = m_1 \oplus m_2(\{S1, S2\}) + m_1 \oplus m_2(\{S1\}) + m_1 \oplus m_2(\{S2\}) =$$

$$0.18 + 0 + 0.42 = 0.60$$

$$\begin{aligned} \text{Bel}_1 \oplus \text{Bel}_2(\{S2, D1, D2\}) &= m_1 \oplus m_2(\{S2, D1, D2\}) + m_1 \oplus m_2(\{S2, D1\}) + \\ & m_1 \oplus m_2(\{S2, D2\}) + m_1 \oplus m_2(\{D1, D2\}) + \\ & m_1 \oplus m_2(\{S2\}) + m_1 \oplus m_2(\{D1\}) + m_1 \oplus m_2(\{D2\}) = \end{aligned}$$

$$0.28 + 0 + 0 + 0 + 0.42 + 0 + 0 = 0.70$$

$$\text{Bel}_1 \oplus \text{Bel}_2(\{S1, S2, D2\}) = \text{Bel}_1 \oplus \text{Bel}_2(\{S1, S2\}) = 0.6.$$

$$\text{deoarece } m_1 \oplus m_2(\{S1, S2, D2\}) = m_1 \oplus m_2(\{S1, D2\}) = m_1 \oplus m_2(\{S2, D2\}) = 0$$

Se observa ca , in acest exemplu, functia $m_1 \oplus m_2$ satisface definitia unei functii de atribuire probabilistica de baza, i.e. $\sum_{X \subseteq \theta} m_1 \oplus m_2(X) = 1$ cu $m_1 \oplus m_2(\phi) = 0$. Conditia (2) din definitia unei

atribuirii probabilistice de baza este intotdeauna satisfacuta datorita formulei (26). Conditia (3) din definitie, $m(\phi) = 0$, este o conditie problematica in cazul in care tabloul de intersectie contine intrari nule. Aceasta situatie nu a aparut in exemplul de mai sus deoarece orice doua multimi cu valori nenule ale atribuirii probabilistice de baza au avut intotdeauna cel putin un element in comun. In general este posibil sa existe multimi X_i si Y_j fara elemente comune, i.e. $X_i \cap Y_j = \phi$, dar cu valori ale functiilor m_1 si m_2 diferite de zero. In acest caz ar rezulta o functie $m_1 \oplus m_2(\phi) \neq 0$, ceea ce contrazice definitia.

Teoria Dempster-Shafer rezolva aceasta problema prin normalizarea valorilor atribuirii probabilistice de baza combinate astfel incit $m_1 \oplus m_2(\phi) = 0$ si toate valorile functiei sa ramina in continuare in intervalul $[0,1]$. Normalizarea se face prin definirea sumei tuturor valorilor nenule asociate multimii vide in tabloul de intersectie. In acest fel, formula (25) de calcul a atribuirii probabilistice de baza combinata este inlocuita cu formula

$$m_1 \oplus m_2(A) = \frac{\sum_{\substack{X_i, Y_j \subseteq \theta \\ X_i \cap Y_j = A}} m_1(X_i) \cdot m_2(Y_j)}{1 - \sum_{\substack{X_i, Y_j \subseteq \theta \\ X_i \cap Y_j = \phi}} m_1(X_i) \cdot m_2(Y_j)} \quad (27)$$

Se poate demonstra ca utilizind formula de mai sus conditiile din definitia functiei de atribuire probabilistica de baza sint indeplinite.

5.3.4 Intervale de incredere

Functia de incredere $\text{Bel}(A)$ asociata unei submultimi de ipoteze A reprezinta masura increderii totale in submultimea A dupa ce au fost considerate toate probele in favoarea ipotezelor din A . Dar aceasta functie contine si alte informatii despre A , respectiv $\text{Bel}(\sim A)$, i.e. masura increderii cu care probele confirma negarea ipotezelor din A , i.e. ipoteza $\sim A$. Cantitatea $1 - \text{Bel}(\sim A)$ exprima plauzibilitatea lui A , numita si *prag de semnificatie* a lui A .

Informatia reprezentata de functia de incredere $\text{Bel}(A)$ poate fi exprimata convenabil printr-un interval, numit *interval de incredere* al submultimii de ipoteze A sau *increderea* in A . Intervalul de incredere al multimii A este

$$[\text{Bel}(A), 1 - \text{Bel}(\sim A)]$$

Se poate arata ca $1 - \text{Bel}(\sim A) \geq \text{Bel}(A)$ sau, echivalent $\text{Bel}(A) + \text{Bel}(\sim A) \leq 1$, deoarece $\text{Bel}(A) + \text{Bel}(\sim A) \leq \sum_{X_i \subseteq \theta} m(X_i) = 1$, iar A si $\sim A$ nu au submultimi comune.

In modelul Bayesian $\text{Bel}(A) + \text{Bel}(\sim A) = 1$ ceea ce inseamna ca lungimea intervalului de incredere este zero. In teoria Dempster-Shafer lungimea intervalului de incredere este de obicei diferita de zero si reprezinta masura increderii in faptul ca daca multimea de ipoteze A nu este sigura, nici negarea multimii de ipoteze, $\sim A$, nu este sigura. Se observa ca lungimea intervalului de incredere al multimii de ipoteze A este suma functiilor de incredere asociate acelor submultimi de ipoteze din θ care intersecteaza A dar care nu sint submultimi ale multimii A . Daca A este o ipoteza individuala, toate aceste submultimi sint si superseturi ale lui A , dar acest lucru nu mai este valabil pentru cazul in care A contine mai multe ipoteze. Lungimea intervalului de incredere poate fi interpretata si ca masura incertitudinii unei ipoteze pe baza unei probe date. In continuare se dau exemple de interpretari ale unor intervale de incredere:

[0,1] reprezinta nici o incredere in ipoteza

[0,0] reprezinta increderea ca ipoteza este falsa

[1,1] reprezinta increderea ca ipoteza este adevarata

[0.3,1] reprezinta increderea partiala in ipoteza

[0,0.8] reprezinta neincrederea partiala in ipoteza

[0.2,0.7] reprezinta atat increderea cit si neincrederea in adevarul ipotezei

Teoria Dempster-Shafer modeleaza procesul acumularii de probe in favoarea unei multimii de ipoteze competitive pornind de la un interval de incredere [0,1] asociat fiecarei ipoteze, corespunzator situatiei initiale in care nu exista nici o proba. Pe parcursul acumularii probelor, intervalul de incredere asociat se va reduce reprezentind increderea crescuta intr-o ipoteza sau o multime de ipoteze. Se observa ca aceasta abordare difera de modelul Bayesian in care probabilitatile se distribuie, la inceput, egal intre ipoteze. Intervalul de incredere pune in evidenta clar faptul ca nu exista nici un fel de informatie la inceput. Acest lucru nu se intimpla in modelul Bayesian deoarece, dupa considerarea unui numar oarecare de probe, se pot obtine in final aceleasi probabilitati cu cele atribuite initial ipotezelor. Aceasta diferenta este importanta in momentul in care sistemul de rationament incert trebuie sa decida daca mai este nevoie sa considere in continuare noi probe sau nu.

5.4 Retele Bayesiene

Modelul retelelor Bayesiene, introdus de Judea Pearl [1988], porneste de la modelul probabilistic Bayesian, dar elimina numarul enorm de calcule necesare in acesta prin considerarea caracteristicilor de modularitate si de cauzalitate ale domeniului problemei. Ideea de baza a retelelor Bayesiene este aceea ca, pentru a descrie domeniul problemei, nu este necesar sa se considere probabilitatile tuturor perechilor de evenimente (fapte) posibile. Cele mai multe evenimente sint independente intre ele si interactiunile dintre acestea nu trebuie considerate, deoarece nu exista. Modelul retelelor Bayesiene foloseste un graf orientat aciclic [Sedgewick,1990] pentru a reprezenta gradele de incredere in faptele din baza de cunostinte si dependentele cauzale existente intre aceste fapte.

Realizarea inferentelor într-o astfel de rețea revine la propagarea probabilităților faptelor date și/sau inferate în rețea spre nodurile concluzie. Reprezentarea cunostintelor sub forma de rețele va fi discutată pe larg în Capitolul 6. Modelul rețelilor Bayesiene este, evident, foarte potrivit pentru a realiza inferențe incerte într-o astfel de reprezentare. Eliminând calculele laborioase și cantitățile mari de informații necesare în modelul Bayesian, rețelele Bayesiene au avantajul posibilității modelării ipotezelor dependente, deci elimină condiția de independență necesară în modelul factorilor de certitudine din MYCIN și în teoria Dempster-Shafer.

Exemplul ierbii ude din finalul Secțiunii 5.2 a pus în evidență faptul că există două moduri în care o propoziție poate influența o altă propoziție. Primul mod sugerează simptome pe baza cauzei acestora, iar cel de-al doilea sugerează cauza pe baza simptomelor. În rețelele Bayesiene se face clar distincția între aceste două tipuri de relații pe baza direcției arcelor din graf. Fiecare nod din graf corespunde unei propoziții, care poate lua valorile adevărat sau fals, sau unui obiect (obiect-atribut) care poate lua valori din domeniul de valori asociat obiectului. Exemple de astfel de obiecte și valori sunt, pentru domeniul medical: o boală specifică, temperatura pacientului, rezultatul unei analize. Direcțiile arcelor din rețea indică întotdeauna o legătură cauzală directă, deci (cauza→efect). Pentru exemplul ierbii ude, reprezentarea sub forma de rețea Bayesiană este dată în Figura 5.4.

Pe lângă cele trei noduri corespunzătoare celor trei fapte din exemplu, se observă că s-a adăugat un al patrulea nod, propoziția sezon ploios care poate lua valorile adevărat sau fals.

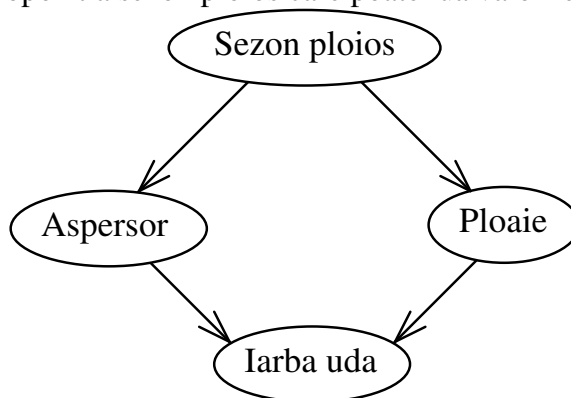


Figura 5.4 Reprezentarea cauzalității într-o rețea Bayesiană

Pentru realizarea inferentelor incerte, se asociază probabilități *a priori* faptelor din rețea și probabilități conditionate fiecărei legături cauzale (arc) din rețea. Pentru exemplul ierbii ude, probabilitățile considerate sunt indicate în Figura 5.5. În această figură se observă, de exemplu, că probabilitatea *a priori* ca sezonul să fie ploios este de 0.5 și, într-un astfel de sezon, probabilitatea (conditionată) de producere a ploii este 0.9.

Probabilitate	Valoare
$P(\text{Iarba udalAspersor}, \text{Ploaie})$	0.95
$P(\text{Iarba udalAspersor}, \sim \text{Ploaie})$	0.9
$P(\text{Iarba udal} \sim \text{Aspersor}, \text{Ploaie})$	0.8
$P(\text{Iarba udal} \sim \text{Aspersor}, \sim \text{Ploaie})$	0.1
$P(\text{Aspersor} \text{Sezon ploios})$	0.0
$P(\text{Aspersor} \sim \text{Sezon ploios})$	1.0
$P(\text{Ploaie} \text{Sezon ploios})$	0.9
$P(\text{Ploaie} \sim \text{Sezon ploios})$	0.1
$P(\text{Sezon ploios})$	0.5

Figura 5.5 Probabilitati pentru o retea Bayesiană

Pentru a putea realiza inferente într-un astfel de model este nevoie de un mecanism care să permită calculul influenței unui nod asupra celorlalte. Există trei clase de algoritmi pentru realizarea acestor calcule: metoda transmiterii mesajelor, metoda triunghiulară și algoritmi stocastici. Ideea tuturor celor trei clase de algoritmi este aceea de a exploata domeniul limitat de influență al nodurilor. Astfel, deși operația de actualizare a probabilităților în rețea este complexă computațional, complexitatea poate fi redusă în practică. Detalii asupra acestor algoritmi pot fi găsite în Pearl [1988].

Reprezentarea cunostintelor incerte prin rețele Bayesiene a fost utilizată într-o serie de sisteme bazate pe cunostințe, în special în domeniul diagnosticării medicale unde s-au construit sistemele CASNET [Weiss, s.a., 1978] și INTERNIST/CADUCEUS [Pople, 1982].

5.5 Exerciții și probleme

1. Folosind modelul Bayesian să se determine probabilitatea unui eveniment A în condițiile producerii unui eveniment B, știind că $P(B|A) = 0.84$, $P(A) = 0.2$ și $P(B) = 0.34$.
2. În modelul Bayesian să se demonstreze că dacă evenimentele A și B sunt independente, i.e. $P(A \& B) = P(A) \cdot P(B)$, atunci $P(A|B) = P(A)$.
3. Considerând teoria clasică a probabilității, este posibil să se calculeze $P(A|\sim B)$ când se cunosc $P(A)$, $P(B|A)$ și $P(B)$? Justificare.
4. Să se folosească modelul coeficientului de certitudine din MYCIN pentru rezolvarea următoarelor probleme. Trei reguli R1, R2 și R3 concluzionează despre ipoteza h pe baza probelor e1, e2 și, respectiv e3. Regula R1 are asociat coeficientul de certitudine $CF1 = 0.3$, regula R2, $CF2 = 0.2$ și regula R3, $CF3 = -0.1$. Să se calculeze măsura încrederii (MB), măsura neîncrederii (MD) și coeficientul de certitudine (CF) asociate ipotezei h pe baza:

- (a) obtinerii probei e1
 - (b) obtinerii probelor e1 si e2
 - (c) obtinerii probelor e1, e2 si e3.
5. In modelul MYCIN se cunosc urmatoarele: exista o regula care contine in partea stinga conditii despre s1, s2 si s3 si concluzioneaza despre ipoteza h1 cu coeficientul de certitudine $CF[h1, s1 \wedge s2 \wedge s3] = 0.7$. Ipoteza h1 este "identitatea organismului este streptococ", s1 este observarea faptului ca organismul este gram-negativ, s2 ca organismul are morfologia coc si s3 ca organismul se dezvoltă in lanturi. Se stie de asemenea ca $MB[lanuri, e] = 0.6$ si $MD[lanuri, e] = 0$. Sa se calculeze $MB[h1, s1 \wedge s2 \wedge s3]$, $MD[h1, s1 \wedge s2 \wedge s3]$ si $CF[h1, s1 \wedge s2 \wedge s3]$ in conditiile in care se cunoaste cu certitudine ca organismul este gram-pozitiv si ca are morfologia coc.
6. Se considera exemplul din Sectiunea 5.3.3 si o a treia proba m_3 care confirma ipoteza de vinovatie a organizatiei S1 cu valoarea 0.8. Sa se calculeze $m_3 \oplus m_4$ si $Bel_3 \oplus Bel_4$, unde $m_4 = m_1 \oplus m_2$ si $Bel_4 = Bel_1 \oplus Bel_2$.
7. Fie multimea de ipoteze de diagnosticare a unui pacient: alergie, gripa, raceala, pneumonie. Notind cu Al alergia, cu Gr gripa, cu Ra raceala si cu Pne pneumonia, se obtine cadrul de selectare $\theta = \{Al, Gr, Ra, Pne\}$.
- (a) Sa se calculeze valorile atribuirii probabilistice de baza pentru aceste ipoteze in cazul in care nu exista nici o proba.
 - (b) Sa se calculeze valorile atribuirii probabilistice de baza m_1 si a functiei de incredere Bel_1 pentru cazul in care se obtine o proba care sustine multimea de ipoteze $\{Gr, Ra, Pne\}$ cu valoarea atribuirii probabilistice de baza 0.7, de exemplu faptul ca pacientul are febra.
 - (c) Sa se calculeze valorile functiei de incredere $Bel_1 \oplus Bel_2$ in cazul in care o a doua proba, de exemplu pacientului ii curge nasul, sustine multimea de ipoteze $\{Al, Gr, Ra\}$ cu $m_2 = 0.8$.
 - (d) Sa se calculeze valorile functiei de incredere combinate $Bel_4 \oplus Bel_3$, unde $Bel_3 = Bel_1 \oplus Bel_2$, in cazul in care se obtine o a treia proba care sustine ipoteza diagnosticului de alergie cu valoarea $m_4 = 0.9$.
 - (e) Sa se comenteze si sa se interpreteze toate rezultatele obtinute.

Capitolul 6

Modelul cunostintelor structurate

Modelele de reprezentare prezentate anterior, respectiv logica cu predicate de ordinul I si regulile de productie, sint abordari caracterizate de putere expresiva, metode de inferenta puternice si validitate. Aceste modele nu dau, insa, nici o indicatie asupra modului de organizare si reprezentare interna a cunostintelor in sistem. Construirea bazelor de cunostinte mari si a sistemelor care exploateaza aceste cunostinte trebuie sa tina cont de eficienta reprezentarilor interne asociate unui model, de capacitatea de acces si modificare a cunostintelor. In plus, s-a constatat ca si la nivel simbolic, achizitia cunostintelor, intelegerea, utilizarea si intretinerea bazelor de cunostinte este dificila in lipsa unei organizari conceptuale adecvate. Aceste motive au dus la aparitia unei noi clase de modele de reprezentare a cunostintelor: cunostintele structurate. Spre deosebire de logica cu predicate de ordinul I in care exista o sintaxa si semantica unica, precisa si unanim acceptata, modelul cunostintelor structurate include mai multe paradigme de reprezentare si o adevarata multitudine de limbaje de descriere a cunostintelor.

Cele mai importante reprezentari structurate ale cunostintelor sint: retelele semantice si grafurile conceptuale, unitatile, numite in diverse abordari si cadre sau prototipuri, dependentele conceptuale si scenariile. Toate aceste modele prezinta o caracteristica comuna: ofera posibilitatea de a organiza si structura cunostintele in functie de relatiile existente intre obiectele universului problemei.

Paradigmele de reprezentare se numesc modele de cunostinte structurate tocmai datorita accentului fundamental pe care il pun pe structura cunostintelor in reprezentare, prin indexarea cunostintelor in functie de obiectele importante in sistem. Daca sistemul bazat pe cunostinte are nevoie de una din informatiile asociate unui astfel de obiect, structura de cunostinte asociata obiectului este regasita si toate faptele relevante despre acel obiect sint identificate deodata. Modelele cunostintelor structurate au asociate metode de inferenta specifice, incluse direct in reprezentare. In acest fel, anumite operatii care ar fi necesitat scrierea si executia unor reguli (logice sau de productie) explicite in alte reprezentari, sint executate automat de mecanismele inferentiale asociate structurii reprezentarii.

Modelul *retelelor semantice*, primul model structurat de reprezentare a cunostintelor, a fost introdus de Quillian in 1967 pentru a descrie semantica cuvintelor si a propozitiilor limbajului natural. Ulterior, retelele semantice au fost folosite ca metoda de reprezentare a cunostintelor in numeroase sisteme bazate pe cunostinte. De exemplu, Winston [1984] foloseste retelele semantice pentru a reprezenta si invata informatii despre lumea blocurilor, iar Hendrix [1977] foloseste modelul retelelor semantice partitionate pentru a obtine intreaga putere expresiva a calculului cu predicate. Sistemul SNePS, prescurtare de la Semantic Network Processing System, a fost conceput de Shapiro [1979] si a fost dezvoltat ulterior prin introducerea unor mecanisme

suplimentare de reprezentare si rationament, cum ar fi contextele multiple si utilizarea convingerilor in rationamentul nemonoton [Kumar,1990].

Conceptul de *cadru* ca unitate fundamentala de reprezentare a unei structuri de cunostinte a fost propus pentru prima oara de Minsky [1975] intr-un sistem de intelegere a imaginilor. Primul limbaj bazat pe cadre a fost limbajul KRL, prescurtare de la numele Knowledge Representation Language, propus de Bobrow si Winograd [1977]. Limbajul a fost urmat de o multitudine de alte limbaje de acelasi tip, cum ar fi FRL, prescurtare de la numele Frame Representation Language, proiectat de Roberts, Goldstein in 1977, RLL [Greiner,Lenat,1980], KL-ONE [Brachman, Schmolze,1985], CYC [Lenat,s.a.,1990] si altele.

Dependentele conceptuale au fost introduse de Schank in 1973 pentru reprezentarea semnificatiei limbajului natural. Dependentele conceptuale reprezinta o teorie de modelare a cunostintelor despre evenimentele referite, de obicei, in propozitiile limbajului natural. *Scenariile*, introduse de Schank si Abelson in 1977 reprezinta o combinatie intre modelul cadrelor si dependentele conceptuale, permitind descrierea unei secvente tipice de evenimente intr-un context particular. O prezentare detaliata a diverselor modele structurate de reprezentare a cunostintelor poate fi gasita in Barr, s.a [1982], Patterson [1990], Luger si Stubblefield [1993], Rich si Knight [1991].

Retelele semantice si unitatile pot fi caracterizate drept "structuri slabe" de reprezentare a cunostintelor deoarece modelul nu contine, in sine, informatii despre cunostintele specifice pe care le va reprezenta. Aceste structuri slabe pot fi puse in corespondenta cu strategiile de cautare neinformata, in care tehnica de cautare este independenta de caracteristicile problemei de rezolvat. Spre deosebire de aceste modele, dependentele conceptuale si scenariile pot fi caracterizate drept "structuri tari" de reprezentare a cunostintelor, deoarece ele includ notiuni specifice despre tipul obiectelor si relatiilor existente intre acestea. Structurile tari de reprezentare a cunostintelor pot fi puse in corespondenta, la nivelul controlului, cu strategiile de cautare informata, in care fiecare algoritm de cautare include informatie euristica specifica domeniului problemei.

6.1 De la reprezentarea logica la modelul cunostintelor structurate

Fie urmatoarele propozitii, care trebuie reprezentate intr-o baza de cunostinte:

Radu i-a trimis Ioanei o scrisoare.

Radu este student.

Ioana este eleva.

Adresa lui Radu este Luterana, 15.

Considerind logica cu predicate de ordinul I, aceste cunostinte pot fi reprezentate astfel:

Ocupatie (radu,student)

Ocupatie (ioana,eleva)

Trimate (radu, ioana, scrisoare)

Adresa (radu, luterana -15)

Din punct de vedere al facilitatii de acces, este convenabil sa se grupeze faptele care refera un obiect important intr-o singura entitate. Acest lucru se poate face, pentru exemplul de mai sus, astfel:

Radu

Ocupatie (radu,student)

Trimite (radu, ioana, scrisoare)

Adresa (radu, luterana - 15)

Ioana

Ocupatie (ioana,eleva)

Trimite (radu, ioana, scrisoare)

O schema de reprezentare in care faptele sint indexate in functie de entitati sau obiecte considerate importante in problema, se numeste o *reprezentare orientata pe obiecte*. Modelul cunostintelor structurate se bazeaza pe aceasta idee, avind deci mai multe caracteristici in comun cu limbajele orientate pe obiecte. Utilizind o reprezentare structurata a cunostintelor, informatiile continute de o predicatelor Ocupatie si Adresa, pot fi exprimate astfel:

Radu

Ioana

Ocupatie: student

Ocupatie: elev

Adresa: luterana-15

Cele mai multe limbaje de descriere a cunostintelor structurate folosesc o forma de predicate binare pentru a reprezenta fapte despre obiecte, asa cum se vede in exemplu. In cazul in care exista predicate de aritate mai mare decit 2, se poate folosi o metoda simpla de transformare a acestora in predicate binare. In exemplul anterior, predicatul Trimite (radu, ioana, scrisoare) poate fi reformulat ca o conjunctie de predicate binare prin postularea existentei unui "eveniment de trimitere", astfel:

$$(\exists x)(\text{ISA}(x, \text{eveniment - trimitere}) \wedge \text{Expeditor}(x, \text{radu}) \wedge$$

$$\text{Destinatar}(x, \text{ioana}) \wedge \text{Obiect}(x, \text{scrisoare}))$$

Predicatul ISA este folosit pentru a exprima apartenenta unui obiect (particular) la o multime. Aplicind skolemizarea in formula de mai sus, se inlocuieste variabila cuantificata existential x cu o constanta t_1 si se obtine:

$$\text{ISA}(t_1, \text{eveniment - trimitere}) \wedge \text{Expeditor}(t_1, \text{radu}) \wedge$$

$$\text{Destinatar}(t_1, \text{ioana}) \wedge \text{Obiect}(t_1, \text{scrisoare})$$

Aceasta exprimare a predicatului Trimite poate fi convenabil reprezentata ca un obiect structurat. In plus, daca se considera ca Radu si Ioana sint persoane, se poate folosi predicatul ISA pentru a-i caracteriza suplimentar ca membrii ai multimii tuturor persoanelor existente. Astfel, se obtine reprezentarea structurata a cunostintelor din Figura 6.1(a).

Entitatile descrise de aceasta reprezentare sint *obiecte particulare* sau *instante*. Se observa ca structurile care descriu cele doua persoane, Radu si Ioana, sint relativ asemanatoare. Daca ar trebui descris un alt eveniment de trimitere, cum ar fi trimiterea unei carti de catre Ioana lui Radu, acesta ar avea o structura asemanatoare cu T1. Pe baza acestor observatii, se constata necesitatea crearii unor *obiecte generale*, numite si *obiecte generice* sau *clase*, care sa descrie toti membrii

unei multimi de obiecte. Pentru exemplul anterior se pot defini doua obiecte generice Eveniment-trimitere si Persoana, obiecte indicate in Figura 6.1(b).

Radu	Ioana
ISA: Persoana	ISA: Persoana
Ocupatie: student	Ocupatie: elev
Adresa: luterana-15	

T1

ISA: Eveniment-trimitere

Expeditor: Radu

Destinatar: Ioana

Obiect: scrisoare

(a)

Eveniment-trimitere	Persoana
AKO: Eveniment	AKO: Fiinta
Expeditor: Persoana	Ocupatie: (student, elev, inginer)
Destinatar: Persoana	Adresa: sir
Obiect: ClasaObiect	

(b)

Figura 6.1 O reprezentare structurata a cunostintelor

In definitia de mai sus se observa aparitia unui alt predicat important, AKO, care descrie incluziunea unei multimi intr-o alta multime. Predicatul AKO, specifica faptul ca obiectul generic Eveniment-trimitere face parte din clasa mai larga a evenimentelor si ca obiectul generic Persoana (deci multimea tuturor persoanelor) este inclus in clasa Fiinta (deci in multimea tuturor fiintelor). In logica cu predicate, aceste incluziuni s-ar fi exprimat astfel:

$(\forall x) (\text{Eveniment - trimitere } (x) \rightarrow \text{Eveniment } (x))$

$(\forall x) (\text{Persoana } (x) \rightarrow \text{Fiinta } (x))$

Cele doua predicate de apartenenta si de incluziune, ISA si respectiv AKO, definesc doua tipuri de relatii fundamentale in reprezentarea structurata a cunostintelor. Relatia de apartenenta a unui obiect la o multime defineste o *relatie individual-generic*, sau instanta-clasa si va fi notata in continuare, prin conventie, ISA (prescurtare de la IS A). Relatia de incluziune a unei multimi de obiecte intr-o alta multime defineste o *relatie generic-generic*, sau subclasa-clasa si va fi notata in continuare, prin conventie, AKO (prescurtare de la A Kind Of).

Semnificatia relatiilor ISA si AKO a fost mult discutata in literatura de specialitate deoarece aproape fiecare sistem bazat pe cunostinte structurate a folosit aceste relatii, sub un nume sau altul, cu semnificatii sau nuante semantice diferite. Intr-un articol foarte cunoscut, Brachman [1983] identifica urmatoarele interpretari al acestor relatii, existente in diverse sisteme.

Relatie generic-generic

- subclasa-superclasa (exemplu: masini de curse-automobile)
- specializare-generalizare (exemplu: magazin cu autoservire-magazin)
- un fel de (exemplu: un catel este un fel de animal)

- incluziune conceptuala (exemplu: un triunghi este un poligon)
- multimi si tipul lor (exemplu: elefanti si o multime de elefanti)
- restringerea valorilor (exemplu: trompa unui elefant este un cilindru de 1.3 metri)

Relatie individual-generic

- apartenenta la o multime
- proprietati ale obiectelor (exemplu: camila este maro)
- abstractizarea (exemplu: ornitorincul este o specie pe cale de disparitie)

Reprezentarea structurata a cunostintelor poate fi deci privita ca o multime de *obiecte particulare* si/sau *generice*, fiecare obiect fiind descris printr-o multime de atribute (proprietati), numite de obicei *sloturi*, si valorile lor asociate. In exemplul anterior, obiectul particular Radu este descris prin atributele ISA, Ocupatie si Adresa, cu valorile asociate Persoana, student si luterana-15. Se observa ca valorile atributelor pot fi atat obiecte atomice (nestructurate) cit si obiecte structurate ale reprezentarii (instante sau obiecte generice). Se va vedea mai tirziu ca exista si alte tipuri de valori pentru atribute.

O astfel de reprezentare a cunostintelor poate fi imbogatita cu inferente specifice care opereaza pe structura reprezentarii. Aceste inferente au ca suport organizarea obiectelor bazei de cunostinte in clase si a claselor in ierarhii de clase, pe baza relatiilor ISA si AKO.

Forma de inferenta specifica fundamentala cunostintelor structurate este *mostenirea proprietatilor*, in care elemente ale unei clase mostenesc atribute si valori de la clasa sau de la superclasele in care aceasta clasa este inclusa. Mostenirea proprietatilor (atributelor) poate avea doua forme:

- Aplicarea mostenirii proprietatilor de la clasa la instanta reprezinta corespondentul regulii Modus Ponens din logica cu predicate in reprezentarea structurata a cunostintelor. Aceasta inferenta specifica are urmatoarea forma: Daca un obiect O_1 este o particularizare (legat prin relatia ISA) a unui obiect generic O si obiectul O are un atribut (proprietate) A , atunci si instanta O_1 are atributul A .
- Aplicarea mostenirii proprietatilor intre o clasa si o superclasa, deci de-a lungul unei relatii sau a unui lant de relatii AKO insemna mostenirea atributelor de la clasa (obiect generic) la subclasa (obiect generic cu grad de generalitate mai mic). Daca o clasa C_1 este o subclasa a unei clase C (legata prin una sau mai multe relatii AKO) si clasa C are proprietatea A , atunci clasa C_1 are de asemenea proprietatea (atributul) A .

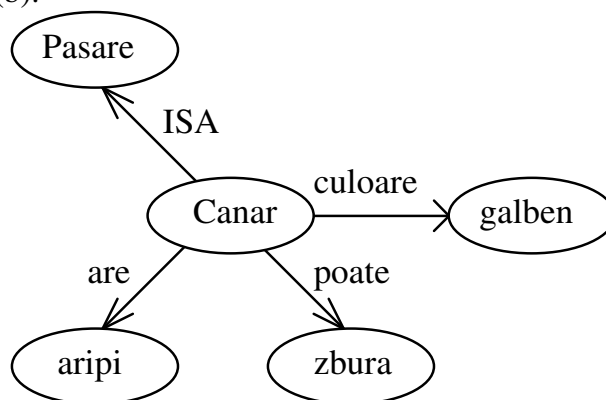
Mostenirea proprietatilor poate fi realizata mult mai eficient in cadrul modelului cunostintelor structurate decit in cazul altor modele, cum ar fi modelul logic in care aceste relatii trebuie definite explicit. Structurile descrise permit realizarea atat a mostenirii monotone (rationament monoton) cit si a mostenirii nemonotone prin prezenta valorilor implicite sau a exceptiilor (rationament nemonoton). Detalii suplimentare despre mostenirea proprietatilor si algoritmi utilizati vor fi prezentate in sectiunile urmatoare.

6.2 Retele semantice

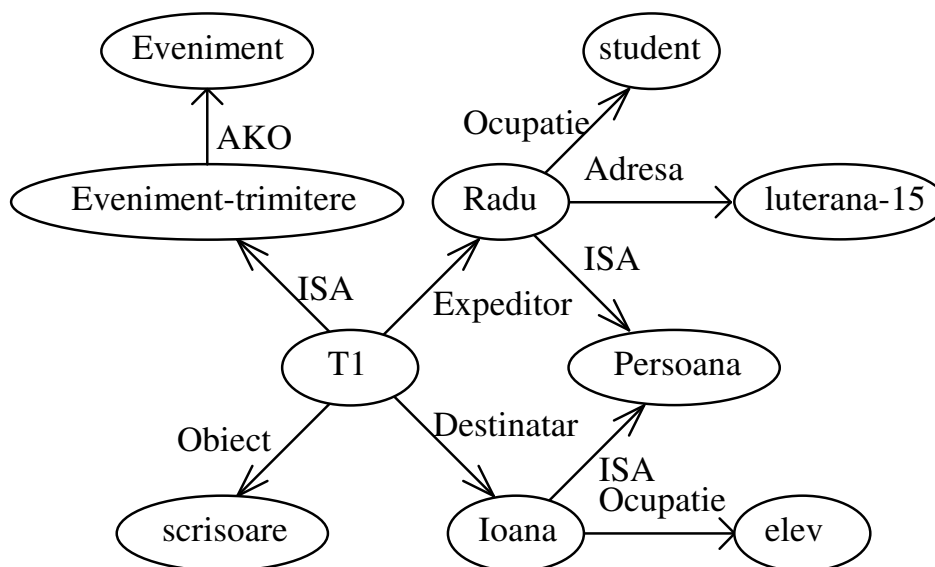
Retelele semantice, numite si retele asociative, ofera o modalitate de structurare a cunostintelor pe baza unei reprezentari grafice asociate obiectelor si atributelor universului problemei. Ideea de baza a reprezentarii cunostintelor prin retele semantice este de a explica intelesul unui concept prin modurile in care acest concept este legat de alte concepte. Denumirea de retea semantica, mult mai des folosita decit cea de retea asociativa este justificata prin faptul ca acest model a fost folosit, pentru prima oara, in descrierea semanticii limbajului natural.

6.2.1 Reprezentarea cunostintelor prin retele semantice

Retelele semantice sint grafuri orientate in care nodurile si arcele sint etichetate cu nume ale obiectelor, atributelor si relatiilor intre obiectele si attributele universului problemei. De obicei nodurile retelei reprezinta obiecte generice sau particulare, iar arcele reprezinta relatii intre acestea, sau attribute ale obiectelor punctind spre noduri ce contin valorile acestor attribute. Un exemplu de retea semantica este cel prezentat in Figura 6.2(a). Cunostintele continute de reprezentarea structurata din Figura 6.1 pot fi reprezentate sub forma unei retele semantice asa cum se vede in Figura 6.2(b).



(a)



(b)

Figura 6.2 Retele semantice

Intr-o retea semantica, la fel ca in orice reprezentare structurata a cunostintelor, pot exista noduri asociate obiectelor individuale (instante), de exemplu nodurile Canar, Ioana, Radu, si noduri asociate obiectelor generice (clase), de exemplu nodurile Pasare si Eveniment-trimitere, Eveniment, Persoana din Figura 6.2. Cunostintele asociate arcelor intr-o retea semantica pot reprezenta relatii specifice problemei de rezolvat, cum ar fi relatiile: are, culoare, poate, Obiect, Expeditor, Destinatar, sau relatii generale care se pot intilni, sub o denumire sau alta, in orice retea semantica. Cele mai frecvente relatii generale sint ISA (instanta-clasa) si AKO (subclasa-clasa); alte relatii generale des intilnite sint: SUBSET-OF (submultime a), HAS-PARTS (este format din), AGENT (agent), OBJECT (obiect), ATTRIBUTE (atribut).

Una dintre primele aplicatii ale reprezentarii cunostintelor prin retele semantice a fost gasirea relatiilor intre semnificatia cuvintelor in limbaj natural prin propagarea activarii nodurilor prin retea de-a lungul arcelor si determinarea nodului in care activarile se intilnesc. Acest proces se numeste *cautarea intersectiei* si a fost folosit de Quillian [1969].

Programul lui Quillian defineste intelesul cuvintelor din limba engleza in acelasi fel in care acesta este definit de un dictionar: un cuvint este definit pe baza altor cuvinte, componentele definitiei sint explicate pe baza altor cuvinte, si asa mai departe. Fiecare nod din retea semantica corespunde unui *concept-cuvint* cu legaturi (arce) spre alte concepte-cuvinte care formeaza definitia conceptului-cuvint. Baza de cunostinte astfel construita poate fi folosita pentru a gasi relatii intre perechi de cuvinte din limba engleza. Procesul de cautare a intersectiei se desfasoara astfel:

- Nodurile asociate celor doua concepte-cuvint initiale se activeaza si devin sursa de propagare a activarii.
- Activarea unui nod se propaga pe arcele care pleaca din acel nod spre alte noduri, acestea devenind la rindul lor surse de activare. Propagarea activarii se face pe nivel.
- Procesul se opreste in momentul in care un nod, numit si concept comun sau nod intersectie, se gaseste la intersectia a doua cai distincte de propagare a activarii.
- Caile descoperite in retea catre conceptul comun reprezinta relatia semantica existenta intre cele doua cuvinte-concept initiale.

Procesul de cautare a intersectiei descris reprezinta o forma de rationament specifica retelelor semantice propuse de Quillian in care inferenta specifica este activarea unui nod si propagarea activarii de-a lungul unui arc. Cu toate acestea, o astfel de reprezentare nu aduce avantaje semnificative fata de calculul cu predicate, de exemplu, din punct de vedere al capacitatilor de rationament incluse in reprezentare. Puterea expresiva a retelelor semantice consta in definirea relatiilor asociate arcelor si a regulilor de inferenta asociate.

Asocierea unor relatii de tip ISA si AKO arcelor din retea, asa cum s-a vazut la inceputul acestei sectiuni, a permis atasarea unor inferente specifice puternice care, pe linga modularitatea si indexarea cunostintelor, constituie al doilea mare avantaj al reprezentarii structurate a cunostintelor. Definirea unor reprezentari structurate tari, prezentate in Sectiunea 6.4, a constituit urmatorul pas in extinderea modelului initial propus de Quillian.

6.2.2 Inferente specifice rețelelor semantice

Mostenirea proprietatilor (atributelor), forma de inferenta specifica reprezentarilor structurate a cunostintelor, poate fi implementata in cadrul unei retele semantice in care exista relatii (arce) de tip ISA si AKO. Considerind o retea semantica cu noduri asociate obiectelor particulare si generice, si cu relatii de tip ISA, AKO si relatii care descriu attributele asociate obiectelor (in retea pot exista si alte tipuri de relatii sau noduri), instantele vor mosteni attributele claselor carora apartin, iar nodurile clasa vor mosteni attributele nodurilor superclasa de care sint legate prin relatia AKO. Revenind la exemplul din Figura 6.2(b) si presupunind ca orice persoana are un atribut Inaltime, nodurile asociate instantelor Radu si Ioana vor mosteni acest atribut. Considerind exemplul din lumea blocurilor prezentat in Figura 6.3, se poate deduce, pe baza mostenirii proprietatilor, ca Piramida18 are forma triunghi iar Caramida12 are forma dreptunghi. De asemenea, se poate infera ca atit obiectele generice Piramida si Caramida, cit si instantele au valoarea atributului Consistenta mare.

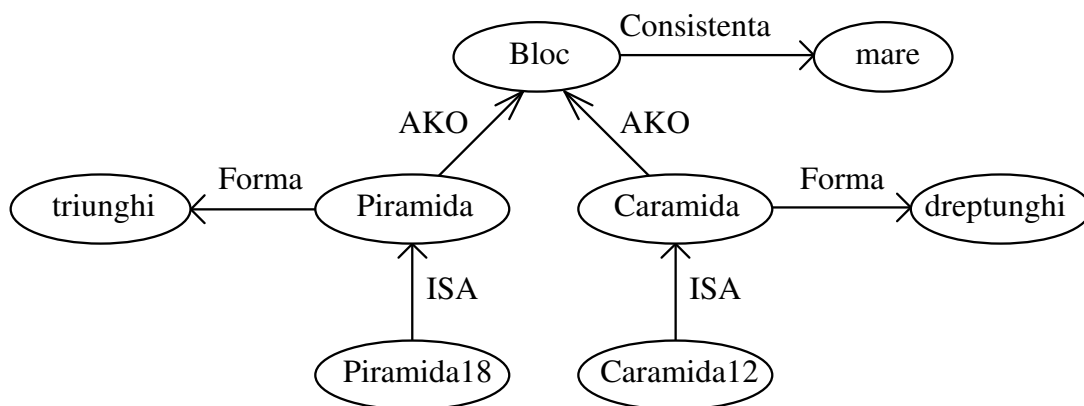


Figura 6.3 Mostenirea valorilor in retele semantice

Valorile atributelor Forma si Consistenta au fost mostenite de-a lungul relatiilor instantă-clasă si clasă-superclasă. Procedura de mostenire a valorilor de attribute este prezentata in continuare.

Algoritm: Mostenirea valorilor atributelor intr-o ierarhie de clase

Algoritmul determina valoarea unui atribut A al unei instante O

DetVal (O, A, V)

1. Formeaza o lista L cu nodul O si toate nodurile legate de O prin relatia ISA

2. **cit timp** L $\neq \{ \}$ **executa**

2.1. Elimina primul nod, N, din lista L

2.2. **daca** atributul A al nodului N are valoarea V

atunci

2.2.1. Depune V in nodul punctat de atributul A al obiectului O

2.2.3. **intoarce** SUCCES

2.3. Adauga toate nodurile legate prin relatia AKO de nodul N, la sfirsitul listei L

3. **intoarce** INSUCCES

sfîrsit.

Observatie. In algoritmul prezentat se considera ca ierarhia de clase are o forma de arbore. Problemele care apar in cazul unei ierarhii de tip graf si posibile rezolvări ale acestor probleme vor fi prezentate in Sectiunea 6.3.

Dupa cum se vede si din algoritmul prezentat, un obiect poate fi instanta a mai multor obiecte generice, asa cum o clasa poate apartine mai multor superclase. De exemplu, un cub particular poate fi atit o instanta a unei clase Bloc cit si o instanta a unei clase Jucarie. O varianta a acestei situatii este surprinsa de unele modele de retele semantice in care se introduce notiunea de perspectiva. O *perspectiva* indica faptul ca un obiect poate avea sensuri diferite in contexte diferite, asa cum se vede in Figura 6.4.

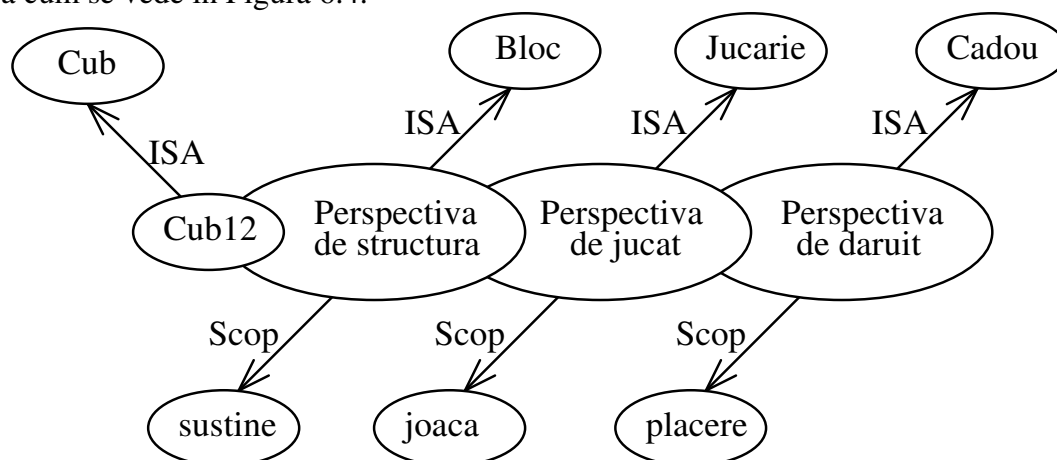


Figura 6.4 Utilizarea perspectivelor in retelele semantice

Daca se doreste aflarea valorii unui atribut pentru o anumita instanta atunci fie exista un atribut unic asociat unui obiect generic si se aplica algoritmul anterior, fie atributul apare in diverse perspective, cum ar fi atributul Scop in Figura 6.4, caz in care trebuie specificata si perspectiva din care intereseaza valoarea atributului. Notiunea de perspectiva este utilizata atit in modelul retelelor semantice cit si in modelul cadrelor.

Retelele sementice au fost extinse si prin introducerea unor caracteristici asociate atributelor din retea, numite *fatete*. Fatetele reprezinta diverse modalitati de considerare a valorilor unor attribute. Cele mai intilnite fatete in retelele semantice sint: fateta valoare, fateta valoare implicita si fateta procedura necesara. *Fateta valoare* este valoarea obisnuita a unui atribut, considerata in exemplele de pina acum. *Fateta valoare implicita* este utilizata pentru a caracteriza tipic valoarea unui atribut. De exemplu, se stie ca elefantii au in general culoarea gri, deci se poate considera ca valoarea implicita a atributului culoare a clasei elefantilor este gri. In lipsa unor informatii particulare despre elefantul Jumbo, se poate infera ca Jumbo are culoarea gri. Dar daca se stie ca elefantul Maia este alb, aceasta valoare a atributului culoare va fi introdusa si se va neglija fateta valoare implicita a clasei elefantilor. Utilizarea fatetei valoare implicita este o inferenta specifica a retelelor semantice care poate modela o forma de rationament nemonoton. In lipsa unor informatii contrarii, valoarea unui atribut este data de fateta valoare implicita. Daca ulterior se constata ca valoarea reala a atributului este alta, aceasta noua valoare devine valoarea atributului si valoarea implicita inferata anterior pentru acel atribut este retractata.

Tot in lumea blocurilor se considera exemplul din Figura 6.5. Caramida12 va mosteni valoarea implicita a atributului Culoare de la clasa Caramida, dar pentru Caramida0, despre care se stie ca are culoarea alba, aceasta inferenta implicita nu se mai aplica. In continuare se prezinta algoritmul de mostenire a valorilor implicite intr-o retea semantica.

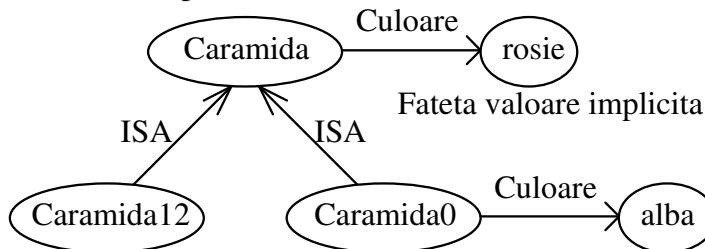


Figura 6.5 Mostenirea valorilor implicite in retele semantice

Algoritm: Mostenirea valorilor implicite ale atributelor intr-o ierarhie de clase

Algoritmul determina valoarea unui atribut A al unei instante O

DetValImp(O, A, V)

1. Formeaza o lista L cu nodul O si toate nodurile legate de O prin relatia ISA
 2. **cit timp** $L \neq \{ \}$ **executa**
 - 2.1. Elimina primul nod, N, din lista L
 - 2.2. **daca** atributul A al nodului N are valoarea implicita V **atunci**
 - 2.2.1. Depune V in nodul punctat de atributul A al obiectului O
 - 2.2.3. **intoarce** SUCCES
 - 2.3. Aadauga toate nodurile legate prin relatia AKO de nodul N, la sfirsitul listei L
 3. **intoarce** INSUCCES
- sfirsit.**

Fateta procedura necesara asociata unui atribut contine o procedura sau o functie care poate calcula valoarea atributului pe baza altor valori de attribute sau informatii existente in retea sau furnizate de utilizator. Aceasta fateta introduce o componenta procedurala in modelul descriptiv al reprezentarii structurate a cunostintelor. Fateta procedura necesara poate fi mostenita de-a lungul ierarhiei de clase la fel ca si fatele valoare si valoare implicita. Algoritmul utilizat este similar cu cel prezentat anterior, in care se considera insa fateta procedura necesara in loc de fateta valoare implicita, si va fi numit DetProcNec (O,A,V).

De exemplu, pentru cazul prezentat in Figura 6.6, greutatea instantei Caramida12 poate fi calculata pe baza valorilor atributelor Volum si Densitate, mostenind fateta procedura necesara a atributului Greutate de la superclasa Bloc de-a lungul unei relatii ISA, apoi a unei relatii AKO.

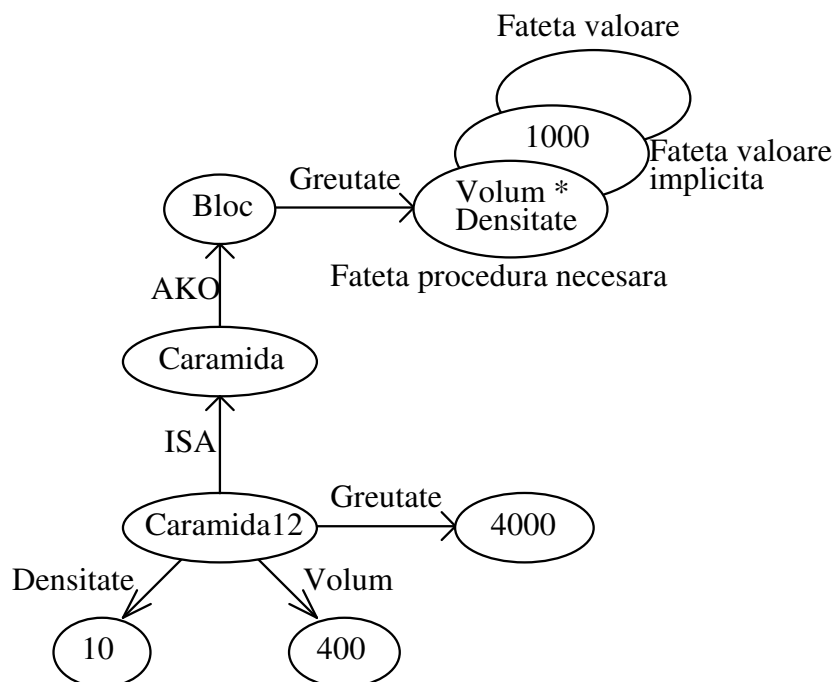


Figura 6.6 Mostenirea procedurilor necesare in retele semantice

Deși algoritmul de mostenire a fatetei procedura necesara este asemanator cu cei doi algoritmi prezentati anterior, pot apare complicatii suplimentare in cazul in care valorile de atribut utilizate in procedura de calcul nu se cunosc. In acest caz este necesar apelul subprogramelor de determinare a valorilor sau a valorilor implicite ale acestor atribut. In exemplul anterior, daca nu exista o valoare a atributului Volum asociat instantei Caramida12, procedura de calcul a greutatii trebuie intrerupta temporar pentru a determina valoarea acestui atribut pe baza unuia din algoritmi de mostenire prezentati pentru fatetele valoare sau valoare implicita.

Intr-o retea semantica imbogatita cu fatete ale nodurilor, mostenirea fatetelor valoare, valoare implicita si procedura necesara reprezinta forme de inferenta specifice. In functionarea generala a sistemului se pune insa problema executarii repetate, intr-o anumita ordine, a acestor inferente specifice. Strategia de control utilizata poate indica ordinea de aplicare a inferentelor si modul de inspectare a retelei. Exista doua strategii de control de baza: strategia N si strategia Z, a caror nume provine tocmai din disciplina de parcurgere a retelei semantice in inspectarea celor trei fatete existente. In continuare se prezinta algoritmi generali ai celor doua strategii, cititorul fiind indemnat sa descopere de ce ele se numesc N si Z.

Algoritm: Strategia N de determinare a valorii unui atribut

Algoritmul determina valoarea unui atribut A al unei instante O utilizind strategia N.

DetValN (O, A, V)

1. **daca** DetVal (O,A,V) = SUCCES
 atunci intoarce SUCCES
2. **daca** DetValImp (O,A,V) = SUCCES
 atunci intoarce SUCCES

3. **daca** DetProcNec (O,A,V)

atunci intoarce SUCCES

4. **intoarce** INSUCCES

sfirsit.

Algorithm: Strategia Z de determinare a valorii unui atribut.

 Algoritmul determina valoarea unui atribut A al unei instante O utilizind strategia Z.

DetValZ (O, A, V)

1. Formeaza o lista L cu nodul O si toate nodurile legate de O prin relatia ISA

2. **cit timp** $L \neq \{ \}$ **executa**

 2.1. Elimina primul nod, N, din lista L

 2.2. **daca** fateta valoare a atributului A a nodului N este V

atunci

 2.2.1. Depune V in nodul punctat de atributul A al obiectului O

 2.2.2. **intoarce** SUCCES

 2.3. **daca** fateta valoare implicita a atributului A a nodului N este V

atunci

 2.3.1. Depune V in nodul punctat de atributul A al obiectului O

 2.3.2. **intoarce** SUCCES

 2.4. **daca** fateta procedura necesara a atributului A a nodului N este proc $(A_1, \dots, A_n, V)$

atunci

 2.4.1. Determina valorile atributelor $A_1, \dots, A_n$ ale instantei O

 2.4.2. **daca** s-au gasit valori pentru $A_1, \dots, A_n$

atunci

 i. **executa** proc $(A_1, \dots, A_n, V)$

 ii. Depune V in nodul punctat de atributul A al obiectului O

 iii. **intoarce** SUCCES

3. **intoarce** INSUCCES

sfirsit.

Strategiile N si Z nu sint singurele strategii de control utilizate in retelele semantice, aceste strategii fiind utilizate numai pentru controlul mostenirii fatetelor valoare, valoare implicita si procedura necesara de-a lungul ierarhiilor de clase definite de retea. In functie de modelul particular de retea utilizat, diverse alte forme de inferenta si strategii de control asociate pot fi utilizate, cum ar fi cautarea intersectiei, pentru a aminti numai un exemplu simplu. Ideea introducerii fatetelor atributelor ce descriu obiecte cit si a mecanismelor de mostenire a valorilor

acestor fatete nu este particulara retelelor semantice, ea fiind intilnita si in paradigma unitatilor, in forme mai elaborate, asa cum se va vedea mai tirziu.

Retelele semantice au un corespondent direct in calculul cu predicate de ordinul I si, de altfel, modelul logic a fost intens utilizat pentru a explica semantica diverselor variante de retele semantice propuse. Consideratiile asupra legaturii dintre logica cu predicate si modelul reprezentarii structurate a cunostintelor, prezentate in Sectiunea 6.1, se aplica si la retele semantice. Se aminteste ca exemplul din Figura 6.2(b) este o rescriere folosind retele semantice a cunostintelor structurate prezentate in Figura 6.1, cunostinte care au fost initial exprimate utilizind formalismul logic.

Pentru rezolvarea problemelor utilizind retele semantice, este necesar sa se defineasca notiunea de identificare a doua retele semantice. In acest scop, se poate descrie un obiect (rete) scop care trebuie sa identifice cu un obiect (rete) din baza de cunostinte. In timpul procesului de identificare diverse noduri ale retelei semantice scop si ale retelelor din baza de cunostinte vor primi valori, de cele mai multe ori pe baza algoritmilor de determinare a valorii unui atribut, prezentati anterior. Identificarea a doua retele semantice se defineste pe baza corespondentei existente intre modelul structurat si logica cu predicate de ordinul I. O prima definitie a identificarii retelelor semantice este urmatoarea.

Definitie. Doua obiecte reprezentate prin retele semantice identifica daca si numai daca formulele bine formate asociate celor doua retele unifica.

In general, operatia de identificare a retelelor semantice nu este simetrica deoarece, asa cum s-a spus, exista un obiect scop care trebuie identificat cu un obiect al bazei de cunostinte. Din acest motiv se considera urmatoarea definitie relaxata a identificarii a doua retele semantice.

Definitie. Un obiect scop identifica un obiect fapt daca formula bine formata asociata obiectului scop unifica cu o subconjunctie a formulelor asociate obiectului fapt.

Aceasta a doua definitie revine la a afirma ca identificarea reuseste numai daca formula asociata obiectului scop poate fi demonstrata pe baza formulelor asociate obiectului fapt. De exemplu, pentru retelele semantice prezentate in Figura 6.7, reseaua semantica fapt definita de evenimentul de trimitere T2 identifica cu reseaua scop definita de T3 dar nu identifica cu reseaua semantica scop definita de evenimentul T4.

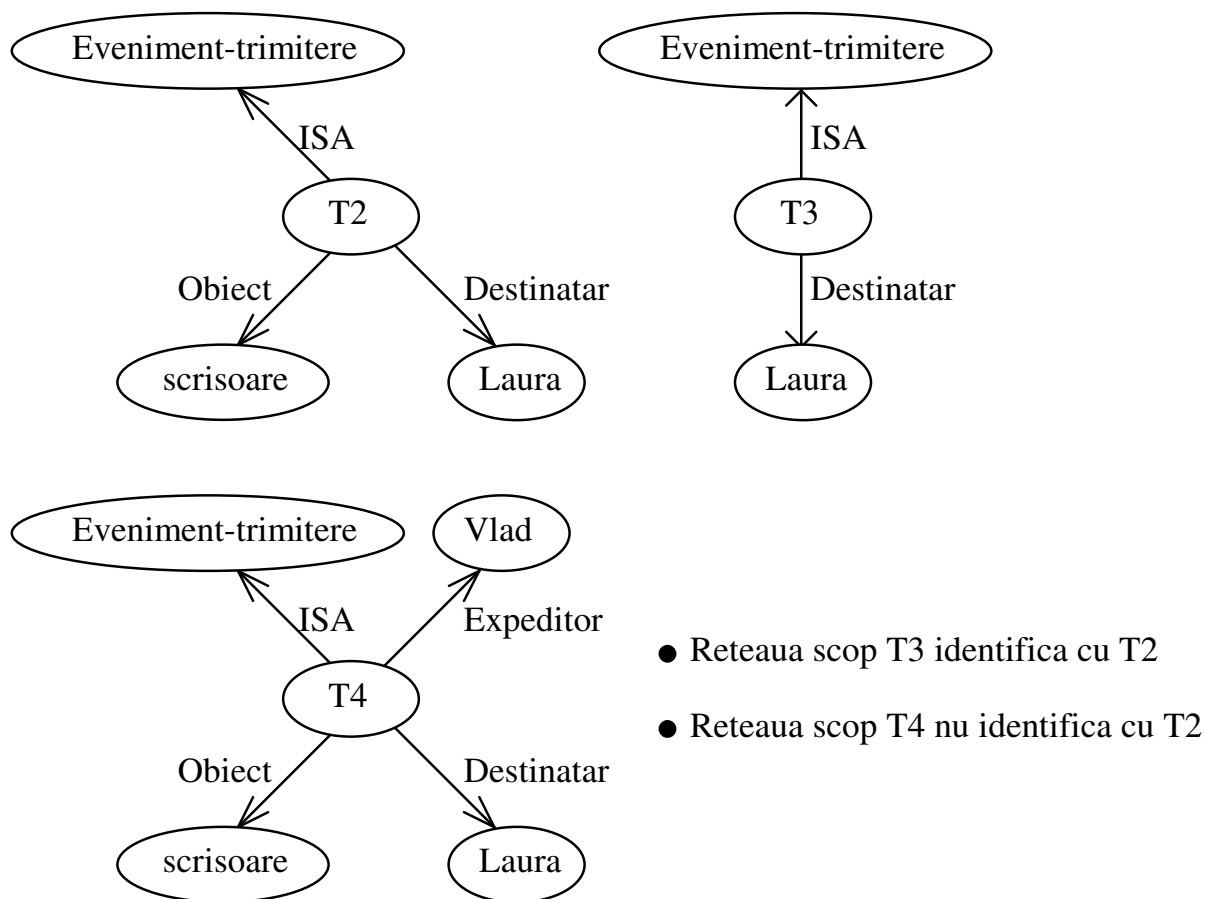


Figura 6.7 Identificarea rețelilor semantice

Cu toate avantajele prezentate ale rețelilor semantice, formalismul nu reușește să aibă toată puterea expresivă a logicii cu predicate de ordinul I. Pentru a elimina acest dezavantaj, Hendrix [1977] propune modelul *rețelilor semantice partitionate* ce are ca scop principal posibilitatea reprezentării într-o rețea a variabilelor cuantificate universal. În acest model extins, rețeaua este partitionată într-o multime de spații, fiecare spațiu fiind format din anumite noduri ale rețelei și reprezentând contextul unei variabile. În plus, există o clasă specială a asertiunilor generale cu două sau mai multe atribute: o formă, care specifică relația despre care se vorbește și una sau mai multe relații ($\forall$) care indică variabilele cuantificate universal.

6.2.3 Grafuri conceptuale

Prezentarea anterioară a rețelilor semantice a fost făcută utilizând numai o reprezentare grafică asociată, fără a specifica un limbaj de reprezentare a cunostintelor. Până în prezent, s-au dezvoltat o serie de limbaje de descriere a modelului rețelilor semantice, neexistând însă o sintaxă unanim acceptată, așa cum există, de exemplu, în cazul logicii cu predicate. Printre limbajele propuse, cel care pare să se fi impus este teoria grafurilor conceptuale introdusă la începutul anilor '80 de John F. Sowa. De atunci și până în prezent, grafurile conceptuale au fost extinse și aplicate în diverse domenii cum ar fi prelucrarea limbajului natural, modelarea bazelor de date și învățarea automată.

Un graf conceptual este o reprezentare grafică a percepției mentale care constă într-o multime de concepte primitive și relațiile existente între aceste concepte. Un astfel de graf este reprezentat printr-un graf finit conex bipartit în care nodurile reprezintă fie *concepte*, fie *relații*

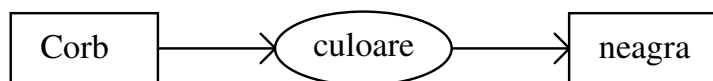
conceptuale (relatii între concepte). Grafurile conceptuale nu folosesc arce etichetate ca în modelul obisnuit al rețelelor semantice. Relatiile între concepte sunt reprezentate tot ca noduri, legate prin arce de conceptele implicate în relație. Unul dintre avantajele reprezentării relațiilor sub forma de noduri în loc de arce, deci al considerării unui graf bipartit, este acela de a simplifica reprezentarea relațiilor de diverse arități.

Pentru a putea face distincția între cele două tipuri de noduri dintr-un graf conceptual, prin convenție, nodurile concept sunt reprezentate prin dreptunghiuri, iar nodurile relații conceptuale prin ovale. Un singur graf conceptual este echivalent, în mare, cu reprezentarea unei propoziții în limbaj natural. Mai multe astfel de grafuri pot fi legate între ele pentru a reprezenta structuri complexe ale discursului. Figura 6.8 prezintă trei exemple de grafuri conceptuale, asociate propozițiilor "Coco zboară", "Culoarea corbilor este neagră" și "Matei mănincă supă cu lingură". Se observă apariția relațiilor conceptuale de aritate diferită și modul de reprezentare a acestora în graf.

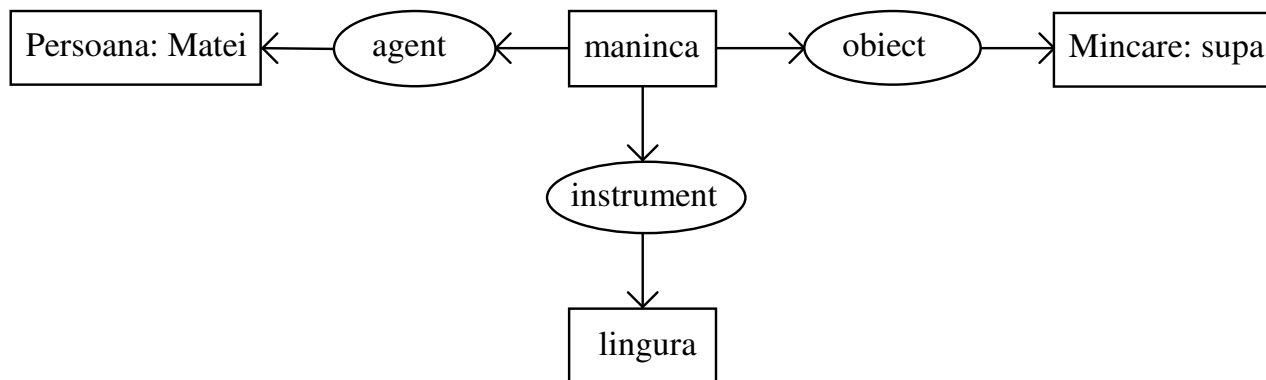
În grafurile conceptuale, nodurile conceptuale pot reprezenta atât *concepte concrete* (obiecte), cum ar fi Coco, Corb, supă, lingură, cit și *concepte abstracte* cum ar fi iubire, frumusețe, credință și, în exemplu, mănincă. Modelul de reprezentare permite atât reprezentarea conceptelor individuale, deci a instanțelor, cit și a conceptelor generice, deci a claselor. Fiecare nod concept reprezintă o instanță aparținând unei anumite clase, deci un *concept individual*. *Conceptul generic* care definește instanța apare în interiorul nodului sub forma unei *etichete de tip* a conceptului individual respectiv. Conceptul individual se mai numește și *referent*. În exemplul din Figura 6.8 conceptul individual Coco face parte din clasă (tipul) Pasare, iar tipul instanței Matei este conceptul generic Persoană. În cazul în care într-un nod concept apare numai conceptul generic fără specificarea conceptului individual, cum ar fi în exemplul anterior Corb sau lingură, acest lucru indică un concept individual nespecificat de acel tip, deci un oarecare corb sau o lingură oarecare.



(a) Coco zboara



(b) Culoarea corbilor este neagra



(c) Matei maninca supa cu lingura

Figura 6.8 Diferite grafuri conceptuale

Relatiile conceptuale agent, obiect si instrument sint relatii standard, care fac parte din multimea de concepte predefinite in reprezentare. In functie de problema particulara de reprezentat, se pot defini noi relatii conceptuale, cum ar fi in exemplu relatiile zboara si culoare.

Limbajul de descriere a grafurilor conceptuale este o liniarizare a reprezentarii grafice prezentate. Forma echivalenta a grafurilor din Figura 6.8 in acest limbaj este urmatoarea:

- (a) [Pasare: Coco]→(zboara)
- (b) [Corb]→(culoare)→neagra
- (c) [Persoana: Matei]←(agent)←[maninca]→(obiect)→[Mincare: supa]
→(instrument)→[lingura]

Limbajul grafurilor conceptuale contine urmatoarele conventii sintactice. Parantezele patrate corespund nodurilor concept, iar parantezele rotunde corespund nodurilor relatii conceptuale. Caracterul - indica continuarea unui graf conceptual liniar pe urmatoarea linie. Limbajul mai contine si alte semne speciale, cu semnificatii bine definite, cum ar fi: #, *, ?, @, ∇ si altele. Caracterul # introduce un marcaj al conceptelor individuale (marcaj individual) care pot fi astfel identificate printr-un numar. De exemplu:

[Papagal: #120]→(culoare)→[galbena]

indica faptul ca o pasare particulara, identificata prin marcajul #120, are culoarea galbena. Marcajele individuale sint diferite de numele conceptelor individuale deoarece, spre deosebire de nume, ele identifica unic acea instanta. In plus, aceste marcaje permit atit exprimarea numelui instantei ca o relatie in graf, cit si asocierea a mai multor nume unei instante, de exemplu:

[Papagal: #120]→(culoare)→[galbena]

$\rightarrow(\text{nume})\rightarrow["\text{Coco}"]$
 [Papagal: #121] $\rightarrow(\text{culoare})\rightarrow[\text{galbena}]$
 $\rightarrow(\text{nume})\rightarrow["\text{Coco}"]$
 $\rightarrow(\text{nume})\rightarrow["\text{Chanel}"]$

Semnul întrebării plasat într-un nod concept în locul referentului indică o întrebare referitoare la acel concept; de exemplu [Casa: ?] reprezintă întrebarea "Care casă?".

Caracterul * este un alt marcaj care indică o variabilă sau un concept individual nespecificat (marcaj generic). De exemplu, [Casa: *x] indică o casă oarecare. De multe ori acest marcaj este omis, deci considerat implicit, dar el este necesar în cazul în care două noduri concept diferite indică aceeași instanță nespecificată. Folosind această facilitate reprezentarea frazei "Papagalul își ciugulește penele cu ciocul." este următoarea:

[Papagal: *x] $\leftarrow(\text{agent})\leftarrow[\text{ciugulește}]\rightarrow(\text{obiect})\rightarrow[\text{pene}]\rightarrow(\text{parte})\rightarrow[\text{Papagal: *x}]$
 $\rightarrow(\text{instrument})\rightarrow[\text{cioc}]\rightarrow(\text{parte})\rightarrow[\text{Papagal: *x}]$

Caracterul @ indică o cantitate, [Casa: @n] reprezentând n case. Caracterul $\forall$ are aceeași semnificație ca în logica cu predicate de ordinul I, indicând cuantificarea universală. Similar, caracterul $\sim$ indică negația logică.

Conceptele generice într-o reprezentare prin grafuri conceptuale sunt organizate într-o ierarhie de clase, numită în acest formalism *ierarhie de tipuri*. Această ierarhie de tipuri formează o latice pe baza relației de ordine parțială definită de incluziunea între tipuri (clase). De exemplu, tipul Papagal este inclus în tipul Pasare care este inclus în tipul Fiintă. Laticea de tipuri include *tipul universal*, notat cu T, care este un supertip al tuturor tipurilor existente, și *tipul absurd*, notat cu $\perp$, care este un subtip al tuturor tipurilor.

Inferențele specifice grafurilor conceptuale sunt reprezentate printr-o mulțime de operații de transformare a grafurilor care permit crearea de noi grafuri din cele existente, și printr-o serie de inferențe inspirate din logica cu predicate de ordinul I. Există patru operații de bază pentru transformarea grafurilor conceptuale: copiere, restricție, reuniune și simplificare. *Operația de copiere* permite obținerea unui nou graf ca o copie exactă a unui graf dat. *Operația de restricție* permite înlocuirea unor noduri conceptuale din graf prin noduri specializate ale acestora, în două moduri: înlocuirea marcajului generic printr-un marcaj individual și înlocuirea etichetei de tip, deci a tipului unui concept cu un subtip al acestui tip (în ierarhia definită de laticea de tipuri). De exemplu, fiind dat graful conceptual

g1: [Pasare: Coco] $\rightarrow(\text{poziție})\rightarrow[\text{palmier}]$
 $\rightarrow(\text{culoare})\rightarrow[\text{galbena}]$

se poate obține următorul graf, g2, prin operația de restricție

g2: [Papagal: Coco] $\rightarrow(\text{poziție})\rightarrow[\text{palmier}]$
 $\rightarrow(\text{culoare})\rightarrow[\text{galbena}]$

Operația de reuniune permite combinarea a două grafuri conceptuale într-unul singur. Dacă există un nod concept C1 în graful g care este identic cu nodul concept C2 din graful g', atunci se poate forma un graf g'' prin eliminarea nodului C2 din g' și redirectarea tuturor relațiilor ce referă C2 în g' spre nodul C1 din g. Operația de reuniune este o regulă de specializare deoarece graful

rezultat g'' este mai puțin general decât fiecare din grafurile g și g'. De exemplu, prin operația de reuniune, din grafurile g2 și g3 descrise mai jos se obține graful g4.

g2: [Papagal: Coco]→(poziție)→[palmier]
→(culoare)→[galbenă]

g3: [Papagal: Coco]←(agent)←[manincă]→(obiect)→[banană]
→(instrument)→[cioc]
→(culoare)→[galbenă]

g4: [Papagal: Coco]←(agent)←[manincă]→(obiect)→[banană]
→(instrument)→[cioc]
→(poziție)→[palmier]
→(culoare)→[galbenă]
→(culoare)→[galbenă]

Operația de simplificare este operația prin care se elimină dintr-un graf relații duplicate, deci porțiuni identice din graf. Relațiile duplicate apar de obicei ca rezultat al unei operații de reuniune. De exemplu, aplicând operația de simplificare grafului g4 obținut anterior se obține graful g5:

g5: [Papagal: Coco]←(agent)←[manincă]→(obiect)→[banană]
→(instrument)→[cioc]
→(poziție)→[palmier]
→(culoare)→[galbenă]

Operația de restricție a unui graf conceptual poate fi utilizată pentru a identifica două noduri concept din două grafuri diferite, pentru ca apoi să se poată aplica operația de reuniune asupra acestor două grafuri. Dintr-un anumit punct de vedere, această operație poate fi privită ca un echivalent al procesului de unificare din logica cu predicate de ordinul I. Restricția combinată cu reuniunea permite implementarea mostenirii proprietăților în cadrul grafurilor conceptuale. De exemplu, înlocuirea unui marcaj generic cu unul individual implementează mostenirea atributelor pe baza unei relații ISA. Înlocuirea tipului individual cu un subtip în ierarhia de tipuri definește mostenirea proprietăților pe baza unei relații AKO.

Restricția și reuniunea sunt operații de specializare a grafurilor conceptuale. Dacă se definesc operațiile inverse se obțin operații de generalizare, pe baza cărora grafurile conceptuale pot fi utilizate ca instrument în învățarea automată (Capitolul 9).

Alte reguli de inferență utilizate în grafurile conceptuale reprezintă reformulări ale regulilor de inferență din logica cu predicate de ordinul I. Acestea sunt:

- *Stergere*. Orice graf conceptual inclus între un număr par de negații poate fi sters;
- *Inserare*. Orice graf conceptual poate fi inserat în contextul altui graf dacă acest context este între un număr impar de negații;
- *Iterație*. O copie a oricărui graf conceptual C poate fi introdus într-un context al altui graf în care apare C sau în care C este dominat de alt concept;
- *Deiterație*. Orice graf conceptual care ar putea fi rezultatul unei iterații poate fi sters din graful în care apare;
- *Negația dublă*. Orice dubla negație poate fi stearsă sau adusă în fața unui graf conceptual.

De exemplu, orice graf g poate fi inclus in implicatia $\sim[u \sim[v]]$ pentru a deriva $\sim[u g \sim[v]]$. Graful $\sim[u \sim[v]]$ este considerat o implicatie deoarece $P \rightarrow Q$ este echivalent cu $\sim(P \wedge \sim Q)$. Orice graf g poate fi sters din consecinta unei implicatii, deci din $\sim[u \sim[v g]]$ se poate obtine $\sim[u \sim[v]]$. Orice graf g poate fi sters din antecedentul unei implicatii numai daca a fost declarat explicit, adica din graful g si graful $\sim[g v \sim[w]]$ se obtine graful $\sim[v \sim[w]]$.

Se observa ca deiteratia si negatia dubla sint reguli de inferenta echivalente cu regula Modus Ponens din logica cu predicate de ordinul I. Fiind dat graful p si graful $\sim[p \sim[g]]$, deiteratia permite stergerea lui p si obtinerea grafului $\sim[\sim[g]]$ si, aplicind dubla negatie, se obtine graful rezultat g .

Aceste reguli de inferenta impreuna cu operatiile de transformare a grafurilor constituie un sistem inferential complet al modelului grafurilor conceptuale. In plus, exista reguli precise de transformare a unei reprezentari prin grafuri conceptuale intr-o reprezentare in logica cu predicate de ordinul I si invers. De altfel, modelul grafurilor conceptuale este considerat a fi o reprezentare grafica a unei formule logice. De exemplu, formula bine formata din calculul cu predicate

$$(\forall x)(\exists y) (\text{Papagal}(x) \rightarrow (\text{Cioc}(y) \wedge \text{Parte}(x,y)))$$

care exprima propozitia "Orice papagal are un cioc", poate fi exprimata in modelul grafurilor conceptuale prin

$$[\text{Papagal}: \forall] \rightarrow (\text{parte}) \rightarrow [\text{Cioc}]$$

Graful g_2 prezentat anterior poate fi exprimat in logica astfel:

$$(\exists x)(\exists y) (\text{Papagal}(\text{coco}) \rightarrow (\text{Palmier}(x) \wedge \text{Pozitie}(\text{coco}, x) \wedge \text{Galbena}(y) \wedge \text{Culoare}(\text{coco}, y)))$$

Regulile de inferenta de stergere, inserare, iteratie, deiteratie si negatie dubla sint inferente valide, provenind direct din logica cu predicate de ordinul I. Operatiile de transformare a grafurilor sint reguli de inferenta invalide dar utile si care, in plus, pastreaza sensul. Grafurile conceptuale au fost intens utilizate in reprezentarea cunostintelor necesare prelucrarii limbajului natural. Una din caracteristicile grafurilor conceptuale, care le-au facut interesante in astfel de aplicatii, este capacitatea de a exprima presupunerile implicite continute in propozitiile limbajului natural. Detalii suplimentare asupra modului de utilizare a grafurilor conceptuale in acest domeniu de aplicatie cit si regulile de transformare a grafurilor in logica cu predicate de ordinul I si invers pot fi gasite in Sowa [1984], Patterson [1990], Luger si Stubblefield [1993].

6.3 Unitati

Pe masura cresterii complexitatii problemelor abordate creste complexitatea cunostintelor necesare si apare nevoia unei structurari crescute a acestor cunostinte. Din acest motiv, retelele semantice au evoluat spre o alta paradigma de reprezentare structurata a cunostintelor: unitatile, numite si cadre, scheme, prototipuri sau clase. Desi nu exista o distinctie neta intre sistemele bazate pe retele semantice si sistemele bazate pe unitati, cu cit exista o structurare mai accentuata a cunostintelor in sistem, cu atit este mai probabil ca sistemul sa fie considerat un sistem bazat pe unitati. In plus, diversele limbaje bazate pe cadre propuse au dezvoltat o serie de caracteristici suplimentare ale acestei reprezentari fata de retelele semantice, cum ar fi: utilizarea extinsa a

sloturilor, atasarea procedurala, metaunitatile si includerea altor paradigme de reprezentare in modelul general al cadrelor. Cadrele au fost propuse de Minsky [1975] pentru a reprezenta si intelege obiectele unui scenariu intr-un sistem de perceptie vizuala automata. In lucrarea lui publicata in 1975, Minsky defineste cadrele astfel [Minsky,1975]:

"Esenta teoriei cadrelor este urmatoarea. Cind cineva intilneste o situatie noua (sau face o modificare esentiala a punctului de vedere asupra unei probleme), persoana respectiva selecteaza din memorie o structura numita 'cadru'. Aceasta este o structura amintita si modificata astfel incit sa se potriveasca realitatii prin schimbarea detaliilor in functie de necesitati."

6.3.1 Descrierea modelului unitatilor

O *unitate* este o colectie de attribute, numite de obicei *sloturi*, cu valorile lor asociate si posibile restrictii impuse asupra acestor valori, care descriu un obiect (entitate) al universului problemei. O singura unitate este rareori folositoare; sistemele bazate pe unitati folosesc o colectie de astfel de structuri legate intre ele pe baza faptului ca valoarea unui atribut al unei unitati poate fi o alta unitate. Cunostintele descrise in Figura 6.1 sint un model simplu de reprezentare bazata pe unitati.

La fel ca si in cadrul oricarei paradigme structurate de reprezentare a cunostintelor, unitatile pot desemna *obiecte generice*, deci multimi de obiecte cu caracteristici comune, sau *instante*, deci obiecte particulare ale acestor multimi. Relatiile ISA si AKO descrise in Sectiunea 6.1 se reprezinta in cadrul unitatilor sub forma unor attribute speciale si permit organizarea unitatilor unei baze de cunostinte in taxonomii. In continuare, pentru ilustrarea facilitatilor de reprezentare a unitatilor, se va utiliza un limbaj de reprezentare asemanator limbajului KEE [Kikes,Kehler, 1985], dar mult simplificat. Sistemul KEE este un sistem cadru de dezvoltare a sistemelor bazate pe cunostinte impreuna cu un mediu integrat, bazat in principal pe modelul unitatilor si care include cele mai multe din facilitatile propuse pentru acest model. Sistemul KEE s-a impus in ultimul timp datorita dezvoltarii a numeroase aplicatii scrise in KEE si a incorporarii unor tehnici noi de inteligenta artificiala cum ar fi: reprezentarea si utilizarea regulilor de productie prin unitati, compilarea bazei de cunostinte, utilizarea unui sistem de mentinere a consistentei datelor bazat pe presupuneri si integrat intr-un model de lumi posibile [Filman,1988;Filman,s.a.,1992].

In limbajul de descriere a unitatilor folosit in continuare, fiecare unitate este precedata de cuvintul cheie **Unit**, urmat de numele unitatii. Relatia de incluziune subclasa-clasa existenta intre unitati generice (AKO) este reprezentata prin doua attribute speciale, numite **SuperClasses** si **SubClasses**, ale caror valori pot fi alte unitati generice. Relatia de apartenenta a unui membru la o clasa, deci stabilirea unei instante a unei unitati generice (ISA), este reprezentata de atributul special **MemberOf**. Un obiect particular poate fi instanta a mai multe unitati generice, asa cum o unitate generica poate fi subclasa a mai multe clase. In consecinta, taxonomia de unitati nu mai este un arbore ci un graf (de cele mai multe ori o latice), asa cum se permite in cele mai multe limbaje bazate pe unitati existente la ora actuala. Inferentele specifice de mostenire a atributelor descris de in Sectiunea 6.1 pot fi astfel implementate in modelul unitatilor.

Pe langa cele trei attribute speciale **SuperClasses**, **SubClasses** si **MemberOf**, o unitate este descrisa printr-o multime de sloturi. Fiecare slot are un nume si una sau mai multe valori. Intr-o

unitate generica exista doua tipuri de sloturi: *sloturi membru*, numite **MemberSlot**, care descriu attributele fiecarui membru al clasei, si *sloturi proprii*, numite **OwnSlot**, care descriu attributele ce caracterizeaza clasa ca un intreg. In Figura 6.9 se prezinta o astfel de reprezentare, in care unitatile Camion, CamionMare, CamionMareRosu sint unitati generice, organizate ierarhic, iar unitatile CMR1 si CMR2 sint instante ale clasei CamionMareRosu. Sloturile membru ale unitatii generice CamionMareRosu devin sloturi proprii in cele doua instante CMR1 si CMR2.

Unit Camion

SuperClasses: Vehicol

SubClasses: CamionMare, CamionMediu, CamionMic

MemberOf: ObiecteFizice

Unit CamionMare

SuperClasses: Camion

SubClasses: CamionMareRosu, CamionMareRemorca

Unit CamionMareRosu

SuperClasses: CamionMare

MemberSlot: Sofer

Value: necunoscut

MemberSlot: Inaltime

Value: necunoscut

MemberSlot: Culoare

Value: rosie

MemberSlot: Pret

Value: necunoscut

OwnSlot: CelMaiMare

Value: CMR10

OwnSlot: CelMaiScump

Value: CMR210

Unit CMR1

MemberOf: CamionMareRosu, ProprietateFirmaX

OwnSlot: Sofer

Value: Paul

OwnSlot: Inaltime

Value: 2.00

OwnSlot: Culoare

Value: rosie

OwnSlot: Pret

Value: 3 000 000

OwnSlot: Proprietar

Value: X

Unit CMR2

MemberOf: CamionMareRosu

OwnSlot: Sofer

Value: Toderita

OwnSlot: Inaltime

Value: 2.00

OwnSlot: Culoare

Value: rosie

OwnSlot: Pret

Value: 10 000 000

Figura 6.9 Descrierea unor unitati generice si a unor instante

Sloturile proprii CelMaiMare si CelMaiScump ale unitatii generice CamionMareRosu caracterizeaza clasa in ansamblu si nu mai apar in instantele acesteia. Deci, in urma mostenirii

atributelor de la clasa la instanta, sloturile membru ale clasei devin sloturi proprii ale instantei iar sloturile proprii ale clasei nu se mostenesc la instante. Orice slot membru al unei clase este mostenit de subclasele descendente din acea clasa, in urma mostenirii atributelor de la clasa la subclasa. Se observa de asemenea existenta sloturilor membru cu valori "necunoscut", care vor capata valori in instante, cit si existenta sloturilor membru cu valori fixate, cum ar fi slotul Culoare, care indica ca toti membrii acelei clase trebuie sa aiba acea valoare pentru slot.

O unitate generica poate fi vazuta atat ca o clasa, cit si ca o entitate de sine statatoare. Asa cum s-a vazut, o unitate generica poate avea proprietati (attribute) care nu apartin instantelor individuale ale clasei, ci intregii clase (sloturile proprii). Din acest punct de vedere, o unitate generica poate fi vazuta ca o instanta a unei alte unitati generice speciale, numita de obicei *metaunitate* sau *metaclasa*. In acest fel, o unitate generica poate mosteni sloturile membru ale metaunitatii a carei instanta este, sub forma de sloturi proprii.

Unit Clasa

SuperClasses: Clasa

MemberOf: Clasa

MemberSlot: NumarMembrii

Unit ObiecteFizice

SuperClasses: Clasa

MemberOf: Clasa

OwnSlot: NumarMembrii

MemberSlot: CelMaiMare

Unit ProprietateFirma

SuperClasses: Clasa

MemberOf: Clasa

OwnSlot: NumarMembrii

Value: necunoscut

MemberSlot: ValoareTotala

Value: necunoscut

Unit ProprietateFirmaX

MemberOf: ProprietateFirma

OwnSlot: NumarMembrii

Value: 100

OwnSlot: ValoareTotala

Value: 100 000 000

MemberSlot: Proprietar

Value: X

Figura 6.10 Unitati generice si metaunitati

In exemplul considerat clasa Camion poate fi o instanta a metaclasei ObiecteFizice de la care mosteneste sloturile CelMaiMare si CelMaiScump, (prezentate de abia in unitatea CamionMareRosu din considerente de claritate). In orice sistem bazat pe unitati exista o cea mai generala metaunitate, fiecare clasa (unitate generica) putind fi o instanta a acestei metaunitati. In Figura 6.10 se prezinta o ierarhie de metaunitati. In plus, se detaliaza si unitatea

ProprietateFirmaX a carei instanta este unitatea CMR1 (CMR1 este o instanta a doua unitati generice, CamionMareRosu si ProprietateFirmaX, putind mosteni sloturi de la ambele unitati).

Metaclasele reprezinta o alta forma de metacunostinte, alaturi de metaregulile descrise in Capitolul 4. Spre deosebire de metareguli care specifica cunostinte de control, metaclasele sint metacunostinte care descriu structura reprezentarii, altfel spus reprezentarea reprezentarii.

Modelul unitatilor este de obicei extins prin modalitati de reprezentare a proprietatilor atributelor, numite *fatete*. Deci orice slot dintr-o unitate poate fi caracterizat printr-o multime de fatete. Dintr-o anumita perspectiva, un slot poate fi vazut ca o unitate ale carei sloturi sint fatetele. Cele mai frecvente fatete utilizate in paradigma unitatilor sint:

- *fateta valoare* care indica una sau mai multe valori ale slotului; in cazul in care nu se stie valoarea unui slot, fateta valoare indica necunoscut;
- *fateta domeniu de valori* care indica tipul admisibil al valorilor unui slot;
- *fatete* care descriu anumite *restrictii* ale valorilor unui slot, cum ar fi numarul de valori (attribute monovaloare sau multivaloare), limitari ale acestor valori sau conditii pe care aceste valori trebuie sa le indeplineasca;
- *fateta valoare implicita* care are acelasi rol cu cel descris in cazul retelelor semantice: atita timp cit nu se cunoaste valoarea unui slot se poate presupune ca valoarea acestui slot este valoarea implicita;
- *fateta mostenire* care indica explicit regulile de mostenire a valorii slotului de la diverse unitati; de obicei aceasta mostenire se realizeaza implicit de-a lungul relatiilor **SuperClasses** (AKO) si **MemberOf** (ISA) dar aceasta fateta poate indica si alte relatii pe baza carora se fac mosteniri de valori, cit si diverse prioritati in mostenire;
- *fateta valoare activa*, echivalenta fatetei procedura necesara din modelul retelelor semantice, si *fateta metoda* (sau slotul special metoda), fatete care ataseaza componente procedurale unui slot; aceste fatete vor fi discutate in sectiunea urmatoare;
- *fateta comentariu* care descrie semnificatia slotului.

In cazul fatetelor restrictii, se observa regasirea ideii de atribut monovaloare si multivaloare mentionate in Capitolul 4 sub forma valorii unice sau a valorilor multiple ale unui slot. Utilizind fatetele prezentate mai sus, descrierea unitatii CamionMareRosu prezentata in exemplul anterior poate fi reformulata asa cum se vede in Figura 6.11.

Unit CamionMareRosu
SuperClasses: CamionMare
MemberSlot: Sofer
Value: necunoscut /*fateta valoare */
ValueClass: Persoana /*fateta domeniu de valori; indica unitatea Persoana */
Cardinality: 2 /*fateta numar de valori; un camion poate avea doi soferi posibili */
Default: Paul /*fateta valoare implicita*/
Restrict: (oneof Paul, Toderita, Gelu, Mihai, Barbu) /* fateta de descriere a restrictiei */
MemberSlot: Inaltime
Value: necunoscut
ValueClass: real
Cardinality: 1
Restrict: X.Inaltime > 1.50
MemberSlot: Culoare
Value: rosie
ValueClass: string
Cardinality: 1
Comment: "Culoarea tuturor membrilor unitatii" /* fateta comentariu */

Figura 6.11 Sloturi si fatete intr-o unitate

In sistemele actuale bazate pe unitati, sloturile sint instante ale unor metaunitati speciale care le descriu in termenii fatetelor. Aceste metaunitati reprezinta o alta forma de metacunostinte de reprezentare, inasa discutarea acestei probleme depaseste cadrul prezentarii de fata.

6.3.2 Inferente specifice unitatilor

In cadrul modelului unitatilor, *mostenirea atributelor* este o forma de inferenta specifica, la fel ca si in cazul retelelor semantice. Daca ierarhia de unitati este arbore, algoritmi de mostenire sint identici cu cei prezentati la retelele semantice, pentru mostenirea valorilor, a valorilor implicate si a procedurilor necesare (Sectiunea 6.2.2). Spre deosebire de retelele semantice, cele mai multe limbaje de reprezentare bazate pe unitati permit mosteniri multiple asa cum s-a vazut in exemplul anterior, forma taxonomiei de unitati fiind in acest caz graf. Acesta este un graf orientat aciclic, in care exista o ordine partiala impusa de relatiile AKO. Existenta unei astfel de ierarhii induce o serie de complicatii conceptuale in realizarea mostenirii valorilor, a caror rezolvare necesita un nou algoritm de mostenire.

Fie exemplul din Figura 6.12, in care se revine la o notatie grafica a unitatilor pentru a simplifica prezentarea. Pe baza ierarhiei existente si a relatiei de mostenire se intreaba daca Fifi poate zbura. Raspunsul corect la aceasta intrebare este "nu" deoarece, chiar daca pasarile in general zboara, struti nu zboara (decit daca sint struti magici, dar in acest exemplu nu ne ocupam de struti magici). Aplicind algoritmul de mostenire a valorilor prezentat anterior pentru slotul Zboara, raspunsul depinde de ordinea de considerare a unitatilor de care este legata instanta Fifi prin relatia ISA (**MemberOf**), deci poate fi si "da", i.e. Fifi zboara. Este evident ca nu acesta este

raspunsul cautat si ca este nevoie de un algoritm de traversare a relatiilor ISA si AKO care sa garanteze prioritatea cunostintelor specifice fata de cele mai generale.

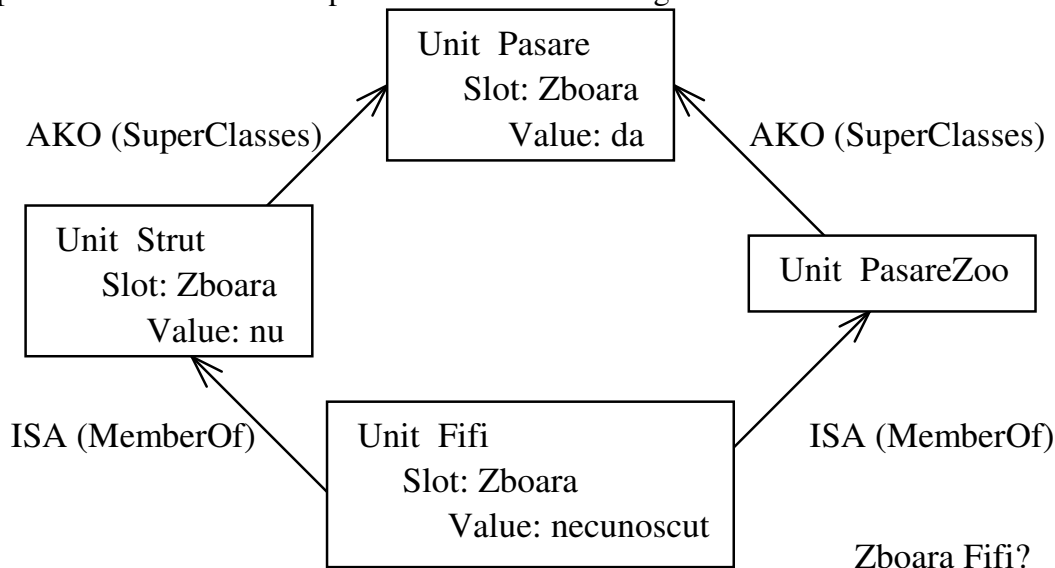


Figura 6.12 Taxonomie de unitati de tip graf orientat aciclic

O posibilitate de rezolvare a acestei probleme este specificarea explicita a ordinii de mostenire cu ajutorul fatetei mostenire a slotului. Acest lucru este posibil in anumite limbaje bazate pe unitati dar introduce destul de multe complicatii in construirea si specificarea bazei de cunostinte. O alta posibilitate este construirea unui algoritm inteligent care sa rezolve astfel de probleme.

O posibila idee care ar sta la baza unui algoritm de mostenire capabil sa rezolve mosteniri multiple de attribute este folosirea lungimii caii intre unitatea curenta U pentru care se doreste aflarea valorii slotului (atributului) S si unitatea U' unde s-a gasit aceasta valoare, considerind valoarea corecta valoarea slotului din unitatea cea mai apropiata de unitatea U. Acest algoritm ar putea fi implementat intr-o maniera similara cu cea prezentata in Sectiunea 6.2.2, selectind din lista unitatilor competitive unitatea cea mai apropiata de unitatea de interes U care are o valoare pentru slotul S. Un astfel de algoritm functioneaza pentru exemplul lui Fifi dar nu functioneaza pentru exemplul descris in Figura 6.13 in care unitatea Pasare se afla la o distanta mai mica de Fifi decit de unitatea Strut din care trebuie obtinuta valoarea corecta a slotului Zboara.

Functionarea incorecta a algoritmului propus provine din faptul ca lungimea caii intre doua unitati intr-o ierarhie nu corespunde intotdeauna nivelului de generalitate al unitatilor. De multe ori, nivelul de generalitate al unei unitati generice este dat de gradul de elaborare a unitatilor in baza de cunostinte. Solutia acestei probleme este construirea algoritmului de mostenire pe baza distantei inferentiale [Touretzky,1986] si nu pe baza distantei simple intre unitati.

Definitie. *Distanta inferentiala* intre clase (unitati generice si instante) se defineste astfel: Clasa1 este mai aproape de Clasa2 decit de Clasa3 daca si numai daca Clasa1 are o cale inferentiala care trece prin Clasa2 spre Clasa3. Cu alte cuvinte, Clasa1 este mai aproape de Clasa2 decit de Clasa3 daca Clasa2 este intre Clasa1 si Clasa3 de-a lungul unui lant de relatii ISA si AKO.

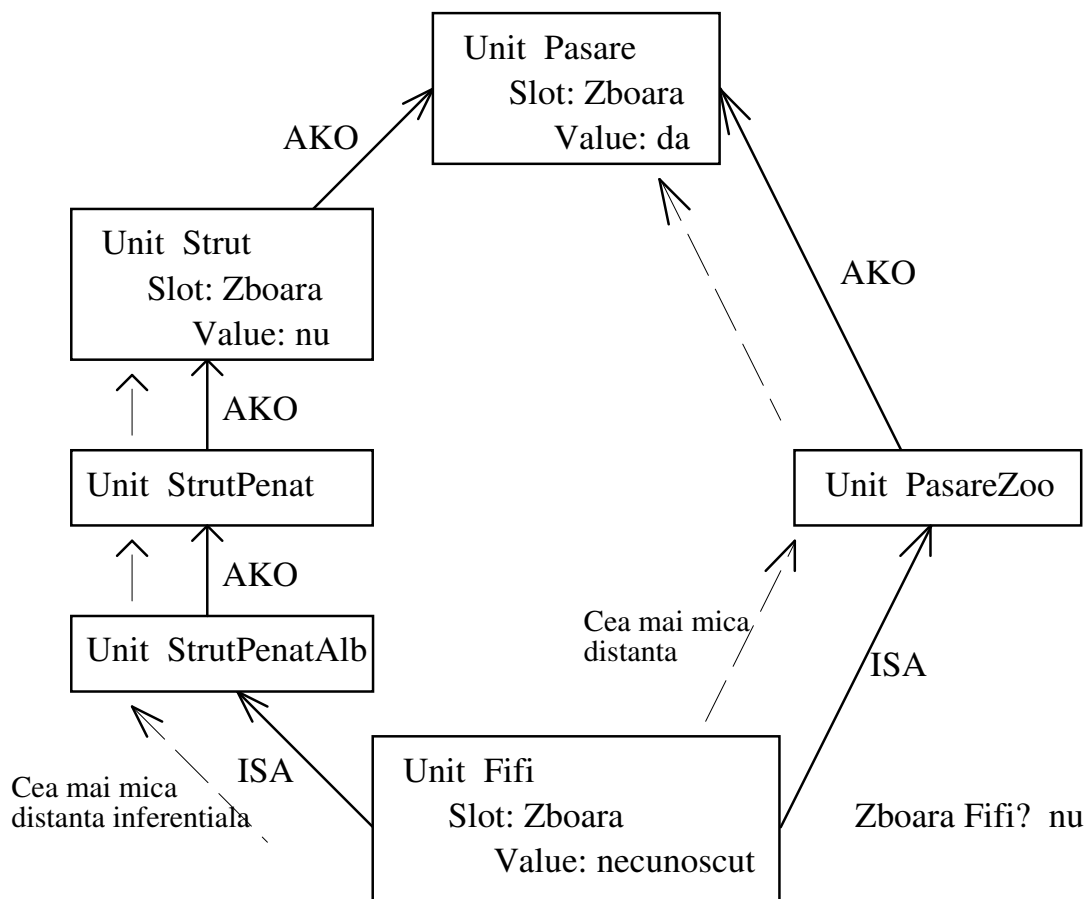


Figura 6.13 Distanța și distanța inferențială între unități

Folosind distanța inferențială, se poate defini mostenirea proprietăților astfel: mulțimea valorilor competitive ale unui slot S într-o unitate U conține acele valori care pot fi derivate dintr-o unitate X care este mai sus decât U în ierarhie și care nu sunt contrazise de valorile slotului S dintr-o unitate Y care are o distanță inferențială mai mică decât distanța inferențială a unității X față de U . Folosind această definiție pentru exemplul din Figura 6.13, se deduce valoarea corectă a slotului $Zboara$ al lui $Fifi$ deoarece unitatea $Strut$ are o distanță inferențială mai mică față de unitatea $Fifi$ decât distanța inferențială a unității $Pasare$ față de $Fifi$. Acest lucru este adevărat deoarece există o cale inferențială între $Fifi$ și $Pasare$ care trece prin unitatea $Strut$.

În continuare se prezintă algoritmul de mostenire a proprietăților bazat pe distanța inferențială pentru cazul în care sloturile unităților sunt monovaloare, deci admit o singură valoare.

Algoritm: Mostenirea proprietăților bazată pe distanța inferențială.

Algoritmul determină valoarea V a slotului S al unității U

1. Formează o listă L cu unitatea U și toate unitățile legate de U prin relația $MemberOf$
2. Formează o listă de candidați $CAND \leftarrow \{ \}$
3. **cit timp** $L \neq \{ \}$ **execută**
 - 3.1. Elimină prima unitate, X , din listă L
 - 3.2. **dacă** slotul S al lui X are valoare
atunci $CAND \leftarrow CAND \cup \{X\}$

3.3. **altfel** adauga in lista L toate unitatile legate de X prin relatia SuperClass

4. **pentru** fiecare unitate $C \in \text{CAND}$ **executa**

4.1. Verifica daca exista un alt element $C' \in \text{CAND}$ cu o distanta inferentiala fata de U mai mica decat aceea a lui C

4.2. **daca** C' exista
atunci elimina C din CAND

5. **daca** $\text{card}(\text{CAND}) = 0$
atunci intoarce INSUCCES /* nu s-a gasit valoare pentru S */

6. **daca** $\text{card}(\text{CAND}) = 1$
atunci

6.1. Fie C1 unicul element al listei CAND

6.2. Depune valoarea slotului S al lui C1 ca valoare a slotului S al lui U

6.3. **intoarce** SUCCES

7. **daca** $\text{card}(\text{CAND}) > 1$ /* contradictie, S este monovaloare */
atunci intoarce CONTRADICTIE

sfirsit.

Observatii:

- Algoritmul este garantat sa se termine deoarece ierarhia de relatii AKO este reprezentata de un graf aciclic orientat.
- Distanța inferentială definește o relație de ordine parțială între unități. Din această cauză pot exista unități generice între care nu se poate defini o distanță inferentială, cum ar fi exemplul din Figura 6.14. Pentru exemplul lui Cici algoritmul va deduce mai multe valori pentru slotul monovaloare UndeEste, deci se obține o contradicție, conform pasului 7. Acest lucru este corect deoarece există două extensii posibile a bazei de cunoștințe prezentată în Figura 6.14: una în care Cici este la Polul Nord și alta în care Cici este pe Raft.
- În cazul în care unitățile admit sloturi multivaloare, deci se accepta mai multe valori pentru un slot, algoritmul trebuie modificat în consecință.

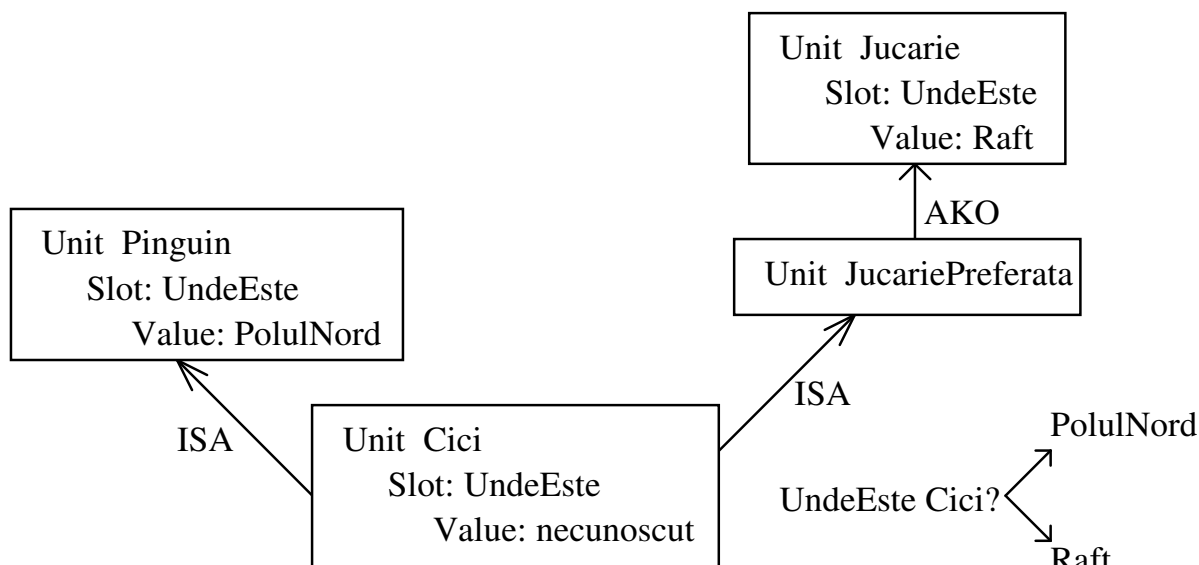


Figura 6.14 Situatie contradictorie pentru obtinerea valorii unui slot prin mostenire

O alta forma de inferenta specifica unitatilor este *atasarea procedurala* pe fatetele sloturilor si impunerea unei politici de executie a acestor componente procedurale. Exista doua tipuri de atasari procedurale in cazul modelului unitatilor: valori active si metode.

Valorile active, numite si *demoni*, corespund fatetelor procedura necesara prezentate la retele semantice. Valorile active sint proceduri sau functii atasate sloturilor sub forma de fatete, numite in limbajul utilizat in aceasta descriere **ActiveValue**, care sint automat invocate la fiecare obtinere sau modificare de valoare a unui slot. Aceste fatete se mai numesc si demoni deoarece ele se comporta ca niste spiridusi ce monitorizeaza orice utilizare sau schimbare de valoare a unui slot. Considerind unitatea CamionMareRosu imbogatita cu doua noi sloturi, asa cum se vede in Figura 6.15, se poate observa existenta fatetei valoare activa Actual.Localizare care reprezinta o procedura (demon) ce va actualiza pe o harta pozitia camionului mare si rosu de fiecare data cind aceasta pozitie se va schimba.

Metodele sint functii sau proceduri atasate sloturilor a caror executie este declansata de transmiterea unor mesaje catre acele sloturi. Un mesaj transmis trebuie sa specifice unitatea si slotul la care se refera mesajul si eventualii parametri necesari, intr-o maniera similara cu mecanismul transmiterii de mesaje din limbajele orientate pe obiecte. Considerind exemplul din Figura 6.15, in cazul in care camionul CMR2 este defect se transmite mesajul de diagnosticare catre slotul Diagnostic din unitatea CMR2. Valoarea acestui slot este obtinuta prin mostenire de la unitatea generica CamionMareRosu si are ca valoare o metoda, i.e. functia de stabilire a defectului. Efectul transmiterii mesajului, care poate contine eventual si parametri, cum ar fi simptomele constatate, este executia metodei si obtinerea diagnosticarii defectului camionului CMR2. In limbajul propus, componentele procedurale de tip metoda sint reprezentate prin sloturi speciale dar in alte limbaje ele pot fi si fatete ale sloturilor.

```

Unit CamionMareRosu
  SuperClasses: CamionMare
  ...
  MemberSlot: Localizare
    Value: necunoscut
    ValueClass: Oras
    ActiveValue: Actual.Localizare /* fateta valoare activa */
  MemberSlot: Diagnostic
    Value: funct.diag.Camion
    ValueClass: method /* slot metoda */
Unit CMR1
  MemberOf: CamionMareRosu, ProprietateFirmaX
  ...
  OwnSlot: Diagnostic
    Value: funct.diag.Camion
  OwnSlot: Defect
    Value: /* se va completa prin executia functiei de
diagnosticare din slotul Diagnostic */

```

Figura 6.15 Atasarea componentelor procedurale unitatilor

Rezolvarea problemelor utilizind reprezentarea cunostintelor pe baza unitatilor necesita descrierea bazei de cunostinte in termenii unitatilor generice si a unor instante partial sau total specificate, instantele avind rolul datelor initiale ale problemei. Solutia problemei va fi reprezentata de obtinerea uneia sau a mai multor instante sau a valorilor necunoscute de sloturi din instantele partial specificate. Deoarece valorile de sloturi pot fi la rindul lor unitati, deci pot implica noi procese de instantiere, rationamentul intr-un astfel de proces de rezolvare consta din multiple sarcini de obtinere a instantelor, care pot fi eventual intrerupte pina la terminarea executiei altor sarcini de instantiere si reluate ulterior. Structura de control utilizata frecvent pentru un astfel de mecanism de rezolvare a problemelor este controlul bazat pe agenda al carui algoritm a fost prezentat in Capitolul 4.

6.4 Structuri "tari" de reprezentare a cunostintelor

Modelele de reprezentare structurata a cunostintelor descrise in sectiunile anterioare sint modele generale. Retelele semantice sau unitatile particulare construite pentru o anumita clasa de probleme, pot defini relatii si inferente specifice dar nu contin reguli rapide, integrate in reprezentare, referitoare la tipul obiectelor si la relatiile descrise de structura. Stabilirea acestor reguli este lasata pe seama celui care construiesc sistemul bazat pe retele semantice sau cadre.

Cele doua modele de reprezentare structurata prezentate in aceasta sectiune, dependentele conceptuale si scenariile, sint considerate a fi structuri "tari" de reprezentare deoarece ele ofera notiuni specifice de descriere a tipului obiectelor si relatiilor existente intre acestea. Aceste reprezentari sint un exemplu de teorie a inteligentei artificiale care permite reprezentarea si utilizarea cunostintelor obisnuite, de bun simt, despre situatii comune intilnite in viata de zi cu zi. Dependentele conceptuale reprezinta actiuni sau evenimente singulare iar scenariile folosesc

dependentele conceptuale ca elemente componente pentru a reprezenta secvențe de evenimente corelate cauzal, întâmplări sau povestiri. Scenariile pot fi văzute drept un tip special de unități ale caror sloturi sînt fie obiecte fie structuri de dependente conceptuale.

6.4.1 Dependente conceptuale

Dependentele conceptuale sînt un model structurat care permite reprezentarea cunoștințelor conținute în propozițiile limbajului natural. Scopul dependentelor conceptuale este acela de a reprezenta cunoștințele astfel încît:

- reprezentarea să fie independentă de limbajul în care au fost formulate propozițiile;
- propoziții diferite, dar avînd aceeași semnificație, să aibă aceeași reprezentare sub forma dependentelor conceptuale;
- reprezentarea semnificației propozițiilor să fie neambiguă;
- să faciliteze executia inferențelor determinate de informațiile conținute în propoziții.

Din aceste motive, dependentele conceptuale folosesc o reprezentare a propozițiilor limbajului natural care nu se bazează pe cuvintele propoziției ci pe o mulțime de entități primitive conceptuale care pot fi combinate pentru formarea semnificației cuvintelor și a propoziției. Acest model a fost propus de Schank [1972] și a fost folosit în diverse programe de înțelegere a limbajului natural.

În teoria dependentelor conceptuale se disting cinci tipuri de elemente componente (primitive ontologice) care constituie componentele constructive ale reprezentării. Aceste elemente de bază sînt: entități, acțiuni, cazuri conceptuale, timpuri conceptuale și dependente conceptuale, fiecare element avînd o serie de subtipuri prezentate în continuare.

Entități

- *Obiectele* sau producătorii de scenarii, numite pe scurt PP, reprezintă persoanele (actorii) sau obiectele fizice (inclusiv memoria umană) care acționează în universul discursului.
- *Atributele* sau ucenicii scenariilor, numite pe scurt PA, reprezintă proprietățile producătorilor de scenarii.

Acțiuni

- *Acțiunile primitive*, numite pe scurt ACT. În reprezentarea dependentelor conceptuale acțiunile sînt formate dintr-o mulțime de acte primitive. Un set tipic de astfel de acțiuni primitive este următorul:
ATRANS Transferul făcut de o relație abstractă, de exemplu "a da"
PTRANS Transferul locației fizice a unui obiect, de exemplu "a merge"
PROPEL Aplicarea unei forțe fizice unui obiect, de exemplu "a împinge"
MOVE Mișcarea părții unui corp de către acel corp, de exemplu "a lovi"
GRASPA Preluarea unui obiect de către un actor, de exemplu "a smulge"
INGEST Înghețarea unui obiect de către o ființă, de exemplu "a minca"
EXPEL Expulzarea unui obiect din corpul unei ființe, de exemplu "a plinge"
MTRANS Transferul unei informații mentale, de exemplu "a destăina"
MBUILD Construirea informațiilor noi din cele vechi, de exemplu "a decide"

SPEAK Producerea sunetelor, de exemplu "a vorbi"

ATTEND Concentrarea atentiei unui organ de simt asupra unui stimul, de exemplu "a asculta"

- Ucenicii actiunilor, numiti pe scurt AA, reprezinta atribute sau proprietati ale actiunilor primitive

Cazuri conceptuale

- Caz obiect (o)
- Caz direct (D)
- Caz instrument (I)
- Caz recipient (R)

Timpuri conceptuale

- Conditional (c)
- Continuu (k)
- Prezent (nil)
- Trecut (p)
- Viitor (f)
- Negativ (/)
- Interogativ (?)
- Tranzitie (t)
- Inceputul tranzitiei (ts)
- Sfirsitul tranzitiei (tf)
- Atemporar (delta)

Timpurile conceptuale ofera o modalitate de modificare a descrierii evenimentelor prin indicarea timpului, modului sau aspectului unui verb, deci a informatiilor care apar de obicei in limbajul natural.

Dependente conceptuale

Aceste dependente reprezinta reguli semantice de formare a structurilor de dependente pe baza entitatilor si a actiunilor, cum ar fi relatia intre un actor si un eveniment sau relatia intre o actiune primitiva si un instrument. Exista o serie de astfel de dependente conceptuale stabilite, o parte dintre cele mai importante fiind prezentate in Figura 6.16. In aceasta figura prima coloana contine forma dependentei conceptuale, a doua coloana contine un exemplu de utilizare a dependentei conceptuale si a treia coloana contine exprimarea in limbaj natural a acestei dependente.

Elementele componente prezentate sint folosite pentru crearea *structurilor conceptuale*, numite si structuri de dependente sau conceptualizari, care pot descrie adecvat intelesul unei propozitii in limbaj natural. O astfel de structura conceptuala se reprezinta printr-o forma particulara de graf. In reprezentarea grafica a dependentelor conceptuale nodurile reprezinta

entitati sau actiuni, sagetile indica directia dependentelor, sagetile duble indica legaturi bidirectionale intre actori (entitati) si actiuni. In plus, toate legaturile (arcele) din graf au asociate etichete corespunzatoare relatiilor de caz, deci cazurilor conceptuale, sau relatii temporare, deci timpurilor conceptuale. De exemplu, structurile conceptuale din Figura 6.16 reprezinta urmatoarele reguli:

- Regula 1 descrie relatia intre un actor si evenimentul cauzat de acesta. Relatia de dependenta este biderctionala deoarece nici actorul nici evenimentul nu pot fi considerate mai importante in relatie. Litera p cu care este etichetata legatura indica timpul trecut.
- Regula 2 descrie relatia intre un producator de scenarii si un ucenic de scenarii. Multe descrieri de stari, cum ar fi inaltime, sint reprezentate in dependentele conceptuale prin valori cuantificate.

1.	PP $\longleftrightarrow$ ACT	Vlad $\xrightarrow{p}$ PTRANS	Vlad a fugit
2.	PP $\longleftrightarrow$ ACT	Vlad $\longleftrightarrow$ inalt($>$ medie)	Vlad este inalt
3.	PP $\longleftrightarrow$ PP	Vlad $\longleftrightarrow$ doctor	Vlad este doctor
4.	PP $\uparrow$ PA	baiat $\uparrow$ frumos	Un baiat frumos
5.	PP $\uparrow\uparrow$ PP	catel $\uparrow\uparrow$ Posedat-de Vlad	Catelul lui Vlad
6.	ACT $\xleftarrow{o}$ PP	Vlad $\xrightarrow{p}$ PROPEL $\xleftarrow{o}$ carut	Vlad a impins carutul
7.	ACT $\xleftarrow{R}$ PP $\swarrow$ PP	Vlad $\xrightarrow{p}$ ATRANS $\xleftarrow{R}$ Vlad $\swarrow$ Ana $\uparrow_o$ carte	Vlad a luat cartea de la Ana
8.	ACT $\xleftarrow{I}$ $\updownarrow$	Vlad $\xrightarrow{p}$ INGEST $\xleftarrow{I}$ Vlad $\uparrow_o$ inghetata $\uparrow_o$ executa lingurita	Vlad a mincat inghetata cu lingurita
9.	ACT $\xleftarrow{D}$ PP $\swarrow$ PP	Vlad $\xrightarrow{p}$ PTRANS $\xleftarrow{D}$ catel $\swarrow$ cutie $\uparrow_o$ hrana	Vlad a hranit catelul
10.	PP $\xrightarrow{R}$ PA $\swarrow$ PA	flori $\xrightarrow{R}$ dimensiune $>$ x $\swarrow$ dimensiune $=$ x	Florile au crescut
11.	$\xrightarrow{R}$ sau $\xrightarrow{R}$	Radu $\xrightarrow{p}$ PROPEL $\xleftarrow{o}$ glont $\xleftarrow{R}$ Mihai $\swarrow$ pusca Mihai $\xrightarrow{p}$ sanatate(-10)	Radu l-a impuscat pe Mihai
12.	$\downarrow$	Vlad $\xrightarrow{p}$ PTRANS	Vlad a fugit ieri
13.	$\downarrow$	Eu $\xrightarrow{p}$ PTRANS $\xleftarrow{o}$ Eu $\xleftarrow{D}$ acasa $\swarrow$ Eu Eu $\xrightarrow{p}$ MTRANS $\xleftarrow{o}$ iepure $\xleftarrow{R}$ CP $\swarrow$ ochi padure	Ducindu-ma acasa, am vazut un iepure
14.	$\downarrow\downarrow$	Eu $\xrightarrow{p}$ MTRANS $\xleftarrow{o}$ iepure $\xleftarrow{R}$ CP $\swarrow$ ochi padure	Am vazut un iepure in padure

Figura 6.16 Dependente conceptuale tipice

- Regula 3 descrie relatia dintre doua obiecte (producatori de scenarii), unul dintre acestea apartinand multimii definite de celalalt.
- Regula 4 defineste relatia intre obiect si un atribut al acestuia. Sageata este orientata spre obiectul care este descris.
- Regula 5 descrie relatia intre doua obiecte, unul dintre obiecte oferind o informatie specifica despre celalalt. Cele mai frecvente tipuri de informatie care sint reprezentate in acest fel sint relatiile de posesiune (Posedat-de), de localizare (Loc) si de continut fizic (Cont). Directia sagetii este spre conceptul descris.
- Regula 6 descrie relatia intre o actiune primitiva si producatorul de scenarii care este obiectul actiunii. Sageata este orientata spre actiune deoarece contextul actiunii particulare determina semnificatia relatiei fata de obiect.
- Regula 7 descrie relatia intre o actiune primitiva, sursa si destinatia actiunii.
- Regula 8 descrie relatia intre o actiune primitiva si instrumentul cu care se executa acea actiune. Instrumentul trebuie sa fie intotdeauna o conceptualizare, i.e. trebuie sa contina o actiune si nu un simplu obiect fizic.
- Regula 9 descrie o relatie intre o actiune primitiva, sursa si destinatia acelei actiuni.
- Regula 10 reprezinta relatia intre un obiect, starea initiala in care s-a aflat obiectul si starea finala in care se afla.
- Regula 11 descrie relatia intre o conceptualizare si o alta conceptualizare care a cauzat-o pe prima. Se observa ca sagetile indica dependenta unei conceptualizari de o alta si de aceea au o directie opusa directiei implicatiei existente intre concepte. Cele doua forme ale regulii descriu cauza unei actiuni (a) si cauza unei schimbari de stare (b).
- Regula 12 reprezinta relatia intre o conceptualizare si momentul de timp la care s-a produs evenimentul descris de conceptualizare.
- Regula 13 descrie relatia intre o conceptualizare si o alta conceptualizare ce reprezinta timpul in care s-a produs prima. Exemplul acestei reguli pune in evidenta modul in care dependentele conceptuale imita o prelucrare informationala specifica modelului mental uman: actiunea "a vedea" este reprezentata ca un transfer de informatie intre ochi si un element de prelucrare constient.
- Regula 14 descrie relatia intre o conceptualizare si locul in care aceasta s-a produs.

Utilizind aceste structuri conceptuale de baza, se pot construi structuri complexe corespunzatoare semnificatiei propozitiilor in limbaj natural. Se poate observa existenta unei asemanari intre modelul dependentelor conceptuale si cel al grafurilor conceptuale prezentat in Sectiunea 6.2.3. In ambele modele exista concepte primitive si relatii intre aceste concepte si ambele accepta o sintaxa grafica precisa. Spre deosebire de grafurile conceptuale, care sint structuri generale de concepte, strins legate de modelul logic de reprezentare, dependentele conceptuale sint orientate spre semantica cunostintelor pe care le descriu. Din punct de vedere al reprezentarii cunostintelor si a facilitatilor de rationament, dependentele conceptuale ofera urmatoarele avantaje:

- (1) Numarul de reguli de inferenta necesare rezolvarii problemei scade.
- (2) Multe reguli de inferenta sint deja incluse in reprezentare.

- (3) Structura initial construita pentru reprezentarea informatiei dintr-o propozitie contine sabloane care trebuie completate. Aceste sabloane pot fi folosite ulterior ca un mecanism de focalizare a atentiei programului pentru intelegerea propozitiilor urmatoare.

Fiecare dintre aceste trei avantaje sint discutate in continuare. In primul rind, utilizarea structurilor de granularitate fina a dependentelor conceptuale permite descrierea unui numar minim de reguli de inferenta referitoare la aceste structuri, de exemplu cite o regula pentru fiecare actiune primitiva. Apoi, ori de cite ori o astfel de structura apare, se vor folosi aceleasi reguli de inferenta. De exemplu, urmatoarele verbe: a da, a lua, a fura, a dona, implica toate transferul unui obiect de la o persoana la alta. In orice propozitie ar apare un astfel de verb, inferentele despre cine are obiectul dupa executia actiunii si cine a avut obiectul inaintea actiunii sint aceleasi. Aceste inferente pot fi asociate, de exemplu, cu actiunea primitiva ATRANS.

Un al doilea avantaj al reprezentarii sub forma dependentelor conceptuale este acela ca, pentru a construi reprezentarea, se utilizeaza atit informatiile explicite existente in propozitiile date cit si regulile de inferenta asociate reprezentarii. Odata ce aceste reguli au fost aplicate, rezultatele obtinute devin parte a reprezentarii si pot fi utilizate ulterior fara a mai exista necesitatea reaplicarii regulilor de inferenta. Pentru a clarifica acest aspect se considera propozitia:

"Tudor l-a amenintat pe Cornel ca ii sparge nasul."

Reprezentarea acestei propozitii folosind dependentele conceptuale este data in Figura 6.17. Pentru simplificarea expunerii, actiunea "crede" este reprezentata ca o simpla unitate (de fapt aceasta actiune ar trebui reprezentata in termenii altor actiuni primitive). Reprezentarea indica faptul ca Tudor l-a informat pe Cornel ca el (Tudor) va face ceva pentru a-i sparge nasul lui Cornel. Tudor a facut aceasta in scopul de a-l face pe Cornel *sa creada* ca daca el (Cornel) face un anumit lucru (care nu ii convine lui Tudor) atunci Tudor ii va sparge nasul. Ideea de a crede poate fi reprezentata in dependente conceptuale ca o actiune primitiva MTRANS a unui fapt in memoria lui Cornel. Actiunile a1 si a2 sint sabloane care se refera la actiuni viitoare, nespecificate inca.

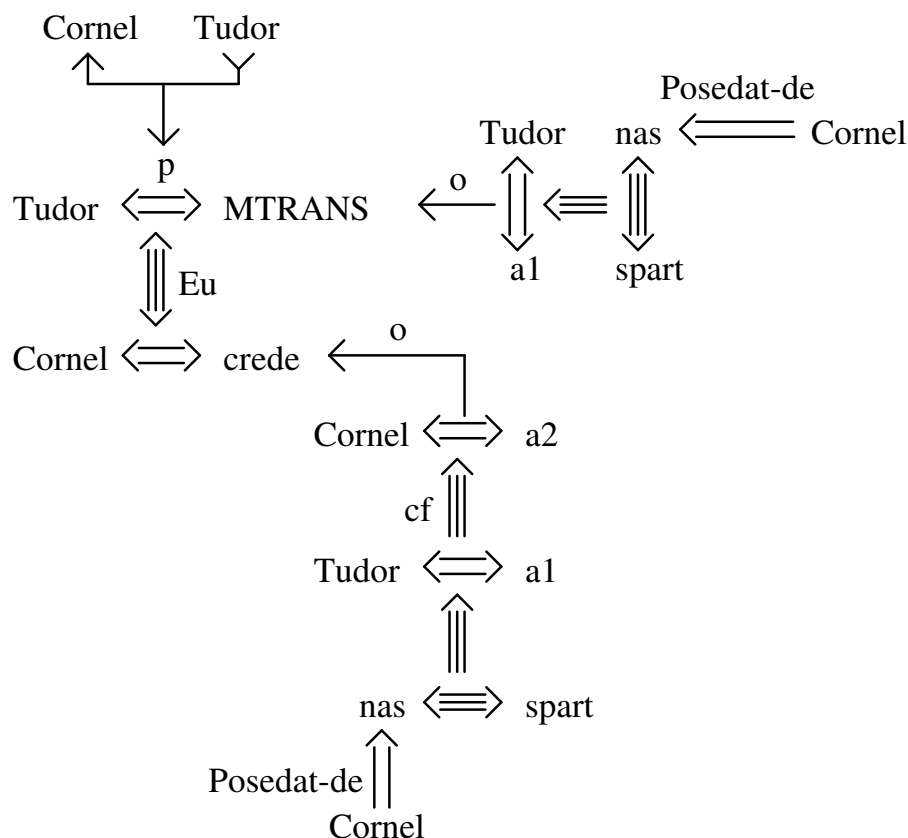


Figura 6.17 Reprezentarea sub forma dependentelor conceptuale a unei amenintari

Cel de-al treilea avantaj al dependentelor conceptuale este acela ca elemente nespecificate ale reprezentarii pot fi utilizate drept cheie a focalizarii atentiei pentru intelegerea evenimentelor ulterioare, pe masura producerii lor. De exemplu, dupa ce s-a introdus si inteles propozitia anterioara, "Tudor l-a amenintat pe Cornel ca ii sparge nasul.", se poate presupune ca se va afla in curind care este lucrul (actiunea) pe care Cornel l-ar fi putut face si care nu ii convine lui Tudor. Aceasta actiune poate fi utilizata pentru a inlocui actiunea necunoscuta a2. Prezenta sabloanelor in reprezentare ofera indicatii despre evenimentele sau obiectele de interes pentru intelegerea completa a unui anumit eveniment.

Pe langa avantajele enuntate, modelul dependentelor conceptuale prezinta si o serie de dezavantaje. In primul rind, reprezentarea cunostintelor in acest model necesita descompunerea cunostintelor intr-un numar mare de primitive la un nivel de granularitate scazut. Reprezentările obtinute pot fi deosebit de complexe pentru domenii mari ale discursului. Un al doilea dezavantaj al acestui model este faptul ca permite numai reprezentarea cunostintelor despre evenimente. Cunostinte cum ar fi descrierea obiectelor fizice sau actiuni sociale nu pot fi reprezentate adecvat cu ajutorul dependentelor conceptuale, desi au existat unele propuneri de extindere a modelului in acest sens.

6.4.2 Scenarii

Asa cum s-a vazut, dependentele conceptuale permit reprezentarea si rationamentul asociat evenimentelor. De cele mai multe ori insa, intereseaza reprezentarea unei succesiuni de evenimente, corelate intre ele. O astfel de reprezentare este posibila cu ajutorul scenariilor. Modelul de reprezentare a cunostintelor prin scenarii a fost introdus initial de Schank si Abelson

[1977] pentru a descrie intelesul unei povestiri sau intimplari exprimate in limbaj natural, in acelasi fel in care scenariul unei piese sau a unui film descrie actiunea piesei sau a filmului respectiv.

Un *scenariu* este o structura care descrie o secventa tipica de evenimente petrecute intr-un anumit context. Modelul scenariilor se bazeaza pe o structura predefinita asemanatoare unitatilor, formata din sloturi si care contine valori presupuse, inferente specifice si alte cunostinte relevante unei situatii stereotipice. Fiecare eveniment dintr-un scenariu este descris prin una din structurile conceptuale prezentate in sectiunea anterioara. Din punct de vedere al capacitatii de descriere a unor situatii comune, stereotipice, deci situatii ale caror caracteristici generale sint cunoscute, scenariile se aseamana cu cadrele, mai ales in sensul definitiei lui Minsky. Cele mai multe persoane stiu ce trebuie sa gaseasca si cum trebuie sa se comporte intr-un restaurant: la intrarea in restaurant chelnerul indica o masa libera, aduce meniul, preia comanda si asa mai departe. Pe baza unui astfel de scenariu tipic, predefinit, o povestire sau o secventa de evenimente referitoare la o persoana care intra intr-un restaurant poate fi inteleasa.

Spre deosebire de unitati, scenariile au o structura si o semantica specializata atit la nivelul organizarii generale datorita componentelor scenariului, cit si la nivelul elementelor constructive deoarece acestea sint dependente conceptuale. Componentele unui scenariu sint:

- *Conditii initiale.* Secventa de evenimente descrisa de un scenariu poate avea loc daca aceste conditii de intrare sint satisfacute, deci adevarate. De exemplu, in scenariul restaurantului conditiile de intrare pot fi: persoana este flaminda, persoana are bani, restaurantul este deschis, etc.
- *Rezultate.* Rezultatele sint conditiile care vor fi in general adevarate dupa petrecerea evenimentelor din scenariu. In scenariul restaurantului rezultatele pot fi: persoana nu mai este flaminda dar este mai saraca si proprietarul restaurantului are mai multi bani.
- *Proprietati.* Proprietatile sint sloturi care reprezinta obiecte implicate in evenimentele scenariului. Prezenta acestor obiecte in scenariu poate fi inferata chiar daca ele nu sint explicit mentionate in povestire. In cazul restaurantului se poate face presupunerea ca persoana va gasi in local mese, scaune, meniu (daca are noroc) si mancare.
- *Roluri.* Rolurile sint sloturi care reprezinta persoanele implicate in evenimentele scenariului. Prezenta acestor persoane in scenariu poate fi de asemenea inferata implicit, chiar daca nu este explicit mentionata. In cazul in care povestirea indica persoane specifice, acestea vor apare ca valori ale sloturilor roluri ce inlocuiesc valorile implicite. In scenariul restaurantului aceste persoane pot fi: clientul, chelnerul, bucatarul, proprietarul restaurantului.
- *Particularizari.* Particularizarile sint variatiile specifice ale unui scenariu general. Diferite particularizari ale unui scenariu pot avea atit componente comune cit si componente diferite. Dintr-un anumit punct de vedere, particularizarile pot fi vazute ca instante ale unor scenarii generale dar, spre deosebire de instantele unitatilor generice, particularizarile pot avea si componente diferite de scenariul general.
- *Scene.* Un scenariu este alcatuit dintr-o secventa de scene, fiecare scena corespunzind unui eveniment. Evenimentele individuale sint reprezentate prin dependente conceptuale.

Scenariile pot fi vazute ca un lant de relatii cauzale intre evenimente [Rich,Knight,1991]. Agentii scenariilor vor executa anumite actiuni care le vor permite executia altor actiuni si asa mai departe. Inceputul lantului cauzal este specificat de conditiile de intrare iar sfirsitul acestuia este caracterizat de rezultatele scenariului. In Figura 6.18 se poate vedea exemplul unui scenariu intr-un restaurant.

Scenariu: RESTAURANT Particularizare: bistro Proprietati: mese, meniu, M=mincare, bani Roluri: C=Client, H=Chelner, B=Bucatar, A=Casier, P=Proprietar Conditii initiale: C este flamind. C are bani. Rezultate: C nu are bani. P are mai multi bani. C nu mai este flamind. C este multumit (optional).
Scenariu 1: Intrare
C PTRANS C in restaurant C ATTEND ochi la mese C MBUILD unde sa se aseze C PTRANS la masa C MOVE C in pozitie asezat

Figura 6.18 Un scenariu intr-un restaurant

Scenariu 2: Comanda
(meniu pe masa) (H aduce meniu) (C cere meniu) C PTRANS meniu lui P C MTRANS semnal lui H H PTRANS H la masa C MTRANS 'vreau meniu' lui H H PTRANS H la meniu H PTRANS H la masa H ATRANS meniu lui C C MTRANS H la masa *C MBUILD alegerea lui M C MTRANS semnal lui H H PTRANS H la masa C MTRANS 'Eu vreau M' lui H

H PTRANS H lui B	
H MTRANS (ATRANS M) lui B	
H PTRANS H lui C	C EXECUTA (prepara scenariu M)
H MTRANS 'nu M' lui C	pentru Scena3
(salt la *) sau	
(salt la Scena4 la Fara plata)	
Scena3: Maninca	
B ATRANS M lui H	
H ATRANS M lui C	
C INGEST M	
(Optiune: salt la Scena2 pentru alta comanda;	
altfel salt la Scena4)	
Scena4: Iesire	
C MTRANS lui H	C ATRANS bacsis lui H
C PTRANS C la A	
C ATRANS bani lui A	
(Fara plata) C PTRANS C afara din restaurant	

Figura 6.18 (continuare) Un scenariu intr-un restaurant

Un program care utilizeaza reprezentarea cunostintelor sub forma de scenarii are o baza de cunostinte in care exista o multime de scenarii predefinite la care se adauga scenariile construite pe baza analizei textului povestirii, pornind de la scenariile predefinite. Pentru a utiliza un scenariu existent in baza de cunostinte, acesta trebuie cautat si activat. Exista doua posibilitati de activare a scenariilor, corespunzatoare importantei acestora in discurs la un moment dat. Scenariile care sint doar mentionate intr-o povestire nu trebuie activate total ci este suficienta mentinerea unei referinte catre scenariu. Acesta este cazul scenariului restaurantului in urmatoarea secventa de evenimente:

"Gabriela a trecut pe linga restaurantul ei preferat in drum spre facultate. Ea trebuia sa ajunga la cursul de inteligenta artificiala."

Scenariile importante, care reprezinta esenta discursului la un moment dat, trebuie activate total si programul trebuie sa incerce instantierea sloturilor scenariului cu obiectele si persoanele specifice mentionate in situatia curenta. Activarea totala a scenariilor se face pe baza sloturilor relevante din scenariu, cum ar fi conditiile initiale, proprietatile, rolurile sau evenimentele scenariului. Pentru reducerea numarului de activari ale unui scenariu, s-a constatat ca este necesar ca un context sa contina cel putin doua valori de sloturi relevante pentru ca scenariu sa fie total activat.

Odata ce un scenariu a fost activat si sloturile lui au fost instantiate (nu neaparat toate), acesta poate fi utilizat pentru a raspunde la diverse intrebari in legatura cu situatia descrisa de scenariu. Cea mai importanta proprietate a scenariilor este capacitatea de a da raspunsuri referitoare la cunostinte si evenimente care nu au fost explicit indicate in povestire. Se considera urmatoarea povestire:

"Aseara Vlad s-a dus la restaurant si a comandat o friptura. Dupa ce a platit-o, a observat ca a ramas fara bani. El s-a grabit spre casa deoarece incepuse sa ploua."

Scenariul asociat acestei povestiri este cel prezentat in Figura 6.18. Utilizind acest scenariu, programul poate da raspunsuri corecte la intrebari de tipul "A mincat Vlad aseara?", "Cum a platit Vlad la restaurant?", "Ce a mincat Vlad aseara?", desi nici unul din raspunsurile la aceste intrebari nu este explicit indicat in povestire.

O a doua proprietate importanta a scenariilor este posibilitatea de a construi o interpretare coerenta a unei multimi de evenimente observate. Deoarece un scenariu este un lant cauzal de evenimente, el poate oferi informatii despre legaturile existente intre evenimente. Fie urmatoarea povestire:

"Maria s-a dus la cina. Ea s-a asezat la masa si a chemat chelnerita. Chelnerita i-a adus meniul si ea a comandat o friptura."

Daca se pune intrebarea "De ce i-a adus chelnerita meniul Mariei?", programul poate sa dea doua raspunsuri pe baza informatiilor continute in scenariul asociat. Primul raspuns este "Deoarece Maria i-a cerut-o." si este obtinut prin parcurgerea inversa a lantului cauzal si aflarea evenimentului care a generat evenimentul curent. Al doilea raspuns posibil este "Pentru ca Maria sa-si aleaga felul de mancare." si este obtinut prin parcurgerea inainte a lantului cauzal si aflarea evenimentului a carui producere este posibila datorita evenimentului curent. Considerind prima proprietate a scenariilor, se pot da raspunsuri si la urmatoarele intrebari: "A fost Maria la restaurant?", "Cine a platit friptura?", "Cine a comandat friptura?". Ultima intrbarea este mai dificila deoarece ultima femeie mentionata inaintea utilizarii pronumelui "ea" este chelnerita si nu Maria. Rolurile din scenarii pot fi utilizate pentru a rezolva referintele pronominale si alte ambiguitati de acest tip.

O alta caracteristica importanta a scenariilor este faptul ca permit focalizarea atentiei programului asupra evenimentelor neobisnuite. Acest lucru se poate observa in exemplul povestirii urmatoare:

" Vlad s-a dus la restaurant. A fost condus la o masa. El a comandat o friptura. A stat la masa si a asteptat mult timp. Vlad s-a enervat si a plecat."

Partea importanta a acestei povestiri este momentul in care ea se abate de la seria de evenimente obisnuite intr-un restaurant. Vlad nu s-a enervat deoarece a fost condus la o masa ci deoarece a trebuit sa astepte mult pina sa fie servit. Odata ce secventa tipica de evenimente a fost intrerupta, scenariul nu mai poate fi folosit pentru a prezice alte evenimente. De exemplu, in aceasta povestire nu se poate infera ca Vlad a achitat nota de plata dar se poate infera ca a consultat meniul deoarece acest eveniment apare in scenariu inaintea intreruperii scenariului.

Reprezentarea secvențelor de evenimente prin scenarii pune anumite probleme, cum ar fi problema identificării [Luger,Stubblefield,1992]. Fie următorul exemplu de povestire:

"În drum spre concert, Vlad a fost la restaurantul lui preferat. Lui nu i-a parut rău de banii plătiți deoarece îi place Mozart."

Această povestire poate să activeze atât scenariul restaurant cât și scenariul concert deoarece elementul "banii plătiți" poate fi atât la restaurant (apare ca slot relevant în scenariul restaurant), cât și la concert (se presupune că acest slot apare și în scenariul concert). În plus, scenariile sunt reprezentări structurate mai puțin generale decât entitățile și, din acest motiv, nu sunt adecvate pentru reprezentarea tuturor tipurilor de cunoștințe. Cu toate acestea, scenariile pot fi eficiente în reprezentarea cunoștințelor specifice pentru care au fost proiectate, așa cum a demonstrat programul "Script Applier Mechanism", pe scurt SAM, sau alte programe destinate prelucrării, planificării și înțelegerii povestirilor, cum ar fi PAM, POLITICS, BABEL, CYRUS.

6.5 Exerciții și probleme

1. Să se exprime următoarele enunțuri sub forma unei rețele semantice.
 - (a) Societatea X este o societate de dezvoltare a programelor.
 - (b) Cele trei divizii ale societății sunt: Vinzari, Administrație și Programe.
 - (c) Vlad este șeful diviziei Vinzari, iar Mihai și Elena sunt programatori.
 - (d) Elena este căsătorită cu Radu.
 - (e) Radu este inginer.
 - (f) Ei au trei copii și trăiesc în Timișoara.
 - (g) Elena poartă ochelari și este blondă.
2. Să se reprezinte următoarele enunțuri folosind toate modelele de cunoștințe structurate descrise în acest capitol.
 - (a) Mihai i-a dat Mariei o cutie de bomboane.
 - (b) Radu este inginer.
 - (c) Gelu conduce bine mașina.
 - (d) Gelu conduce o rudă la gară.
3. Să se aleagă un limbaj de programare și să se implementeze algoritmi de moștenire a valorilor, a valorilor implicite și a procedurilor necesare pentru rețele semantice, algoritmi prezentați în Secțiunea 6.2.2.
4. Să se reprezinte sub forma de grafuri conceptuale următoarele formule bine formate din logica cu predicate de ordinul I.

- (a) $(\forall x)(\text{Normal}(x) \wedge \text{Adult}(x) \rightarrow \text{Merge}(x))$
- (b) $(\forall x)(\forall y)(\text{Casatorit}(x,y) \rightarrow \text{Casatorit}(y,x))$
- (c) $(\forall x)(\text{AreAripi}(x) \wedge \text{FaceOua}(x) \rightarrow \text{Pasare}(x))$

5. Sa se indice forma logica echivalenta a urmatoarelor grafuri conceptuale.

- (a) [Persoana: Ela] $\leftarrow$ (Agent) $\leftarrow$ [Bea] $\rightarrow$ (Obiect) $\rightarrow$ [Mincare:lapte]
 $\rightarrow$ (Instrument) $\rightarrow$ [Pahar]
- (b) [Camila: Cleo] $\leftarrow$ (Agent) $\leftarrow$ [Bea] $\rightarrow$ (Obiect) $\rightarrow$ [Apa] $\rightarrow$ (Atribut) $\rightarrow$ [50-litri]

6. Utilizind modelul unitatilor, sa se reprezinte urmatoarele cunostinte sportive. Nelu este pivot si membru al echipei PicioareIuti. Echipa PicioareIuti este o echipa de baschet. O echipa de baschet are un antrenor si un numar de membri. Echipa de baschet este o echipa sportiva. Metaclasa echipa sportiva este caracterizata de numarul total de echipe sportive existente. Pivotul unei echipe de baschet este un jucator de baschet care la rindul lui este un barbat, deci o persoana.

7. Sa se modifice algoritmul de mostenire a proprietatilor bazat pe distanta inferentiala, prezentat in Sectiunea 6.3.2, pentru cazul in care unitatile contin si sloturi multivaloare (sloturi cu mai multe valori posibile). Tipul unui slot (monovaloare sau multivaloare) este indicat de fateta cardinalitate asociata slotului.

8. Sa se construiasca o reprezentare sub forma de dependente conceptuale a propozitiei

Vlad a rugat-o pe Ana sa-i dea un creion.

Cum poate fi folosita aceasta reprezentare pentru a raspunde la intrebarea

I-a vorbit Vlad Anei?

9. Sint dependentele conceptuale o forma de reprezentare adecvata pentru a descrie cunostintele dintr-un articol de istorie? Dar pentru unul de geografie?

10. Sa se propuna structuri de date pentru reprezentarea dependentelor conceptuale. Considerind un text format din propozitii care pot fi descrise de regulile conceptuale 1, 2, 3 si 6 (Figura 6.16) sa se scrie un program care primeste la intrare acest text si construiesc dependentele conceptuale asociate.

Anexa

Dictionar roman-englez de termeni semnificativi

Capitolul 1

Ce este inteligenta artificiala?

cunostinte	knowledge
ingineria cunostintelor	knowledge engineering
sistem bazat pe cunostinte	knowledge based system
stiinta cunoasterii	cognitive science
limbaje de nivel foarte inalt	very high level languages
retele neuronale	neural nets / neural networks

Capitolul 2

Strategii de rezolvare a problemelor

rationament direct	forward reasoning
rationament invers	backward reasoning
cautare directa	forward search
cautare inversa	backward search
cautare bidirectionala	bidirectional search
control condus de date	data-driven control
control condus de scopuri	goal-driven control
strategie de cautare	search strategy
strategie de cautare de baza	basic / uninformed search strategy
strategie de cautare pe nivel	breadth first search
strategie de cautare in adincime	depth first search
cautare in adincime cu nivel iterativ	depth first iterative deepening
nod rezolvat	solved node
nod nerezolvabil	unsolvable node
functie euristica	heuristic function
graf SI/SAU	AND/OR graph
nod explorat	explored node
nod expandat	expanded node
strategie de cautare neinformatata	uninformed search strategy
strategie de cautare informatata	informed / guided / directed search strategy

strategia alpinistului	hill-climbing strategy
strategie de cautare de cost uniform	uniform cost strategy
FRONTIERA	OPEN
TERITORIU	CLOSED
analiza bazata pe modalitati	means-end analysis
taierea spatiului de cautare	pruning
factor de ramificare	branching factor
graf orientat aciclic	directed acyclic graph / DAG

Capitolul 3

Modelul logicii simbolice

rationament de bun simt	common sense reasoning
respingere prin rezolutie	refutation resolution
clauza	clause
clauza de baza	ground clause
clauza unitara	unit clause
clauza (Horn) distincta	definite (Horn) clause
unificator	unifier
expresii unificabile	unifiable expressions
cel mai general unificator	most general unifier / mgu
strategia dezvoltarii pe latime	level saturation strategy / breadth strategy
strategia rezolutiei semantice	semantic resolution
strategia rezolutiei liniare	liniar resolution
strategia multimii suport	set of support strategy
rezolutie de intrare liniara	linear input resolution
rezolutie unitara	unit resolution

Capitolul 4

Modelul regulilor de productie

sisteme de productie	production systems
reguli de productie	production rules
sistem bazat pe reguli / SBR	rule-based system / RBS
partea stinga a regulii	left-hand side / LHS
partea dreapta a regulii	right-hand side / RHS
unitate de cunostinte	chunk of knowledge
identificarea regulilor	rule match
ciclu de inferenta	recognize-act cycle
multime de conflicte	conflict set

rezolvarea conflictelor	conflict resolution
identificare	match
selectie	select
actiune	act
inlantuire inainte	forward chaining
inlantuire inapoi	backward chaining
focalizarea atentiei	attention focussing
criteriu de specificitate	specificity
cunostinte de control	control knowledge
incearca toate regulile	try-all
factor / coeficient de certitudine	certainty factor
accumulare de probe	evidence gathering
sarcina	task
sistem bazat pe agenda	agenda driven systems / agenda based system
metacunostinte	meta-knowledge
metareguli	meta-rules

Capitolul 5

Cunostinte incerte si rationament statistic

eveniment	event
spatiu de esantionare	sample space
ipoteza statistica	hypothesis
proba	evidence
incredere / convingere	belief
masura a increderii	measure of belief / MB
masura a neincrederii	measure of disbelief / MD
coeficient / factor de certitudine	certainty factor / CF
teoria matematica a probelor	mathematical theory of evidence
ipoteza individuala	singleton
cadru de selectare	frame of discernment
atribuire probabilistica de baza	basic probability assignment / bpa
functie de incredere	belief function / Bel
increderea in A	confidence in A
interval de incredere	confidence interval
plauzibilitate / prag de semnificatie	plausibility
metoda transmiterii mesajelor	message-passing method
metoda triunghiulara	clique triangulation method

Capitolul 6

Modelul cunostintelor structurate

rețele semantice / rețele asociative	semantic nets / associative nets / semantic networks / associative networks
structuri "slabe" de reprezentare	weak slot-and-filler structures
structuri "tari" de reprezentare	strong slot-and-filler structures
unitati / cadre / scheme	units / frames / schemas / prototypes / prototipuri / clase
clase / slot-and-filler structures	
obiecte particulare / instante	instances
fateta	facet
mostenirea proprietatilor	property inheritance
este un / este o	is a / ISA
un fel de	a kind of / AKO
grafuri conceptuale	conceptual graphs
dependente conceptuale	conceptual dependencies
scenarii	scripts
reprezentare orientata pe obiecte	object oriented representation
sistem cadru	system shell
demoni	demons
distanța inferentială	inferential distance
cautarea intersecției	intersection search
concept-cuvînt	word concept
valoare implicită	default
procedura necesară	if-needed procedure
rețele semantice partitionate	partitioned semantic nets
operație de copiere	copy
operație de restricție	restrict
operație de reuniune	join
operație de simplificare	simplify
marcaj	marker
referent	referent
stergere	erasure
inserare	insertion
iteratie	iteration
deiteratie	deiteration
cazuri conceptuale	conceptual cases
timperi conceptuale	conceptual cases
obiect / producator de scenarii	object / picture producer / PP
atribute / ucenicul scenariului	attribute / picture aiders / PA
actiune primitiva	primitive action / ACT
ucenicul actiunii	action aiders / AA

caz obiect	objectiv case
caz direct	directive case
caz instrument	instrumental case
caz recipient	recipient case
roluri	roles